Criterios del workflow tal como están implementados y corridos (2026-10-12)

Para quién: un agente estadístico que debe decidir si cada criterio del workflow (pasos 1-26) se puede mejorar. Este documento describe lo que el CÓDIGO aplica y lo que las CORRIDAS REALES han hecho, no lo que proponen los dossiers antiguos de esta carpeta. Donde dossier y código discrepan, manda el código y la diferencia va marcada con ⚠.

Cómo se hizo: lectura de código, `ledger/thresholds.yaml`, `pipeline/autopilot/criteria.yaml`, `assets/*.yaml` y resultados en `AlgoData`, en el árbol de trabajo del 2026-10-11/12 (incluye cambios aún sin commit). Cada sección la redactó un agente de lectura (sin ejecutar SQX); lo «no verificado» está dicho así. Los criterios del paso 8 en adelante se contrastaron con corridas de 2026-10-08 a 2026-10-11.

Lo que hay que saber antes de leer nada

1. **Los umbrales viven en tres sitios.** `pipeline/autopilot/criteria.yaml` (pasos 6-23 del autopiloto, sin congelar; 16 cambios en `criteria-history.jsonl` solo para el paso 8), `ledger/thresholds.yaml` (los congelados con `ledger:`) y `assets/_study.yaml` / `_build.yaml` / `_policy.yaml` (build y ventanas). Casi todos los umbrales de los pasos 10, 14, 16, 18.5 y 20 están **sin calibrar**: no hay población nula ni plantada que mida su tasa de falsos positivos.
2. **Los criterios casi no discriminan hasta ahora.** CSCV (18) da 10 PASA de 10; superficies por mercado (18.5) 0 NO PASA en 13; WFM (19) 0 NO PASA en 10; T3 del MC Retest (14) vale 1,000 en 58 de 58 casos; el paso 17 da 0 PASA en 12 lotes. El paso 12 vivo no puede eliminar ninguna madre. El paso 16 y el 11.5 no cortan (`rules: []`).
3. **El modo `dev` del paso 8** (`criteria.yaml`, `dev.on: true`, `sample: 3`) sorteaba 3 estrategias entre las que no caen: en USATEC la puerta dejaba 169 y siguieron 3. Casi todas las corridas de 2026-10-10/11 pasaron por ese sorteo, así que **no hay evidencia a escala de población** de los pasos 9-12 (entre 1 y 25 estrategias por corrida).
4. **Nulos y calibraciones desfasados.** El control `falsePositives` del paso 8 da 0,98 % de la población nula que sigue frente a 1,69 % de la real (cota gruesa: ≈ 58 % de los 169 serían ruido). El registro de calibración del 17 se midió con volteo de signos por día y el código vuela por semana. Las cifras de `_study.yaml` vienen de `Test_Calib_*`, borrada el 2026-10-04.
5. **Los costes reconstruidos no reproducen el NetProfit de SQX** (`descuadre` 0,925-1,0 en las 13 corridas del 18.5; alto en 8 de 12 lotes del 17). Es el punto estadístico más serio de la cadena de variantes y no tiene causa verificada.
6. **Eliminados el 2026-10-11 (sin commit):** pasos 22 (mapa condicional), 23 (estructura) y 24 (stop ATR; su medición preregistrada dio NO PASA: el stop a k=2 toca el 11,55 % de las operaciones, límite 1 %). El 25 está fuera de la cadena del autopiloto; solo marca. El 21 solo describe. El 20 **corta** desde 2026-10-11 lo que `blindJoint` descarta (`criteria.yaml:305-314`), pero ninguna corrida real usa aún la lectura `peor` ni el benchmark de trader al azar.
7. **Tiempos, cores y RAM:** solo existen donde se citan (p. ej. USATEC: gate 77 s, `falsePositives` 4.170 s, el 95 % del paso 8; monkey 26,9 GB de RAM medidos con 300 estrategias). Para la mayoría de pasos solo hay segundos de reloj, sin cores ni RAM: «sin dato».
8. **Los dossiers `criterios-paso-NN-*` y `criterios-pasos-6-*` que se citan en el texto están borrados** (2026-10-12; siguen en el historial de git). Describían diseños ya reemplazados y este documento los sustituye.

Índice

parte	pasos
A	1-3: idea, vocabulario, plantilla
B	4-5: preflight y creación del proyecto
C	6-7: build y retest OOS
D	8: puerta IS/OOS
E	9-12: transferencia (otros mercados y temporalidades)
F	13-16: MC Retest y SPP
G	16.5-17: variantes, nube y WFC
H	18-19: CSCV, superficies por mercado, WFM
I	20-23: análisis conjunto, exposición (22 y 23 eliminados)
J	24-26: stop ATR (eliminado), edge por coste, validación MT5

Pasos 1-3 — criterios tal como están implementados (corte 2026-10-12)

Convenciones: «CORTA» = elimina o impide; «MARCA» = deja una etiqueta pero la población pasa; «INFORMA» = solo se imprime. Verificado = leído en código o recalculado leyendo datos de AlgoData en esta sesión; «dossier» = tomado de `docs/AgentPDFs/criterios-paso-0N-*-2026-10-08.md` y no re-verificado (se indica). Donde dossier y código discrepan manda el código y va con ⚠. Estado del árbol: mucho del paso 2-3 está sin commitear (`git status` : `sqx/blocks/install.py` , `sqx/inspect/vocabulary.py` , `sqx/blocks/taxonomy.py` , `sqx/projects/buildcensus.py` , `builder.py` , `buildrules.py`).

Ninguno de los tres pasos tiene clave en `pipeline/autopilot/criteria.yaml` ni umbral en `ledger/thresholds.yaml` (verificado por el dossier; el código de autopiloto no lee nada de estos pasos). Son pasos de autoría y descubrimiento; los únicos números estadísticos viven en el perfil de mercado del paso 1.

PASO 1 — La idea (`studies/research/` , agentes `researchDirector` / `ideaExpert` / `ideaReviewer`)

1.0 Propósito, entradas, salidas

- Propósito: decidir qué celda (activo × TF × dirección × familia) merece CPU y producir tres ideas exactas, con hoja de hipótesis y revisión, que el dueño veta o contesta. No toca SQX.
- Entradas: barras M1 del tramo `build` de 19 activos (cortadas antes de remuestrear, `marketProfile/inputs.py:36-48` , test `tests/test_marketprofile.py::test_build_only` envenena lo posterior), `assets/*` (costes), `board/prior.yaml` (prior del dueño, transcrita de un documento del 2026-10-02), memoria de corridas (`studies/research/memory/`), cola de bibliografía (`literature/`).
- Salidas: `AlgoData/research/profiles/{measures,scores}.csv` (+ `run.json`), `board/board.json` , `proposals/<id>.json|.md` , `ideas/<ACTIVO>/*.md` , hoja de hipótesis sellada.
- Orden de aplicación: (a) perfil mide y corrige (BH) → (b) 4 filtros → (c) puerta del tablero → (d) puntos del tablero → (e) director elige UNA celda → (f) ideaExpert propone 3 ideas → (g) ideaReviewer anota → (h) `proposal.check` (negativas) → (i) `stamp` → (j) el dueño veta/contesta (`launchable`).
- No existe PASA/LIMBO/NO PASA: hay tres niveles de «no»: el perfil no deja entrar la celda, `proposal.check` no acepta el borrador, el dueño veta o deja preguntas abiertas.

1.1 Estado de los datos hoy (distinto del dossier)

- ⚠ El mapa se recalculó el 2026-10-11 08:35 (`run.json` : código `abca93c-dirty` , seed 593561232, `tests: 16188` , `wall_s: 5287.9`). El dossier describe el del 2026-10-08 (16.036 pruebas, código `7203f4c-dirty`). Recalculado ahora sobre `measures.csv` : 16.188 filas, 111 medidas distintas, p≤0,05 en 1.319 filas, 504 nombradas por BH (`significant`), p cruda más grande nombrada 0,0013329 (= 4/3001), `passes` = 2 medidas, 2 celdas-familia de 1.064 (DJ30 H1 largo ruptura `channel55_up200` múltiplo 3,143; EURUSD M15 corto momentum `bar3atr` múltiplo 2,303, 66,9 oper/año, 8 de 10 años). Es decir: más pruebas (16.188 vs 16.036) y el mismo veredicto (2 celdas) y el mismo corte (4/3001).
- ⚠ `board/board.json` , `memory/*.csv` y `proposals/` en disco son del 2026-10-02 (20:19-20:32); el tablero se recalcula en memoria cada vez, el fichero en disco está viejo (359 celdas en el JSON).
- Solo existe UNA propuesta: `proposals/20261002-203224-DJ30-H1-long-ruptura.json` (ideas `donchian55BreakTightRange` , `donchian55BreakUptrendD1` , `dayOpenNoiseBreakUp`); `estado.json` paso 6 de 6. Ninguna construida. Ficheros de idea: `ideas/DJ30/2026-10-02-ruptura-h1-largo.md` , `ideas/XAUUSD/2026-10-01-impulso-h4.md` . `memory.ideas.index()` ejecutado hoy: XAUUSD 29,29,29; DJ30 vacío,vacío,vacío.

1.2 Criterios (numerados)

C1.1 Significancia del perfil: p por conteo contra nulo + BH al 5 %

- Pregunta: ¿esta regla condicional produce, en `build` , un estadístico mayor que el de series sin estructura?
- Definición: estadístico = t de los rendimientos log apertura-a-apertura de las operaciones que dispara la regla (`measure/trades.py:26-36` ; 0 si <30 operaciones). Nulo = 3.000 series fabricadas remuestreando barras (4 distancias al cierre previo: O,H,L,C) con reposición y bloque = 1 barra en los 4 TF (`marketProfile/config.yaml:13,23` ; `series.py:123-151` , `engines/resample/draws.py:84-100`); el calendario queda fijo. p = (1 + nº nulos ≥ real)/(3000+1) (`one.py:89-92`) → p mínima 1/3001 = 0,00033. Lado corto: se da la vuelta a la serie. <30 operaciones → `p = 1` puesto a mano (`one.py:151,159` , `config.yaml:33`). Semilla `null` = entropía del SO por corrida (`config.yaml:16`); los 3.000 sorteos se comparten entre las medidas de una celda activo×TF (`one.py:82-86`). z = (real − media nulo)/sd nulo (`one.py:158`) solo alimenta la puntuación.
- Corrección: Benjamini-Hochberg al 5 % sobre TODAS las filas del mapa juntas (`many.py:70` , `engines/inference/fdr.py:10-44` ; `alpha: 0.05` , `config.yaml:26`). `q` = p·N/puesto monótona. Al lado, solo informativo: `q_family` , `significant_family` y `passes_family` (BH dentro de cada familia, `many.py:71-72`). CORTA (es el primer filtro de `passes`). Población: tramo `build` (30-

09-2013 a 31-12-2019 para DJ30; ~6,25 años), 19 activos × M15/M30/H1/H4 × medidas (111) × lado(s).

- Justificación declarada: nulo de bootstrap de 1 barra porque con bloques mayores el nulo hereda la estructura que quiere romper y rechaza de menos (`blocklen.py`, 1-10-2026: 0,3-0,8 % de falsas alarmas al 5 % con 24 barras frente a 4,0-7,5 % con 1; `config.yaml:17-23`). 3.000 sorteos porque con 14.364 pruebas y 1.000 sorteos solo la p mínima sobrevivía a BH (`config.yaml:14-15`).
- Puntos débiles: (1) la comprobación del bloque se hizo al 5 %, con 200 sorteos, 6 celdas y las 31 medidas originales; el perfil decide a p≤0,00133 donde las falsas alarmas del nulo no están medidas (dossier §2.2; en XAUUSD H1 el tamaño medido fue 7,45 % nominal 5 %); las 80 medidas añadidas después no se han comprobado ni al 5 % (dossier 5.14.7-8). (2) El nulo conserva la deriva media pero no su reparto temporal: una regla larga que opera solo en rachas alcistas (p. ej. filtro D1>SMA200) le gana al nulo sin aportar entrada (dossier §2.2; DJ30 `channel55_up200`: nulo media 0,601 sd 1,043, z 3,08, p=3/3001). (3) Resolución de p: saltos de 1/3001; el corte (4/3001) cae entre saltos; con 2 nulos por encima la p verdadera tiene IC95 % [0,00008; 0,0024] (dossier 5.16, calculado por el revisor): la celda del DJ30 repetida con otra semilla quedaría fuera 1/3 a 1/2 de las veces. (4) Supuesto de BH (independencia o dependencia positiva y lista fijada de antemano): la lista crece cada mañana (14.364 el 2-10 → 16.036 el 8-10 → 16.188 el 11-10) y las pruebas vecinas se parecen (correlación mediana del z entre marcos contiguos 0,83; entre celdas cualquiera 0,38; dossier 5.18, «calculado por el revisor», no re-verificado); las 16.188 equivalen a bastantes menos pruebas efectivas (orden de cientos a ~1.000+, estimación gruesa). (5) 2.330 filas con n<30 tienen p=1 y entran en N de BH (conservador; sacarlas deja las mismas 2 celdas pero cambia el corte de 4/3001 a 5/3001: dossier 4.3, cifras del 8-10). (6) Selección: la celda elegida es la que pasó; la selección no se corrige en ningún sitio (dossier 5.2).
- Ejemplo real (mapa del 8-10, verificado por el dossier; el DJ30 sigue pasando hoy): DJ30 H1 largo `channel55_up200`: 445 operaciones en build (71,2/año), t real 3,816, z 3,08, p = 3/3001, puesto 486 entre 16.036, q = 0,0330. Hoy el mapa tiene 16.188 pruebas y la celda sigue pasando (`passes` la incluye). Efecto: 504 nombradas sobre 16.188 (familias: reversión 247, patrón 113, volatilidad 76, sesión 61, momentum 6, ruptura 1, tendencia 0, verificado hoy).
- Coste: `run.json` `wall_s: 5287,9 s` en total con 4 procesos (uno por activo; `config.yaml: run.workers 4`); por activo y TF hay tiempos en `run.json` (p. ej. DAX40 M15 244,9 s, M30 132,4, H1 77,4, H4 40,8; AUDUSD M15 495,0 s). RAM: sin dato.

C1.2 «Paga»: múltiplo ≥ 2× el coste de ida y vuelta (bruto)

- Pregunta: ¿el efecto medio por operación supera el coste?
- Definición: efecto = media (precio apertura salida − apertura entrada)·signo, en unidades de precio, bruto y sin swap; coste = spread + 2×deslizamiento + comisión de la ficha del activo en el tramo `build` (`inputs.py:73-90`); múltiplo = efecto/coste. `pays = multiple >= 2.0` (`config.yaml:27`, `many.py:73`). Nulo se trata como −inf. CORTA (3er filtro de `passes`).
- Justificación: diseño, el efecto debe cubrir el coste con holgura. Punto débil: bruto y sin swap (DJ30 ~3,5 puntos/noche = 0,67 costes según el ideaExpert, dossier 5.8); coste del tramo `build`, el de `oos1` es un 48 % mayor en DJ30 (7,7 vs 5,21 puntos; calculado por el revisor): una celda que paga 2,0× en build paga ~1,35× con el coste de oos1.
- Ejemplo: DJ30 H1 largo `channel55_up200`: efecto 16,37 puntos, coste 5,21 (spread 2,60 + 2×1,30 + comisión 0), múltiplo 3,14 (dossier §3.1.6). Hoy, `pays` = 3.465 filas del mapa (verificado).
- Maldición del ganador: el múltiplo que se enseña al cruzar un corte de z≈3 viene inflado (con exceso verdadero 0, 1× o 2× se vería 2,82×-3,09× al pasar; dossier 5.17, calculado por el revisor).

C1.3 «Estable»: signo positivo en MÁS de la mitad de los años de `build`

- Definición: `stability = years_with_sign/years`; `stable = stability > 0.5` (`config.yaml:28`, `many.py:74`). CORTA (filtro de `passes`). Se cuenta el signo del efecto, no que pague el coste; un trimestre parcial (2013) cuenta como año entero.
- Punto débil: filtro laxo (una regla sin contenido tiene ~50 % de sacar ≥4 años positivos de 7; en índices con deriva más); en DJ30 `channel55_up200` 2014 y 2015 dan 2,83 y 1,59 puntos (< coste 5,21) y votan a favor (dossier §3.1.6). Hoy `stable` = 6.110 filas.
- Ejemplo: DJ30 6 de 6 años; EURUSD M15 corto `bar3atr` 8 de 10 años (verificado hoy).

C1.4 «Frecuente»: ≥ 40 operaciones/año; suelo 35

- Definición: `trades_per_year = n_trades/años`; `frequent = trades_per_year >= min_trades(symbol)` con `min_trades_per_year: 40` (`config.yaml:31`), excepciones por activo en `min_trades_per_year_by_asset` (hoy `{}`), y `FLOOR = 35` (`many.py:10`) que lanza `ValueError` si algo baja de 35. CORTA. (Dueño, 2-10-2026.) Hoy `frequent` = 7.644 filas.
- Punto débil: en las medidas «base» cada barra con señal abre operación aunque la anterior siga abierta (solape; `measure/trades.py:22-23`), así que la frecuencia cuenta operaciones solapadas: `channel55` DJ30 da 145/año en el perfil y 95 con una posición a la vez (cifra del ideaExpert, dossier 5.6). Las medidas tipo `rule` llevan una posición.
- ⚠ Suelo de operaciones inconsistente: CLAUDE/memoria del dueño (`minimo-operaciones-por-ano.md`): 35/año en M15, M30, H1 y 20 en H4 «en el builder y en el paso 8»; el perfil usa 40 sin distinción de TF (`many.py` / `config.yaml`), y `ideaReviewer.md:55` (modificado, sin commitear) ya dice 35 en M15/M30/H1 y 20 en H4 (el dossier decía «40-50»: ⚠ dossier 5.9 ya resuelto en el árbol de trabajo). El perfil sigue pidiendo 40 también en H4: no verificado si es intencionado.

C1.5 «Pasa los cuatro» (`passes`) y puntuación de familia

- `passes = significant & pays & stable & frequent` (`many.py:77-78`). Una familia-celda pasa si alguna medida con operaciones pasa (`scores` , `many.py:98-143`); líder = la que mejor paga entre las que pasan, si no la de q mínima (`_lead` , `many.py:82-88`); una medida de reloj nunca lidera mientras la familia tenga otra (`_tradable`). Puntuación 0-100 = 100 × media(z recortado a [0,5])/5 (`many.py:129` , `config.yaml: score.z_cap 5.0`). La puntuación «ya no ordena nada» según el README del perfil. INFORMA.
- Resultado hoy: 2 de 1.064 celdas-familia pasan. Sobre las 276 medidas con operaciones, sin reloj, nombradas por BH: mediana del múltiplo 0,28; solo 5 pagan ≥2×; de esas, 2 son frecuentes (verificado hoy). Quitando la significancia, `pays&stable&frequent` da 311 medidas (hoy; 267 el 8-10) → la significancia es el filtro que más elimina.
- Alternativa en el CSV (no decide): `passes_family` = 3 medidas hoy, todas de reversión; ninguna coincide con las dos oficiales (dossier 3.1.8; hoy 3 medidas, verificado).
- Punto débil crítico (dossier 5.15, «calculado por el revisor»): en tendencia+ruptura+momentum hay 7 pruebas con p≤4/3001 y el azar solo daría ~9; las 2 celdas pasan porque BH sube el listón por las ~504 nombradas, casi todas reversión/patrón/volatilidad/sesión (mediana 0,28× el coste). Con `q_family` ninguna de las dos pasa. Que pasen 2 no distingue el mapa de uno sin nada dentro (el azar daría ~3,8 entre 2.864 pruebas frecuentes donde z=3 vale ≥2×).
- Potencia (`profilePower/` , `pooledProfile/` , del 2-10; no recalculados con el corte de hoy): simulación en H4 sobre copias de signo aleatorio de XAUUSD/EURUSD/USA500: efecto mínimo detectable 1,2× (run3_follow) a no alcanzado (tsmom_d60_run), `channel55` 18,5×; aproximación normal sobre todo el mapa: en H1 podría ver 2× el coste solo en el 14-36 % de las pruebas (techo, `power/calibration.csv` muestra que la aproximación promete de más). «No pasa» en H4 suele ser «no se puede saber». Perfil agrupado: 142 grupos nombrados, 0 celdas nuevas con los 4 filtros estrictos; solo informa.

C1.6 Puerta del tablero («qué celdas entran»)

- Definición (`board/many.py:run` , gate dueño 2026-10-02): parte de `scores.csv` sin `needs_clock` ; una celda entra si (a) la prior del dueño la da por Alta (matriz `board/prior.yaml` , `prior.py`), o (b) `passes` en la familia (evidencia «desnuda»), o (c) una variante del barrido pasa en meseta (`sweep/best.csv: plateau_any` ; ≥2 vecinas a un paso también pagan, `config.yaml:88`). CORTA lo demás.
- Resultado (8-10, dossier): 359 de 1.064 entran: 357 solo por la prior, 2 por lo medido, 0 por el barrido (el barrido del 2-10: 0 en meseta). 22 llevan marca de costes provisionales (XAGUSD, UKOIL, USOIL; `board/config.yaml marks`). USDCHF, UKOIL y USOIL no tienen prior. `board.json` en disco (2-10) tiene 359.
- Punto débil: la prior se transcribe de un documento fechado el 2-10-2026, DOS días después del fin de `oos2` (30-09-2026), y la estadística es «un extra, no un requisito» (palabras del dueño en `board/config.yaml:3-7`); no se puede evaluar si la prior guía (el autoexamen necesita 20 corridas cerradas; hay 0).

C1.7 Puntos del tablero (orden, no filtro)

- `puntos = 100 × freno × (×0,5 si «en contra») × (0,30·prior + 0,30·evidencia + 0,10·señal + 0,15·hueco + 0,15·pasado)/1,0` (`board/factors.py:69-73` , pesos `board/config.yaml:8-13`); prior Alta 1 / Media 0,5 / Baja 0; evidencia desnuda 1,0, meseta 0,8, débil 0,5 (significativa y estable), ninguna 0,25, en contra 0 (+freno ×0,5; en contra = significativa con múltiplo <0,5, o p cruda ≥0,95 con múltiplo ≤−1, o barrido máx <0,5×; `factors.py:51-66`); señal = min(múltiplo/10,1) (solo si hay medida); hueco = 1/(1+corridas de esa familia en esa celda); pasado = tasa Beta encogida hacia la global con 4 pseudo-intentos (`factors.py:21-48`); freno = 1/(1+ideas gastadas/6). INFORMA / ordena. La página imprime 40 filas (`board/config.yaml page.rows`).
- Hoy (dossier, recalculado en memoria el 8-10, no reverificado hoy): 20 celdas a 67,5 (prior Alta + evidencia débil), 276 a 60,0 (prior Alta sin evidencia) → las 2 celdas medidas están en los puestos 297 y 313, fuera de la página de 40; los empates se deshacen por orden alfabético (AUDJPY ocupa 20 de 40 líneas). `past` = 0,5 constante en las 359 (0 corridas cerradas con familia). Hueco/pasado no ven las 34 de 44 corridas de Estrategia Simple (sin familia).
- Punto débil: el orden depende de la prior (357/359 celdas) y no de la medición; un empate masivo se deshace por alfabeto.

C1.8 Negativas de `proposal.check` (`board/proposal.py:46-90`)

CORTAN el borrador (el director debe rehacerlo). Lista implementada (verificada leyendo el fichero): faltan campos (`cell` , `diagnosis` , `measures` , `ideas` , `prior_failures`); `reviewer` ausente o igual a quien propone (`PROPOSERS = ideaExpert, researchDirector`); un fracaso cerrado de la celda sin postura `adopta/rechaza` con razón (`stances`); no hay exactamente 3 ideas con 3 nombres distintos; a una idea le falta alguno de los 12 campos (`IDEA_KEYS`); dirección de la idea ≠ la de la celda (regla 13); cantera no en (`perfil` , `libros` , `propia`); la paleta del hueco usa la familia de la condición fija o una «igual» (`palette.alike` , regla 15 «nunca dos iguales»); una pregunta con <2 lecturas (regla 11); hoja de hipótesis incompleta (`sheet.problems` : sus 4 campos y dos números >0) o revisión sin veredicto de {resiste, dudosa, tumbada} o con un eje (mecanismo, coste, parecido, ambigüedad) vacío. `stamp` (`:109-123`) aborta si la celda no está en el tablero y añade id, fila de tablero, ideas gastadas, marca de costes provisionales, sello de hoja y dos costes fijos de texto. - NO mira: nº de condiciones (regla 14; `over_the_cap` actúa luego, en el paso 5), reloj/calendario (regla 15), si son tres mecanismos o uno en tres periodos, si las hipótesis medidas están contadas, si se leyó algo fuera de `build` . - `launchable` (`proposal.py:98-106`): no vetada, sin

preguntas abiertas y, solo si `review.removes` fuese `true`, no «tumbada». ⚠ `memory/config.yaml:172` `removes: false`: «tumbada» solo anota (decisión del dueño); en el caso real quedaban 40 preguntas abiertas (A 16, B 13, C 11) y 0 lanzables. - Sin corrida real posterior: ninguna idea revisada se ha construido; no hay dato de acierto del revisor.

C1.9 Hoja de hipótesis y contraste con el build (`memory/sheet.py`)

- Hoja = 4 campos (operaciones/año y duración mediana esperadas, etc.), sello hash de 16 caracteres (`sheet.py:33-37`). Tras el build, si las operaciones/año o la duración construidas difieren de las predichas en más de un factor 2 (`memory/config.yaml:166` `tolerance: 2`), la corrida se cierra con causa `hypothesis_mismatch` (`sheet.py:78-99`). CORTA indirectamente (marca el cierre de la corrida, alimenta `queries.failures`). Sin corrida real (ninguna hoja sellada tiene build).
- Punto débil: factor 2 sin justificar estadísticamente (encargo 42).

C1.10 Contabilidad de «miradas» (no filtro)

- `ledger.report` suma a la N de un estudio las hipótesis que el ideaExpert midió (`memory/trials.py:18-43`, `ledger/report.py:91`), leyendo «Hipótesis medidas: N» del fichero de idea (`memory/ideas.py:11`). Las pruebas del perfil y las variantes del barrido se imprimen «aparte, sin sumar» (`trials.py:46-61`; para DJ30 H1 largo: 202 pruebas del perfil + 702 variantes = 904). INFORMA.
- ⚠ El regex actual (modificado, sin commitear) `Hip[oó]tesis medidas(?: en total)?[ \t*]*:?[ \t*]*(\d+)` acepta «en total» pero NO atraviesa saltos de línea; el fichero del DJ30 pone `## 3. Hipótesis medidas en total` y la cifra («**68 estadísticas condicionales**») en otra línea → sigue sin casar: verificado hoy `ideas.index()` devuelve `hypotheses_measured = ''` para las 3 ideas del DJ30 y 29 para las 3 del oro; `ideas_spent.csv` dice 0 para DJ30 H1 largo. Además el fichero da dos cifras (68 estadísticas, 21 hipótesis) y la ficha no dice cuál es «la mirada». Efecto sobre el listón: nulo cuando N son miles (N 15.967 → 15.996, listón 0,083981 → 0,083990), apreciable solo con N de decenas (`tests/test_ledger_ideas.py`: DSR 0,4025 → 0,3764 con 29 sobre N=50).

C1.11 Filtros duros del scout de bibliografía (`literature/schema.py:22-25,60-80`)

- Rechazan un paper o una entrada antes de medirla: `clock` (regla 15; regex CLOCK en `schema.py:35`, y `needs_clock` no «no»), `not_sqx_at_market` (regla 15 stop/limit), `conditions` (≠1 o 2 condiciones de entrada: regla 14, `schema.py:79`), `two_legs` (regla 13), más otros cinco hasta nueve filtros (no listados aquí; ver `literature/README.md`). Cada campo extraído debe anclar a una cita literal. CORTA (rechazo con la frase que lo muestra). Una regla con dos lecturas lleva pregunta y no se mide hasta que el dueño contesta (regla 11).
- Corridas recientes (`AlgoData/research/literature/runs.jsonl`, verificado hoy): 2026-10-09 13 papers, 9 rechazados, 4 aceptados, 11 entradas, 170.825 tokens; 2026-10-10 13/10/3/3 entradas, 109.073 tokens; 2026-10-11 13/9/4/4 entradas, 174.547 tokens (modelo opus+haiku). Las entradas aceptadas se miden en el perfil (`report --top-up`), que **vuelve a juzgar el mapa entero** → el veredicto de las celdas ya medidas cambia cada mañana (⚠ el número de pruebas pasó 14.364 → 16.036 → 16.188 en 9 días; con el puesto de DJ30 fijo en 486, la celda dejaría de pasar cuando N > ~24.300, dossier 5.4, no re-verificado).
- Punto débil: `selection.py:16-37` solo compara la muestra del paper con `build` (`overlaps_is`), nunca con `oos1`/`oos2`; 13 de 53 entradas con año final llegan a ≥2020 (dossier 5.19); la idea C viene de un paper de 2025 con `sample: n/a`.

C1.12 Instrucciones a los agentes que el código no comprueba

- `ideaExpert.md` (123 líneas; Opus; todas las herramientas): condición simple con mecanismo económico; medir solo en `build`; estadísticas condicionales rápidas, nunca optimización; contar cada hipótesis medida (línea 116); cada lectura ambigua listada (regla 11). `ideaReviewer.md` (107 líneas; solo Read/Grep/Glob, no puede medir): 4 ataques (mecanismo, coste, parecido, ambigüedad), veredicto de tres. Solo se fuerza en código lo de §C1.8. «Solo `build`» está forzado para el perfil (test) pero no para el experto, que escribe scripts sueltos; el umbral «9,0 × ATR» de la idea A del DJ30 salió de probar umbrales de 7 a 10 sobre `build` (selección dentro de muestra, dossier 5.10).
- Reglas duras en este paso (dossier §3.9, verificado en código lo de `check`): 11 → fichas + `check` + `launchable`; 13 → `check` (dirección de la celda) + perfil mide largo/corto aparte + `two_legs` scout; 14 → solo aplica el scout (`conditions`) y de palabra el experto (A y B: 2 fijas, sin hueco); en código lo cuenta `buildrules.over_the_cap` al crear el proyecto; 15 → medidas de reloj no lideran/entran; filtros scout; `check` prohíbe repetir la familia fija en la paleta. ⚠ `prev_day_break` lidera sesión en 152 celdas y `needs_clock` es falso en las 1.064 celdas (dossier decisión 12); el dueño no ha decidido si cuenta como calendario.

1.3 Ejemplo de cadena (dossier + datos de hoy)

16.188 pruebas (111 medidas × 76 celdas × lados) → 504 nombradas por BH → de ellas 5 pagan ≥2× → 2 además operan ≥40/año y son estables → 2 celdas-familia de 1.064 pasan los 4 filtros → 359 celdas entran al tablero (357 por prior) → la página imprime 40 (ninguna medida) → una propuesta DJ30 H1 largo ruptura (tomada del 2-10, cuando era puesto 1 con 70,6 puntos) → 3 ideas: 0 resiste, 2 dudosas, 1 tumbada → 40 preguntas abiertas → 0 lanzables → 0 construidas.

1.4 Tiempos/recursos

Perfil: 5.288 s totales con 4 procesos (11-10-2026); recarga diaria de bibliografía: 110-175 k tokens por día (arriba). Director + experto + revisor: una sesión Opus por propuesta, sin dato de tokens en disco (`proposals/` no los guarda; no verificado). Python del tablero: ~50 ms (dossier). SQX: ninguno. El «custodio estimado 0,8 h por idea» de la propuesta es una estimación declarada, no una medida.

1.5 Discrepancias del paso 1

- ⚠ Cifras del dossier (16.036 pruebas, `7203f4c`) ≠ disco de hoy (16.188, `abca93c-dirty` , 111 medidas); el veredicto (2 celdas) y el corte (4/3001) no cambian.
- ⚠ `memory/ideas.py:11` ya acepta «en total» pero sigue contando 0 para el fichero del DJ30 (cifra en otra línea).
- ⚠ `ideaReviewer.md:55` ya dice 35/20 (el dossier decía 40-50); el perfil sigue en 40 para todos los TF.
- ⚠ `WORKFLOW.md` fila 1 «Idea en el chat | — | —» (no verificado hoy; lo afirma el dossier 5.12).
- ⚠ Textos con cifras viejas (README del perfil «189 tests/celda, 14.364»; hoy 211 por celda y 16.188; manual cap. 82), `series.py:137` «in blocks, in another order» (es con reposición), `proposal.py:145` «five steps» (son seis).
- ⚠ No existe corrida cerrada con familia: `selfcheck` → `{"enough": false, "closed": 0, "needed": 20}` ; no se sabe si perfil ni prior guían.

PASO 2 — Vocabulario (`sqx/inspect/vocabulary.py` , `sqx/blocks/install.py` , `sqx/blocks/taxonomy*` , skills `sqx-custom-block` / `sqx-random-group`)

2.0 Propósito, entradas, salidas

- Propósito: ¿la lógica exacta del dueño ya existe como bloque SQX utilizable? Si no, autorarlo, validarlo e instalarlo en conductor y custodio. No arranca SQX ni gasta CPU; exige el worker parado.
- Entradas: `config.xml` , `user/settings/customBlocks.xml` , `blockGroups.xml` de cada instalación; la idea (paso 1) con lecturas fijadas.
- Salidas: XML `Item` / `Group` instalado en `SQX_w1` y `SQX_w2` (nunca el maestro), copia en `<fichero>-backups/<ms>.xml` , taxonomía refrescada, instantánea opcional `AlgoData/templates/vocabulary/<install>-<fecha>.json` (solo existen las del 2026-09-22).
- Orden: buscar `vocabulary <término>` → decidir (persona/agente) si existe → autorar (skill; 7 comprobaciones de `validate.py` para bloques, 8 para grupos; `assess.py`) → `install.py` en conductor y custodio → `vocabulary --diff custodian` .
- Sin criterio estadístico, sin umbral, sin ledger.

2.1 Inventario hoy (ejecutado hoy con `python3 -m sqx.inspect.vocabulary` , solo lectura)

`SQX_w1` : 849 nativos + 179 propios, 25 grupos (20 de condición, 4 de valor, 1 vacío `RandConditions`). `--diff custodian` : 0 bloques y 0 grupos faltan. `--diff master` : faltan 8 bloques (`CBlock_CloseCrossesAbove/Below` de DCUpper/DCLower, EMA, HMA, KCUpper/KCLower) y 3 grupos (`crossAboveHMA_v1Signal` , `donchianUpperCrossUpSignal` , `emaCloseAboveSignal`): deliberado (regla 3). Taxonomía `sqx/blocks/taxonomy.yaml` : 767 filas, 171 propios, es decir 8 propios por detrás del conductor (⚠ no refrescada todavía; el instalador modificado la refresca en la siguiente instalación).

2.2 Criterios (todas CORTAN salvo donde se indica)

C2.1 «¿Existe el bloque?» — `find()` (`vocabulary.py:110-126`)

- Pregunta: ¿hay un bloque nativo o propio para esta lógica?
- Definición hoy (árbol de trabajo): subcadena, en minúsculas, en clave, texto de pantalla, categoría o ayuda (`b.get("category")` , `b.get("help")`). Devuelve `pooled_by` (grupos que lo contienen). INFORMA: la decisión final «existe / es estado o transición» es de una persona o agente, no del código.
- ⚠ Dossier 3.2/5.1 (commit `b2b4c8c`) decía que solo miraba clave y texto de pantalla (buscar `keltner` daba 18 nativos y 0 propios); el árbol actual ya busca también categoría y ayuda (cambio sin commitear). No re-ejecutado `find("keltner")` tras el cambio.
- Punto débil: una subcadena no distingue estado de transición («cierra por encima» ≠ «cruza por encima»); la regla 11 obliga a preguntar, y el matiz lo decide el humano. Una respuesta «no existe» sigue sin ser prueba (ayuda larga, nombres sin el término).
- Ejemplo real: `CBlock_CloseCrossesAboveKCUpper` (plantilla `keltnerUpperCrossUp` , brief 2026-09-22): la categoría nativa Keltner trae 16 condiciones y ninguna es un cruce sobre el cierre; la más cercana `KCBarClosesAboveUpper` es un estado; el dueño eligió «cruce, no estado». Sin corrida estadística.
- Coste: instantáneo (lectura de 3 XML).

C2.2 «¿Lo alcanza un grupo?» — `pooled_by()` y mensaje «NO GROUP POOLS IT — unusable in a template» (`vocabulary.py:95-107, :206`)

- INFORMA. Hoy 808 de 849 nativos y 130 de 179 propios no están en ningún grupo (cifras del dossier 2026-10-08, con 179 propios idénticas hoy). ⚠ El mensaje es engañoso: un hueco libre (`#Group#` vacío) sortea todo lo encendido en la tarea Build, incluidos nativos (`knowhow/authoring/holes-groups-randomcondition.md`); solo es cierto para un hueco atado a un grupo.
- Avisos de `report()` / `usable_groups()` (`:129-184`): grupo vacío («EMPTY, unusable»: construye y no sortea nada, sin error) y ausencia de grupos de valor («every stop-entry shape is ungenerable here»).

C2.3 Misma instalación en conductor y custodio — `missing()` / `--diff` (`vocabulary.py:150-163`)

- Definición: claves de bloque propio y nombres de grupo que el destino no tiene frente a una referencia. INFORMA (no hay puerta que lo exija antes del paso 3). Unidireccional (no dice lo que sobra), solo por nombre (dos definiciones distintas con la misma clave pasan). Dos órdenes de instalación sin transacción: el conductor puede quedar instalado y el custodio no → una plantilla «desaparece del constructor sin error».
- Hoy: 0 faltantes en custodio (verificado hoy).

C2.4 El instalador (`sqx/blocks/install.py`)

CORTA (SystemExit) en este orden, verificado leyendo el fichero modificado: 1. Rol `master` → «the master is the owner's install» (`:129`, regla 3). 2. Puerto del worker contesta (`worker.listening(port)`, `:131`): solo mira el puerto; no mira el fichero `OWNER` ni PID (regla 4: SQX reescribe settings al salir). 3. NUEVO sin commitear: `validate()` (`:58-79`): XML ilegible o almacén ilegible; la etiqueta de un hijo no es una de las que ya tiene el almacén; un hijo sin atributo (`key` o `name`) o cuyo `key` no empieza por `CBlock_`; XML sin elementos. Escribe nada si falla. 4. Copia `backup()` antes de escribir (`:32-43`); añade o sustituye por clave (quita la vieja y añade la nueva AL FINAL; no «en su sitio»), NUEVO: escritura atómica (`.tmp` + `os.replace`); reescribe todo el fichero con `ElementTree.indent`; NUEVO: llama a `taxonomy.refresh(role)` y, si falla, avisa de ejecutar `python3 -m sqx.blocks.taxonomy --role`. - ⚠ Siguen sin cubrirse (dossier 3.7, no reverificado en el árbol actual): `present` se calcula una vez antes del bucle, así que un lote con la misma clave dos veces la duplica o falla a medias tras la copia; `replaced` no distingue la misma definición de otra; no comprueba que el grupo apunte a bloques existentes. - Ejemplo: `keltnerUpperCrossUp` (autorado 2026-09-22, grupo de un solo ítem desde 2026-09-26): 30 estrategias generadas en 29 s en `SQX_w2` (`smoke_keltnerUpperCrossUp`, XAUUSD H1, 2008-2017), las 30 con el bloque; humo USDJPY H1 tras pasar a grupo: 15/15 con periodos variados (`manifest.json`). Coste de la instalación: segundos; `SQX_w1` guarda 14 copias de `customBlocks` (15 MB) que nadie poda (dossier).

C2.5 Comprobaciones de la skill `sqx-custom-block` (`~/.claude/skills/sqx-custom-block/engine/validate.py`; viven en `tools/sqx-lab/plugins/sqx-lab/skills/`)

- 7 para bloques: (1) XML bien formado; (2) sin claves duplicadas en el lote; (3) pares opuestos simétricos (si A apunta a B, B apunta a A y viene en el mismo lote); (4) todo `#ParamN#` usado está declarado; (5) claves `<Param>` únicas por bloque; (6) el nombre no acaba en `_<dígitos><letras>` (AlgoWizard recorta ese sufijo y rompe el enlace con el opuesto); (7) con catálogo: todo átomo de varias salidas lleva `#Line#` y ningún átomo `talib_*` (evita el NullPointerException del Stockpicker). 8 para grupos (no vacío, tipo coherente, ítems planos, `CBlock_*` resuelto contra la instalación viva…; cabecera `:7-27`). CORTAN solo «si alguien las corre»: el instalador no las invoca (salvo lo nuevo de C2.4).
- ⚠ Sin `--catalog` la comprobación 7 se salta y el resultado sigue siendo «ALL CHECKS PASSED», código 0. `assess.py` (higiene de ventaja: CRÍTICO repintado/`shift 0`/umbral fuera de rango, AVISO, INFO) solo califica bloques con `categoryType="Custom blocks"` y con ninguno imprime «no custom blocks found» y sale 0 incluso con `--strict` (dossier 5.2): el bloque del ejemplo no fue calificado. No verificado hoy.
- Lo que NADIE comprueba: que el bloque haga lo que dijo el dueño (transición vs estado; opuesto realmente espejo; `shift` correcto). Lo cierra la construcción de humo y la lectura humana.
- Regla de oro de autoría (`tools/sqx-lab/overlays/sqx-custom-block.md`): preguntar siempre (regla 11); lo que dijo el dueño, exacto — si falta se autora, nunca se aproxima; lo nombrado es fijo, los números no nombrados quedan optimizables; escribir también el opuesto.

C2.6 Regla 15 en el paso 2 — clasificación de «bloque de reloj» (`sqx/projects/buildcensus.py:14-20,25-40`) y negativa al aplicar la paleta (`buildingblocks.py:rewrite`)

- NO se aplica al autorar ni al instalar. Se aplica en el paso 6 al escribir la paleta en la tarea Build: una entrada de `<BuildingBlocks>` clasificada `clock` o `level` (stopLimitBlocks) se deja apagada aunque la paleta la pida y se imprime «refused» (`buildingblocks.py:62-66`).
- Definición hoy (árbol de trabajo): `CLOCK = ^(Current|Bar)(Hour|Minute|DayOf|WeekOf|Month|Time)|^IsMonth` sobre la parte de la clave tras el último punto, MÁS `CLOCK_NAMES = {CBlock_TimeBreakoutLong, CBlock_TimeBreakoutLong_2, SessionHigh, SessionLow, SessionOpen}` (dueño, 2026-10-11, añadido sin commitear). `clock` gana sobre la categoría.

- ⚠ Dossier 5.3 (del 8-10) decía que la lista dejaba fuera `CBlock_TimeBreakoutLong(_2)` y `SessionHigh/Low/Open`: eso ya está arreglado en el árbol de trabajo. Siguen fuera: `SessionClose`, `VolumeProfileCustomHours`, `…MultiSession` (nativos), y cualquier otro bloque propio `CBlock_*` que lea el reloj y no esté en `CLOCK_NAMES` (la regex nunca atrapa `CBlock_*`). Un bloque de reloj dentro de un grupo o FIJO en una plantilla no pasa por esta negativa de paleta (ver C3.6).
- Punto débil: lista nominal, no semántica; si el dueño decide que «sesión» no es reloj, `SessionHigh/Low/Open` y las paletas `ruptura_base` / `sesion_base` / `timeRangeBreakout` cambian.

C2.7 Taxonomía atrasada → bloque instalado no sorteable

- `buildingblocks.rewrite` cuenta como `outside` toda clave que `taxonomy.yaml` no lista y la deja apagada (`:45-46,60`). Con la taxonomía en 767 filas (171 propios) frente a 179 instalados, los 8 propios nuevos (incluido `CBlock_CloseCrossesAboveKCUpper`) no los alcanza una paleta hasta regenerar (`python3 -m sqx.blocks.taxonomy`). Corregido por el código nuevo de `install.py` (refresco automático) pero la taxonomía en disco aún no incorpora esos 8 (verificado hoy: 171 propios). El efecto en una construcción real no medido.

2.3 Ejemplo de cadena

849 nativos + 179 propios en conductor y custodio (idénticos); maestro 171 propios y 22 grupos (le faltan 8 bloques y 3 grupos, por diseño) → una idea nueva pasa por `find` (hoy con categoría y ayuda) → si falta, se autora (7+8 comprobaciones si se corren) → instala en 2 workers (3 negativas duras + validate nuevo) → 0 faltantes en `--diff custodian` → la taxonomía debe refrescarse para que el paso 6 pueda sortear el bloque. Sin corrida estadística.

2.4 Tiempos/recursos

CPU ~0 (lectura/escritura de XML de 1,3 MB con re-sangrado). Sin datos de `perf/` para este paso (no verificado). Backups 15 MB en `SQX_w1`.

2.5 Discrepancias del paso 2

- ⚠ `find()` y el instalador cambiaron en el árbol de trabajo (sin commitear) respecto al dossier de 8-10: ahora busca categoría y ayuda, valida el XML y la clave `CBlock_`, escribe atómico y refresca la taxonomía.
- ⚠ `CLOCK_NAMES` (2026-10-11) cubre ya `CBlock_TimeBreakoutLong(_2)` y `SessionHigh/Low/Open`; el dossier lo daba como fallo abierto.
- ⚠ Persisten: lote con claves repetidas, `assess.py` ciego a bloques sin `categoryType`, `--diff` unidireccional y por nombre, el mensaje «unusable», textos con cifras de 2026-09-22 (manual cap. 21 «849+171, 20 grupos»; hoy 179 y 25), `~/.sqx-lab` apunta al custodio y no al conductor (según el dossier).
- ⚠ Las skills `sqx-custom-block` y `sqx-random-group` no están en `.claude/skills/` (viven en `tools/sqx-lab/plugins/sqx-lab/skills/`; solo `sqx-doctor/sqx-setup` y `sqx-wait` coexisten con ellas en `~/.claude`); aparecen en la lista de skills del sistema.

PASO 3 — La plantilla (`sqx/templates/{build,holes,registry}.py`, `sqx/projects/{builder,buildrules}.py`, skill `sqx-strategy-template`, agente `templateArchitect`)

3.0 Propósito, entradas, salidas

- Propósito: fijar la parte de la idea que llevan TODAS las estrategias y dejar al sorteo el resto, y garantizar que cumple regla 13 (una dirección) y 14 (≤2 condiciones por lado). No arranca SQX.
- Entradas: la idea con lecturas fijadas, el bloque (paso 2), un esqueleto probado (`market_long` del kit).
- Salidas: carpeta `AlgoData/templates/library/<nombre>/` con `template.sqx`, `brief.md`, `manifest.json`, `deps/`; fila en `templates/registry.csv`; (al correr) fila en `templates/runs.csv`. Las negativas reales ocurren después, en `sqx.projects.builder` (paso 5).
- Orden: `sqx.templates.build` (esqueleto → bloque → grupo de 1 → atar hueco 1 → vaciar hueco 2; comprueba XML, clave del bloque, 2 huecos, `build.py:172-174`) → archivo + `registry.py` → [al crear proyecto] `registry.check_name` → fichero existe → `directions` (regla 13) → `early_gates` (nuevo) → `over_the_cap` (regla 14) → coste/RAM → `what_to_build` escribe el rango.
- Defaults del dueño aplicados por la skill: larga, una condición libre, lo nombrado fijo, números no nombrados aleatorios, `Shift` 1.

3.1 Biblioteca hoy (ejecutado hoy con el código del paso)

61 plantillas en `library/` (registry: 61 `draft`, 0 `validated`, 0 `buildConfirmed`). 38 solo largas, 18 solo cortas, 5 con ambas (`gbpRandomEntry`, `gbpRandomEntryExit`, `gbpRandomExit`, `spRSI2`, `xauWeekend`). Construibles: 55 (37 con 1 fija + 1 hueco libre de entrada, 18 con 1 hueco atado + 1 libre; 27 con hueco libre de salida, 27 sin, 1 con 1 salida fija = `indicesIBS`). Rechazadas: 6 (`gbpRandomEntry`, `gbpRandomEntryExit`, `gbpRandomExit`, `gbpjpyEndMonth`, `spRSI2`, `xauWeekend`): 5 por dirección + `gbpjpyEndMonth` por tope («2 fijas + 1 hueco»; además `gbpjpyEndMonth` y `xauWeekend` salen en `clock_fixed`).

3.2 Criterios (numerados)

C3.1 Una dirección por plantilla y por build (regla 13) — `builder.directions` (`builder.py:34-42`) y negativa en `:213-218`

- Pregunta: ¿abre la plantilla operaciones en una sola dirección?
- Definición: busca las reglas «Long entry» / «Short entry» del XML y devuelve las que contienen una acción cuya clave empieza por `Enter`. CORTA con `SystemExit` si el proyecto construye (`a.workflow` o `Build` en `--tasks`) y el número de lados ≠ 1. El paso 26 (`mt5.verify`) pasa por el mismo constructor una estrategia ya generada con `--tasks Retest`, que trae `Enter` en las dos reglas: por eso la condición «si construye» (`:209-212`).
- Justificación (dueño, 2026-10-01, «regla dura»): una idea corta es otra plantilla; mezcladas no se sabe de qué lado viene el resultado.
- Punto débil: sin estadística; mide la estructura, no que la lógica sea simétrica o asimétrica.
- Ejemplo: biblioteca: 38 L, 18 S, 5 ambas (rechazadas).

C3.2 Cuenta de condiciones por lado — `holes.conditions` (`sqx/templates/holes.py:69-110`)

- Definición: por cada señal recorre el árbol atravesando `AND` / `OR` / `Not` (`LOGIC`): un `randomBlock` con `#Group#` relleno = `bound`; con `#Group#` vacío = `free`; cualquier otra hoja = `fixed`, SALVO que su «huella» (tupla de las claves de todos los `Item` de la hoja, `holes.py:78-82`) ya haya aparecido en esa señal: entonces no suma (decisión del dueño 2026-10-05: la misma condición dos veces sobre dos periodos — doble ROC, doble Vortex — cuenta como UNA fija). Lado = `Exit` en el nombre de la variable → salida, si no entrada; si hay dos señales del mismo lado cuenta la más llena (`:106-108`). INFORMA (alimenta C3.3 y C3.4).
- ⚠ El dossier (§5.7) avisa: la huella mira solo las claves de bloque, no los parámetros, así que dos `ROCAboveLevel` con niveles distintos o 3-4 copias también cuentan como una; no verificado que SQX permita el mismo bloque dos veces en un hueco. Sin límite de copias.
- Ejemplo real: `doubleROCLong` → entrada {fixed 1, bound 0, free 1}, salida {0,0,0}; sus 41 estrategias guardadas (supervivientes del paso 8, no la población) tienen 3 condiciones de entrada (2 ROC + 1 hueco), 38 de 41 con el hueco = `CBlock_WassersteinShiftHigh` (dossier 3.3).

C3.3 Tope de condiciones y rechazo de dos huecos — `buildrules.over_the_cap` (`buildrules.py:179-201`) y `assets/_build.yaml:23-28`

- Definición: tope `complexity.max_entry_conditions: 2`, `max_exit_conditions: 2`, `simple: {entry: [1,2], exit: [1,2]}`. Por lado: si `free > 1` → rechazada SIEMPRE («hasta 4 condiciones; una plantilla de solo huecos es una Simple Strategy»); si no, si `fixed + bound + free > 2` → rechazada («con 2 fijas no cabe hueco»). CORTA con `SystemExit`.
- Justificación (dueño, 2026-10-02, «fundamental»; dossier §4.5, §3.8): cada condición de más eleva el espacio al cuadrado (1 fija + hueco 0..1 con C=103 → 104 lógicas; un hueco 1..2 → 5.356, cincuenta veces más; dos huecos 1..2 → 4.603.379 como conjuntos de 2 a 4 bloques, cuentas del dossier «calculado por mí»).
- ⚠ Matiz del dossier §2.4: el rechazo de «dos huecos libres» es política de la regla 14, no aritmética (con rango 1..1 saldrían exactamente 2). Una plantilla con un SOLO hueco libre y nada fijo se acepta (rango 1..2) aunque la regla diga que eso es Simple Strategy (dossier 5.7; hoy ninguna de la biblioteca tiene esa forma en la entrada).
- Ejemplo real: biblioteca hoy: 3 plantillas fallan el tope (`gbpRandomEntry`, `gbpRandomEntryExit`, `gbpjpyEndMonth`: 2 fijas + 1 hueco), 3 más fallan solo por dirección.

C3.4 Rango «What to build» — `buildrules.what_to_build` (`buildrules.py:120-145`)

- Definición (verificado en código): sin plantilla (Simple Strategy): `complexity.simple` tal cual (entrada 1..2, salida 1..2). Con plantilla: por lado `hi = max(cap − fixed − bound, 0)`; si hay algún `bound` → `(1,1)`; si no → `(min(0 si hay fija, si no suelo del lado), hi)` con suelo entrada 1 y salida `int(exits.condition_required)` (hoy falso → 0). `set_complexity` lo escribe en `<Chart name="Main chart">` (`minConditions/maxConditions/minExitConditions/maxExitConditions`) de la tarea Build; lookback 1 (`_build.yaml:30`). No CORTA: escribe. Es lo que hace cumplir la regla 14 «por hueco»: SQX aplica el rango a CADA `RandomCondition` y lo suma a lo fijo.
- Justificación: medido con rango 1..2: con 2 huecos libres, 69 % de una población USDJPY H1 traía ≥3 condiciones de entrada; con atado + libre, 1.962 de 5.026 traían un `AND` anidado (`knowhow/authoring/condition-count-simple-vs-template.md`).

- Tabla de casos (dossier §3.5, 17 comprobaciones en `tests/test_condition_count.py`): 1 fija + 1 hueco libre → 0..1 → 1 o 2 condiciones; fija gemela (doble ROC) + hueco → 0..1 → 2 o 3; 1 atada + 1 libre → 1..1 → exactamente 2; solo 1 hueco libre → 1..2; 2 fijas sin hueco → 0..0; salida sin nada → 0..2.
- Ejemplo real: `Test_USDJPY_M30_doubleROCLong_10k_B5` (build 2026-10-08 01:24-02:57): tarea Build `minConditions=0 maxConditions=1 minExitConditions=0 maxExitConditions=2 minPeriod=2 maxPeriod=200 minShift=1 maxShift=1`; censo: solo `EnterAtMarket`, salidas `ExitAfterBars` 100 % (4 a 48 velas) y `_ExitRule_` 50 %, 103 condiciones encendidas de 575, 0 niveles stop/limit, 0 de reloj (paleta `doubleROCLong_USDJPY_M30`) (dossier; no reverificado hoy).
- Punto débil / no medido (dossier 5.9): con qué frecuencia el hueco 0..1 queda vacío en la población construida (en las 41 supervivientes nunca); la forma «atado + libre → 1..1» (18 plantillas) no se ha construido nunca con ese rango (la última con hueco atado, `donchianUpperCrossUp`, es del 2-10 y anterior al arreglo); no se sabe si el rango de salida 0..2 hace algo cuando la plantilla no tiene hueco de salida (0 condiciones de salida en 41/41 supervivientes). Un hueco atado a un grupo de 2-13 bloques (18 plantillas: `MeanReversionFilters` 2, `Structural…` / `Volatility…` 4, `TrendRegime…` / `DirectionalMomentum…` 6, `BookTriggers` 13, `EntropyFilter` 2, `AcceleratorFilter` 4) es un segundo sorteo que la paleta no puede estrechar; el código lo cuenta como idea fija (`buildrules.py:128`).

C3.5 Rango de condiciones: coherencia con el censo — INFORMA / no comprobado automáticamente tras el build

- El control posterior vive en el paso 8 y `memory.sheet` (§C1.9), no en el paso 3; no se mide cuántas lógicas distintas tiene la población construida (en las 41 guardadas son 3; los `.sqx` de un build de 10.000 no se archivan; dossier 5.9).

C3.6 Regla 15 sobre lo que la plantilla FIJA — `buildrules.clock_fixed` + `early_gates` (`buildrules.py:148-176`; llamado en `builder.py:219`)

- Definición (NUEVO en árbol de trabajo, sin commitear): para cada bloque fijo de la plantilla aplica `buildcensus.kind` y rechaza si es `clock` (regex CLOCK o `CLOCK_NAMES`). CORTA con `SystemExit` junto a otros dos portones tempranos: el proyecto ya existe vivo (carpeta del worker o fila no retirada de `registry.csv`), y, si es workflow, un timeframe sin `crosstf.timeframes` en `_build.yaml`.
- ⚠ El dossier (§5.6, §6.1) decía «en ningún sitio»: ya existe para bloques fijos. Resultado hoy: `clock_fixed` marca `gbpjpyEndMonth` (`IsMonthFirstTradingDay`, `IsMonthLastCustomTradingDay`) y `xauWeekend` (`BarDayOfWeekIs`) (ambas ya rechazadas por otras reglas). Punto débil: depende de que la lista de reloj sea completa (C2.6); `CBlock_*` de calendario no listados pasan.
- ⚠ Sigue sin comprobarse que un bloque de la plantilla lea un segundo gráfico: `filterMALong`, `filterMAShort`, sus `_withExit` y `spRSI2` abortan el build (ficha `template-second-chart-aborts-build.md`); el builder no las rechaza de antemano (dossier 5.5).

C3.7 Nombre, existencia del fichero y marca de estado

- `registry.check_name`: prefijo `Test_` / `Trade_` / `Research_`, solo guiones bajos (regla 6) — `builder.py:207-208`; plantilla existe (`:209-210`). CORTAN.
- Estado en `registry.csv`: `draft` → `validated` (XML resuelve) → `buildConfirmed` (un build real sacó estrategias con el bloque, vía `/template-run` y `sqx.inspect.template_check`, o la ventana `ui/daemon/writes.py:10-30` sin comprobar). MARCA: hoy 61 de 61 `draft`, aunque `doubleROCLong` se ha construido y tiene 3 proyectos reales; `bin/workflow.sh`, autopiloto y campaña no tocan el estado.

C3.8 Registro de lo ya probado (cinco libros; solo uno impide repetir)

- `templates/runs.csv` (`registry.py:14,18-52`): clave (plantilla, símbolo, TF), se actualiza en su sitio → pierde la historia; NO impide repetir. Hoy 7 filas (verificado): `indicesIBS|USA500|H1`, `doubleROCLong|USDJPY|M30` (10.000 construidas), `squeezeBBLong|XAUUSD|H1`, `doubleVortexLong|EURJPY|H1` (10.000), `aroonLong|NIKKEI225|H1` (10.000), `doubleROCLong|USATEC|M15` (500), `filterMALong|EURUSD|M30` (10 construidas, 0 pasan el paso 8; verdict relleno a mano). ⚠ El dossier hablaba de 3 filas; ahora 7; solo la última tiene `verdict` y `report`.
- `AlgoData/campaign/ledger.jsonl` (54 líneas, 3 celdas en último estado: indicesIBS USA500 H1 `done`, doubleROCLong USDJPY M30 `done`, squeezeBBLong XAUUSD H1 `interrupted`; verificado hoy): lo que SÍ impide repetir. `pipeline/campaign/plan.py:candidates` (`:51-85`) descarta una celda si su clave está en el ledger sin marca `test`, o si un proyecto `Trade_`/`Research_` la corrió (`_launched`, `:35-39`); solo largas, sin avisos en `manifest.json.flags` (10 plantillas con aviso: las 6 rechazadas, `acceleratorEMALong` y `_withExit` por grupo no instalado en SQX_w1, `squeezeBBShort` y `_withExit`) y con coste completo y `feedQuality/summary.json`. CORTA candidatas. ⚠ Las tres celdas son de prueba (`Test_`), así que para la campaña ninguna celda está probada (dossier 5.4; en el ledger hoy el campo `test` no aparece al leer la última línea por clave; no reverificado).
- `projects/registry.csv`: una fila por alta de proyecto (11 altas de `doubleROCLong × USDJPY × M30` del 05-10 al 08-10, todas `Test_`); `ledger/<activo>_<TF>_<plantilla>.jsonl`: cuenta N, no impide. Hoy 109 filas en `USDJPY_M30_doubleROCLong.jsonl` (dossier del 8-10: 83), de las que 8 de paso 6 suman `n_scored` = 4.880.561 (verificado hoy); por pasos: 17: 34, 18: 18, 18.5: 18, 24: 16, 6: 8, 19: 6, 8: 4, 10: 2, 20: 2, 12: 1.

- ⚠ Un intento que falla no queda en ningún registro (`Test_EURUSD_M30_filterMALong_10k` del 7-10 aborta en 1 s; solo en `projects/registry.csv` y en la carpeta del autopiloto; `runs.csv` hoy sí tiene la fila `filterMALong|EURUSD|M30` de otra corrida del 9-10, `…_charts`).

C3.9 Tamaño del espacio y N que se cobra (informa; no es criterio de código)

- Tres N distintas (dossier §3.8, «calculado por mí»): lógicas posibles (104 para `doubleROCLong` con paleta de 103 + hueco vacío; nadie la calcula ni la anota), estrategias generadas (4.880.561, lo que suma `ledger/trials.py:13-58` como N en el paso 6; verificado hoy), N efectiva (`population_n_eff`, `trials.py:78`, solo sobre supervivientes). Valor esperado del máximo del ruido (normal, independiente): N=104 → 2,52σ; 5.356 → 3,70σ; 165.600 → 4,49σ; 4,88 M → 5,17σ (cota √(2 ln N) 3,05/4,14/4,90/5,55; dossier, integración numérica propia, coincidencia con dos revisores).
- Punto débil: el supuesto de independencia falla en ambos sentidos (38 de las 41 guardadas de la misma lógica tienen correlación semanal mediana 0,73 ≈ 2 independientes, calculado por el revisor, sobre supervivientes del mismo cribado; y la asimetría de las operaciones sube el máximo hasta 6,58σ en una simulación de juguete). ⚠ `ledger.accumulated` se llama desde `ledger/report.py`, `portfolio/common/construct/oos2.py`, `core/archive/manifest.py` y la ventana, pero ningún fichero de `pipeline/autopilot/` (grep del dossier, no repetido hoy): los pasos 6-25 no deciden con N. La N va por plantilla × activo × TF: otra plantilla sobre USDJPY M30 empieza de cero. Hoy `trials.accumulated("USDJPY_M30_doubleROCLong")` falla con `TypeError: string indices must be integers` (`ledger/trials.py:40`) al llamarla con el nombre del estudio; no investigado si es mi llamada (argumento) o un fallo; el dossier la llamó sin error.
- Tramos gastados de esa celda (según el libro; dossier §3.7/5.12 con datos del 8-10): `oos1` cribado 4 veces con 4 `config_hash` distintos (19.000 estrategias → 5, 3, 3, 56), 24 filas sobre `oos2` (pasos 17, 18, 18.5, 24, 19, 20; el paso 20 ciego ventana 2023-01-01 → 2026-10-01); para la campaña la celda está sin probar. Hoy las filas de pasos 17+18+18.5+24+19+20 suman 94 (verificado), es decir, `oos2` sigue consumiéndose sobre la misma celda.

3.3 Ejemplo de cadena

Biblioteca 61 → (dirección ≠ 1) −5 → (tope/hueco) −1 más (`gbpjpyEndMonth`) = 55 construibles (37 «fija + hueco» + 18 «grupo + hueco») → 4 con proyecto alguna vez al corte del dossier (hoy `runs.csv` registra 6 plantillas distintas con corrida: indicesIBS, doubleROCLong, squeezeBBLong, doubleVortexLong, aroonLong, filterMALong) → 0 `buildConfirmed`. Un caso: `doubleROCLong` → conditions {1 fija,0,1 libre} → rango escrito 0..1 → paleta de 103 → 8 builds, 4.880.561 generadas, 324.271 aceptadas (dossier) → 41 guardadas tras el paso 8.

3.4 Tiempos/recursos

`sqx.templates.build` y `holes`: segundos, sin SQX ni RAM relevante; sin dato en `perf/` (no verificado). El coste real está después (ver paso 6): p. ej. el build `…_10k_B5` tardó 01:24-02:57 (93 min, dossier). La campaña bloquea candidatas leyendo `board.json` y `manifest.json` (instantáneo).

3.5 Discrepancias del paso 3

- ⚠ Dossier 5.1 (la aritmética de la regla 14 no estaba en el commit): hoy `holes.py`, `test_condition_count.py` y `sqx/templates/` ya no salen en `git status` (commiteados); `buildrules.py` (35 líneas nuevas: `clock_fixed`, `early_gates`) y `builder.py` siguen sin commitear.
- ⚠ Dossier 5.6/6.1 («reloj fijo en la plantilla: en ningún sitio») ya no es cierto: `clock_fixed` + `early_gates`.
- ⚠ `runs.csv` tiene 7 filas (dossier 3); el veredicto lo rellena a mano solo la de `filterMALong`.
- ⚠ Sin cambio: 61/61 `draft`; dos plantillas largas con segundo gráfico entran a la campaña sin aviso (`filterMALong`, `_withExit`); `sqx/templates/README.md:23` y la skill dicen «500 condiciones» (son 575) y «copia keltnerUpperCrossUp» (retirada); ficha `condition-count-simple-vs-template.md` describe la librería anterior; `CLAUDE.md` regla 14 no menciona «atado+libre → 1..1».
- ⚠ Hueco atado a grupo de varios bloques tratado como idea fija (18 plantillas) y rango 1..1 nunca construido.
- ⚠ N efectiva de lógicas no medida; `trials.accumulated` falla hoy con mi llamada de nombre de estudio (posible mal uso mío).

Pasos 4 y 5 — Preflight y creación del custom project (estado del árbol de trabajo, 2026-10-12)

Convenciones: «CORTA» = el código aborta; «MARCA» = deja una marca/aviso; «INFORMA» = sólo imprime. Manda el código del árbol de hoy (con cambios sin commit); donde el dossier del 2026-10-08 discrepa lo anoto con ⚠. Ningún paso de esta sección juzga estrategias: no hay regla en `pipeline/autopilot/criteria.yaml` ni hechos en `facts.py`. Lo que se «criterio» aquí son (a) las comprobaciones bloqueantes y (b) las cifras de coste/ventana que el preflight deja escritas y que todos los pasos 6-26 heredan. Corridas observadas hoy (lectura, 2026-10-12): `python3 -m core.assets <S>` sobre las 28 fichas de `assets/symbols/` (27 + `XAUUSD_SYN01`): todas salen 0.

PASO 4 — Preflight: costes, ventanas, rangos y calidad del feed

4.0 Propósito, entradas, salidas

- **Pregunta:** antes de crear nada para un activo, ¿están decididos sus costes, caben sus ventanas en los datos que SQX tiene, y qué cifras exactas llevará cada tarea?
- **Comando:** `python3 -m core.assets <SÍMBOLO>` (`core/assets.py:140 main`). Lo llaman: `bin/workflow.sh:59` (entero, «no se crea nada» si falla), `ui/daemon/launch/run.py`, `pipeline/campaign/plan.py`, y sólo `pending()` el builder (`sqx/projects/builder.py:222-225`), los configuradores MC Retest/SPP/WFM/rankings y `pipeline/stages/gates.py`. El guard de hooks no vigila la regla 5.
- **Entradas (3 ficheros fusionados por `core/assetdata.py:85 load`):** `assets/symbols/<S>.yaml` (clase, feeds, instrumento, `costs`, `mc_retest`, `mt5` / `mt5_point`, `session`); `assets/_policy.yaml` (tramos por activo, `segments_default`, `data:` = rango que SQX tiene, `swap`, `mc_retest.default_multiples`); `assets/_classes.yaml` (esquema de campos por clase). Además `AlgoData/feedQuality/<feed>/summary.json` (aviso A4) y `assets/_markets.yaml` (universo de retest).
- **Salidas:** no escribe nada. Imprime el informe (`report()`, `core/assets.py:51`) y un código de salida. Lo que las tareas SQX llevarán realmente lo produce `sqx_settings(data, tramo)` (`core/assetdata.py` ~191-215), que llama cada configurador.
- **Orden de aplicación en `main()` (`core/assets.py:140-176`):** (1) `validate()` → si algo, imprime «ESQUEMA ROTO» y **sale 3 sin imprimir el informe**; (2) `pending()` calculado antes de imprimir (cambio nuevo sin commit); (3) `report()` en `try/except (TypeError, KeyError)`: si hay costes pendientes se traga la traza; (4) `feed_quality()` (A4); (5) `past_data()` → A1 a **stderr**; (6) `segments_pending()` → A2; (7) `mc_pending()` → A3 (sólo si no hay `missing`); (8) si `missing` → «BLOQUEADO: … Pregúntale al dueño», **sale 2**.
- **Códigos:** 0 = informe (con o sin avisos); 1 = activo desconocido/sin argumento/traza no cubierta; 2 = coste obligatorio sin valor; 3 = ficha rota o ventana sin datos.

4.1 Criterios que CORTAN

C4.1 — Esquema de la clase (`validate`, `core/assetcheck.py:37-55`; salida 3)

- **Pregunta:** ¿la ficha cumple lo que su clase (`forex` / `no_forex`) exige?
- **Definición:** una línea de error por: campo de coste que la clase exige y falta; campo que sobra; falta `mc_retest.spread` o `mc_retest.slippage` (deben existir aunque sus bordes sean `null`); `instrument.*` no numérico (un comentario pegado lo vuelve texto); cualquier hoja de la ficha que sea un texto que se lee como número (`numeric_strings`, líneas 14-34; `1e-05` leído como texto rompió una vez el paso 26); más `before_data` (C4.2). Los campos exigidos salen de `core.assetdata.fields()`: spread y slippage por cada mitad de tramo (`is`, `oos`, `oos2`, desde 2026-09-30 en las dos clases), comisión, y los dos swaps. Corta; población: la ficha entera.
- **Por qué:** una ficha incompleta produce tareas con coste a cero sin que nadie lo vea. Punto débil: es sólo estructura; no mira que el valor sea cierto ni plausible.
- **Corrida:** 28/28 fichas pasan (2026-10-12). Sin corrida de fallo reciente.
- **Coste:** milisegundos (lectura YAML con caché por mtime, `assetdata._parse`). Sin dato de perf/ propio.

C4.2 — Tramo que pide histórico que SQX no tiene (`before_data`, `core/assetcheck.py:113-131`; salida 3)

- **Definición:** si `data:` del activo es `null` → «SQX no tiene datos de `<feed>`» (corta, ninguna ventana es válida). Si no: por cada tramo con `from` no nulo, corta cuando `año(from) < año(primer dato)`.
- **Por qué:** SQX corre una ventana más corta sin avisar, y entonces las operaciones/año salen falsas. Ejemplo documentado: DAX40 con datos desde 2013-09-30 y una construcción desde 2008.
- **Punto débil verificado en código:** compara **años**, no días; `build: {from: 2011}` con datos desde 2011-09-19 pasaría y la ventana se quedaría 8 meses corta. Hoy no ocurre: los activos con datos a mitad de año llevan fecha exacta en `_policy.yaml` (NIKKEI225 2011-09-19, DAX40/DJ30 2013-09-30, USA500/USATEC 2012-01-19, HK50 2012-03-29, UK100 2011-09-19, AUS200 2013-10-09, XCUUSD 2012-03-05). Tampoco comprueba `from < to`, solapes ni contigüidad de tramos. `data:` es una foto: se refresca con `python3 -m core.assets --dataranges` (`core/assets.py:179`); si no se ejecuta tras actualizar datos, C4.2 y A1 comparan contra un rango viejo.
- **Corrida:** 28/28 pasan. `SP500ft` aparece en `_policy.yaml` con `data: null` y todos los tramos `null` (feed inexistente en SQX) pero ya no tiene ficha en `assets/symbols/` (no es corrible).

C4.3 — Coste obligatorio sin valor (`pending`, `core/assetcheck.py:59-79`; salida 2)

- **Definición:** obligatorios idénticos en las dos clases (`REQUIRED`, líneas 9-10): `spread_is`, `spread_oos`, `spread_oos2`, `commission`. `commission.use` es un dict `{tramo: {method, value}}`; bloquea si **cualquier** tramo de `data["segments"]` carece de `value` (`(use.get(seg) or {}).get("value") is None`; cambio sin commit: antes `v["value"]`, que ahora tolera un tramo ausente). Los demás (slippage, `swap_long`, `swap_short`) **no** bloquean.
- **Por qué:** una corrida sin coste son días de CPU sobre backtests sin fricción. Slippage = 0 es una decisión declarada («a diferencia de un spread sin pactar», `_classes.yaml`). Para el swap no hay razón escrita de por qué no bloquea.
- **Corrida:** 28/28 pasan. ⚠ El dossier §5.7 decía que una comisión sin decidir daba traza de Python (`TypeError` en `report()` → `mc_retest()` → `sqx_settings`); el árbol de hoy lo cubre con el `try/except` de `core/assets.py:158-163` y `sqx/CLAUDE.md` lo recoge («exit 2 + BLOQUEADO … no traceback since 2026-10-11»). No verificado ejecutando (no se tocan fichas).

4.2 Criterios que sólo AVISAN / INFORMAN

id	qué	dónde	salida	hoy (2026-10-12)
A1	tramo cuyo fin es posterior a la última vela de SQX (`past_data`, `assetcheck.py:135-152`)	`core/assets.py` → **stderr**	aviso, no bloquea	ninguno (stderr vacío en 28/28)
A2	tramo con `from` / `to` nulo (`segments_pending`)	informe	aviso	ninguno
A3	rango MC Retest sin resolver (`mc_pending`, `assetcheck.py:156-172`)	informe	aviso	`XAUUSD` y `XAUUSD_SYN01`: `min_distance` sin decidir (sólo importa con órdenes stop/limit; con órdenes a mercado la tarea `MCR 4 MinDist` ni se escribe)
A4	calidad del feed (C4.7)	informe	aviso; **nunca bloquea** (respuesta 3.1 del dueño, encargo 17)	15 activos con aviso de «construcción anterior al año estable» (lista en C4.7)
A5	tramo reservado `reserved_for` (oos2: WFC, CSCV, MarketSurfaces, WFM, BlindJoint, ATRStop)	informe	sólo con `ALGO_AUTONOMOUS=1` (`assetdata.enforced`)	nadie pone la variable: protección de documento, no de código
—	`provisional()` (campos cuyo `why` contiene «PROVISIONAL», `assetcheck.py:83`)	no lo imprime el comando; marca `assets/INDEX.md`, el registro del builder (`builder.py:166`) y el ledger del pipeline	marca	ningún activo la lleva hoy

⚠ A1 va a stderr y el informe no lo incluye; `ui/daemon/launch/run.py` sólo enseña stderr si el comando falla, así que desde la ventana ese aviso no se ve nunca (dossier §5.6; no reverificado el fichero de la ventana).

4.3 Los tramos y las ventanas (no son un criterio que juzgue, pero fijan la muestra de todo lo demás)

- **Definición** (`assets/_policy.yaml:25-48` `segments_default`, fechas por activo `:50-198`): tres tramos con año completo, **ambos extremos inclusivos** (`core/assetdata.py:135-170` `_ms`: `to: 2017` → dateTo = 2018-01-01 00:00 UTC; `to: 2026-09-30` → 2026-10-01 00:00 UTC). Papel: `build` (única muestra que ve el generador; spread `is`), `oos1` (retest hasta SPP: decaimiento, Monte Carlo, MC Retest, cross-market, SPP; spread `oos`), `oos2` (último fuera de muestra: WFC, CSCV, superficies de mercado 18.5, WFM, BlindJoint, stop ATR; spread `oos2`).
- **Familias:** forex y metales construyen 2008-2017 / oos1 2018-2022 / oos2 2023-2026-09-30 (aunque haya datos desde 2003; «por qué 2008» no está escrito en ningún fichero: ventana heredada del oro). Índices, crudos y cobre: desde su primer dato hasta 2019 / 2020-2023 / 2024-2026-09-30. Reparto en NIKKEI225 55 %/27 %/18 %; USDJPY 53/27/20 (años 8,3+4+2,75 y 10+5+3,75).
- **El final de oos2 se mueve solo:** el primer sábado de cada mes `bin/monthly-oos2-roll.sh` lo lleva al último día del mes anterior si los datos llegan (`assets/RULES.md:161-165`; no releído en esta sesión).
- **Punto débil:** `build` y `oos1` se gastan por selección (el generador y los pasos 7-16 eligen con ellas); sólo `oos2` queda «limpio», y se mira muchas veces (el ledger cuenta cada mirada). Los costes de los tres tramos salen de ticks 2025-26, es decir de dentro de `oos2` (ver C4.5): los tramos no son comparables en coste.
- ⚠ Nota de convención: el proyecto XAUUSD del master tiene `dateTo` = 2017-12-31 00:00 (un día menos que la convención inclusiva); sus databanks vienen de esa ventana (comentario de `_policy.yaml`).

4.4 Criterio: spread por tramo (cifra escrita en la ficha, `costs.spread_is/oos/oos2`)

- **Pregunta:** ¿qué spread (puntos de SQX) debe cobrar el backtest en cada tramo?
- **Regla vigente** (dueño, 2026-10-04; `assets/RULES.md:93-100`; el `why` de cada ficha): spread medio **por tiempo** de la firma de fondeo **más ancha** (la mayor entre las medias de 12 y de 3 meses) de FTMO/Hantec/FundingPips, expresado como % del precio, × **precio medio de cada tramo** ÷ `tick_size`, **sin factor de seguridad** (`ledger/thresholds.yaml:1030` `spread.safety.factor = 1.0`; antes 1,25 desde 2026-09-27). Fuente: ticks de las firmas muestreados 3 días/mes (martes-jueves de la segunda semana), tope de 60 s por cotización. `knowhow/costs/funded-tick-spread-vs-price.md`.
- **Decide / efecto:** fija el coste de TODAS las tareas de TODOS los pasos; no corta nada. Si cambia, cambia qué estrategias sobreviven en 6, 8, etc.
- **Ejemplos reales** (fichas hoy): NIKKEI225 304,11 / 474,91 / 796,12 puntos (= 0,017352 % × precios medios 17.525,7 / 27.368,7 / 45.879,8 ÷ 0,01; firma ancha = FundingPips ≈ 10,0 unidades de precio fijas). USDJPY 0,51 / 0,58 / 0,77 (0,005135 % FTMO; precios 98,909 / 113,513 / 149,483). XAUUSD 13,33 / 16,93 / 31,42 (0,0105 % FTMO sobre 1.265,27 / 1.606,85 / 2.981,88; media de 3 meses).
- **Por qué (declarado):** «la historia paga la fricción de 2026 en proporción al precio» (decisión explícita del dueño, «% a propósito»), una sola regla para los 28 activos y priorizando el fondeo.
- **Puntos débiles estadísticos (del dossier, parte reverificada):** 1. Se extrapola un % de 2025-26 a 2008-2023. Donde hay medida de Darwinex (único con ticks 2017-2023) la ficha del NIKKEI225 queda debajo: ficha/Darwinex-medido = 0,41 en `build` (304 vs 747, sólo 2017-10..2019, 577 días), 0,65 en `oos1` (475 vs 732, 1.034 días; la ficha < percentil 2,5 de lo cotizado, 501) y 1,74 en `oos2` (796 vs 458). El coste relativo sube en escalón en las fronteras de tramo: la caída de `oos1` a `oos2` mezcla pérdida de filo con subida de coste (no se pueden separar). La tarjeta que justifica el cambio dice lo contrario en su título («los spreads de los índices son casi constantes en **puntos**»; llevar el % de hoy atrás «infravalora entre 2 y 4 veces»). Cifras «ficha vs medido» calculadas por el revisor del dossier, no reverificadas aquí. 2. Muestra de ticks: ~36 días por firma y activo (ningún lunes, viernes ni primera semana de mes); IC95 % bootstrap de meses ±7-9 % (NIKKEI225 0,0162-0,0185 %, USDJPY 0,0047-0,0056 %). El error grande no es de muestreo sino de llevar el % 15 años atrás. 3. Ninguna fuente de ticks de firma llega antes de 2017; Hantec desde 2023, FTMO 2024, FundingPips 2025 (no medido antes). `build` de forex (2008-2017) es modelo en el 97,6 % de sus días del estudio Darwinex. 4. La media no describe ninguna hora real en forex (hora 0 del servidor = 3,3 % del tiempo y 29 % de la media en USDJPY/FTMO; spread 9× la media, el resto de horas 0,73×): una estrategia que entra en la vela de las 00:00 paga ~9-10× lo que la ficha cobra (0,77 vs 7,2 puntos en `oos2`). 5. Hay tres cifras de spread para el mismo tramo (NIKKEI225 `build`: 304 ficha / 607 propuesta de septiembre ×1,25 / 747 Darwinex medido).
- **Corrida real sobre el efecto:** no medido cuántas estrategias de una población cambian de signo entre 304 y 747 puntos (se propone leerlo con el «coste de equilibrio» c* del paso 8 sobre una cosecha existente). Sin corrida real.
- ⚠ **El método vigente no tiene comando.** En `grep` de `*.py` sólo `studies/data/spread/onboard.py` menciona «más ancha/widest»; ningún módulo calcula «firma más ancha × precio medio del tramo». Las 28 fichas llevan el `why` de «recalibrado 2026-10-04» hecho por un script que no está en el repo (commit `3b71384`). Atenuante nuevo en el árbol: `onboard.apply` (`studies/data/spread/onboard.py` ~110-118) **se niega a sobrescribir** un spread cuyo `why` contenga «sin factor» («28 of 28 cards carry it»), así que el riesgo del dossier §5.1 («`--write` pisa las fichas con Darwinex+modelo») queda tapado por un marcador de texto, no por una definición del método. Un activo **nuevo** dado de alta con `/asset-onboard` sale con el método antiguo (Darwinex medido/modelado, `safety.factor` 1,0, slippage 35 %, comisión forex 8 USD de `config.yaml: onboard.commission_usd_per_lot`, ≠ 5 USD de las fichas forex).
- **Estudio de apoyo `studies/data/spread/`:** mide el spread de Darwinex minuto a minuto desde 2017-10, lo reconstruye hacia atrás con uno de cuatro modelos (`relativo`, `puntos`, `volatilidad`, `volatilidad_precio`; `model.py`), elige el de menor error en validación cruzada con `split: 2022` salvo que el dueño lo fije (`relativo` para DAX40, DJ30, NIKKEI225, USA500, USATEC, UK100, AUS200; `config.yaml: model.fixed`) o si el relativo es constante (±20 % por año completo, `ledger spread.constancy.tolerance`; `verdict.choose`); cifra por tramo = media del relativo × precio medio ÷ tick × factor (`verdict.points`). Calcula también percentiles 2,5-97,5 del cociente día/modelo (`verdict.mc_multiples`: NIKKEI225 0,27×-1,99×; USDJPY 0,65×-1,47×) y la banda `spread = a·precio^b` (NIKKEI225 b = −0,63; USDJPY b = +2,26). Hoy ninguno de esos dos últimos alimenta las fichas. Cubre 23 activos (`config.yaml: band.assets`).
- **Coste de cómputo:** sin dato (no hay `timing` de AlgoData para el estudio de spread; `AlgoData/spread/<feed>_TICK/` guarda `minutes.parquet`, `daily.parquet`, etc.).

4.5 Criterio: deslizamiento (`costs.slippage_is/oos/oos2`)

- **Definición:** 35 % del spread de su tramo, por pata (dueño 2026-10-04; antes la mitad; `studies/data/spread/config.yaml: safety.slippage_share`). SQX lo cobra en entrada **y** en salida, así que una operación paga spread × (1 + 2 × 0,35) = **×1,7**. Ejemplos: NIKKEI225 106,44 / 166,22 / 278,64; USDJPY 0,18 / 0,20 / 0,27; XAUUSD 4,67 / 5,93 / 11,00.
- **Por qué:** 600 operaciones reales de FTMO, Hantec y FundingPips costaron de media 1,33× el spread medio de 12 meses de la firma más ancha (IC95 % 1,08-1,72). El 1,7 queda justo debajo del extremo alto. Por firma: FTMO 1,17, FundingPips 1,64, Hantec 1,13. Por pata: salida por stop 1,23-1,24 spreads, por señal 0,44-0,60; entrada por señal + salida por stop = 1,68 (FTMO) a 1,83

(FundingPips), es decir más de lo que SQX cobra justo en las perdedoras.
- **Puntos débiles:** el único margen sobre la media (el «sin factor») es la diferencia 1,7 vs 1,33 (+28 %), y se aplica sobre el spread de la ficha, no sobre el medido; el IC es de 600 operaciones mezclando 3 firmas; el coste real depende de stop vs señal y no se modula por tipo de salida. No bloquea.
- **Corrida real:** 600 operaciones reales de 2026 (tarjeta `funded-tick-spread-vs-price.md`).

4.6 Criterio: comisión y swap

- **Comisión** (un `{method, value}` por tramo; `core/assetdata.py` ~210-213): decisión del dueño 2026-10-01 = peor caso de FTMO y Hantec. Índices: `PercentageBased 0` (confirmado en operaciones reales: sin comisión en FTMO ni Hantec; dueño 2026-09-29). Forex: `SizeBased 5,0` USD por lote (ida y vuelta; operaciones reales 2,50 USD/lote/lado). XAUUSD: `SizeBased 5,0` (Hantec, 2,50/lado verificado en `XAUUSD.h`; FTMO 0,0014 % ≈ 0,0040/0,0029/0,0019 % a la mediana de cada tramo, Hantec es el peor en los tres). `PercentageBased` se cobra **una vez** por operación sobre el precio de apertura (verificado operación a operación 2026-09-27, `knowhow/costs/commission-methods.md`); la comprobación de `SizeBased` operación a operación **sigue pendiente** (si SQX lo cobrase por pata, USDJPY pasaría de 10,49 a 15,49 USD por ida y vuelta, y la comisión del 47,7 % al 65 %). Bloquea si falta un tramo (C4.3).
- **Swap** (`studies/data/spread/fundedswap.py:29-70`; dueño 2026-10-01): cada lado es el **peor** de FTMO y Hantec leído en vivo del servidor MT5 ese día. Índices en % anual sobre nocional (`puntos MT5 × point ÷ precio × 36.000`; NIKKEI225 −4,78 % largo (FTMO), −1,87 % corto (Hantec)); forex/metales en puntos SQX por noche (`$/lote-noche ÷ (point_value × paso del tick)`; USDJPY +2,57 largo, −22,15 corto; el swap forex usa el paso del tick (`TICKSTEP`, 0,001), diez veces menor que `tick_size`, 0,01). Triple swap miércoles (resto) o viernes (índices). No bloquea.
- **Puntos débiles:** (a) **swap de un día aplicado a toda la historia**: 16 de los 27 activos tienen un lado abonado hoy (+1,62 USD/lote/noche = +0,59 % anual en USDJPY largo, abonado también en 2008-2015 con diferencial ≈ 0); el dueño puso 0 en el lado abonado del crudo por esa razón, en los demás no. En NIKKEI225 largo una noche cuesta el 45 % de una ida y vuelta completa y una semana (7 cargos con el triple) 3,1× la fricción de entrar y salir, así que en largas de varios días el swap pesa más que el spread. (b) No hay serie histórica de swap en el proyecto. (c) Única verificación operación a operación del swap (2026-10-04, 1.050 largas de USDJPY, `knowhow/costs/swap-types.md`): residuo +26,1 USD por operación frente a +5,4 predicho, sin explicación; si hay sesgo es a favor de los largos. (d) `sqx_now` de las fichas es lo que llevaba el master al escribirlas, no una lectura actual.
- **Ejemplos de coste total, un lote, ida y vuelta en `build`** (cálculo del dossier con las fichas): NIKKEI225 0,3264 USD = 0,0295 % del nocional (spread 0,1920 + slippage 2×106,44 → 0,1344; comisión 0); USDJPY 10,49 USD = 1,66 pips (spread 3,22 + slippage 2,27 + comisión 5,00).

4.7 Criterio: rango de perturbación del MC Retest (lo consumen los pasos 13 y siguientes)

- **Pregunta:** ¿entre qué spread y slippage (puntos absolutos) sortea el MC Retest?
- **Definición** (`core/assetranges.py:10-43`): si la ficha escribe `min` y `max` mandan («declarado»). Todas los dejan `null` desde 2026-10-04, así que el rango = `multiples` propio de la ficha (`mc_retest.spread|slippage.multiples`) o, si no tiene, `mc_retest.default_multiples` de `_policy.yaml:253-259` = **0,5× a 2×** del coste con que corre el backtest en el tramo `build` (el informe sólo imprime `build`; MCR 8 llevaba el de `oos1`). Lo que se escribe como máximo es `max(factor.max × coste, mín + MIN_SPAN)` con `MIN_SPAN = 1,0` punto (`assetranges.py:7`, línea 41; SQX sortea en rejilla de ~0,1 punto: con <10 pasos el rango no tiene dispersión, `knowhow/costs/mc-retest-ranges.md`). `min_distance` es null en todas y sólo cuenta con órdenes stop/limit.
- **Por qué:** 0,5×-2× = «la firma más barata en un mes tranquilo y el peor mes de la más ancha» (dueño 2026-10-04; antes 1×-4×). El `multiples` propio sale de «el peor mes de la firma más ancha».
- **Corrida real hoy** (`mc_retest(load(S))`, 2026-10-12; spread min–max / slippage min–max, tramo build): NIKKEI225 152,06–608,22 / 53,22–212,88; XAUUSD 6,67–26,66 / 2,34–9,34; USDJPY 0,255–1,377 / 0,09–**1,09**; EURUSD 0,16–1,92 / 0,055–**1,055**; AUDUSD 0,315–2,457 / 0,11–**1,11**. Activos con `multiples` propio: AUDJPY 2,8, AUDNZD 3,1, AUDUSD 3,9, DAX40 2,3, EURAUD 2,9, EURGBP 2,7, EURUSD 6,0, GBPUSD 2,7, USDCAD 3,2, USDCHF 2,8, USDJPY 2,7; el resto (índices, crudos, metales, CADJPY, EURJPY, GBPJPY, NZDUSD, AUS200, HK50…) usa 0,5×-2×.
- **Puntos débiles / ⚠:** (1) en forex el `MIN_SPAN` convierte el rango de slippage en otro: USDJPY `max = max(2,7×0,18; 0,09+1) = 1,09` = 0,5×-6,06× (el preflight imprime «0.5x-2.7x»); lo mismo en AUDUSD (×5,05), CADJPY (×2,77), EURGBP (×4,35), EURJPY (×2,88), EURUSD (×9,59), GBPUSD (×3,62), NZDUSD (×3,28), USDCAD (×5,5), USDCHF (×3,36); la etiqueta `source` del informe miente en esos casos (cifras del dossier; confirmadas en EURUSD/AUDUSD/USDJPY en la corrida de hoy). Media del sorteo en USDJPY-slippage 3,3× y el 91 % de repeticiones más caras que el backtest. (2) El sorteo uniforme es un nivel constante para todo el backtest: no simula un spread que se abre a la hora de entrada (rollover, apertura, noticia). (3) En índices el techo no alcanza lo medido: NIKKEI225 608 en `build` < media Darwinex 747 (la prueba más dura sigue siendo más barata que la realidad medida); el peor mes muestreado de FundingPips (marzo 2025, 15 unidades a 36.740) es 2,35× su media de 12 meses, por encima del techo 2×; lo medido en Darwinex día a día da 1,99×. (4) No reproducido: de dónde sale el `multiples` por par (USDJPY 2,7 y EURUSD 6,0; el revisor obtiene 1,7 y 6,7 con sus muestras). (5) `verdict.mc_multiples` calcula un rango medido por activo que nadie lee.
- **Coste:** sin dato.

4.8 Criterio: conversión de puntos MT5→SQX (informativo)

`to_sqx = point MT5 ÷ tick_size SQX`, guardado en `mt5_point.<firma>.to_sqx` de cada ficha. NIKKEI225 ×1; USDJPY ×0,1 (spread MT5 8 = 0,8 SQX); EURUSD ×0,1; UKOIL FTMO ×1 pero **Hantec ×10**. **Ningún código lee `to_sqx`** (es una nota para configurar a mano; el paso 26 usa el `point` en vivo y avisa si difiere, `mt5/verify/conditions.py:84-88`; no reverificado).

4.9 Criterio: calidad del feed (aviso A4; `core/assets.py:105-137 feed_quality`, `studies/data/feedQuality/`)

- **Pregunta:** ¿los feeds M1 sobre los que se construye tienen anomalías (picos que se deshacen, velas congeladas, huecos, caídas del proveedor), y desde qué año son estables?
- **Qué imprime:** por feed con `summary.json` (nada si nunca se escaneó): `calidad del feed X: N anomalías marcadas por año (mediana desde 2013, K = k), estable desde Y (escaneado fecha)`; si `año(build.from) < stable_from` → `AVISO: la construcción empieza en …` con los grados de los años afectados; y los «episodios» por columna (primeros 12 meses). **Avisa, nunca bloquea** (respuesta 3.1 del dueño).
- **Detección** (`detect.py`, `summary.py`, números en `ledger/thresholds.yaml` clave `feedQuality.*`, todos del dueño, 2026-09-26): escala robusta por hora de la semana = MAD × 1,4826 sobre las 52 semanas previas (`scale.window_weeks 52`, arranque desde 13 semanas, suelo 3 ticks y 0,25 × mediana de casillas); un minuto es **pico** si |movimiento del cierre| / escala ≥ K, y se **marca** sólo si se deshace (`spike.m 3` velas, `rho 0,8` de vuelta del salto, de cierre a cierre); el pico que no se deshace es «movimiento extremo» y no se marca (es mercado). **Congelado** = ≥ 10 velas M1 seguidas con OHLC idéntico (`frozen.length`); **hueco** = ≥ 5 min sin vela dentro de la sesión deducida (`gap.minutes 5`); **caída del proveedor** = silencio de 5-179 min compartido por todos los feeds (`partial_close_minutes 180` ⇒ ≥ 180 = cierre parcial, se anota); las horas del feed 23, 0, 1 (rollover de Nueva York) no cuentan para congelados ni huecos (se cuentan aparte, nunca marcan). Sesión deducida del feed (`session.share 0,5` de semanas con vela, años 2015-2019). **K por feed** = el menor de {15, 20, 25, 30} cuya mediana de picos de cierre en los años tranquilos 2013-2019 queda < 52 al año (`k.max_per_year 52`; `summary.k_star`). Marcadas = cierre y vuelta + mecha y vuelta + congelado + hueco + caída del proveedor (`summary.MARKED`).
- **Año estable** (`summary.stability`): G(año) = huecos propios + caídas del proveedor; base = mediana de G desde 2013 (`stable.base_year`); un año es «ok» si G ≤ max(3 × base, 30) (`stable.factor 3`, `stable.floor 30`); `stable_from` = el primer año a partir del cual **todos** los siguientes son ok; los anteriores son «calidad B» si G ≤ max(10 × base, 30) (`b_factor 10`) o «inestable». **Episodio** = mes con congelados/huecos > 5 × la mediana de los 36 meses previos y ≥ 5 sucesos (`episode.*`). El veredicto del feed (`feed.py`) marca «sesión MAL declarada» si los huecos propios residuales por año superan 30 (`gap.max_residual_per_year 30`, número distinto aunque de igual valor que el `floor`).
- **Ejemplo real NIKKEI225** (scan 2026-10-04, K = 20, base 238, umbral estable 714, umbral B 2.380): 2011 (desde sept.) 13 huecos B; 2012 887 B; 2013-2016 200-309 B; **2017 2.429 inestable**; 2018 357 B; **2019 2.026** (rompe la racha; 2.026 > 714) → primer año estable **2020**; 2020 543; 2021-23 134-290; 2024-26 13-38. Salida: `291 anomalías marcadas por año … estable desde 2020`, `AVISO: la construcción empieza en 2011, antes del año estable 2020: esos años son calidad B y inestable`, episodios de congelado 2014-09..12 y de hueco 2017-05..08, 2017-10, 2019-04, -06, -07, -11, -12, 2023-02. La mediana de huecos propios es 245/año (3 picos de cierre y 1 de mecha que se deshacen al año): es más un problema de sesión mal deducida (el viernes sale partido en dos trozos con un minuto entre medias; no validado) que de precios falsos.
- **Tabla real (summary.json hoy, 27 feeds escaneados):** `stable_from` posterior al inicio del `build` en **15 activos**: AUDNZD (2009 vs 2008), AUDUSD (2025), AUS200 (2018 vs 2013; 559 huecos/año), DAX40 (2019), DJ30 (2019), NIKKEI225 (2020), NZDUSD (2025), UK100 (2025), UKOIL (2018), USA500 (2020), USATEC (2019), USOIL (2020), XAGUSD (2020), XAUUSD (2014 vs 2008), XCUUSD (2018). Huecos propios por año mayores que 30: AUS200 559, HK50 351*, NIKKEI225 245, XCUUSD 162, UKOIL 109, USOIL 148, UK100 51, XAGUSD 33, DAX40 31, USA500 30 (*HK50 sin aviso porque `stable_from` 2012 = su build.from). Los feeds `*_TICK`, `EURUSD_M1_dukas`, `XAUUSD_SYN01` no tienen escaneo (el informe no imprime nada de ellos).
- **Puntos débiles / ⚠:** (a) **`stable_from` depende de un solo año malo reciente**: AUDUSD y NZDUSD salen con `stable_from` 2025 con 3 y 6 huecos propios por año de mediana porque 2024 supera el listón (AUDUSD 2024 «inestable», NZDUSD 2024 «calidad B»); UK100 igual (2012 «inestable» y 2025). Resultado: el aviso salta para toda una construcción 2008-2017 por un evento de 2024 (conclusión mía, a partir de `grade` en `summary.json`; la causa concreta del 2024 de AUDUSD no verificada). (b) La frase «mediana desde 2013» está escrita a mano en `core/assets.py:~122` (el año viene del ledger). (c) El preflight no imprime los huecos propios por año ni el veredicto de sesión (`feed.py`), sólo `marked_per_year`. (d) El listón de 30 son dos parámetros con el mismo valor. (e) Una sola base (mediana desde 2013, 2013-2026 incluyendo año parcial) para todos los años. (f) Anomalías marcadas ≠ impacto en una estrategia: existe `alarm.py` / `one.py` / `many.py` (barajado: B = 10.000 permutaciones de subconjuntos del mismo tamaño, sobre la suma del neto de las operaciones marcadas, unilateral, p < 0,01 sin corrección por multiplicidad, mínimo 10 marcadas; `feedQuality.alarm.*`), pero **no forma parte del preflight**.
- **Coste:** escaneo por feed, sin dato; el preflight sólo lee un JSON.

4.10 Cadena y resumen de la corrida del paso 4

28 fichas entran → 28 salen con código 0; 0 bloqueos (B1/B2/B3); avisos de hoy: A3 en 2 (XAUUSD, XAUUSD_SYN01), A4 en 15 (construcción anterior al año estable), A1/A2/provisional 0. Los pasos 5-26 heredan las cifras sin volver a comprobarlas: el preflight comprueba que las cifras **existen** y las ventanas **caben**, no que sean **ciertas** (un spread de 304 y uno de 747 pasan igual); sólo el paso 26 contrasta con una cuenta real.

4.11 Tiempos y recursos del paso 4

El comando es una lectura de YAML/JSON: sin dato de perf; en la práctica instantáneo (la caché `_PARSED` por mtime se creó porque `symbol_for()` parseaba cada ficha 1.971 veces: 54 s de los 60 de crossTF, 2026-09-25). `--dataranges` pregunta al conductor (`exportdrv.symbols()`), sólo ahí hay coste de SQX.

4.12 Discrepancias del paso 4 (⚠ resumen)

1. ⚠ El método del spread vigente no tiene comando en el repo (ver C4.4); `onboard` calcula otra cosa (Darwinex medido/modelado) y sólo está protegido por el marcador «sin factor» en el `why`.
2. ⚠ Cinco textos describen el método viejo (Darwinex ×1,25, slippage la mitad, 8 USD/lote): `.claude/skills/asset-onboard/SKILL.md` (modificado hoy sin commit, no releído), manual cap. 59 (`59-spread-real.md`), `assets/RULES.md:52-58` (clases con 1 spread en forex vs 3 reales) y `:238-240` (UKOIL 90 puntos), notas de `_classes.yaml` (slippage «la mitad»; fórmula del swap de forex con `tick_size` en vez de `TICKSTEP`, que daría 16,2 USD/noche en vez de 1,62 si alguien la aplicase) y la fila del paso 4 de `WORKFLOW.md` (dice «hoy XAUUSD y USDJPY», cuando el estudio cubre 23). Según el dossier; no reverificados aquí uno a uno.
3. ⚠ Dossier §5.7 (traza al faltar comisión) resuelto en el árbol de hoy.
4. ⚠ Dossier habla de 27 activos; hay 28 fichas (la 28ª, `XAUUSD_SYN01`, gemelo sintético del oro para el test de viabilidad MCPT).
5. ⚠ Etiqueta `source` del MC Retest falsa donde actúa `MIN_SPAN` (10 pares forex).
6. ⚠ Residuos en fichas: `broker: Infinox` en NIKKEI225 y `broker: the5ers` en USDJPY; nota de USDJPY del 2026-09-23 («defaults de fábrica … comisión CERO») sigue saliendo en cada lectura aunque ya no es verdad (no reverificado hoy); `XCUUSD` lleva `session: XAGUSD_ftmo` (nadie comprueba que la sesión sea la del activo); la sesión ni se imprime ni se comprueba en el preflight (la comprueba el builder, `doctrine.blockers`).

PASO 5 — Creación del custom project (`sqx/projects/builder.py`)

5.0 Propósito, entradas, salidas

- **Pregunta:** ¿está bien montado el proyecto donde vivirá todo el workflow? Un único `project.cfx` con todas las tareas, cada una con su ventana, sus costes, su entrada y su salida; sin pérdida de estrategias.
- **Comando:** `python3 -m sqx.projects.builder <Test_|Trade_|Research_ nombre> --purpose "…" (--template <.sqx> | --simple long|short) --symbol S --role custodian --timeframe TF --workflow [--max-strategies N --minutes M] [--random]`. En la práctica lo lanza `bin/workflow.sh simple|template` (regla 19), que antes valida el nombre (`^(Test|Trade|Research)_[A-Za-z0-9_]+$`), rechaza si el `project.cfx` ya existe (`[[ -e $CFX ]] && die`, línea ~58) y pasa `core.assets` (línea ~59); siempre añade `--workflow`; después aplica la paleta (paso 6) y, con `--until`, `workflow.trim`.
- **Entradas:** plantilla `.sqx` de la biblioteca (o Simple Strategy); ficha del activo; `_policy.yaml`; `_build.yaml` (doctrina); `_study.yaml` (filtros del Build); `stages.yaml`; **donante congelado** `AlgoData/projectsBackup/XAUUSD_base_2026-09-21/project.cfx` (`builder.py:29`; copia byte a byte del XAUUSD del master del 2026-09-21, MD5 `3f997754…` coincide con su `LEEME.md` según el dossier; **el código no comprueba el hash**, `--donor` lo sustituye y el registro no anota cuál se usó).
- **Salidas:** `<install>/user/projects/<nombre>/project.cfx`; copia de la plantilla en `<install>/user/settings/StrategyTemplates/authored/<carpeta>.sqx`; fila en `AlgoData/projects/registry.csv` (12 columnas: `name, kind, install, created, purpose, symbol, timeframe, template, workflow, retired, archive, acceptance`, bajo `flock`, `registry.py`); resumen por pantalla (`summary.say`) o JSON (`--json`).

5.1 Orden de aplicación en `main()` (`builder.py:175-246`) — criterios que CORTAN (`SystemExit`)

#	comprobación	dónde	regla que aplica	estado
G1	nombre `Test_/Trade_/Research_` + `[A-Za-z0-9_]`	`registry.check_name` · `builder.py:206`	regla 6 (la API HTTP de SQX parte por espacios)	igual que el dossier
G2	la plantilla existe	`:208`	—	igual
G3	**una sola dirección**	`directions()` · `:30-42`, `:212-217`	regla 13 (dueño 2026-10-01): lee `strategy_Portfolio.xml` de la plantilla y cuenta las reglas `Long entry` / `Short entry` con acción `Enter…`; ≠ 1 ⇒ corta. Sólo si el proyecto construye (`--workflow` o `Build` en `--tasks`; `mt5.verify` pasa estrategias terminadas con `--tasks Retest`)	igual
G4	**`early_gates`** (nuevo, sin commit; `sqx/projects/buildrules.py:164-176`)	`:218`	(a) el proyecto **no existe vivo**: carpeta en el install o fila viva del registro (`exists`, `:154`); (b) con `--workflow`, el timeframe ∈ `crosstf.timeframes` de `_build.yaml` (hoy **M15, M30, H1, H4**; antes sólo M30 y H1); (c) regla 15: ninguna condición **fija** de la plantilla lee el reloj o el calendario (`clock_fixed`, `kind(k)=="clock"`)	**nuevo**
G5	tope de condiciones	`buildrules.over_the_cap` · `:220`	regla 14: >1 hueco aleatorio libre en un lado (0..4 condiciones) ⇒ corta; fijas + hueco > `complexity.max_entry|exit_conditions` (2) ⇒ corta. Una plantilla de sólo huecos es una Simple Strategy	igual
G6	costes pactados	`assetcheck.pending` · `:222-226`	regla 5: C4.3	igual
G7	ningún install lo tiene abierto	`cfxio.held` · `:227-230`	regla 4: el puerto del install contesta o hay un proceso corriendo desde él	igual
G8	sesión y timeframe conocidos	`doctrine.blockers` · `doctrine.py:63-81`, `:231`	falta `session:` en la ficha, o timeframe ∉ `TF_MINUTES` {M5, M15, M30, H1, H2, H4, D1}	igual
G9	de dónde se toma la sesión	`source.pick` · `:234`, `source.py:30-51`	«ninguno» sólo si el donante define feed **y** sesión; si no, el `project.cfx` más reciente de los tres installs que defina ambos, si no el que defina el feed (y la sesión sale de la BD de SQX); sólo lectura; no inventa	igual

Dentro de `build()` (`builder.py:45-172`), antes de mover el zip al install:

#	rechazo	dónde	qué lo dispara
G10	falta la sesión	`doctrine.borrow_session` (`doctrine.py:84-124`)	ni un proyecto ni la BD de SQX (`sessiondb`) la definen: «no se inventan horarios»
G11	ninguna tarea define la sesión	`configure.py:136-142`	`unify_sessions` no la encuentra
G12	sin filtro para activo/timeframe/modo	`core/buildfilters.py:_need`	falta `trades_per_year.<TF>.min`, `profit_factor_min`, `net_profit_min` en `_study.yaml`; o `sample` ≠ `IS` (selección con OOS la gasta). Hoy `_study.yaml` define M15, M30, H1 (35/año) y H4 (20/año). **No hay filtro por omisión**
G13	plantilla ignorada	`resources.refuse` (`:143-144`)	una tarea nombra plantilla pero declara `type="simple"` (SQX construiría genérico en silencio; OPEN #9: 0 de 642)
G14	tarea sin costear	`refuse` (`:145-149`)	una tarea sin ningún `<Setup>` sobre el feed del activo (operaría el mercado del donante con sus costes; OPEN #34)
G15	feed del donante vivo	`refuse` (`:150-156`)	tras la sustitución de feed, `XAUUSD_M1` aparece todavía en algún XML (OPEN #35)
G16	**MonteCarloManipulation encendido** (nuevo, sin commit)	`refuse` → `crosschecks.mc_manipulation_on`	regla del dueño 2026-10-11: ese cross-check no se usa nunca; `_build.yaml: crosschecks.build` pasa de `[MonteCarloManipulation]` a `[]`. `python3 -m sqx.projects.rankings <cfx> --mc-off` lo apaga en un `.cfx` existente (rehúsa si un install lo tiene abierto)

Todo `refuse` ocurre sobre un zip **staged en `tmp_dir()` fuera del install**; no queda un `.cfx` a medias en `user/projects` (OPEN #34).

5.2 Qué se clona y construye (lo que hace, pieza a pieza)

1. Lee el donante (17 tareas: 1 `Build` `CONSTRUCCION`, 14 `Retest` — `OOS`, `Retest Markets - Family`, `MC Trades`, `MCR 1…8`, `SPP IS`, `SPP OOS`, `WFM` —, 1 `ClearDatabanks`, 1 `GoToTask`). **No conserva ni `GoToTask` (bucle de generación perpetua) ni `ClearDatabanks`** (vacía databanks mientras se autosincronizan: mecanismo de la regla 1): `keep(members, {"Build","Retest"})`, `builder.py:99-102`.
2. Traduce nombres de feed obsoletos (donante anterior al cambio de feeds 2026-10-01): `core.symbols.current`.
3. `wf.kept_members` (`workflow.py:37-48`): se queda con las tareas que algún paso corre (`stages.yaml` + `wfc`) → 9: `CONSTRUCCION`, `OOS`, `Retest Markets - Family`, `MCR 1 Bar`, `MCR 4 MinDist`, `MCR 7 OHLC`, `SPP IS`, `SPP OOS`, `WFM`. Se van `MC Trades`, `MCR 2 Spread`, `3 Slippage`, `5 Params`, `6 Exits`, `8 Stress` y las dos de control (decisión del dueño 2026-10-07: el paso 13 corre sólo el control T3 y la historia T2; T1 va por la fábrica de variantes). Selección **por título**: un donante con un título cambiado pierde una tarea sin error.
4. `wf.complete` añade 4: `CrossTF` (copia de `OOS`) y `WFC 1 IS`, `WFC 2 OOS1`, `WFC 3 OOS2` (títulos en `_build.yaml: wfc.tasks`).
5. `borrow_session` / `borrow_symbol`; ficha de registro SQX (`refresh_all`); `sync_databanks` (de «Auto-sync never» a «every 1 hour»: el donante construye pero deja la carpeta vacía).
6. Por tarea: `set_template` / `set_simple` (fuerza `type="template"`; con `simple` SQX construiría genérico), `set_generation`, `set_caps`, `crosschecks.silence` en los `Retest`.
7. `chain_databanks` (cada tarea lee la salida de la anterior en orden de `config.xml`, la primera conserva su entrada) y, con `--workflow`, `wf.rewire`: `CrossTF` lee los supervivientes cargados (`CrossTF_Input`); las 3 MCR leen las mismas madres (`CrossTF_Mothers`, no encadenadas entre sí); las 3 patas del WFC leen el mismo lote (`WFC_Variants`). `SPP IS` queda leyendo la tarea anterior en orden (`MCR 7 OHLC`), por eso `stages.yaml` deja `MCR 7 OHLC` la última.
8. Sobre el zip staged: `configure` (costes y ventana por tarea), `rankings.apply` (filtros del Build), `wf.finish` (seis tareas finales), y sólo entonces `refuse` y `shutil.move` al install. `wf.finish` llama `stage.apply(cfx, ["build","oos"])`: sólo `CONSTRUCCION` y `OOS` quedan encendidas. - **Resultado:** 19 tareas y 30 databanks declarados (21 del donante − 4 que `keep` descarta + 13 de `wfc.declare()`; tres databanks quedan declarados sin tarea que los escriba). - ⚠ **Todavía válido en el código de hoy:** `install_template` (`builder.py:87-91`) escribe en el install (trunca y reescribe `authored/<carpeta>.sqx`, compartido por todos los proyectos de esa plantilla en ese install) **antes** de las puertas `refuse` / `configure` /filtros; el comentario de `:134-138` («Nothing under `install` is touched until every gate…») es falso para la plantilla. `early_gates` reduce el caso de rechazo temprano pero no el de los rechazos de `build()` (G10-G16).

Qué tramo y qué precio deja el builder en cada tarea

`configure.BY_TASK = {"Build": "build"}`, el resto `oos1` (`configure.py:25-26`): `Build` en `build` (USDJPY 2008-01-01..2017-12-31, spread 0,51, slippage 0,18); **toda tarea `Retest` en `oos1`** (2018-01-01..2022-12-31, 0,58 · 0,20) aunque luego la reescriba el configurador de su paso (13 → `build`, 16.5 → `build` / `oos1` / `oos2`, 15 → `build`, 19 → `build..oos2` con costes de `oos1`, 9 → `build..oos1`). Las 3 tareas `CrossTF <TF> build` nacen en `build`. En **todos** los tramos: comisión de la ficha por tramo, swap de la ficha, triple swap miércoles (viernes en índices), cierre 23:00. Sólo reescribe los `<Setup>` que cotizan el activo del proyecto (`setups.set_costs`); uno que cotice otro mercado se deja como está (ver ⚠ abajo). ⚠ El blanco se mueve: ninguna puerta comprueba que la ventana escrita en cada tarea sea la de su tramo (`result["segments"]` se calcula en `builder.py:164` y nadie lo lee), y `registryxml.clip` recorta **sin avisar** el rango de datos de cada símbolo a las barras del registro de SQX mientras los `<Setup>` conservan las fechas.

Las 19 tareas (lectura del dossier sobre la copia viva y B5; no reverificada en esta sesión)

`CONSTRUCCION` (6) → `OOS` (7) → `Retest Markets - Family` (9) → `MCR 1 Bar`, `MCR 4 MinDist`, `MCR 7 OHLC` (13) → `SPP IS`, `SPP OOS` (15) → `WFM` (19); `CrossTF` (11), `WFC 1 IS` / `2 OOS1` / `3 OOS2` (16.5), `CrossTF M15|H1|H4` (11; para madres M30; hoy `crosstf.timeframes`: M15→[M10,M30,H1], M30→[M15,H1,H4], H1→[M30,H4], H4→[M30,H1,H12]), `CrossTF M15|H1|H4 build` (11.7). Sólo `CONSTRUCCION` y `OOS` encendidas al crear.

5.3 Criterio: filtros de aceptación del `Build` (lo único estadístico del paso)

- **Pregunta:** ¿qué estrategia entra en el databank mientras se genera?
- **Definición** (`core/buildfilters.py`, `sqx/projects/rankings.py`, `assets/_study.yaml`): `rankings.apply` **sustituye ENTERAS** las condiciones de «Ranking» del donante (`NumberOfTrades > 500`, `SortinoRatio > 0.6`, `Stagnation < 540`, que nadie había decidido) por: `NumberOfTrades >= ceil(trades_per_year[TF].min × años_build)`, opcionalmente `NumberOfTrades <= floor(max × años)` (hoy **sin techo**), `ProfitFactor >= profit_factor_min (1,0)`, `NetProfit > net_profit_min (0)`; todas **sobre IS** (`sample: IS` → `sampleType` 10 de SQX, el único valor admitido). `años_build = round((dateTo − dateFrom)/365,25 d, 2)` (`buildfilters.py:64`; USDJPY 10,0014 se redondea a 10,0 para que el suelo no salga 101). Valores vigentes `_study.yaml` (dueño 2026-10-09): **35 operaciones/año en M15, M30 y H1; 20/año en H4; sin techo; PF ≥ 1,0; NetProfit > 0**. Ejemplo: XAUUSD H1, 10 años ⇒ `NumberOfTrades >= 350`. Modo `calibration` (`--acceptance calibration`): sólo `NumberOfTrades >= 100` y `NetProfit > 0`.

- **Corta:** sí, pero **no mata nada que SQX no haya generado ya**: es un filtro de entrada al databank (SQX borra del databank lo que no cumple y no lo cuenta como superviviente de nada). Todo `Retest` se silencia (ver 5.4).
- **Por qué (declarado):** filtro mínimo a propósito: medido 2026-10-03 (mismo activo/familia/lado/caja de build, filtro dentro vs control sin filtrar) ningún filtro de IS probado da más estrategias robustas por minuto; todos dan entre un 70 y un 98 % menos (robustas/min filtro/control: suelo 35 ops/año 5,4/32,4; PF≥1,2 y Ret/DD≥2 7,1/33,6; % aciertos ≥55 10,6/32,4; PF≥1,3 y Ret/DD≥3 4,8/35,8; PF≥1,5 y Ret/DD≥5 0,6/35,8; PF≥1,8 y Ret/DD≥8 no llega a 300 estrategias en 4 min). El suelo 35/año se subió de 10 a 35 (2026-10-09) para coincidir con el R4a del paso 8, porque el paso 8 mandaba a Limbo entre el 31 % y el 99 % de un databank de 10.000 (6 runs, 2026-10-08); coste medido: 85 de las 179 que pasaron R3 tenían menos de 35/año, y plantillas de pocas señales en H1 casi no construyen (105 y 15 de 10.000). Techo 75/año retirado: por encima las poblaciones daban 0,41 veces lo que da el azar (2026-10-03).
- **Puntos débiles:** (a) todo el fichero está marcado «PROPUESTA DE AGENTE, 2026-10-04 — PROVISIONAL hasta que el dueño la apruebe» (aunque el dueño fijó los valores 35/20 y quitó el techo el 2026-10-09; el encabezado no se actualizó, ⚠ no hay constancia explícita de aprobación del resto); (b) las cifras de «robustas por minuto» son comparaciones de un puñado de builds (203 poblaciones + 7 rondas según el encabezado), sin intervalos publicados salvo uno (pureza ×1,54, IC 0,86-2,43) y «NO probado en builds largos con el databank lleno»; (c) filtrar en el generador sesga la búsqueda (más trials hasta llenar el databank) y SQX no dice cuántas rechazó; (d) los años nominales del tramo, no los cargados: un feed que empezara tarde daría un Build de menos años con el suelo de diez; (e) los umbrales de operaciones se aplican al total IS, no por régimen.
- **Corrida real:** B5 `Test_USDJPY_M30_doubleROCLong_10k_B5` (2026-10-08 01:24, archivado 03:09, tar.gz en `AlgoData/projects/retired/SQX_w2/`): topes 10.000 estrategias / 90 min, `NumberOfTrades >= 100 & <= 750 & PF >= 1,0 & NetProfit > 0` (valores anteriores al cambio de 2026-10-09; ⚠ el dossier §3.5 aún lo describe como vigente). Más reciente en el registro: `Test_XAUUSD_H1_ruptura_mcptS` y `…_mcptR` (2026-10-11 21:05, SQX_w2, `simple:long`, XAUUSD_SYN01/XAUUSD, H1): `acceptance = study @IS: NumberOfTrades >= 350 & ProfitFactor >= 1.0 & NetProfit > 0`. Entran/salen estrategias: no es una medida de este paso (sin corrida real contada).

5.4 Otros criterios de configuración (reglas, no estadística)

- **Silenciar (`crosschecks.silence`, `builder.py:99-101`)**: con `--workflow`, las 18 tareas de tipo `Retest` pierden todas sus condiciones de aceptación y `DeleteFailedStrategies` pasa a falso (el `Build` es la 19.ª y conserva las suyas). Por qué: una tarea que borra lo que falla convierte la medida en selección y sólo se podría decir cuántas sobrevivieron; el veredicto lo da Python (dueño 2026-09-24/25). Comprobado en la copia viva (18 tareas, 0 condiciones activas) según el dossier.
- **Doctrina `_build.yaml` aplicada a todas las tareas (`doctrine.apply_doctrine`)**: mismo timeframe en todos los gráficos, `precision` 1 (3 en el cross-check de alta precisión de la construcción, `crosschecks.precision`), motor `MetaTrader5 (hedged)`, sesión del activo, sizing por riesgo ATR (`Amount 1000`, `ATRMult 4`), salida el viernes a las 21:00; sólo el `Build` lleva generador: órdenes sólo `EnterAtMarket`, salida por barras obligatoria (rango en horas→barras, p. ej. 2-24 h en M30 = 4..48 barras), salida por condición optativa, sin SL/PT/trailing/break-even; `genetic-evolution` (o `random-generation` con `--random`); «What to build» por hueco (regla 14: p. ej. doubleROCLong entrada 0..1 y salida 0..2). Cross-checks: en el Build `[]` hoy (⚠ el dossier decía `MonteCarloManipulation` encendido: cambiado 2026-10-11); `RetestOnAdditionalMarkets` apagado; en el resto ninguno (`default: []`).
- **Topes de generación** (`doctrine.caps`, `_build.yaml:81-84`; por prefijo): `Trade_` y `Research_` 10.000 estrategias y 180 min; `Test_` 300 y 10 min; los argumentos `--max-strategies/--minutes` mandan. Por qué esos números: no consta justificación escrita (no verificado; la regla 19 de CLAUDE.md cita 160 min para 1.000 estrategias y 10 madres, 230 a mano, que es otro número).
- **Encendido por paso (`stage.only/apply/just/own`, `stage.py:44-133`; `stages.yaml`)**: `action=start` corre todas las tareas con `active="true"`; cada configurador deja encendida sólo su tarea. `stage.only` reescribe `active=` por regex según el título; **una tarea sin título (un `ClearDatabanks` / `GoToTask` ajeno) se deja siempre apagada**. `apply/just` rechazan con el install abierto y **fallan** si falta alguna tarea pedida. ⚠ `stage.own` exige que estén los títulos de **todos** los pasos: en un proyecto recortado con `trim` devuelve un aviso y **no cambia nada**, de modo que `CONSTRUCCION` y `OOS` siguen encendidas y un `action=start` a mano relanzaría el Build (lo evita el `stage.apply` de los lanzadores, `ui/daemon/advance/run.py:206`, `ui/daemon/launch/run.py:163`; según Taleb en el dossier, no reverificado).
- **Recorte `--until`** (`workflow.trim`, `workflow.py:115-137`; `bin/workflow.sh:62-64`): borra tareas y XML de los pasos posteriores (oos|crossmarket|crosstf|mcretest|spp|wfm); sus databanks quedan declarados y vacíos. ⚠ No quita las seis hermanas CrossTF ni las gemelas «build» (no figuran en `stages.yaml`); el registro no anota el recorte (consta `workflow=yes`); sólo `crosstf` tiene test (`tests/test_autopilot_crosstf_cascade.py:190-205`).

5.5 Registro, retirada y barrido

- **`registry.record`** (`builder.py:242`; `registry.py:87-106`): añade fila bajo candado, escribiendo a un fichero vecino que luego se mueve encima; **borra la fila viva anterior del mismo nombre e install** (pierde `purpose` y `created`; ahora poco probable porque `early_gates` ya rechaza un nombre vivo en el builder). El resultado completo del builder no se guarda en ningún sitio, sólo lo que va

a la fila. **Estado real hoy (2026-10-12): 187 filas** (165 `test`, 21 `legacy` anteriores a los prefijos, 1 `trade`; 0 `research`), 173 retiradas, 14 vivas; `workflow=yes` en 79, `no` en 86, vacío en 22 (anteriores a la columna). (El dossier del 2026-10-08 contaba 161/150/11.)

- **`retire`** (`sqx/projects/retire.py:55-104`, `python3 -m sqx.projects.retire <P> --role <rol> --yes`; sin `--yes` es ensayo): se niega si es proyecto de serie (`Builder`, `Retester`, `Optimizer`, `PortfolioComposer`, `PortfolioMaster`), sin `project.cfx`, install abierto, zip ilegible, modificado hace < 60 min (`--own` lo salta), `--keep` pide un databank inexistente, o (cambio sin commit) hay supervivientes del paso 8 que el almacén no tiene enteros. Archiva **sólo el `project.cfx`** (más databanks de `--keep`) en `AlgoData/projects/retired/<install>/<nombre>-<fecha>.tar.gz`, borra la carpeta del install y marca la fila.
- **`sweep`** (`sweep.py:35-81`, lunes 02:50 por cron `50 2 * * 1`, agente `projectJanitor`), orden de decisión por proyecto: install encendido → se queda; zip ilegible → se queda; tocado hace < 24 h → se queda; en `retire-queue.txt` → se retira; `Trade_` → se queda; `Research_` → se queda; `Test_` → se retira; < 10 tareas → se retira; si no, se queda («workflow entero»). Del master sólo se mira la cola. ⚠ **No distingue un `Test_` terminado de uno parado entre pasos**: lo retira a las 24 h de inactividad con el install apagado y `sweep_all` llama a `retire(…, [], True)` sin `--keep`: se borran los databanks de todos los pasos; la guarda de supervivientes mira sólo las del paso 8, así que lo que producen CrossTF, MC Retest, SPP, WFC y WFM no lo guarda nada. Hoy el registro tiene 14 proyectos vivos (no verificado cuántos superan las 24 h de inactividad).

5.6 Cadena de ejemplo y recursos del paso 5

- **Cadena (B5, USDJPY M30, `doubleROCLong`, 2026-10-08):** 17 tareas del donante → 9 conservadas → +4 añadidas (CrossTF, 3 WFC) → +6 al final (3 `CrossTF <TF>` + 3 `build`) = 19 tareas, 30 databanks; recortado con `--until crosstf` a 10 tareas (`CONSTRUCCION`, `OOS`, `Retest Markets - Family`, `CrossTF` y las seis hermanas). Con 18 `Retest` silenciados. Filtro de Build entonces 100-750 operaciones / PF ≥ 1,0 / NetProfit > 0.
- **Tiempo/recursos del paso 5:** sin dato propio (no hay timing del builder en perf/, AlgoData/logs ni registro; la fila de B5 dice `created 01:24` pero el builder no registra su duración). Es una operación de ficheros (zip + XML) fuera del install: sin CPU significativa ni SQX; el coste de ese proyecto vive en el paso 6 (90 min de tope en B5).

5.7 Discrepancias del paso 5 (⚠ resumen, doc↔código de hoy)

1. ⚠ **Resueltas en el árbol de hoy, vigentes en el dossier §5/§6:** (a) nombre repetido → `early_gates` lo rechaza; (b) M15/H4 con `--workflow` → `crosstf.timeframes` ya incluye M15, M30, H1 y H4 y `early_gates` da un mensaje claro (el `KeyError 'H4'` ya no debería ocurrir; no ejecutado); (c) bloque de reloj en una plantilla fija → rechazado (regla 15, `clock_fixed`); (d) `MonteCarloManipulation` ya no se enciende en el Build y `refuse` lo prohíbe; (e) el filtro del Build ya lleva el suelo 35/20 (corregido el 2026-10-09; el ejemplo B5 del dossier §3.5 aún muestra 100-750).
2. ⚠ **Vigente:** la plantilla se copia al install antes de pasar `refuse` (puede pisar la que usa otro proyecto vivo); no hay hash del donante ni columna que lo anote; el registro guarda `simple:long` en la columna de plantilla y `pipeline/autopilot/resume.py:85` se lo pasa a `--template` (un proyecto simple no se puede reanudar; el builder rechaza con «simple:long does not exist»); `registryxml.clip` recorta sin avisar; ninguna puerta compara la ventana de cada tarea con su tramo; `Retest Markets - Family` y los `<Setup>` ajenos (p. ej. `AUDJPY_TICK` o `XAGUSD`/`UKOIL.cash` con spread 0, slippage 0 y comisión 0) heredados del donante quedan a coste cero hasta que un paso los reescribe (`set_costs` los deja; no comprobado que SQX no cargue sus datos al correr el Build); `oos2` no lo escribe el builder (su `--segment` sólo admite `build|oos1`), pero la protección de oos2 del resto del código sólo muerde con `ALGO_AUTONOMOUS=1`.
3. ⚠ `sweep.py:10` dice «15 a 18 tareas» (hoy 19; el umbral 10 vale); `retire-queue.txt` dice «lunes 03:00» y el cron es 02:50.
4. ⚠ Dos reglas de coste para ventana larga conviven: el WFM la cobra al precio de `oos1` (`wfm.py:120-134`) y la docstring de `setups.span` dice «último tramo».
5. ⚠ `_study.yaml` conserva el encabezado «PROPUESTA DE AGENTE … PROVISIONAL» aunque los valores vigentes son del dueño (2026-10-09).

5.8 Cierre de la sección

Pasos 4 y 5 son mecánica con tres puntos estadísticos reales: (1) la **cifra de spread/slippage/swap por tramo** (extrapolación del % de 2025-26 a toda la historia; sin comando que la reproduzca; swap de un día en toda la historia); (2) el **rango del MC Retest** (0,5×-2× salvo `multiples` propios; ancho mínimo 1 punto que lo distorsiona en 10 pares forex; el techo en índices queda por debajo de lo medido en Darwinex) y (3) los **filtros de aceptación del Build** (35/20 operaciones por año, PF ≥ 1, NetProfit > 0 sobre IS, sin techo). El resto son negativas de código que protegen contra pérdida silenciosa de datos o de coste. Todo lo que los pasos 6-26 digan de una estrategia está dicho a los costes de estas fichas.

Pasos 6 y 7 — Build y Retest OOS en SQX (estado implementado a

2026-10-12)

Fuentes: código y `assets/` del árbol de trabajo de hoy; corridas reales en `AlgoData/` (autopilot, metrics, ledger); dossiers `criterios-paso-06-build-2026-10-08.md` y `criterios-paso-07-retest-oos-2026-10-08.md` solo para el porqué y las cifras que cito de ellos (marcadas «dossier 10-08»). Manda el código. Nada de lo que sigue lo he ejecutado: solo lectura.

0. Hechos transversales (valen para los dos pasos)

- **Ninguno de los dos pasos juzga con estadística.** `pipeline/autopilot/criteria.yaml` no tiene bloque «6» ni «7» (verificado: solo hay «8» y siguientes; R1a en `criteria.yaml:73-74`); `ledger/thresholds.yaml` no tiene umbrales suyos. Lo único que corta algo en el paso 6 son los filtros de aceptación escritos en la tarea Build de SQX (apartado 6.C1-C4). El paso 7 no corta nada por diseño.
- **Cambios de código/doctrina posteriores a los dossiers del 10-08 (⚠ los dossiers ya no describen lo cargado):** 1. `assets/_study.yaml` (sin confirmar, modificado): suelo de operaciones **35/año en M15, M30 y H1 y 20/año en H4**, **sin techo** (dueño, 2026-10-09). Los dossiers dicen 10 al año y techo 75/100. Los `symbols:` y `classes:` están vacíos: se quitaron los suelos propios de índices/crudo (13-16/año). 2. `assets/_build.yaml` (sin confirmar): `crosschecks.build: []`. `MonteCarloManipulation` **no se enciende nunca** (dueño, 2026-10-11); `tasksettings.set_crosschecks` (`sqx/projects/tasksettings.py:149-150`) lanza `SystemExit` si se lo piden; `resources.refuse` también. El dossier del paso 6 describía el MC encendido con 30 simulaciones: ya no. 3. `sqx/projects/setups.py` (`OOS_RANGE`, `:139`, `:175`) **ya está confirmado en git** (git status limpio para ese fichero); el dossier del paso 7 lo daba como sin confirmar. 4. `assets/_policy.yaml`: añadido el gemelo sintético `XAUUSD_SYN01` (ventanas idénticas a XAUUSD) — irrelevante para el criterio.
- **Corridas reales recientes (post-cambio 35/año, 2026-10-10/11, todas `Test_`, custodio `SQX_w2`, Simple Strategy long, build de 10.000 + retest de 10.000)**: `Test_DAX40_H1_reversion_wf2`, `Test_NIKKEI225_M30_volatilidad_wf3`, `Test_XAUUSD_H1_ruptura_wf4` (y su repetición `…_wf4s`), `Test_USATEC_H1_ruptura_leaseWeb`. Resúmenes en `AlgoData/autopilot/<P>/<ts>/resumen.md`, tiempos en `tiempos.md` / `fases.csv`, métricas en `AlgoData/metrics/<P>/{Results,OOS}/metrics.csv` (separador `;`). Las cifras del dossier 10-08 (USDJPY M30 `doubleROCLong`, B5) son de antes del cambio de filtros y de plantilla; las cito aparte.
- ⚠ **Las corridas Simple Strategy no dejan fila de paso 6 en el ledger.** `ledger/steprows.py:14-27` (`signed`) devuelve `None` si el registro no trae plantilla (`row["template"]`) y `log_build` imprime «Ledger: sin plantilla en registry.csv…» sin escribir. Verificado: `AlgoData/ledger/NIKKEI225_M30_simple_long.jsonl` (700 filas), `XAUUSD_H1_simple_long.jsonl` (58) y `USATEC_H1_simple_long.jsonl` (49) tienen pasos 8, 10, 12, 14, 17, 18, 18.5, 19, 20, 24 y **ningún 6 ni 7**. Consecuencia: para las corridas actuales **no existe en el ledger el número de estrategias generadas (N de ensayos del build)**; solo existe en el log de SQX (que se retira con el proyecto). Las únicas filas «step 6» que hay son de las plantillas (p. ej. `USA500_H1_indicesIBS.jsonl`: 1.442.623 generadas / 84.305 aceptadas el 2026-10-05; y 4 builds más de 346k-362k generadas, 74k-81k aceptadas). No he podido verificar si `ledger/trials.py` suma algo para las Simple.

PASO 6 — Configuración del build (y la tarea Build corriendo)

6.1 Propósito

Fijar qué forma pueden tener las estrategias que SQX genera, con qué piezas, cuánto tiempo busca y cuáles de las generadas se guardan. El paso no emite veredicto: escribe una tarea de SQX (`Build`) y la deja correr.

6.2 Entradas

- Ficha del activo (`assets/symbols/<S>.yaml`: sesión, costes `spread_is`, `slippage_is`, comisión, swap), `assets/_build.yaml` (doctrina), `assets/_study.yaml` (filtros de aceptación), `assets/_policy.yaml` (ventana `build` del activo), la paleta de bloques (`sqx/blocks/palettes/*`, taxonomía `sqx/blocks/taxonomy.yaml`), el proyecto donante congelado `XAUUSD_base_2026-09-21` (hereda el genético `buildmode_model.xml`, la métrica de ranking, el descarte de similares y el rango de periodos).
- Argumentos: `--max-strategies N`, `--minutes M` (`bin/workflow.sh:59-61`; si no se dan, `doctrine.caps`, `sqx/projects/doctrine.py:155-163`, por prefijo: `Trade_` / `Research_` 10.000 y 180 min, `Test_` 300 y 10 min — `_build.yaml:caps`).

6.3 Salidas

Databank `Results` en SQX (las N guardadas, con 38 columnas de métricas al exportar), `metrics.csv` + `trades.parquet` por `export` al parar, fila de `registry.csv` con los filtros en texto (ej. `study @IS: NumberOfTrades >= 350 & ProfitFactor >= 1.0 & NetProfit > 0`), línea de log de SQX `Strategies generated: …, Accepted: …, Rejected: …` y (solo con plantilla) fila `step 6` del ledger.

6.4 Orden de aplicación

1. `core.assets <S>` (regla 5; sale ≠0 → no se crea nada). 2. `sqx.projects.builder … --workflow` clona el donante y escribe: doctrina (`doctrine.apply_doctrine`, todas las tareas), forma de la estrategia (`buildrules.py`, solo la tarea que genera), filtros (`rankings.apply` → `rankings.set_rankings`, `sqx/projects/rankings.py:19`), caps. 3. `sqx.projects.buildingblocks --palette` (`bin/workflow.sh:67`). 4. `pipeline.autopilot.run` arranca SQX; el Build corre hasta su tope; SQX encadena el Retest OOS en la misma sesión («6+7»). Dentro de SQX, por estrategia generada: backtest sobre `build` → ¿cumple los filtros? (C1-C4) → descarte de «demasiado parecidas» (heredado) → entrada al databank; si el databank está lleno entra solo si mejora a la peor en la métrica de ranking (C5).

6.5 Criterios

Todos los filtros C1-C4 se miden **sobre el tramo `build` (IS, `sampleType=10`)**, nunca OOS (`buildfilters.py:14` `SAMPLES={"IS":"is"}`, `:61` rechaza cualquier otro valor). Todos **CORTAN** (la estrategia que no los cumple no entra al databank). Ninguno marca ni solo informa.

C1 — Suelo de operaciones por año (el único de los filtros que muerde de verdad)

- **Pregunta:** ¿tiene el build operaciones suficientes para que su t/PF sea algo más que ruido de unas pocas?
- **Definición:** `NumberOfTrades >= ceil(min_por_año × años_build)` (`core/buildfilters.py:36-78`). `años_build = round((fin − inicio)/365,25 días, 2)` sobre la ventana `build` del activo (`_policy.yaml`). Umbral vigente (`assets/_study.yaml`, `defaults.trades_per_year`): **M15 35, M30 35, H1 35, H4 20**. Un timeframe ausente del fichero = no se construye (`_need`, `:18-23`). Sin nulos: SQX cuenta operaciones del backtest.
- **Cifras por activo (registro de proyectos, verificadas contra la fórmula):** XAUUSD H1 350 (10,0 años), NIKKEI225 M30 290 (35 × 8,28 = 289,8), DAX40 H1 219 (35 × 6,25 = 218,75), USATEC/USA500 H1 279 (35 × 7,95 = 278,25). El mismo suelo exige R4a del paso 8 (`criteria.yaml:79-80`: pasa ≥35, limbo ≥20 operaciones/año del build).
- **Por qué:** el paso 8 mandaba a Limbo todo lo que quedaba por debajo de 35/año (entre el 31 % y el 99 % de un databank de 10.000 según seis runs de 2026-10-08, comentario de `_study.yaml`); coste medido «85 de las 179 que pasaron R3 tenían menos de 35/año; las plantillas de pocas señales en H1 casi no construyen (105 y 15 de 10.000)». Es decisión del dueño del 2026-10-09 (no hay calibración vigente detrás).
- **Punto débil:** (a) umbral por **año**, no por significancia; con t = media/error, 350 operaciones en 10 años y 220 en 6 años no equivalen en potencia, y el suelo en operaciones/año con ventanas de longitud distinta da suelos totales distintos (219 vs 350). (b) Selecciona sobre la propia muestra del build (la muestra inflada ya es la que mira). (c) Sin techo: el dossier del 10-03 medía que por encima de 75/año las poblaciones daban 0,41 veces lo que da el azar (calibración borrada; sin medida vigente). (d) Redondeo: ceil en el suelo.
- **Ejemplo real (post-cambio):** `Test_XAUUSD_H1_ruptura_wf4` (XAUUSD H1 long, Simple, 2026-10-11): 10.000 guardadas, operaciones IS mín 350 (el suelo, mordió exacto), p25 409, mediana 495, p75 629, máx 1.417 (mediana ≈ 49,5/año). NIKKEI225 M30 `…_wf3`: mín 290, mediana 437, máx 1.809. DAX40 H1 `…_wf2`: mín 219, mediana 320, máx 2.095. USATEC H1 `…_leaseWeb`: mín 279, mediana 526, máx 2.678. (Calculado por mí sobre `metrics.csv`.) Entradas/salidas del filtro por separado: **sin dato** (SQX solo escribe el total de rechazadas).
- **Coste:** no mide tiempo propio; el filtro se evalúa en cada backtest. Sin dato de cuánto quita.

C2 — Techo de operaciones por año

- **Estado: DESACTIVADO.** `buildfilters.py:68-69` solo añade `NumberOfTrades <= floor(max × años)` si `trades_per_year.<TF>.max` existe; hoy ningún timeframe lo trae (dueño 2026-10-09). El código sigue soportándolo.
- ⚠ Dossiers 10-02/10-08 hablan de techo 75 (H1, M30) y 100 (H4) (y 150/200 en el primero): ya no vale. Verificado: las corridas post-cambio tienen hasta 2.678 operaciones IS en el databank.

C3 — Profit factor mínimo

- **Definición:** `ProfitFactor >= 1.0` (`_study.yaml:profit_factor_min`; `buildfilters.py:71`). PF = suma ganancias / suma pérdidas, en IS.
- **Por qué / punto débil:** es «solo no perder»; es prácticamente redundante con C4 (`NetProfit > 0` ⇔ PF > 1; solo difieren en PF = 1,0 exacto con neto 0). ⚠ El dossier 10-04 §4.1 decía «profit factor: ninguno» pero el fichero lleva 1,0.
- **Ejemplo real:** el PF mínimo del databank es 1,11 (XAUUSD wf4), 1,03 (NIKKEI wf3), 1,12 (USATEC), 1,00 (DAX wf2: «0.9999» de las PF>1; un PF de 1,0 se coló con neto ≥0 apenas); es decir, **no muerde por sí mismo**. En USDJPY B5 (dossier): PF mín 1,14.

- **Coste / entradas-salidas:** sin dato.

C4 — Beneficio neto positivo

- **Definición:** `NetProfit > 0` (estricto; `_study.yaml:net_profit_min: 0`). En IS.
- **Ejemplo real:** neto IS mínimo: XAUUSD wf4 9.046; NIKKEI wf3 3.702; USATEC 13.525; DAX wf2 442 (USD, riesgo fijo 1.000 por operación, capital 100.000). Todas las 10.000 de cada databank tienen neto > 0.
- **Punto débil:** C3 y C4 no añaden información entre sí; ninguno corrige multiplicidad.

C5 — Corte efectivo: databank lleno (métrica de ranking)

- **Definición:** el databank guarda como máximo `MaxStrategies` (10.000 por defecto de `Trade_` / `Research_`; en las corridas de test se pasó `--max-strategies 10000`). Lleno, una estrategia aceptada solo entra si **mejora a la peor en la métrica** `<Ranking type="ReturnDDRatio">` = beneficio neto / drawdown máximo en dinero, sobre `build`. **Nadie escribe esa métrica**: la hereda del donante (`buildcensus.py:101` solo la lee; ficha `knowhow/research/ga-target-choice.md`: elección «sin resolver», repetir la misma métrica varió 16 puntos y cambiar de métrica −1,7 a +6,5; ρ Ret/DD–Sharpe 0,98 en XAUUSD).
- **Por qué:** decisión del dueño 2026-10-01: parar por tiempo (`stop_condition: time-limit`) y que el databank guarde las N mejores. Es la selección que de hecho decide la población que llega al paso 7/8.
- **Punto débil:** es un top-N por métrica del propio `build` tras 10^5-10^6 ensayos (selección máxima sobre la muestra que luego se juzga; el paso 8 corrige juzgando en `oos1`). Métrica de relación (retorno/DD) con DD de un solo camino: muy ruidosa. El corte real depende de cuánto se llenó: en XAUUSD wf4 el Ret/DD mínimo del databank es 2,85 (mediana 4,59, máx 18,1); en USATEC 4,87 (mediana 6,40); **en NIKKEI wf3 0,50 (mediana 1,80) y DAX wf2 0,07 (mediana 1,44)**, es decir, en estos dos el reemplazo por Ret/DD casi no subió el listón (el databank se llenó tarde o el tiempo no alcanzó): **no verificado cuál**.
- **Ejemplo real (dossier 10-08, USDJPY M30 doubleROCLong B5, filtros antiguos 10-75/año):** 832.299 generadas → 47.976 aceptadas (5,76 %) → 10.000 guardadas (corte efectivo Ret/DD ≥ 2,90) → 72 vivas tras el paso 8. Cinco lotes de la misma celda: aceptación 5,8-6,6 % (749.129-832.299 generadas por lote).
- **Coste:** el build llena el databank hacia el minuto ~19-20 a ritmo constante en esa corrida; el resto del tiempo es reemplazo por Ret/DD (dossier).

C6 — Forma de la estrategia (doctrina `_build.yaml`)

No es un filtro estadístico sino lo que SQX puede generar. Cargado (verificado en `assets/_build.yaml`): una dirección (regla 13); órdenes solo a mercado (`order_types: [EnterAtMarket]`); salida obligatoria por barras entre **2 y 24 horas** (convertidas a barras por timeframe; en M30 = 4..48 barras; `buildrules.bar_range`); salida por condición optativa (`by_condition: true`, `condition_required: false`); sin SL, PT, trailing ni break-even; ≤ 2 condiciones por lado (regla 14: Simple 1..2 entrada y 1..2 salida; con plantilla, tope − fijas por hueco); `lookback_bars: 1` (solo la última barra cerrada); gestión de dinero ATRRiskBasedSizingFixedRisk, riesgo 1.000, ATR(20)×4, capital 100.000, DD máx 30 %; cierre viernes 21:00; motor MetaTrader5 (hedged); precisión del build **1** (timeframe elegido, `precision.build: 1`, `default: 1`). - **Punto débil:** (a) ⚠ **con plantilla sin hueco de salida nunca hay condición de salida**: en la corrida USDJPY (dossier) 0 de 3,5 M operaciones cerraron por regla (87 % por barras, 13 % por el viernes); con Simple Strategy sí existe. (b) multiplicador ATR «debatible entre 3, 4 y 5» (comentario). (c) Comentarios de `_build.yaml` contradicen valores (precisión «todo a 1 minuto» frente a `default: 1`; línea ~249 `precision: 1 # un minuto`) — no afectan al cálculo. - **Evidencia de la corrida:** medida en las 4 corridas post-cambio por su registro; recuento de tipos de cierre solo verificado en el dossier 10-08.

C7 — Búsqueda genética (heredada, `sqx/projects/buildmode_model.xml`, copia literal)

20 islas × población 250, 25 generaciones, cruce 89 %, mutación 20 %, migración cada 10 gen. al 3 %, sangre nueva 5 % cada 10 gen., reinicio al terminar y por estancamiento (5 generaciones, `fitnessType=11` «In sample (Training)», reparto Training/Validation 50 %), `DecimationCoef 1`. Solo se cambia a aleatoria con `builder --random` (el comando único `bin/workflow.sh` no lo expone). **Sin comparación vigente genética vs aleatoria** (la de «3 a 18 veces» salió de la calibración borrada 2026-10-04). El «Training» de la estancamiento/«fitness» significa que parte del IS es de validación interna y el resto de entrenamiento: qué mitad es cuál y si el fitness del reemplazo usa Training o el IS entero **no está decodificado**. - **Punto débil estadístico:** las estrategias de un build son parientes, no sorteos independientes; una sola isla da 19/56 y 23/72 de las supervivientes del paso 8 en dos lotes (ficha de las islas, dossier 10-08); los ensayos efectivos están sin medir (las 10.000 guardadas de B5 equivalen a entre ~10 y ~180 resultados independientes, correlación media +0,22 entre dos: cálculo del revisor del dossier, no verificado por mí).

C8 — Paleta de bloques y reglas duras 15

- **Definición:** una paleta (lista de bloques `use=true`) por idea; cualquier bloque no listado se apaga. `sqx/blocks/palette.py:141` (`resolve`): override explícito → peso; si la paleta es lista corta (`unlabelled: off`) lo demás se apaga; si no, peso de la taxonomía (0-3). `buildingblocks.rewrite` (`:37`) además apaga **siempre** (regla 15) los `stopLimitBlocks` y todo bloque de reloj/calendario

reconocido por nombre (`CurrentHour…` , `BarHour…` , `…DayOf…` , `…WeekOf…` , `…Month…` , `…Time…` , `IsMonth…` , regex en `buildcensus.py:14`); bloque desconocido de la taxonomía → apagado. Taxonomía: 767 bloques (588 de condición, 171 de valor, 85 de nivel); la tarea tiene 844 entradas.

- **Banda de tamaño:** `palette.CONDITIONS = (90, 170)` (`palette.py:23`): menos de 90 condiciones ahoga la búsqueda, más de 170 sobreajusta. Solo se avisa («dentro»/«FUERA»), `buildingblocks.py:109-110` **no se niega** a aplicar una paleta fuera de banda (20 paletas en la librería; 7 fuera con 2-12 condiciones). Simple Strategy sin paleta no se lanza (`bin/workflow.sh:51`); con plantilla `-` deja la lista del donante (400 condiciones) y **nada comprueba que la plantilla carezca de hueco**.
- **Por qué:** cifras del dueño 2026-09-24; una paleta por familia de idea (tendencia, ruptura, reversión…, regla 15). El agente `buildingBlocksExpert` (Opus) elige la paleta de una plantilla.
- **Punto débil:** reduce formas de regla pero **no el nº de ensayos** (lo fija el tiempo): con una plantilla de un hueco y paleta de 103 condiciones hay 104 formas de regla (1 + 103) frente a 401 sin paleta; con Simple 1..2/1..2 son 28,7 M frente a 6.432 M (cuenta del dossier, sin contar parámetros). Una paleta corta concentra más ensayos por forma; **no hay ningún par «misma celda con y sin paleta» en disco** (efecto sin medir). Los pesos solo los respeta SQX entre bloques con nombre, no en comparadores genéricos (`knowhow/authoring/builder-block-switches.md`). La paleta se escribe mirando datos previos (notas citan `assets/FAMILIAS.md`) que el recuento de ensayos no suma. La regla de «reloj por nombre» deja abierta la pregunta de `SessionHigh/Low` , `HighestInRange` , `CBlock_TimeBreakout*` (WORKFLOW.md).
- **Evidencia real:** en USDJPY B5 se encendieron 103 de 575 condiciones, 0 valores, 0 niveles, 0 reloj. Las paletas de las corridas post-cambio: **no verificadas** (no he abierto sus `.sqx`).

C9 — Negativas del constructor (previas, no estadísticas)

Nombre `Test_/Trade_/Research_` (`builder.py:207`); una sola dirección (`builder.directions` , `:34` , `:215`); tope de condiciones (`buildrules.over_the_cap` , `:144`); costes decididos (regla 5, `builder.py:222`); sesión y timeframe conocidos (`doctrine.blockers` , `:63`); worker parado (`builder.py:226`); filtro existente para el activo/timeframe (`buildfilters._need`); `sample` solo `IS` ; MonteCarloManipulation rechazado.

6.6 Costes en el Build (spread, deslizamiento, comisión, swap)

Escritos en el `<Setup>` de la tarea Build por `setups.write_setup` (`sqx/projects/setups.py:76-117`) desde la ficha del activo: - **Spread IS** (`spread_is.use` , puntos): % del precio medio del tramo `build` del spread medio por tiempo de FTMO (el más ancho entre FTMO/Hantec/FundingPips), sin factor. XAUUSD: **13,33 puntos = 0,0105 %** del precio medio del `build` (1.265,27); `spread_oos` 16,93 (0,0105 % de 1.606,85); `spread_oos2` 31,42. **El mismo porcentaje en todos los tramos**: los puntos escalan con el precio. - **Deslizamiento:** 35 % del spread del mismo tramo (dueño, 2026-10-04; antes 50 %; «600 operaciones reales costaron 1,33× el spread ida y vuelta, IC 95 % 1,08-1,72»). XAUUSD: IS 4,67; OOS 5,93. - **Comisión:** peor caso de FTMO/Hantec por tramo (XAUUSD: SizeBased 5 USD/lote ida y vuelta; en índices `PercentageBased` 0,0 desde 2026-09-29). **Swap:** % anual del nocional del peor de FTMO/Hantec, **el de hoy en todos los tramos**: XAUUSD largo −7,07 %, corto −0,69 %. - **Punto débil:** (a) el spread «% del precio» infraestima en puntos los tramos viejos de los índices (ficha `costs/funded-tick-spread-vs-price` : elasticidad −0,64 en NIKKEI225; planos USATEC/USA500/DJ30) y reparte el error de forma desigual entre `build` (más barato) y OOS (dossier 10-08, NIKKEI: 304 vs 475 puntos). (b) El swap de hoy sobrecarga/alivia tramos con otros tipos. (c) Las cifras del spread vienen de una medición de 3 meses de ticks en 2026.

6.7 Cifras de tiempo y recursos

- Paso 6+7 en una sola arrancada: 2.379-2.472 s por corrida (DAX wf2 2.379, NIKKEI wf3 2.436, XAUUSD wf4 2.437, wf4s 2.472, USATEC 2.445), de los cuales SQX calculando (de `action=start` a «Project finished»): **1.877 s (31,3 min) en XAUUSD wf4** (build + OOS encadenados); export ≈ 440 s (export.metrics + trades + harvest: OOS 19+79 s, Results 22+120+191 s); sync de cierre 119 s. Son 10.000 estrategias en `Results` y 10.000 en `OOS` . Arranque de la sesión GUI del custodio ≈ 90 s.
- Cuota del tope de minutos usada en esas corridas: **no verificada** (el argumento `--minutes` no consta en `plan.json` / `estado.txt` ; con el `Test_` por defecto serían 10 min, y 31 min de SQX indican otro valor).
- Corrida de referencia (dossier 10-08, USDJPY B5, 90 min de build, 10.000/90 min por argumento): 832.299 generadas (9.240/min), 47.976 aceptadas (533/min), retest OOS de las 10.000 en 67 s, export 7 min. RAM/cores del custodio: JVM 64-79 GB en builds de 4-6 min (`knowhow/sqx-drive/unselected-build-population.md`); heap 80 g (regla 3).
- El `MonteCarloManipulation` ya no corre en el build: el CPU que gastaba (30 simulaciones por estrategia) desaparece; efecto sobre el tiempo **sin medir**.

6.8 Ejemplo de cadena (n entran → n salen)

Post-cambio, `Test_XAUUSD_H1_ruptura_wf4` : N generadas **sin dato** (no hay fila de ledger ni log) → 10.000 guardadas (suelo 350 operaciones; Ret/DD ≥ 2,85) → 10.000 retesteadas (paso 7) → paso 8: 10.000 → **1 por criterios** (en limbo) → paso 9: 1 → 1 → paso 10: 1 → 0 por criterios. (La réplica `wf4s` dejó 0 con los filtros reales y se hizo un sorteo de 3 para el smoke test, «los filtros reales dejaron 0».)

Resto: DAX wf2 10.000 → 3 por sorteo de 3 (sin superviviente por criterio); NIKKEI wf3 10.000 → 3 por sorteo de 3, 3 en limbo → 12 → 3; USATEC 10.000 → 3 por sorteo de 3 → 3 → 2. ⚠ En esas corridas el paso 8 se cortó por «sorteo de 3» (modo desarrollo), no por criterios; solo `wf4` fue por criterios. Con filtros antiguos (dossier, USDJPY B5): 832.299 → 47.976 → 10.000 → 72.

6.9 ⚠ Discrepancias, aspectos sin medida y problemas estadísticos

1. **Número de ensayos del build, mal contado.** El resumen del piloto escribe «CONSTRUCCION: 0 → 10000» y el informe de lotes titula «generadas» las 10.000 guardadas (`studies/screening/batches/report.py:43,53`); las generadas reales fueron 832.299 por lote (dossier). Para Simple Strategy hoy ni siquiera hay fila de ledger (apartado 0). La t esperada del máximo de N ensayos sin ventaja: 3,86 (10.000), 4,23 (47.976), 4,83 (832.299); el dossier avisa de que no son independientes. Ninguna regla de los pasos 8-25 consume esa N, pero `portfolio/common/construct/oos2.py:27,69` deflacta el Sharpe con la N total que suma `ledger/trials.py:43` (incluidas las filas «generated»).
2. **El corte real es un top-N por Ret/DD del propio `build`**, no los filtros. Con 35/año el suelo muerde (mínimo = suelo) pero PF y neto no (mínimos 1,0-1,12, neto > 0 en todas).
3. **La métrica Ret/DD la pone el donante** y la ficha dice «sin resolver».
4. **Cifras de `_study.yaml` sin medida vigente.** El encabezado cita `criterios-pasos-6-a-12-2026-10-04.md` §4 y `AlgoData/scratch/calib2/adaptive/RECIPE.md` (la calibración `Test_Calib_*` se borró el 2026-10-04 por spread mal; la carpeta no existe). El comentario «ningún filtro de IS probado da más robustas por minuto» procede de esa calibración y no se probó con el databank lleno (el caso de trabajo).
5. **Poder del azar:** una población de monos (entrada aleatoria, 5.000, primer lote USDJPY) cruza el paso 8 al 0,66 % (≈66 por 10.000, rango 45-93); las 72 vivas caben dentro del azar (dossier 10-08). Con los filtros de hoy no se ha repetido.
6. **Con plantilla y sin hueco de salida no hay condición de salida** (la doctrina la llama «optativa»).
7. **Paleta fuera de banda no se bloquea**; las `_v3` se nombran «90-150» y el código usa 90-170.
8. **Corrida «dev» del paso 8 («sorteo de 3»)** no es un resultado del criterio.

PASO 7 — Retest OOS en SQX (tarea `OOS`)

7.1 Propósito

Volver a probar, sin tocar su lógica, cada estrategia de `Results` en las fechas que el generador no vio y a los costes de esas fechas; produce el databank `OOS` que el paso 8 contrasta con `Results` . **El paso no calcula estadística ni filtra.**

7.2 Entradas

Databank `Results` (todas, `retestSelected="false"`). Configuración de la tarea desde `assets/_policy.yaml` (tramo `oos1`), la ficha del activo (costes `*_oos`) y `assets/_build.yaml` (motor, precisión, dinero). Va encadenado al Build en la misma arrancada de SQX (`pipeline/autopilot/run.py:83-92` , `merged` → acción «6+7»; el OOS empieza 0,19 s después del Build según la docstring).

7.3 Salidas

Databank `OOS` (en memoria hasta el sync de parada; en disco al parar), exportado por `ui/daemon/loader/afterrun.py:111-135` a `metrics.csv` (21 columnas IS + 13 OOS), `trades.parquet` (columnas Type, Open/Close time, precios, Size, P/L, Balance, Sample type, Close type, MAE, MFE + identity y sample), `equity.parquet` (curva diaria) y la cosecha `harvest/<P>/Results/<día>/` (`studies/screening/gate/harvest.py`) con `missing_oos.csv` y `manifest.json` .

7.4 Criterios

El paso no tiene criterios, ni reglas en `criteria.yaml` , ni filtros (el cfx de la tarea OOS no tiene una sola condición de aceptación viva: `builder.py:99-101` las apaga con `silence` en toda tarea `Retest` de un workflow; motivo, `_build.yaml:125-135` : con una condición viva SQX no escribe la estrategia que no la cumple y Python pierde las muertas, que son la mitad de la evidencia). Lo que hay son configuraciones y negativas, documentadas como «criterios implícitos»:

K1 — Ventana del OOS (`oos1`)

- **Pregunta:** ¿en qué fechas se rejuzga? **Definición:** `oos1` de `_policy.yaml:segments.<S>.oos1` convertido por `setups.bounds` (`setups.py:15-28`) a `dateFrom = 1-ene-from` y `dateTo = 31-dic-to` (inclusivos), sin hueco con el `build` ; la tarea carga barras desde `dateFrom` (`<Symbol dateFrom>`), por lo que no hay calentamiento desde el build. Ventanas vigentes verificadas:

XAUUSD/divisas **build 2008-2017, oos1 2018-2022, oos2 2023-2026-09-30**; índices y crudo (NIKKEI225, DAX40, DJ30, UKOIL…) **build desde 2011-09/2013 hasta 2019, oos1 2020-2023, oos2 2024-2026-09-30**.

- **Rango fuera de muestra:** `OOS_RANGE` (`setups.py:139` , `:175` , ya en git) reescribe `<OutOfSample><Range dateFrom dateTo>` de la tarea al `oos1` del activo; el donante lleva el de XAUUSD (2018-2022) y en un activo con otros años partiría el retest en dos bloques en la fecha equivocada (ficha `sqx-format/oos-range-marker`). No hay comprobación que verifique el rango escrito; `collect.measured` se niega solo si los dos bloques llevan datos.
- **Por qué:** un `<Setup>` de SQX lleva un solo spread/slippage, por eso el OOS es una tarea aparte con sus costes (ficha `per-task-costs`).
- **Punto débil:** `oos1` lo reutilizan todos los pasos 8-16, así que se consume (ver post-selection bias); el paso 7 en sí solo lo lee una vez.
- **Ejemplo real:** NIKKEI225 M30 (cfx del dossier, copia 2026-10-04): Build 2011.09.19-2019.12.31, OOS 2020.01.01-2023.12.31. La primera operación del OOS cae a 7-35 días del inicio (cociente mediano frente a la brecha media 1,06: sin evidencia de calentamiento comido), mientras que en el IS de esa misma corrida el cociente es 9,0 (⚠ dossier: el IS parece empezar tarde, sin investigar `NIKKEI225_M1`).

K2 — Costes del OOS

- Misma doctrina de costes que el Build pero con `*_oos` : spread (`spread_oos.use`), deslizamiento 35 % de ese spread, misma comisión y mismo swap. **El mismo % del precio que el Build** (dueño 2026-10-04: «la historia paga la fricción de 2026 relativa al precio»), por lo que el OOS paga más puntos que el Build cuando el precio ha subido. XAUUSD: spread IS 13,33 → OOS 16,93 puntos (×1,27); slippage 4,67 → 5,93. NIKKEI225 (dossier): 304,11 → 474,91 (×1,56), slippage 106,44 → 166,22; swap −4,78 % largo / −1,87 % corto (el de hoy).
- **Punto débil:** si el spread de puntos de los índices es casi plano con el precio (ficha), tanto el Build como el OOS pagan menos puntos de los medidos (FTMO JP225 487-1.094 pt) y la infraestimación se reparte desigual: el Build queda más barato que el OOS en proporción 1,56, lo que agranda la caída IS→OOS que ve el paso 8. El swap de hoy cobra en 2020-23 ≈ 40 % de los puntos de hoy (NIKKEI: precio 27,4k frente a 68k). Sin medir cuánto pesa en cifras.

K3 — Qué no cambia entre Build y OOS

Motor `MetaTrader5 (hedged)` ; precisión 1 (timeframe elegido, «nada de retest a M1», dueño 2026-10-02); timeframe/feed/sesión; gestión de dinero (riesgo fijo 1.000, ATR 20×4, máx 500 lotes, capital 100.000, DD 30 %); `ExitOnFriday 21:00` , `RealisticGapsHandling` , `ReservedBars 50` . Difieren pero sin efecto en el número (según el dossier): `AutomaticDismissal warnings` , el crosscheck MC (ahora apagado en ambas), `Rankings` (condiciones vivas solo en el Build), `DismissTooSimilarStrategies` (solo Build), métodos de dinero apagados.

K4 — `AutomaticDismissal` (puerta silenciosa de SQX)

Los 10 códigos (1 sin operaciones, 2 muy pocas operaciones, 4 P/L cero, 8 demasiadas cerradas en la misma barra, 16 duración cero, 32 demasiadas abiertas, 64 sin cerrar al final, 256 sin ejecutar, 512 operación atípica, 1024 operaciones ambiguas) están en `dismiss="true"` en las dos tareas (`Retest-Task1.xml:204-215` ; fábrica solo el 1). Pueden dejar una estrategia fuera del `OOS` ; esa falta cuenta como fallo de R1a en el paso 8 (`level0.py` , `criteria.yaml:74` , `missing: fail`). En USDJPY B5: 262 de 10.000 emparejadas con < 35 operaciones OOS, 29 con < 20, mínimo 13 (el código 2 no descartó ninguna con 13). Seis cosechas completas dan **0 sin gemela** (dossier). Qué códigos disparan en OOS de 4 años: **no medido** (propone probar un juguete en el conductor). - Otras vías de pérdida: `MaxStrategies 10000` del Rankings de la tarea corta un retest de más de 10.000; excepción de backtest (código 10001) / filtro de umbral (10000) según el revisor del dossier; sync de cierre (regla 1); duplicados `Strategy X(1)` del reintento (el reintento no vacía `OOS` ; `call.keeper` lo corrige); colapso por identidad en `collect.index` .

K5 — Cosecha y emparejamiento (lo que entrega al paso 8)

`pairing.pair` (`studies/screening/gate/pairing.py:23-31`) empareja **por nombre de fichero** (decisión 2026-09-26) mientras la ficha del 2026-10-08 `sqx-format/names-reused-between-builds` dice «nunca»; devuelve `pairs` , `alias` (identidad cambiada) y `missing` . Tras la limpieza de 2026-10-06 la identidad de un par legítimo no cambia, así que `alias > 0` sería señal fuerte de emparejamiento falso, pero solo se imprime (`harvest.py:96-100` , `:119-125`) y **no entra en** `manifest.json` . `collect.measured` lee de los datos qué bloque (10 IS / 20 OOS) llenó cada databank y se niega si llenaron los dos. Pasa al paso 8: `presencia_passed` , `sanidad_value` (nº de operaciones OOS de las operaciones exportadas), `estaticas_value` (Net profit OOS), `is_trades_per_year` (`level0.py:72-80`). - **Métrica OOS pobre:** SQX solo llena 13 columnas OOS (Annual % Return, CAGR/Max DD %, Calmar, Net profit, PSR, PF, Ret/DD, SQN, Sharpe, Sortino, Winning %, Ulcer, ZScore); faltan nº de operaciones, Drawdown, Max DD %, Stability, RSquared, TRL, R Expectancy, DoF y Param Count (`core/sqxview.py:24-29`). `R Expectancy` redondeada a 2 decimales por la vista. - **Mismo conjunto, misma fecha:** una segunda cosecha del mismo día del mismo conjunto sobrescribe la primera (`harvest.set_aside`).

7.5 Orden de aplicación

Build termina por tope (tiempo/N) → SQX encadena la tarea `OOS` → «Project finished» → `bin/sqx-worker.sh stop` (sync de cierre + `afterrun`: métricas, operaciones, cosecha) → paso 8. Si el Build para por su tope de minutos y el proyecto queda parado, la tarea 7 **no se ejecuta con el 6** y `run.py:164-169` la relanza sola (`ui/daemon/advance/buildcap.py`).

7.6 Ejemplo real

Corridas de 2026-10-10/11: **10.000 `Results` → 10.000 `OOS`** (0 sin gemela) en DAX40 H1, NIKKEI225 M30, XAUUSD H1 y USATEC H1 (`resumen.md` : «CONSTRUCCION: 0 → 10000 · OOS: 0 → 10000»). El `metrics.csv` de `OOS` que queda en disco tiene solo 1 (XAUUSD wf4) o 3 filas (resto): el paso 8 recortó el databank (por criterios o por sorteo de 3), así que **no hay población OOS completa en disco para estas corridas** y no calculo tasas OOS sobre ellas (⚠ las `Results` sí conservan las 10.000). Para una población completa: USDJPY B5 (dossier): el nivel de t pasa de mediana 2,61 en el build a 0,74 en `oos1` , y de 8.514 con t > 2 en el build a 722 en `oos1` ; ρ de Spearman +0,39 entre Ret/DD del build y t en `oos1` . - NIKKEI225 M30 `Trade_…volatilidadL` (cosecha 2026-10-04, 10 pares): OOS 58-187 operaciones en 4 años (14,5-46,8/año); spread 304,11→474,91, slippage 106,44→166,22 (dossier).

7.7 Coste en tiempo y recursos

- Tarea OOS: **144 s de media** (`AlgoData/autopilot/task-durations.json` , clave `OOS` ; `OOS2` 132 s). En el USDJPY B5: 67 s para 10.000. Medición antigua (`knowhow/perf/autopilot-dead-time.md` , 2026-10-01, 571 estrategias): 168 s (27 s preparación + 35 s SQX + 106 s parada y exportación). En la corrida 6+7 de XAUUSD wf4: export.metrics OOS 19 s + export.trades OOS 79 s; sync de cierre 119 s (compartido con el Build). Cores/RAM: los del custodio (80 g, todos los núcleos); sin dato propio del paso.
- Encadenado, SQX arranca (~28 s) y para (~30 s) una vez en vez de dos (`knowhow/sqx-drive/live-chain-without-restart`).

7.8 ⚠ Discrepancias y huecos

1. **`compare` / `restore` de la regla 1 no vigilan `Results` ni `OOS` en una corrida 6+7** (`ui/daemon/launch/run.py:69-70,104` ; en la ruta por defecto `liverun.py` ~130-140 `restore` no se llama nunca). Una pérdida en `OOS` la ve R1a; una en `Results` **no la ve nadie** (la cosecha lee lo que hay en disco): la población que llega al paso 8 sale más pequeña sin rastro (solo el recuento «0 → N» del resumen). Sin incidente observado (dossier).
2. **Emparejamiento por nombre** contra la regla del 2026-10-08; `alias` /«sólo en OOS» no se guardan en `manifest.json` ; `manifest` declara `join: identity… file name as fallback` , al revés que el código.
3. **Fechas de swap de hoy** en un OOS de 2018-2023 y spread en % del precio (K2).
4. **IS que empieza tarde** en NIKKEI (cociente 9,0 vs 1,06 en OOS): sin investigar el feed.
5. **`StopCondition` de 10 minutos de la tarea OOS** (`Retest-Task1.xml:216`) no está comprobado que SQX lo aplique a un retest (las seis cosechas completas sugieren que no).
6. **Fichas desfasadas:** `knowhow/export/databank-metrics-is-oos.md` describe la exportación por vista `.vw` con SQX arrancado; el código lee los `.sqx` sin arrancar (`export_metrics.py:22-29`); `.claude/skills/oos-gate/SKILL.md` dice que la cosecha exige parar el worker, pero solo el sync de cierre lo hace necesario.
7. **Sin dato de pérdida real** de estrategias por `AutomaticDismissal` /MaxStrategies en OOS de 4 años (K4).

Paso 8 — Análisis IS/OOS en Python (la puerta por niveles)

Fuentes verificadas: código `studies/screening/gate/*` y `studies/screening/analysis/*` ; reglas vigentes `pipeline/autopilot/criteria.yaml` (paso `"8"` , líneas 69-145); `studies/screening/gate/config.yaml` ; `ledger/thresholds.yaml` ; corridas reales en `~/Desktop/AlgoData/reports/*/Results/<día>/gate/` y `.../falsePositives/` , y `~/Desktop/AlgoData/autopilot/Test_USATEC_H1_ruptura_leaseWeb/20261011-071421/` . Los dossiers `criterios-paso-08-*` , `puerta-is-oos-paso-08-especificacion-2026-10-06` solo se usan para el porqué. `criterios-pasos-6-y-8-2026-10-02` , `criterios-pasos-6-a-12-2026-10-04` , `criterios-pasos-6-a-19-explicados-2026-10-06` y `puerta-oos-2026-09-23` describen la puerta ANTERIOR a la conmutación del 2026-10-07 (encargo 44) y ya no valen para umbrales (ver §9).

---

1. Propósito, entradas, salidas

Pregunta que contesta: ¿hay en la estrategia una ventaja real y estable en `oos1` (retest fuera de muestra), distinguible del azar y del movimiento del mercado, una vez cobrados los costes? (`puerta-is-oos-...especificacion` §0.1). Cuánto riesgo poner en fondeo se deja a otro módulo.

Entradas (la "cosecha", `harvest.py`): dos databanks de la MISMA proyecto emparejados por nombre y unidos por identidad (`core.sqxfile.identity`, SHA-256 del XML normalizado): el de build (`Results`, ventana IS) y el de retest (`OOS`, ventana oos1). Por cada lado se leen sus métricas (SQStats vía `core/sqxview`, sin SQX), sus operaciones y su equity diaria (un solo `orderstocsv` de `sqcli`). Más: barras M1 remuestreadas al timeframe de la estrategia (`core.barstore`), costes de la ficha del activo para `build` y `oos1` (`engines.market.reprice.costs`), y el inventario de anomalías del feed (`feedQuality`, opcional). `oos2` nunca se lee.

Salidas (`reports/<P>/<D>/<día>/gate/`): `scorecard.parquet` (una fila por estrategia del build, 51 columnas en la corrida USATEC: hechos por nivel + `cayo_nivel`, `cayo_regla`, `survives`, `limbo` + 14 columnas "medidas, no juzgadas" de `measures.py`), `funnel.csv`, `verdict.csv` (contra el databank de retest) y `verdict_build.csv`, `operaciones.parquet` (tabla por operación oos1 repreciada), `retraso.parquet`, `distribuciones.parquet` (nulas del mono, R6, R7), `manifest.json` (con las tres semillas), `gate.md/.html/.json` y una página por estrategia.

Cómo corta de verdad hoy (⚠ clave para el estadístico): el veredicto del gate (`DESCARTAR` = algún Falla; `MANTENER` = Pasa o Limbo) lo relee el juez del autopilot (`pipeline/autopilot/judge.py`) sobre los mismos criterios, y después, mientras `dev.on: true` en `criteria.yaml` (líneas 53-57: `on: true, sample: 3, at: ["8"], seed: null`), **sortea al azar 3 estrategias entre las que no han caído** (Pass y Limbo mezclados, `judge.py:197-203`) y descarta el resto con "fuera del sorteo de desarrollo". El corte real del paso 8 en una corrida autopilot es por tanto 10.000 → 3, no 10.000 → 169. Los 3 MANTENER de la corrida USATEC eran los tres Limbo. Es un modo de desarrollo del dueño (2026-10-01, "no me interesan los filtros, me interesa que el workflow funcione"); el historial `ledger/criteria-history.jsonl` lo apaga/enciende y cambia `sample` (3 → 50 → 10 → 3, 2026-10-09/10). Los criterios siguen sin congelar (`frozen` ausente; cabecera del paso 8 de `criteria.yaml`: "PROVISIONAL (Council 2026-10-04): do not freeze before a null population has run the chain").

Cada decisión se escribe en el ledger (`judge.ledger_row`: `step 8, segment oos1, n_in, n_out, config_hash = sha256(criteria.yaml)`), una fila por estudio (activo×TF×familia).

2. Semántica común: Pasa / Limbo / Falla y la cascada

- Estados por regla (`pipeline/autopilot/criteria.py`, formato descrito en la cabecera de `criteria.yaml`): `pass`, `limbo`, `fail`. Una estrategia sale en su PRIMER Falla; el resultado global es la peor regla. Un Limbo **sigue** y va marcado (`scorecard.limbo`, texto `hecho=valor` por regla en Limbo). `missing:` da el estado de un hecho ausente (por defecto limbo, "raro pero limbo" dueño 2026-10-01); `else: limbo` convierte "no cumple" en Limbo en vez de Falla (reglas que "nunca dan Falla").
- Un hecho NaN se juzga con `missing` (R1a, R1b, R3: `missing: fail`; el resto limbo).
- **Cascada** (`cascade.py`, `cascade.on: true`): los niveles corren en orden (`levels.py`: 0, 1a, 1b×3, 2, 3ab, 3c); tras cada nivel se juzgan las reglas cuyo hecho produce ese nivel; el que cae no recibe cálculo en los niveles siguientes (sus hechos quedan vacíos y su razón es la regla que lo tiró, no "sin dato"). Con `cascade.on: false` (usado por `falsePositives`) todos los niveles corren sobre todos con el mismo veredicto. 17 líneas de regla (el spec habla de 16 filas: R5 son dos líneas de un lado, `criteria.yaml:84-85`).
- El orden de aplicación real es el de la tabla del §4. Efecto estadístico: las reglas posteriores se evalúan sobre una población ya seleccionada por R2/R3 (condicionada a t≥1,65 en oos1), lo que afecta a sus tasas nulas efectivas.
- Semillas: mono, R7 y R6 usan raíz propia (`monkey.seed`, `homogeneity.seed`, `drawdown.seed`; `null` = fresca, guardada en el scorecard y el manifest; `--seed N` fija las tres). Streams por estrategia = BLAKE2b(identity) (`resample.stream`), independientes del orden y de quién esté vivo.

3. Elementos comunes que hay que conocer antes de leer los criterios

Repreciado común (`engines/market/reprice.py`, `reprice.real`): cada operación de oos1 se reprecia desde las aperturas de barra crudas (entrada en la apertura, salida en apertura de barra o a precio de SL dentro de la barra: `AT_PRICE=("SL","PT")`, `AT_OPEN`=resto) con el spread, slippage, comisión y swap de la ficha del activo para el segmento (`reprice.costs(feed,"oos1")`). Da `pnl` (neto de coste) y `cost` por operación. R3 y casi todo el nivel 1b/2/3 leen ESTE P/L, no el de SQX. Excepción: R2 lee `Net profit [OOS]` de SQX (`level0.py:451,520`). ⚠ No verificado en este informe: el docstring del repreciador marca "UNVERIFIED against SQX": cortos, comisión ≠ 0 y swap en puntos (`studies/readings/edgeCost/README.md`), y los costes de `assets/` están marcados PROVISIONAL (WORKFLOW fila 8, OPEN #26).

Exceso por operación (`tradelevel.drift_term`, `config drift.window: year`, `drift.variant: d`, dueño 2026-10-07): `exceso = pnl − drift_term`. Variante `d` = "deriva por ventana de calendario + operación ciega": el término es `lado × lotes × valor_punto × W(h)`, `W(h)` = media de `open[j+h] − open[j]` sobre las barras j de la ventana-calendario de la operación (año natural), ponderadas por 1/ATR(j−1), con h = barras en posición. Quita lo que ganaría una entrada ciega del mismo lado, duración y tamaño (la doctrina dimensiona ∝ 1/ATR). Ventana mínima trimestre (más corta, el exceso → 0 por construcción; `assert` en `tradelevel.calendar`). Medida del sesgo residual (spec §5.5): ≈ 0,017 de t por lado con la variante d (0,09 con la a); sesgo de t de una estrategia sin ventaja ±0,02 a ±0,27 según activo/lado con la a.

La t (`tradelevel.t` / `t_skew_kurt`): `iid = media/sd(ddof=1)·√n` ; `ar1 = iid·√(n_eff/n)` con `n_eff = n(1−ρ)/(1+ρ)` acotado a [2, n] (ρ = autocorrelación lag-1 de las desviaciones) ; con `drift.se_skew_kurt: true` (vigente) `iid = s·√(n−1)/√(1 − γ₃·s + (γ₄−1)/4·s²)` (error estándar tipo PSR), con s = media/sd(ddof=0). El hecho que se juzga es siempre `ar1` .

Escalas de t (`knowhow/research/gate-t-scales.md` , citado en `gate/README.md`): `degradacion_t` (Lo sobre equity diaria) ≈ 0,8× `trade_t` ; no mezclar scorecards anteriores al 2026-10-02.

4. Criterios (orden de aplicación = orden de la tabla)

Convención: "Línea" = lo vigente en `criteria.yaml` . Población a la que se aplica = lo que queda tras la regla previa (cascada), sobre `oos1` salvo R4a (build) y R6 (build como referencia).

Nivel 0 — se lee solo de la cosecha (coste ≈ 0)

R1a presencia — ¿sigue la estrategia en el databank de retest? * Hecho `presencia_passed` (1 si la identidad está en el retest, `level0.py:517`). Línea `>= 1` , `missing: fail` . CORTA. Decisión del dueño 2026-09-23: si SQX la tiró del retest (demasiados trades ambiguos, banderas rojas) esa decisión se respeta. * Débil: depende de reglas internas de SQX, no auditables aquí. Solo un riesgo operativo: la identidad puede cambiar entre build y retest (SQX reescribe `makeExternal`); el dossier lo marca como el primer fallo a arreglar y la cosecha re-keya a la identidad del build. * Corrida USATEC 2026-10-11: 10.000 entran, 0 caen. (XAUUSD wf4, DAX wf2: 0 caen.) Coste ≈ 0.

R1b sanidad y clones — ¿hay algo que medir y no es copia? * Hecho `sanidad_passed` = `n_oos ≥ min_trades` y no ser duplicado de operaciones de otra (`dedupe.trade_duplicates` , sobre la propia lista de operaciones de oos1; la primera del grupo sobrevive). `min_trades = 20` (`ledger/thresholds.yaml:23` , `gate.sanidad.min_trades` , dueño 2026-09-23). Línea `>= 1` , `missing: fail` . CORTA. * Porqué: 4 operaciones no dan error estándar; un clon no es una segunda observación. Débil: el 20 es redundante (R4a y R5b lo cubren: el dossier propone, "opcional", un suelo de 60-80 en oos1; no implementado). * USATEC: 80 caen de 10.000 (0,8 %). XAUUSD wf4: 275 (2,75 %). DAX wf2: 138 (1,4 %).

R2 neto de oos1 — ¿ganó dinero en oos1? * Hecho `estaticas_value` = `Net profit [OOS]` de SQX (métrica del databank retest, a los costes de la tarea de retest). Línea `> 0` . CORTA. Dueño/doctrina; "red por si se afloja la t". * Débil: usa el P/L de SQX, NO el repreciado de R3: puede discrepar con él (coste distinto/redondeos); un neto positivo de SQX con exceso ≤ 0 no importa porque R3 manda. En la población nula de `falsePositives` (monos): el 72 % pasa R2, el 28 % cae; en la real USATEC 78,8 % (el mismo orden de magnitud: R2 apenas discrimina poblaciones sin ventaja). * USATEC: 9.920 entran, caen 2.098 (21,2 %), pasan 7.822. XAUUSD wf4: caen 6.267 (64,4 %), pasan 3.458. DAX wf2: caen 7.040 (71,4 %).

R4a operaciones al año en el build — ¿opera lo bastante para que la t tenga muestra? * Hecho `is_trades_per_year` = nº de operaciones IS / años de build (`market().years["IS"]` ; 7,95 años en USATEC, 10,0 en XAUUSD). Línea: Pasa `>= 35` , Limbo `>= 20` , Falla `< 20` ; en H4: Pasa `>= 20` , Limbo `>= 15` (cambio del dueño 2026-10-09 06:50-09:10, `criteria-history.jsonl` ; el builder lo exige igual: 35/año en M15, M30, H1 y 20 en H4, `assets/_study.yaml`). CORTA. * Porqué: mínimo de muestra; el dueño decidió "35 como mínimo en M15/M30/H1, 20 en H4". Débil: se mide en el BUILD (periodo ya filtrado por el generador con el mismo suelo → con el filtro del builder activo la regla es casi inerte: caen 0 en las tres corridas); no mide muestra en oos1 (eso lo hace R5b). El dossier pedía fijarlo con potencia/s_min económico (pendiente, ⚠ "sin cambios por ahora"). * USATEC/XAUUSD/DAX: 0 caen; 21 en Limbo en XAUUSD wf4, 0 en USATEC. ⚠ El spec proponía R4b (techo ≤75 al año): retirada, no existe en `criteria.yaml` .

R5b operaciones en oos1 — ¿se puede leer la t? * Hecho `sanidad_value` = operaciones OOS. Línea Pasa `>= 100` , `else: limbo` . NUNCA da Falla. MARCA (Limbo). * Porqué: red sin descarte (el spec: cubierta por R4a + R5). Débil: el 100 es una convención; con 100 operaciones y s=0,10 por operación la potencia de R3 a 1,65 es ~26 % (spec §2.6: 12,5 % a s=0,05, 25,8 % a 0,10, 44 % a 0,15). * USATEC/XAUUSD/DAX: 0 en Limbo (todas ≥ 100, el filtro del builder ya empuja a ≥ 35/año×4 años).

Nivel 1a — R3, la regla central

R3 t del exceso — ¿la ventaja se distingue del azar, quitado el movimiento del mercado y con costes pagados? * Hecho `drift_excess_t` = `ar1` de la t del exceso sobre las operaciones de oos1 (variante `d` , ventana `year` , EE con sesgo y curtosis, §3). Línea Pasa `>= 1,65` , **sin Limbo y sin excepciones por timeframe**, `missing: fail` . CORTA. Es la regla que hace casi todo el trabajo: 98-99 % de las que llegan. * Porqué: una cola, p nominal 0,05 por regla única con significado claro (dueño 2026-10-07, "P.5 bis"); antes Pasa ≥ 2,33 y Limbo ≥ 1,65 con excepciones H1/H4. Medido (spec §2.5/§5.7): sin ventaja pasan 3,9 % (celda mediana) a 5,8 % (peor) sin costes; "entrada al azar pagando costes tras el filtro del build" 0,85 %; con la t normal en operaciones con sesgo −1,5 se rechazaba un 6,9 % de las nulas, con el EE corregido 5,1 % (USA500 H1 indicesIBS). * Puntos débiles estadísticos visibles: (1) **multiplicidad no corregida**: se aplica t ≥ 1,65 a 3.458-7.822 estrategias de una misma población generada y seleccionada (no hay FDR; el máximo de t esperable del puro ruido entre 10.000 es ≈ 3,7 y la mejor real del ejemplo era 3,14, spec §15); (2) potencia baja: s=0,10 → ~40 % con 196 operaciones; (3) dependencia entre estrategias de una misma familia (estructuras repetidas, ver `redundancia`) → el nº efectivo de pruebas es menor que 10.000 pero no se estima; (4) ρ AR(1) estimado sobre pocas operaciones; (5) la t compara con una nula de "operación ciega" cuya validez depende del

término de deriva (residuo ±0,017 de t con la variante d); (6) es una sola cola. * Corrida USATEC 2026-10-11 (7.822 entran, oos1 2020-2023, 4,0 años): pasan **173**, caen 7.649 (97,8 %). Mediana de `drift_excess_t` de las 7.822: 0,391 (cuartiles 0,02 / 0,81; máx 2,54). XAUUSD H1 wf4 (3.458 entran): pasan 2. DAX40 H1 wf2 (2.822): pasan 29. Otras (nivel 1a, entran → pasan): NIKKEI225 H1 cal6 8.598 → 1.352 (⚠ atípico: 15,7 %, causa no verificada), NIKKEI M30 wf3 6.117 → 299, USA500 H1 wf1 6.097 → 22, EURUSD H1 chk8 1.095 → 44. * ⚠ Control nulo (`falsePositives` , USATEC, 5.000 monos de entrada aleatoria sobre la huella de las donantes): pasan R3 el **1,0 %**; la población real 1,7 % (173/10.000 con R2 incluido; 1,7 % es sobre las 10.000). Poblaciones con ventaja plantada (Sharpe neto por operación 0,05 / 0,10 / 0,15): 20,0 % / 53,9 % / 83,0 % pasan R3. La fracción de ruido entre los pasantes reales puede estimarse en ≈ 1,0/1,7 ≈ 58 % si la población real fuese de la misma ley nula (cota gruesa, ver §7). * Coste: una pasada vectorizada sobre las operaciones (~7.800 estrategias × ~160-250 operaciones). Medida del catálogo perf (2026-10-07, 300 estrategias, 5.000 sorteos): `gate.level1a` ≈ 6,9 s de reloj, 7,8 s cpu, pico 2,8 GB (etiquetas "wall_s" del `profiling/history.csv` , que parecen acumuladas con `gate.stage` ≈ 6,5 s: no verificado si son incrementales).

Nivel 1b — reglas sobre las operaciones de oos1 (vivas tras R3)

R5 estabilidad de la frecuencia (dos líneas) — ¿sigue operando al mismo ritmo que en el build? * Hecho `z_frec` (`level1b_centred` , `analysis/centred.py`): `log_rate = ln[(n_oos/años_oos)/(n_build/años_build)]` ; `se = √(φ·(1/n_build + 1/n_oos))` , φ = sobredispersión de los recuentos anuales del build (χ² de Pearson contra tasa×exposición, dividida por años−1, acotada a [1,5; 3,5]: `frequency.phi_min/max`); se resta el CENTRO de la población: mediana del `log_rate` de las estrategias del mismo lado (largo/corto/ambos), acotada a ±0,35 (`centring.cap_log_rate`), y 0 si hay menos de 30 estrategias (`centring.min_strategies`). `z = (log_rate − centro)/se` . Líneas: `z <= 1,5` Pasa / `<= 3,5` Limbo y `z >= −1,5` Pasa / `>= −3,5` Limbo (el formato no tiene valor absoluto: dos reglas de un lado). Falla `|z| > 3,5` . CORTA (en principio). * Porqué: una caída fuerte dice que las reglas dejaron de activarse; una subida, que se comporta distinto. El suelo absoluto anterior (15/año) no comparaba con nada. Débil: φ acotada [1,5; 3,5] no estimada en poblaciones nulas; el centro se calcula con TODAS las estrategias del lado (incluidas las caídas), y se limita a ±0,35 (−30 %/+42 %) sin evidencia de qué tope; la hipótesis de Poisson con φ constante; φ por estrategia con ~8-10 recuentos anuales es muy ruidoso. Con R3 antes, la regla ve una población ya seleccionada. * USATEC: 173 entran, 0 caen en ambas líneas (1 en Limbo en la inferior). Mediana `z_frec` de las 7.822: 0,19. XAUUSD/DAX: 0 caen. Esencialmente inerte tras R3 (el nulo de monos también la pasa en un 98 %/97 % porque hereda la huella).

R5c ventaja frente al coste — ¿aguanta un coste peor? * Hechos `edge_mean_oos` = `media(pnl + cost)/media(cost)` por estrategia (`onepass.edge_mean` ; 1 = break-even) y `t_exceso_k` = la t de R3 restando `(k−1)·cost` a cada exceso con `cost_stress.k = 1,25` . Regla compuesta (`all`): `edge_mean_oos` Pasa `>= 2` , Limbo `>= 1,25` , Falla `< 1,25` ; `t_exceso_k` Pasa `>= 1,65` , `else: limbo` (nunca Falla). MARCA con Falla solo por edge < 1,25. * Porqué: ventaja bruta ≥ 2× el coste pagado y que la t aguante +25 % de coste (overkill de costes del dueño, factor 1,25 en SQX). El 2 coincide con `edgeCost.min_edge_spreads = 2.0` del ledger (dueño, confirmado OPEN #48, `action: mark`), la criba antigua del paso 8 (ver §8). Débil: el denominador es el coste del repreciador; con coste ≈ 0 en algunas fichas el cociente explota (inf/NaN); `k = 1,25` es arbitrario; no corrige multiplicidad; es la misma t de R3 (no es una prueba independiente: si R3 pasa por poco, `t_exceso_k` cae por debajo de 1,65 casi seguro → Limbo). * USATEC: 171 entran, 0 caen, **51 en Limbo** (30 %), todas por `t_exceso_k < 1,65` (mediana `edge_mean_oos` = 7,16, mín 4,8: nunca por edge); `t_exceso_k` mediana 1,72. XAUUSD wf4: 2 entran, 1 en Limbo. En el nulo de monos (cascada apagada): 48,3 % caen (edge < 1,25 mayoritario).

R5d degradación dentro de oos1 — ¿la ventaja se apaga con el tiempo? * Hechos `t_b` y `e_fin` (`level1b_centred`): pendiente `b` por mínimos cuadrados del exceso contra el tiempo de cierre dentro de oos1 (años), EE con ρ AR(1) de los residuos (n_eff acotado [2,n]); `t_b = (b − centro_b)/se` , centro = mediana de las `b` del mismo lado (sin tope: `centring.cap_slope` vacío, "lo propone K-m con las poblaciones de prueba", ⚠ pendiente); `e_fin = media_exceso + (b − centro_b)·años_oos/2` (exceso proyectado al final). Línea compuesta: `t_b >= −1,65` Pasa, `else: limbo` (t_b < −1,65 es Limbo); y Falla solo si además `e_fin <= 0` (la parte `any` : `t_b >= −1,65` o `e_fin > 0`). CORTA solo con ambas condiciones. * Porqué: una ventaja que decae durante oos1 es sospechosa de sobreajuste; se centra para no castigar a toda la población si el mercado cambió. Débil: pendiente sobre ~160-250 operaciones tiene poca potencia; la mediana de la población con tope ausente absorbe la parte común (si TODA la población decae, el centro se lleva el efecto y la regla no lo ve); R5d mira solo dentro de oos1, no IS→OOS. * USATEC: 173 entran, caen **2**, 0 Limbo adicional. Mediana `t_b` = 0,00 (por construcción del centrado), mediana `e_fin` = 10,36 USD.

R8 concentración — ¿depende de un trimestre o de tres operaciones? * Hechos `t_sin_mejor_trimestre` (la t de R3 sin las operaciones del periodo de mayor exceso total, periodo = `concentration.drop_period: quarter` por apertura) y `t_sin_3_mejores` (sin las `drop_trades: 3` de mayor exceso). Línea compuesta: ambos Pasa `>= 1,65` , Limbo `> 0` , Falla si alguno `<= 0` . CORTA con t ≤ 0. * Porqué: sustituye al "mejor trimestre ≤ 50 % del beneficio" y a "≥ 3 años positivos" (R7 antigua). Débil: quitar los mejores puntos sesga la t a la baja por construcción (post-selección sobre extremos); no corrige por el nº de periodos de oos1 (de 4 a 20 trimestres según la ventana). * USATEC: 171 entran, 0 caen, **153 en Limbo** (89 %): casi todas tienen alguna t en (0; 1,65). Una regla que deja en Limbo al 89 % de lo que le llega separa poco (ver ⚠ 9). Nulo de monos: 88,6 % caen R8, 0,3 % pasan; plantado s=0,10: 32 % pasan.

R9b MAE típica mala — ¿el riesgo real flotante es el nominal? * Hecho `mae_p95_r` = percentil 95 (`mae.percentile`) de `|MAE ($)|/1R` sobre las operaciones OOS (`1R = money_management.Amount = 1.000 USD` , `assets/_build.yaml`); positivo. Línea Pasa `<= 1,5` , Limbo `<= 2,5` , Falla `> 2,5` . CORTA. Dueño: la doctrina no lleva stop, y la MAE dice lo que cuesta. * Débil: percentil 95 de ~200 operaciones

(~10 puntos) muy ruidoso; unidad "R" nominal (sin stop real); es una regla de riesgo de fondeo, no de ventaja (el spec pasa R9a y R9c, peor día y DD/neto, al módulo de fondeo). * USATEC: 171 entran, 0 caen, **90 en Limbo** (53 %); mediana `mae_p95_r` 1,505 (cuartiles 1,35 / 1,64; máx 1,93) → la línea de Pasa está en la mediana. Nulo de monos: 8,4 % pasan, 5,9 % caen.

Calidad de datos — ¿la ventaja vive de operaciones que tocan un defecto del histórico? * Hechos (`level1b_data.py` , `analysis/dataquality.py`): una operación está "afectada" si entre su barra de entrada y la de salida (ambas incluidas) hay un evento de `events.parquet` del feed (picos, congelaciones, huecos; `skip_cls` vacío = todos); `n_afectadas` ; `ret_datos` = media del exceso sin las afectadas / media del exceso de todas (NaN si la media no es positiva; 1,0 si no hay afectadas); `t_a` = t de Welch (exceso afectadas − resto) (NaN con < 2 operaciones en algún grupo). Línea (`any`): Pasa si `n_afectadas == 0` , o `ret_datos >= 0,60` ; Limbo `ret_datos >= 0,25` ; o, rama `all` : `t_a <= 1,65` Y `ret_datos >= 0,60` / `else limbo` ; **Falla solo con `ret_datos < 0,25` Y `t_a > 1,65`** . Sin inventario utilizable o que termina antes de la última operación: todos los hechos NaN → Limbo, con aviso en el log. * Porqué: atribuir la ventaja al defecto del feed cuando las operaciones afectadas ganan significativamente más. Débil: dos umbrales por regla más una t de Welch con n_afectadas pequeño (a menudo < 10, `feedQuality` lo marca "insuficiente"); el inventario depende de K por activo y de sesiones aún abiertas (Brent). Falla requiere la t de Welch > 1,65: potencia muy baja. * USATEC: 171 entran, 0 caen, 1 Limbo; mediana `ret_datos` 0,907. Nulo de monos: 22,7 % pasan la línea, 0,5 % caen (el sesgo: monos con muchas operaciones afectadas).

Nivel 2 — retraso de una barra

Retraso — ¿sobrevive el exceso si entrada/salida se ejecutan una barra tarde? * Hechos (`level2.py` , `analysis/delay.py`): para cada variante de `delay.variants = ["E+1","S+1","ES+1"]` (E = entrada, S = salida, en la apertura de la barra siguiente; las salidas por SL no se mueven), se reprecia con el mismo repreciador y se recalcula el exceso con el término de deriva de la ventana de la apertura movida. `ret` = media exceso movido / media exceso original (NaN si la original no es positiva); `t_delta` = t pareada (original − movida, iid). Se juzga la variante con menor `ret` (empate: primera de la lista): `retraso_ret_min` , `retraso_t_delta` . Línea (`any`): Pasa `ret >= 0,60` ; Limbo `>= 0,25` ; o rama `t_delta <= 1,65` con `ret >= 0,60` / `else limbo` ; **Falla solo con `ret < 0,25` Y `t_delta > 1,65`** . Con `delay.on_build: false` (vigente) no se calcula sobre el build. * Porqué: exigir `t^(v) ≥ 1,65` obligaría a t≥2,75 de partida y volvería a juzgar la ventaja (spec/4.4 §3.4); la retención y el test pareado aíslan "cuánto depende de la barra exacta". Débil: toma el MÍNIMO de 3 variantes (sesgo hacia Falla), el umbral 0,60 no está calibrado contra una nula, `ret` es un cociente de medias con denominador cerca de 0 para estrategias marginales; E-1 (adelantar) existe pero no está activado. * USATEC: 171 entran, caen **2**, 18 en Limbo; mediana `retraso_ret_min` 0,773 (cuartiles 0,67/0,90). XAUUSD wf4: 2 entran, cae 1. DAX wf2: 29 entran, 0 caen, 2 Limbo.

Nivel 3ab — remuestreo por estrategia

R7 homogeneidad por trimestres — ¿el resultado depende de un régimen? * Hecho `q_p` (`level3ab.py` , `resample.q_null`): Q de homogeneidad del exceso por periodo de apertura (`homogeneity.period: quarter` ; periodos con < `min_trades: 5` operaciones se unen al siguiente, cola corta al anterior) = Σ (suma centrada del periodo)²/tamaño, contra `permutations: 5000` barajados de las operaciones entre periodos (cada periodo conserva su tamaño); `p = (cuenta ≥ observado + 1)/(5001)` (nunca 0); NaN con menos de 2 periodos. Línea Pasa `> 0,05` , `else: limbo` . NUNCA da Falla. * Porqué: sustituye a "≥ 3 años positivos". Débil: es una prueba de homogeneidad (que detecta variación), no de signo; una estrategia homogéneamente mala o buena no se distingue; el p tras seleccionar con t≥1,65 está sesgado; con 5.000 permutaciones por estrategia, p mínimo 1/5001. * USATEC: 169 entran, 0 caen, **7 en Limbo** (4,1 %: justo el 5 % nominal); mediana `q_p` 0,326. Nulo de monos: 87,2 % pasan.

R6 drawdown — ¿el drawdown de oos1 es el esperable dada la distribución del build? * Hecho `dd_p` (`resample.drawdowns`): se toman las operaciones del BUILD repreciadas (`inputs.build_pnl` , `reprice.real` con costes del build), se desplazan a la media de oos1 (`build − media_build + media_oos`), se generan `drawdown.sequences: 5000` secuencias por bootstrap por bloques (`block: 8` operaciones, tantas operaciones como oos1) y `dd_p` = proporción de secuencias con drawdown máximo ≥ el observado en oos1 (drawdown de operaciones CERRADAS, curva que arranca en 0). `dd_esperado_r` = percentil 95 de esos drawdowns en R. Línea Pasa `> 0,10` , Limbo `> 0,01` , Falla `<= 0,01` . CORTA solo con p ≤ 0,01. * Porqué: sustituye a "DD oos1 / DD build ≤ 1,25". Débil: asume que la dependencia temporal cabe en bloques de 8; el DD se mide sobre cerradas (no flotante, a diferencia de `oos_max_dd_r` de `measures.py`); el p alto significa "DD pequeño frente al bootstrap" (sin cota inferior: no detecta DD anormalmente bajo); bloque 8 no calibrado. * USATEC: 169 entran, 0 caen, **8 en Limbo**; mediana `dd_p` 0,751 (mín 0,034). En el nulo de monos 0,3 % caen, plantado s=0,15: 2,8 % caen.

Nivel 3c — el mono

z_mono — ¿gana a entrar al azar en el mismo mercado, sin suponer normalidad? * Hecho `z_mono` (`level3c.py` , `engines.nulls.windowed`): para cada estrategia viva, 5.000 (`monkey.draws`) re-entradas al azar de CADA operación de oos1 en una barra uniforme de su misma ventana-calendario (`drift.window: year`), con el mismo lado y la misma duración (peldaño `timing` , el titular), tamaño por ATR de la barra sorteada, repreciadas con el mismo repreciador; estadístico = P/L medio por operación; `z = (visto − media_monos)/sd_monos(ddof=1)` , NaN si sd=0. Los otros peldaños (`timing_holds` , `timing_sizing` , `free`) se calculan y anotan (`z_mono_<peldaño>`), no se juzgan. `mono_p` (p empírico de `verdict.pvalue(..,"net")`) también anota. Línea Pasa `z_mono >= 1,65` , `else: limbo` . NUNCA da Falla. * Porqué: el control no paramétrico de R3 (el dossier del mono demostró que el peldaño timing ES la deriva de R3 menos costes: de 325 con t≥1,65 sobre una población, 318 ya ganaban al mono con p≤0,05; lift +5 por 1.000). Débil: casi la

misma hipótesis nula que R3, por lo que no añade independencia; sorteo con simplificaciones declaradas (solapes permitidos); la z usa la desviación de la nula sin corrección de cola. Con `mono_p` mínimo ≈ 1/(5.000+1). * USATEC: 169 entran, 0 caen, **104 en Limbo** (61,5 %), 65 Pasa; `z_mono` mediana 1,575 (cuartiles 1,50/1,73; mín 1,23; máx 2,21) → la línea está al 60º percentil de la población. XAUUSD wf4: 1 entra, pasa. DAX wf2: 29 entran, 0 Limbo. * Coste (catálogo perf 2026-10-07, 300 estrategias, 5.000 sorteos): `gate.monkey` ≈ 10,3 s de reloj, 17,3 s cpu, `rss_peak` 26,9 GB (pico del conjunto de procesos fork; no verificado); fork con `core.fanout` sobre `os.cpu_count()` (96 núcleos en esta máquina).

5. Columnas "medidas, no juzgadas" (`measures.py`) y estudios encadenados

14 columnas por estrategia (también las caídas): `trade_t`, `drift_excess_t` (la de R3 recalculada), operaciones/año IS/OOS, `{oos,is}_max_dd_r` (equity diaria flotante de SQX, en R), `_worst_day_r`, `_worst_trade_mae_r`, `_net_per_year_r`, `_dd_over_net_year` (+inf si neto ≤ 0). Ninguna regla las lee hoy. R9a (peor día) y R9c (DD/neto) se retiraron del paso 8 al módulo de fondeo (spec §13). `redundancia` (agrupa por la estructura de bloques del XML; SOFT, no elimina; cálculo sobre equity/estructura) anota.

Qué corre realmente en el paso 8 de una corrida autopilot (fases `tiempos.md`, USATEC): `gate, isOos, filters, decay, falsePositives, edgeCost, feedQuality` (acción "8 python") y, tras el juez, `monkey, entryQuality` ("después del corte": `AFTER_CUT = {"8": ["monkey","monkeyExcess","entryQuality"]}`, `run.py:95`; `monkeyExcess` no se corre). Solo `gate` produce reglas. `edgeCost`, `feedQuality`, `maeDepth` (criba del MAE profundo, encargo 27), `spread` y `snoopingScreen` ya no juzgan nada (`criteria.yaml` cabecera paso 8: "Fuera desde la conmutación: R5 vieja, R6-R8 viejas, R9a y R9c, spread. *y edgeCost.*"); `edgeCost` (modo población) escribe `verdict.csv` con `edge_mean` y `mark` según `min_edge_spreads = 2,0`, sin cortar, y su número es el mismo `edge_mean_oos` del gate (`tests/test_edgecost.py` los ata).

Estudios 4.x del dossier, estado real: 4.2 (espionaje de datos + criba `familia`) y 4.3 (forma del beneficio, `profitShape`) ELIMINADOS (sus t_{-3}, t_{-Q*} y pendiente de R5d viven ya en `level1b_onepass` / `centred`); 4.4 calidad de la entrada → implementado como nivel 2 (retraso) — el e-ratio sigue como lectura posterior al corte (`entryQuality`); 4.5 calidad de datos → regla "Calidad de datos" (nivel 1b); 4.6 spread (R2b neto al spread real de Darwinex) y 4.7 (`edge_median`, `n`) ELIMINADOS. El mono se rediseñó (z continuo, solo oos1, ventana de calendario, tamaño por ATR) y juzga como Limbo.

6. Ejemplo de cadena (cadena real, USATEC H1 `ruptura`, `Test_USATEC_H1_ruptura_leaseWeb`, 2026-10-11)

Build 2012-01-19 → 2019 (7,95 años), oos1 2020 → 2023 (4,0 años), feed `USATEC_M1`, 1R = 1.000 USD, semillas mono 1851582449 / homogeneidad 3929614098 / DD 2340908895, código `abca93c-dirty`, `monkey.timeframe=H1`:

nivel	regla	entran	Limbo	caen
0	R1a presencia	10.000	0	0
0	R1b sanidad/clones	10.000	0	80
0	R2 neto oos1 > 0	9.920	0	2.098
0	R4a ≥ 35/año	7.822	0	0
0	R5b ≥ 100 operaciones	7.822	0	0
1a	R3 t exceso ≥ 1,65	7.822	0	7.649
1b	R5 z_frec ≤ 1,5 / ≥ −1,5	173	0 / 1	0 / 0
1b	R5d e_fin, t_b	173	0	2
1b	R5c edge / t_exceso_k	171	51	0
1b	R8 sin mejor trim. / sin 3 mejores	171	153	0
1b	R9b MAE p95 ≤ 1,5R	171	90	0
1b	calidad de datos	171	1	0
2	retraso de una barra	171	18	2
3ab	R7 q_p > 0,05	169	7	0
3ab	R6 dd_p > 0,10	169	8	0
3c	mono z ≥ 1,65	169	104	0

Resultado del gate: **169 siguen (4 Pasa limpio + 165 en Limbo)**; el juez del autopilot (`paso-8/verdict.csv`: 4 pass, 165 limbo, 9.831 fail) hizo después el sorteo dev: **10.000 → 3** (los 3 elegidos, Strategy 1.11.277, 1.13.407 y 6.22.453, eran Limbo). Otras cadenas recientes (entran → R2 → R3 → final): DAX40 H1 reversion wf2 (2026-10-10) 10.000 → 2.822 → 29 → 29; XAUUSD H1 ruptura wf4 (2026-10-11)

10.000 → 3.458 → 2 → 1; USA500 H1 tendencia wf1 10.000 → 6.097 → 22 → 20; NIKKEI225 M30 volatilidad wf3 10.000 → 6.117 → 299 → 258; DJ30 H1 cal1 10.000 → 2.748 → 16 → 9; EURUSD H1 chk8 10.000 → 1.095 → 44 → 41. Orden de magnitud: entre 0 y 15 % de las que pasan R2 pasan R3; las reglas posteriores a R3 caen solo 0-15 % de ellas, pero dejan en Limbo hasta el 89 % (R8) del resto.

7. Lo que dice el control nulo (`falsePositives`), y lo que significa

`studies/screening/falsePositives` pasa 5.000 monos (entradas aleatorias de la huella de las donantes, a los costes del retest; nunca fueron seleccionados por un build) por las mismas 17 reglas con la cascada apagada, y una población real. Corrida USATEC 2026-10-11 (semilla 3747935572), paso 8:

población	entran	Pasa	Limbo	siguen (Pasa+Limbo)
nula	5.000	2	47	49 (0,98 %)
ventaja 0,05	5.000	22	938	960 (19,2 %)
ventaja 0,10	5.000	88	2.420	2.508 (50,2 %)
ventaja 0,15	5.000	161	3.605	3.766 (75,3 %)
real	10.000	4	165	169 (1,69 %)

Lectura (míos, no del documento, marcados como inferencia): (1) la cadena completa deja pasar el 0,98 % de poblaciones sin ventaja, y la real pasa el 1,69 %: enriquecimiento ≈ 1,7×; (2) la tasa de Pasa limpio (sin Limbo) es idéntica en nula y real (0,04 %), de modo que todo el exceso de la real está en el Limbo; (3) si la población real se pareciese a la nula en su ley de nulos, ≈ 58 % de las 169 serían ruido; hay que tomarlo como cota gruesa (la nula son monos de las donantes: su R2/R3 puede no ser comparable con estrategias de un generador que optimiza el IS); (4) potencia con s = 0,05 de ~19 %, s = 0,10 ~50 % (`hechos_monos.parquet`); (5) las reglas que descartan poco en la nula: R5, R7 y R9b-Falla (≈ 0 %), R6 (0,3-2,8 %) y mono (nunca); las que más descartan: R3 (99 %), R8 (88,6 %), R5c (48 %), R2 (28 %).

Un paso también se mira como tasas por regla en la población real (porcentaje de Pasa / Falla; USATEC): R3 1,7 % / 98,3 %; R5c 1,2 % / 25,6 %; R8 0,2 % / 67,6 %; R9b 45,6 % / 0 %; calidad de datos 51,0 % / 0,4 %; retraso 35,1 % / 6,7 %; mono 0,8 % / 0 % (cascada apagada, sobre las 10.000).

8. Coste en tiempo, núcleos y RAM

- Corrida real (autopilot USATEC, `fases.csv` / `tiempos.md`): `export.harvest` 203,5 s (más `export.trades` 119 s y 88 s y `export.metrics` 20 s y 18 s, ya dentro de los 456 s de "export" del paso 6+7); **`test gate` 77,5 s** para 10.000 estrategias; `isOos` 1,3 s, `filters` 4,2 s, `decay` 45,6 s, `falsePositives` **4.170 s (69,5 min, el 95 % del paso)**, `edgeCost` 5,6 s, `feedQuality` 33,7 s; juez 17,5 s; `monkey` + `entryQuality` 8,4 + 1,3 s. "8 python" total 4.390 s de una corrida de 10.034 s. Sin CPU de SQX (`sqx` = 0 s); el worker está parado.
- Máquina: 96 núcleos, 125 GB (`nproc` , `free`); el monkey usa fork de `os.cpu_count()` ; RAM del gate: pico no medido en esa corrida (sin dato); catálogo `profiling/history.csv` (2026-10-07, commit `48a2f38-dirty` , 300 estrategias USA500 H1): stage 6,5 s, level0 6,5 s, level1a 6,9 s, level1b 7,7 s, level2 7,8 s, level3ab 11,2 s, monkey 10,3 s (17,3 s cpu, 26,9 GB pico), `gate.kept` (3.000 estrategias) 2,5 s; los niveles son lineales en el nº de estrategias vivas (nota de `perf/config.yaml`). Con la cascada, los niveles 1b-3 solo ven 29-173 estrategias.
- Disco: `operaciones.parquet` 55,8 MB, `distribuciones.parquet` 12,1 MB para 10.000 estrategias; `estrategias/` una página por estrategia (≈ 10.000 ficheros).
- El paso original `workflow-2026-09-26/steps.csv` (200 estrategias, regla antigua): 8a harvest 43,3 s, 8b report 3,12 s.

9. Discrepancias doc ↔ código ↔ corrida (⚠)

1. ⚠ **dev.on = true**: el corte real del paso 8 es un sorteo de 3 (10.000 → 3), no los 169 del gate; los dossiers describen el filtro como corte por reglas. Hay que tenerlo en cuenta al interpretar "supervivientes" en pasos 9+ (las 3 de USATEC eran Limbo, ninguna Pasa limpia).
2. ⚠ Los 4 dossiers de 2026-09-23 a 2026-10-06 (puerta-oos, criterios-pasos-6-y-8, 6-a-12, 6-a-19-explicados) describen la puerta anterior: R3 Pasa ≥ 2,33 / Limbo ≥ 1,65 con excepción H4 (2,5/1,9); R4 techo (≤ 75, M30 100, H4 50); R5 absoluta (≥ 15/≥ 10 al año); R6 DD oos1/IS ≤ 1,25; R7 años positivos; R8 mejor trimestre ≤ 0,5; R9a/R9c; R2b spread Darwinex. Nada de eso existe en `criteria.yaml` . Los umbrales hoy están en `criteria.yaml` (paso 8), no en el ledger.
3. ⚠ `gate/README.md` y `config.yaml` dicen que "cada umbral es un `ledger:<clave>` ": solo `gate.sanidad.min_trades` (20) lo es. `config.yaml` guarda directamente `phi_min 1.5, phi_max 3.5, min_strategies 30, cap_log_rate 0.35, k 1.25, drop_trades 3, percentile 95, permutations 5000, block 8, sequences 5000, draws 5000, min_trades 5` . Las líneas de Pasa/Limbo/Falla viven en `criteria.yaml` con historial de cambios (`ledger/criteria-history.jsonl` , 16 líneas), sin congelar (`frozen` ausente).

4. ⚠ `puerta-is-oos-...especificacion` §0.3 y `criterios-paso-08-indice-decisiones` dicen "R5c con coste × 1,5" (índice, propuesta anterior) frente a `k = 1,25` en spec y config: manda el código (1,25).
5. ⚠ WORKFLOW.md fila 8 describe la criba de edge por coste como "el bruto reconstruido de Profit/Loss + comisión + coste de spread modelado, en spreads de hoy (media y mediana) y como c *de breakeven"; el README de* `edgeCost` *y el código dicen lo contrario:* `edge_mean = mean(pnl+cost)/mean(cost)` *del repreciador común, sin mediana ni c* ni desglose por coste, y la regla R5c del gate es el mismo número. Además WORKFLOW dice "□ umbrales laxos de la puerta" y el paso 25 "fuera de la cadena (2026-10-11)".
6. ⚠ `criteria.yaml` dice "16 reglas (R5 son dos líneas)"; el código ejecuta 17 líneas (funnel, `falsePositives`). R4b (techo) y R9a/R9c existen solo en dossiers.
7. ⚠ `centring.cap_slope` está vacío (`config.yaml:248`): el centro de R5d no tiene tope (decisión pendiente "K-m", spec §14.1); `dev.on` y la ventana de la deriva (año vs semestre, `drift.window: year`) son decisiones del dueño con medidas pendientes según el spec (la variante `d` consta como elegida el 2026-10-07).
8. ⚠ Tasa nula declarada de R3: spec dice 3,9 % (mediana)/5,8 % (peor) "sin costes", 0,85 % "con costes tras el filtro del build"; el `falsePositives` real mide 1,0 % en USATEC con costes (consistente con 0,85 %). El 1,65 nominal (5 %) no es la tasa efectiva, que depende del activo y de los costes; no hay medida para cada activo/TF corrida hoy salvo el control por proyecto.
9. ⚠ La regla "mono" (z ≥ 1,65) no puede dar Falla y R5b, R7 y mono solo dan Limbo; R8 deja en Limbo al 89 % y R9b al 53 % en USATEC, de modo que 165/169 de los que "siguen" están en Limbo y la etiqueta no separa nada. El Limbo no tiene efecto sobre el corte (solo se anota), salvo en el sorteo dev, donde cuenta igual que Pasa.
10. ⚠ NIKKEI225 H1 `aroonL_cal6` (2026-10-08): R3 deja pasar 1.352 de 8.598 (15,7 %) y todas siguen hasta el final; no verificado si es un efecto de la deriva de ese activo/lado, de una cosecha sin cascada, o de una versión anterior del código (corrida del día 8; el gate cambió el 7).
11. ⚠ `puerta-oos-2026-09-23.md` y `gate/README` dicen que `sanidad` descarta con `min_trades` ; en el código `sanidad_passed` solo registra `n ≥ 20` y no-clon y R5b (≥ 100) no descarta: coherentes, pero el dossier de 10-06 §4.2 dice que el 20 es redundante y se propone un suelo de 60-80 que NO está aplicado.
12. Limitaciones declaradas del gate (README, spec §15): "pasar el paso 8 hace a una estrategia candidata, no válida"; no deshace la selección post-hoc si las condiciones de aceptación del build leyeron oos; `oos2` no se usa aquí; la multiplicidad se reserva al ledger (`ledger/trials.py` , `N` candidatos + `K` hipótesis) y a los pasos 17-20, no a este.

Pasos 9, 10, 10.5, 11, 11.5, 11.7 y 12 — transferencia (otros mercados y otras temporalidades), tal como está implementado

Convenciones: «Corta» = el juez borra la estrategia del databank del paso (`cut:`); «Marca» = la regla da limbo pero la estrategia sigue (`MANTENER`); «Informa» = solo hecho, ninguna regla lo lee. El juez (`pipeline/autopilot/judge.py:22,210`) conserva todo lo que no sea `DESCARTAR` : pass y limbo siguen, solo fail corta. Una regla compuesta `all` = peor parte, `any` = mejor; un hecho ausente = limbo salvo `missing:` (criteria.yaml, cabecera). Código de referencia, en orden de autoridad: `pipeline/autopilot/criteria.yaml` , `studies/transfer/crossmarket/` (paso 10), `studies/transfer/crossTF/` (11.5 y 12), `sqx/projects/{crossmarket,crosstf,crosstfload}.py` , `sqx/variants/scale.py` ; plan vivo `docs/encargos/51-pasos-10-a-12-transferencia.md` .

Resumen ejecutivo para el estadístico

1. Solo el **paso 10** (9 reglas P1-P9 + control 7.1) y el **paso 12** (una regla: `crossTF.combined.z_new >= 2.33`) tienen criterio. Los pasos 9, 10.5, 11 y 11.7 son SQX/fabricación sin criterio; el 11.5 mide pero **no corta** (`rules: []`).
2. **El paso 12 vivo no puede eliminar ninguna madre**: pass si `z_new >= 2.33` , `else: limbo` , nunca fail (criteria.yaml:195-200). Lo único que quita son las hermanas escaladas (`hermana escalada: solo sigue la madre`). Es una medición, no una puerta.
3. **El paso 10 sí elimina** (fail), pero por muy pocas vías: P1 y P5/P6/P7 nunca dan fail; fail solo en P2, P3, P4, P8, P9 (y 7.1 nunca). En la corrida real más reciente cayó 1 de 3 (por P3, z_conj = -0,24).
4. **Todas las líneas del paso 10 son «PROVISIONAL y SIN CALIBRAR»** (criteria.yaml:131-136; encargo 51 frente 1 pendiente: ni población nula ni con ventaja plantada pasadas por P1-P9). El umbral 2,33 / 1,65 / 0 son convenciones, no medidos contra FP/potencia del paso completo. Medido solo: batería sintética del núcleo (300 estrategias sin ventaja en 11 mercados: media z_conj -0,02, sd 1,08, rechazo 7 % al 5 % nominal y 1 % al 1 %) y paso 12 viejo (300 madres: media z entre -0,09 y +0,02, sd 0,93-1,04, rechazo al 5 % entre 3,0 % y 5,7 %).
5. **Ninguna corrida real ha pasado de 1-25 estrategias por paso 9-12** (dev: `criteria.yaml:55-59` , `dev.on: true, sample: 3, at: ["8"]` : el paso 8 deja pasar 3 al azar). Todas las cifras de «cadena» de abajo son 3 → 2 o similares; no hay población real de cientos.
6. Los dossiers `criterios-paso-10-otros-mercados-2026-10-06.md` y `criterios-paso-12-...-2026-10-06.md` describen el **criterio anterior**; el 10 está obsoleto por completo (ver «Discrepancias»).

PASO 9 — Retest «Cross Market» en SQX (tarea `Retest Markets - Family`)

Propósito. Que SQX reprueba cada superviviente del paso 8 en otros mercados, cada uno a su propia ventana y a sus propios costes, para que el paso 10 mida si la lógica viaja. Es medición: no hay criterio ni corte.

Entradas. Databank `OOS` (supervivientes del paso 8, cortado por el juez). Lista de mercados: `assets/_markets.yaml` (bloque del activo main; categorías `family` + `structural`; «la lista se fija ANTES de mirar resultados»). Costes: ficha de cada mercado en `assets/symbols/<M>.yaml`, tramo `oos1`.

Salidas. Databank `Retest_Markets_-_Family` con una fila por (estrategia × mercado); export de operaciones por mercado (`raw/<P>/Retest_Markets_-_Family/<día>/trades.parquet`).

Implementación (`sqx/projects/crossmarket.py`). - `chosen()` (l.~103): mercados de `family` + `structural` del main; `set_markets()` los escribe como `<Setup>` de la tarea. Un mercado cuyo feed no declara ningún `assets/symbols/*.yaml` o cuya ficha no tiene `build.from` / `oos1.to` se **bloquea** (`blocked`) y el comando aborta salvo `--skip-undeclared` (no se inventan costes, regla 5). - Ventana de cada mercado (`window()`): desde el inicio de SU propio `build` (o su primera barra, lo que sea más tarde, `data_from` de `_markets.yaml`) hasta el final de SU `oos1` (`_build.yaml:152`, `crossmarket.segment: build..oos1`). Nunca la del main (dueño, 2026-10-02: con la ventana del main un índice arrastraba oro y USDJPY por 2023). - Costes (`one_market`): spread, slippage, comisión y swap de la ficha de cada mercado **en `oos1`** (`costs = segment.split("..")[-1]`); un solo precio por Setup aunque la ventana cubra dos tramos. Se hereda del test principal: timeframe, precisión, distancia mínima, sesión (`INHERIT`); `dates="false"`. - `precision: 1` (timeframe elegido, no M1; dueño 2026-10-02), `conditions: []` — si no está vacío el código aborta («esta prueba es una medición, no una puerta»). `silence()` apaga TODAS las condiciones de aceptación de la tarea (un clon traía condiciones vivas que hacen que SQX ignore los bloques extra con `evaluateAll=false`). - Motor: el de la doctrina, `MetaTrader5 (hedged)` (`assets/_build.yaml:72`). - Coste en CPU declarado: «multiplica cada tarea por (1 + nº de mercados)».

Criterios. Ninguno (no corta, no marca). Es una puerta de datos, no estadística.

Ejemplo real. `Test_USATEC_H1_ruptura_leaseWeb` (2026-10-11, custodio): `Retest Markets - Family: 0 → 3` estrategias (las 3 del dev-sample del paso 8). 11 mercados declarados para USATEC (family: AUS200, DAX40, DJ30, HK50, NIKKEI225, UK100, USA500; structural: EURUSD, USDJPY, XAUUSD, XCUUSD).

Coste. Esa corrida: 34 s en total (SQX 5 s, export 20 s, sync 1 s, copias 5 s; `tiempos.md`). Otros: 30-35 s en `Test_XAUUSD_H1_squeezeBBLong` (3 estr.) y `Test_XAUUSD_H1_ruptura_wf4s` (3). Cores/RAM: sin dato propio (custodio, 80 g de heap).

Discrepancias. Sin ⚠ de código. Nota: la ficha `costs_provisional` de algunos mercados se carga sin aviso en la tarea.

PASO 10 — Análisis cross-market en Python («¿la lógica viaja?»)

Propósito. Decidir, por estrategia, si su regla sigue ganando a azar en otros mercados. Nueve hechos P1-P9 frente a corridas al azar conjuntas (encargo 46, reescrito el 2026-10-07; el código antiguo de nulos por mercado, amplitud, Fieller y Timing Alpha se borró).

Entradas. Export del paso 9 (operaciones por estrategia y mercado), barras M1→TF de cada mercado (`barstore`), operaciones del main build+oos1 (`harvest/<P>/Results/<día>/trades.parquet`, para P2 y `e_principal`), `.sqx` de cada estrategia (para P1), fichas de coste, doctrina de tamaño `assets/_build.yaml:60-62` (`Amount 1000, ATRPeriod 20, ATRMult 4`, `Decimals/MaxLots` NO aplicados). Comando: `python3 -m studies.transfer.crossmarket.report --project P --databank D --asset A`.

Salidas. `reports/<P>/<D>/<día>/crossmarket/{verdict.csv, markets.parquet, crossmarket.json, distribuciones.parquet}`; en el autopilot, `paso-10/hechos.parquet` (clave `crossmarket.verdict_csv.<col>`) y `verdict.csv` del juez. Corta el databank `Retest Markets - Family`.

Núcleo común (todos los z salen de un solo tensor)

- **Aparato** (`travel/`): para cada mercado válido se reprecia (`engines/market/reprice`, al precio de apertura de barra, costes de `oos1`) la operación real y `B = 5000` corridas al azar. Cada corrida = las **mismas operaciones desplazadas un número entero de semanas dentro de cada año natural**, un solo desplazamiento por (celda, corrida) **compartido por todos los mercados** (clave = fecha de la celda, no el mercado; `shift.py`, seed 20261007, `config.yaml:6-8`). Tamaño por doctrina en cada barra: `Amount / (ATRMult × ATR(ATRPeriod)(barra cerrada antes de entrar) × valor del punto)`; P/L en **R** (1R = 4 ATR), lotes sin redondear.
- **Celdas** (`cells.py`): años naturales cortados en la frontera del main. Ventanas: **V0** = antes del `build` del main (años que el main no tiene), **V1** = `[build, oos1)` del main (el periodo donde se construyó/seleccionó), **V2** = `[oos1, fin oos1)` (el oos1 del main), `after` = posterior (excluido). Una operación pertenece a la celda donde **cierra**; el desplazamiento lo decide la celda donde **abre**.

- **z y p** (`stat.py`): `z = (real − media(null)) / sd(null, ddof=1)`; `p = (1 + #null >= real)/(B+1)`. NaN = «sin datos» (celda sin barras), 0 = «no pasó nada»; `stat.total` distingue ambos. z se lee sobre **bruto** (`judge_on: gross`, `config.yaml:9`): el coste en R = coste/(4·ATR de entrada) no se cancela contra copias desplazadas en entradas condicionadas a ATR (población sin ventaja con ATR alto leía +1,20 en neto). `e`, `tau_half`, `e_end`, `beta`, `e_end_ratio`, `d_y` siguen en **neto**.
- **Huecos de datos** (`tensor.exclude`, `max_displaced_share 0,20`): un (mercado, celda) donde un desplazamiento uniforme manda >20 % de entradas a una barra posterior (hueco) se pone a NaN en real y en null (HK50 157/76/180 días, UK100 118/90, AUS200 111/92, USA500 y USATEC 89 días en 2011-2015; deja ~10 pares, todos de V1; población uniforme queda en +0,06 ± 0,07). Ficha `knowhow/research/index-feed-holes-and-the-shift-null.md`.
- Lecturas aceptadas por el dueño (dec. 7, encargo 51): las operaciones cerradas tras el fin de oos1 del main quedan fuera; no se reaplica el cierre del viernes ni «plano para entrar» a las desplazadas; una entrada en barra defectuosa pasa a la siguiente válida. Repreciador **no verificado** para cortos, comisión no nula y swap de forex en puntos (`engines/market/reprice.py` docstring; z robusto, `e` en R no).

Orden de aplicación

7.1 → P1 → P2 → P3 → P4 → P5 → P6 → P7 → P8 → P9 (criteria.yaml:127-184). Resultado de la estrategia = su **peor** regla. No hay orden de «cae en la primera»: se calculan todas; el juez lee el peor.

Criterio 7.1 — Mercados válidos de la lista

- **Pregunta:** ¿la lista de mercados es utilizable para medir?
- **Definición:** `invalid_fraction = 1 − markets_valid/markets` (`travel/run.py:346-348`), `markets` = los de la lista (incluye ausentes del export: `universe.absent` → `not_in_export`). Pass `<= 0.25`, `else: limbo` (criteria.yaml:139-140). Nunca fail. **Marca**.
- Un mercado es inválido (`validity.check`, `validity.py:31-69`) si: no hay ficha de costes o está pendiente (`no_cost_card` / `cost_card_pending`); `fill_mismatch` = mediana (firmada por lado) de |precio registrado − apertura de barra| en unidades de ATR mediana > `max_fill_error 0,25` (se mide sobre las operaciones de TODAS las estrategias del export juntas); valor del punto de la ficha vs. pendiente de regresión P/L ~ lado·movimiento·tamaño con diferencia > `point_value_margin 0,02` (2 %); o el mercado no está en el export. Barras planas (ATR < `flat_atr_fraction 0,02` × ATR mediano) se excluyen del tamaño pero no invalidan.
- **Por qué:** control previo (§7.1 de la propuesta del dueño). **Punto débil:** umbrales (0,25 ATR, 2 %, 20 %) fijados sin calibración; la lista se evalúa una vez por export, no por estrategia (todas las estrategias heredan el mismo `invalid_fraction`, no discrimina).
- **Ejemplo real:** USATEC 2026-10-11: 11 mercados, 10 válidos, `invalid_fraction = 0,0909`; XCUUSD inválido por `fill_mismatch 0.406 ATR`. Resultado: pasa. Otras corridas: XCUUSD 0,41-0,58 ATR, XAGUSD 0,48-0,68, AUDNZD 0,27 inválidos «en todas las corridas que los incluyen» (`knowhow/pipeline/step10-p1-p7-never-have-data.md`).
- **Coste:** un cómputo por export (no por estrategia). ⚠ XCUUSD y XAGUSD son estructurales de casi todos los mains y salen siempre inválidos: el control es de hecho una constante.

Criterio P1 — Reglas sin distancias fijas

- **Pregunta:** ¿la regla usa distancias fijas en puntos/pips que no escalan entre mercados?
- **Definición:** `fixed_distance` = nº de distancias fijas del `.sqx` (`sqx/inspect/strategymeta/fixed_distance.fixed_distances`), localizado por `rglob` en `raw/<P>/` y en los databanks de los workers (`travel/units.py`), aceptado solo si la identidad del `.sqx` coincide; NaN si no se encuentra (no 0). Pass `<= 0`, `else: limbo` (criteria.yaml:142-143). Nunca fail. **Marca**. Decisión del dueño 2026-10-07: «P1 se marca y no saca nada» (no hay gancho en el paso 9: 28 de 6.000 estrategias); `DeadZonePip` del WAE y `#StopLossInPips#` no cuentan (350 de 6.000 llevan `DeadZonePip`).
- **Por qué:** una distancia en pips tiene otro significado en otro mercado. **Punto débil:** detector por claves del XML, no verificado frente a todos los bloques; NaN → limbo (sin dato = limbo).
- **Ejemplo real:** USATEC: `fixed_distance = 0` en las 3 → pasa. Histórico: hasta 2026-10-09 P1 era «sin dato» para todas (buscaba solo en `raw/`, donde no hay `.sqx` en el paso 10); arreglado (tarjeta citada).
- **Coste:** abrir cada `.sqx` (zip); sin dato de tiempo propio.

Criterio P2 — Frecuencia de operar en cada mercado

- **Pregunta:** ¿opera con una frecuencia comparable en cada mercado frente a la que tuvo en su main?
- **Definición:** por estrategia y mercado, `z_freq = (ln(r_j/r_main) − m_pop,j) / sqrt(φ (1/N_j + 1/N_main))` (`travel/frequency.py`), `r` = trades/año de la ventana propia de cada mercado (main: build+oos1), `m_pop,j` = mediana de `ln(r_j/r_main)` sobre las estrategias del export (`centred.centre`, mínimo 2 estrategias, si no el centro es 0), `φ` = sobredispersión de los conteos anuales del main en el build, acotada a `[phi_min 1,5; phi_max 3,5]`. Un mercado queda «fuera» si `|z_freq| >`

`3,5` **o** sin ninguna operación (`run.py:258`). `freq_off` = nº de mercados fuera; `freq_off_fraction = freq_off / mercados_válidos` . Regla compuesta `all` : `freq_off <= 0` pass, `<= 2` limbo, más = fail; y `freq_off_fraction <= 0.25` pass, si no fail (criteria.yaml:145-148). **Corta** (fail con ≥3 mercados fuera o >25 %).

- **Por qué:** una estrategia que en otro mercado opera 5× menos no se está comportando como la misma. **Puntos débiles:** (a) centrado en la mediana de la población del export: con 3 estrategias la mediana pasa exactamente por una → `z_freq` de esa = 0,000 (visto en DAX40 y USA500 en la corrida real, ver «Ejemplo»); inútil con poblaciones pequeñas; (b) `φ` fijado en el rango [1,5; 3,5] sin ajuste fino, aproximación normal del log-ratio de Poisson sobredisperso; (c) la fracción 0,25 redunda con el conteo absoluto (con 10 mercados, 3 de 10 = 0,30 ya falla por el conteo; con 11, 3 mercados = 0,27); (d) no corrige por multiplicidad (11 mercados × z>3,5 es conservador).
- **Si faltan las operaciones del main** (`harvest/.../trades.parquet`) `freq_off = NaN` → limbo.
- **Ejemplo real:** USATEC: `freq_off = 0` en las 3 (pasa); z_freq por mercado de 1.11.277: AUS200 -0,11, DAX40 0,000, DJ30 0,15, EURUSD -0,46, HK50 -0,03, NIKKEI +0,19... (`markets.parquet`).
- **Coste:** vectorizado sobre conteos; sin dato propio.

Criterio P3 — Mejor que el azar en el conjunto

- **Pregunta:** sumando todos los mercados, ¿gana más que operar las mismas operaciones en fechas aleatorias?
- **Definición:** `z_conj = stat.joint(tensor, cells = V0+V1+V2)` (`rules.py:154-`), bruto, sobre los mercados válidos, B=5000. Regla: pass `>= 2.33` , limbo `>= 0` , fail `< 0` (criteria.yaml:150-151). **Corta** (fail si z<0). `p_conj` se escribe pero ninguna regla lo lee.
- **Por qué:** una exposición conjunta frente a un nulo con la misma estructura (misma dependencia entre mercados porque el desplazamiento es común). 2,33 = 1 % unilateral (convención del Council), 0 = «al menos no peor que el azar».
- **Puntos débiles:** (a) la ventana `all` incluye **V1 = el build del main**, años donde la estrategia se seleccionó; otros mercados en esos mismos años correlacionan con el main (factor común), de modo que parte de z_conj es una réplica del sesgo de selección, no evidencia nueva; (b) V0 (años anteriores al build del main) es la única ventana realmente «no vista» y ninguna regla la lee (⚠: en la corrida, `z_conj_v0` de 1.11.277 = -2,70 sin efecto); (c) sin corrección por multiplicidad ni por selección previa en el paso 8; (d) con z<0 → fail, así que alrededor del 50 % de una población sin ventaja (z centrada en ≈0, sd≈1,08) cae por esta regla sola; fail es «peor que el azar», no «sin ventaja»; (e) bruto frente a neto: las diez de referencia daban `net_t_pooled` negativo (medias -1,7 a -4,1) mientras el bruto z_conj era positivo (`knowhow/pipeline/step10-p1-p7-never-have-data.md`): P3 aprueba estrategias que en neto pierden en otros mercados.
- **Ejemplo real (USATEC 2026-10-11):** 1.11.277: z_conj 0,138 (limbo); 1.13.407: -0,240 (**fail**); 6.22.453: 0,369 (limbo). Referencia (10 estrategias NIKKEI M30, 2026-10-04): media z_conj +1,45 sobre bruto tras arreglos (+1,72 antes; sesgos del aparato +0,3 a +0,45; población de ATR alto +0,30; V0 -0,56, V1 +0,45, V2 +0,14); cinco de diez ≥2,33 antes del arreglo. Tras el corte: 0 pass, 5 limbo, 5 fail.
- **Coste:** el tensor (runs×mercados×celdas) se reconstruye por estrategia; ver tiempo total al final.

Criterio P4 — Sigue en los años que no vio

- **Pregunta:** ¿la ventaja se mantiene en los años de oos1 del main?
- **Definición:** `z_conj_v2` = `joint` sobre celdas V2 solo. Pass `>= 0` , limbo `> -1.65` , fail `<= -1.65` (criteria.yaml:153-154). **Corta**.
- **Por qué:** oos1 del main es un tramo no usado al construir (pero sí al seleccionar en el paso 8, ⚠ en el sentido de que ya pasó una puerta en esos años). **Puntos débiles:** pocos años (p.ej. 2020-2023 en USATEC = 4 años; celdas anuales); el nombre «años que no vio» solo es cierto para el main: los otros mercados en 2020-2023 son los mismos años calendario en los que el main ya fue filtrado por el paso 8 (dependencia entre mercados); fail con z ≤ -1,65 es unilateral al 5 %, sin multiplicidad respecto a P3.
- **Ejemplo:** 1.11.277 +0,428 (pass), 1.13.407 -0,562 (limbo), 6.22.453 +0,296 (pass).

Criterio P5 — No depende de un solo mercado

- **Pregunta:** quitando cada mercado en turno, ¿sigue la evidencia conjunta?
- **Definición:** `z_minus_market` = el **mínimo** sobre mercados j de `z` del conjunto sin j (`rules.minus_market` , l.44); guarda cuál (`minus_market`). Pass `>= 1.65` , `else: limbo` (criteria.yaml:156-157). Nunca fail. **Marca**. NaN con <2 mercados.
- **Punto débil:** mínimo de 10 z correlacionadas = estadístico sesgado a la baja (extremo de una muestra), y el umbral 1,65 se compara con él como si fuera una z sola; con z_conj ≈ 0 (caso habitual) el mínimo es negativo y siempre limbo. En la corrida, `minus_market` fue USA500_M1 en las tres (el mercado más próximo a USATEC: quitarlo baja la evidencia, lo que es esperable pues es el más correlacionado, no una «dependencia» en sentido de fragilidad).
- **Ejemplo:** -0,151 / -0,836 / -0,032 (los tres limbo).

Criterio P6 — Amplitud entre mercados

- **Pregunta:** ¿la dirección positiva se repite en más mercados de los que da la suerte?

- **Definición:** `rules.breadth` (l.58): `seen` = nº de mercados cuyo total real supera la **mediana** de sus corridas al azar; `p_breadth = (1 + #corridas con C_b >= seen) / (B+1)`, leyendo cada corrida al azar como si fuera la real (misma dependencia). Un mercado sin operaciones nunca supera su mediana (0 no es > 0). Pass `<= 0.10`, `else: limbo` (criteria.yaml:159-160). Nunca fail. **Marca**. Bruto.
- **Por qué:** sustituye el binomial/Katz del dossier viejo (que suponía mercados independientes) por un nulo que conserva la dependencia. **Punto débil:** usa la mediana del nulo, no el cero: es un test de signo robusto pero poco potente; p<=0,10 es laxo (10 %) y como solo marca (no corta) tiene poco coste.
- **Ejemplo:** breadth_count 7, 5, 5 de 10 → p = 0,263 / 0,599 / 0,584 (limbo los tres).

Criterio P7 — Rendimiento mínimo en la mitad de los mercados

- **Pregunta:** ¿al menos la mitad de los mercados retiene una fracción razonable del edge del main?
- **Definición:** `tau_half` (`rules.tau_half`, l.74) = el `ceil(M/2)`-ésimo mayor de `e_j / e_principal`; `e_j` = (real neto − media del nulo neto)/n_j en R por operación sobre la ventana completa; `e_principal` = `e` del main sobre V2 (su oos1), calculado por el propio estudio con el mismo repreciado/desplazamiento (`run._e_principal`, desde 2026-10-09); NaN si `e_principal <= 0` o sin ficha; mercados con <30 operaciones (`min_trades_market`) o `e` no finito = -inf; -inf si menos de la mitad quedan en pie. Pass `>= 0.5`, limbo `> 0`, `missing: limbo`; el resto (<=0) fail (criteria.yaml:162-164). **Corta** (tau<=0).
- **Por qué:** retención del exceso frente a «casa» (la mitad de los mercados a ≥0,5× el edge del main). **Puntos débiles:** (a) el denominador es el edge del main en **oos1 solo** (pocas operaciones, ruidoso) y el numerador en **toda la ventana** (V0+V1+V2) de cada mercado: ventanas y tamaños distintos; (b) cociente de estimaciones ruidosas: un `e_principal` pequeño positivo infla tau; (c) «mediana de ratios» ignora la dependencia; (d) casi nadie pasa: sobre 25 estrategias (DAX40 M30, cal5) 1 pass, 24 limbo, tau mediana 0,19 (max 0,53); 17 (GBPUSD H1, cal4): 0 pass, mediana 0,08 (`knowhow/pipeline/step10-p1-p7-never-have-data.md`). Con `net` (e_end/e) el coste cae sobre el numerador y no sobre el denominador.
- **Ejemplo (USATEC):** e_principal 0,0722 / 0,0933 / 0,0817 R; tau_half 0,254 / 0,070 / 0,089 → limbo, limbo, limbo.
- **Histórico:** hasta 2026-10-09 P7 era «sin dato» para todas (esperaba un productor de `e_principal` del paso 8 que nunca se escribió) → todas limbo por el `missing: limbo`.

Criterio P8 — Mercados en contra

- **Pregunta:** ¿en cuántos mercados lo hace claramente peor que el azar?
- **Definición:** `worse` (`rules.worse`, l.92) = nº de mercados con `trades >= 30` y `z_j <= -1,65` (`worse_z`); `worse_fraction = worse / nº filas de mercados válidos`. `all`: `worse <= 1` pass, `<= 2` limbo, más fail; `worse_fraction <= 0.25` pass, si no fail (criteria.yaml:165-168). **Corta** (≥3 mercados o >25 %).
- **Punto débil:** un z unilateral a -1,65 por mercado entre 10 mercados correlacionados: bajo la nula, el nº esperado de mercados «en contra» es ≈0,5 pero muy correlacionado entre mercados; el umbral «3 o más» no está calibrado bajo la dependencia real. Redunda con P3 (si P3 es negativa, normalmente hay varios mercados en contra).
- **Ejemplo:** worse 0 / 2 / 0; worse_fraction 0 / 0,2 / 0. 1.13.407 queda en limbo por P8 (2 mercados).

Criterio P9 — Degradación en el tiempo (relajada)

- **Pregunta:** ¿la ventaja por operación decae con los años?
- **Definición** (`rules.degradation`, l.110): por año natural de cierre sobre la ventana completa, `d_y = (real_y − media_null_y)/n_y`; años con <30 operaciones se agrupan con el siguiente (cola con el último grupo; grupo en la media ponderada de años); <4 grupos (`degradation.min_years`) = sin dato (NaN en los cuatro). `beta` = pendiente ponderada por n de `d` frente a año; `z_beta = (beta − media(beta_null)) / sd(beta_null)` (cada corrida da su propia pendiente, bruto); `e_end` = ventaja proyectada en el último año con datos (**neto**); `e_end_ratio = e_end / media`. Regla (criteria.yaml:175-184): pasa si `z_beta > -1.65`, o (`z_beta > -2.33` y `e_end > 0`); falla si `z_beta <= -2.33` **y** `e_end <= 0`; limbo lo demás (incluido sin dato). **Corta** solo con ambas condiciones.
- **Por qué:** ventaja que se evapora = muerta. **Puntos débiles:** pendiente lineal sobre ~4-10 puntos, `z_beta` con nulo de pendientes (válido para el signo, pero sensible al agrupado variable); mezcla bruto (z) y neto (e_end); con edge positivo y e_end>0 casi nunca falla.
- **Ejemplo (USATEC):** z_beta 0,991 / 0,518 / 0,550; e_end 0,0234 / 0,0104 / 0,0177 (todos pass).

Orden de aplicación y ejemplo de cadena

Se calculan todas a la vez y se toma la peor. USATEC 2026-10-11 (`paso-10/verdict.csv`): 3 → 2 (1 fail por P3; 2 limbo: 1.11.277 por P3, P5, P6, P7; 6.22.453 por P3, P5, P6, P7; 1.13.407 fail P3 + limbo P4, P5, P6, P7, P8). El juez: 3 → 2 «por criterios, 2 en limbo». Referencia anterior (dossier 51): 10 → 0 pass, 5 limbo, 5 fail.

Coste / recursos del paso 10

`crossmarket.json › wall_s = 17,75 s` para 3 estrategias × 10 mercados × 5000 corridas; paso entero 21 s (`tiempos.md`). Otros: 5-18 s (1-3 estrategias). Escala por estrategia: el tensor se construye por estrategia (`run.py:339-355`) y `e_principal` se recalcula también por estrategia; para cientos de estrategias **sin dato** (nunca corrido); RAM `(5000 runs × 11 mercados × ~100 celdas)` float64 ≈ 44 MB por tensor + `real_net` / `null_net`. Núcleo en Python/numpy, un solo proceso (cores: sin dato).

Discrepancias del paso 10

- ⚠ **El dossier `criterios-paso-10-otros-mercados-2026-10-06.md` (1147 l.) describe el criterio ANTERIOR**: «regla 1 (la mitad de los mercados a 0,5× el original)», «regla 2 (t conjunta)», nulos por mercado, Stouffer, Katz, Fieller. Ese código se borró el 2026-10-07; el criterio vivo son las P1-P9 + 7.1 de arriba. Las cifras de ese dossier (ejemplos NIKKEI M30, 2026-10-04) no corresponden al código actual.
- ⚠ Los encargos 46 y 47 (citados por README, comentarios de `criteria.yaml` y código) **ya no existen** (borrados el 2026-10-09; consolidados en el 51). `criteria.yaml:133` aún cita `criterios-paso-10-otros-mercados-propuesta.md`, que no está en `docs/AgentPDFs/`.
- ⚠ `verdict.csv` de crossmarket trae la columna `verdict = info` por estrategia (el estudio «no decide»); el veredicto real es el del juez. El README anterior decía lo contrario (dossier §5.10).
- ⚠ `z_conj_v0`, `z_conj_v1`, `p_conj`, `net_t_pooled`, `e_end_ratio`, `beta`, `breadth_count` se calculan y se escriben pero **ninguna regla los lee**; en la corrida, `z_conj_v0 = -2,70` (1.11.277) y `net_t_pooled = -1,76 / -2,75 / -1,79` sin consecuencia.
- ⚠ El paso 10 se juzga sobre `segment: oos1` (criteria.yaml:129) pero P3 incluye la ventana V1 = build del main: coherente solo en el sentido de «lo que el juez escribe en el ledger».

PASO 10.5 — Preparación crossTF (fabricar hermanas escaladas)

Propósito. Convertir cada madre superviviente del 10 en «hermanas»: la misma estrategia con parámetros reescalados a otras temporalidades, para que el 11 las reprueba en SQX.

Entradas. `.sqx` de las madres (databank `Retest Markets - Family`, supervivientes del 10), timeframe de la madre (de la tarea `CrossTF`), asset. Doctrina `assets/_build.yaml:157-172` (`crosstf.timeframes`: M15 → M10, M30, H1; M30 → M15, H1, H4; H1 → M30, H4; H4 → M30, H1, H12; **D1 eliminado 2026-10-09**).

Salidas. `AlgoData/crosstf/<P>/<día>/{sqx/*.sqx (madres + hermanas), scaling.parquet}`; `CrossTF_Input` rellenado con madres+hermanas; una tarea SQX por timeframe extra (`CrossTF <TF>`) y su gemela de fase 2 `CrossTF <TF> build`.

Implementación (`sqx/variants/scale.py`, `sqx/projects/crosstfload.py:load_input`). - Cociente de periodos: `ratio = minutos(destino)/minutos(origen)`; cada periodo → `exacto = original/ratio`, redondeado al entero (mitades hacia arriba) con **suelo = `floor`** de `sqx/variants/config.yaml`; se registra `rounding_shift = |escrito − exacto|/exacto` y `clamped` (el suelo rescató el periodo). Desde 2026-10-02 («Redondea donde puedas, ya está») redondeo y suelo **no descartan** la hermana. - Qué se escala (regla del dueño 2026-10-07, encargo 47 F): periodos (incluyendo ParamTypePeriod y las longitudes de bloques custom, `lengths.yaml`), salidas por nº de barras, y los coeficientes que multiplican una **medida por barra** (`ranges.yaml`: tipo `bar`, p.ej. ATRMult de SuperTrend, desviación de Keltner, stop/objetivo `X·ATR`) divididos por `c`; `c` = (ATR mediano en el TF de la hermana con periodo escalado)/(ATR mediano en el TF de la madre) sobre build+oos1 del activo (`ranges.measured`; NIKKEI225: M30→M15 0,704, →H1 1,419, →H4 2,833). Tipos `window` (Bollinger sobre N barras ya escalado) y `other` (umbrales sin unidad de precio) no se mueven; un coeficiente `unverified` o un bloque custom no listado deja la hermana marcada `unverified` (T0 no la lee). Un `bar/c` puede salir de `maxValue` (PMax 10,99 → 15,6). - La ATR scaled necesita `c`: si el símbolo no está en `assets/` los coeficientes no se tocan y se marca. - Sibling name `<madre>_Scaled<TF>`, identificador `CTF<index><TF>`; las madres se copian sin tocar al mismo folder (son la base de la comparación). - Las tareas por TF: un `RetestOnAdditionalMarkets` por timeframe, motor MetaTrader 5 (hedged) para todos (sin motor propio por TF), `precision 1`, ventana `oos1` solo (`crosstf.segment: oos1`, `dates="true"`: manda la ventana del test principal de la tarea), costes de `oos1`.

Criterios. Ninguno (fabricación determinista). Los diagnósticos que dejan (`rounding_shift`, `clamped`, `unverified`, `ratio`, `atr_ratio`) los lee T0 del 11.5/12 y se anotan como hechos.

Ejemplo real (USATEC H1, 2 madres × 2 destinos): M30: `ratio 0,5`, `n_scaled 4`, `rounding_shift 0,0`; H4: `ratio 4,0`, `n_scaled 4`, `rounding_shift 0,2`, `clamped 0`, `unverified 0` para ambas madres. 2 madres × 2 TF → 4 hermanas (6 `.sqx` en `CrossTF_Input`).

Coste. Sin fila propia en `tiempos.md` (de la corrida USATEC se imputa dentro de «11 sqx», ver paso 11); no verificado.

Discrepancias. ⚠ la propiedad «declarada» «el stop no se escala» (dossier 12 §5.15; stop H1 ≈1,4× más ancho que M30) fue **sustituida** el 2026-10-07 por el escalado `bar/c`; el dossier no lo refleja. En este momento los bloques de las plantillas llevan stop/objetivo apagados (`stop_loss` / `profit_target` false; el stop se injerta en el paso 24), así que solo se mueven coeficientes de condiciones.

PASO 11 — Retest crossTimeframes en SQX

Propósito. Que SQX reprueba, en oos1 y a costes de oos1, las madres y sus hermanas (y el «control» = parámetros de la hermana sobre las barras de la madre; y la madre «sin escalar» en otros TF).

Entradas. `CrossTF_Input` (madres + hermanas). **Salidas.** Databank `CrossTF` (el TF de la madre) y `CrossTF_<TF>` por TF extra; export de operaciones; `blocks.json` (mapa tarea → timeframe → databank) junto a `scaling.parquet`.

Implementación. `sqx/projects/crosstf.py` (`set_main` : fija la tarea en `crosstf.segment = oos1` , `precision 1` , cross-check apagado; `wire` : una tarea por TF extra). Cada TF es **una tarea y un databank propio** (dueño, 2026-09-30): como bloques de una sola tarea el export no se puede separar con el mismo símbolo (el k-ésimo ticket etiquetaba el bloque por velocidad, no por TF; `knowhow/export/data-all-blocks.md`).

Criterios. Ninguno. **Ejemplo real** (USATEC): `CrossTF: 0 → 6` (2 madres + 4 hermanas). **Coste:** 88 s (SQX 2 s, export 80 s, copias 4 s; `tiempos.md`); otros 78-83 s.

PASO 11.5 — Criba de fase 1 de crossTF en Python (sin corte)

Propósito. Con lo que ya hay en oos1, calcular T0, T2a y T3 para preparar la fase 2; quitaría las madres inservibles de `CrossTF_Input` antes del 11.7. **Hoy no corta a nadie** (`criteria.yaml:185-194` , `rules: []` , «hasta el "corta" del dueño», encargo 47).

Entradas. Export del paso 11 (operaciones de madres y hermanas en oos1), `scaling.parquet` , barras de cada TF, ficha de costes. **Salidas.** `reports/<P>/CrossTF/<día>/crossTF/{cells.parquet, tf_siblings.parquet, tf_mothers.csv, distribuciones.parquet, verdict.csv, crossTF.json}` ; hechos `crossTF.tf.*` (pass=1, limbo=0, fail=-1; `pipeline/autopilot/tffacts.py`). En la corrida: 476 hechos, «ninguno: el paso no tiene criterio».

El mismo estudio (`python3 -m studies.transfer.crossTF.report --project P --asset A`) se ejecuta dos veces: 11.5 (fase 1, sin build) y 12 (fase 2, con build). Las pruebas T0-T4 se calculan en ambos con los datos que hay; `T1` y `T2b` leen «sin dato» (`no_build_data`) en la fase 1.

Núcleo común de T1-T4 (`studies/transfer/crossTF/tf/` , `config.yaml › tf:`)

- **Nulo conjunto:** una madre y todas sus hermanas pasan por las **mismas** corridas: un desplazamiento de semanas enteras por año natural y corrida (seed 20261007, B=5000), nunca cruzando la frontera build/oos1, repreciadas por el repreciador único al tamaño de la doctrina, en R (`tf/null.py` ; un timeframe ocupa el lugar de un mercado en el tensor de `crossmarket/travel`). Una celda con hueco de datos en CUALQUIER timeframe se descarta en todos (`flat_atr_fraction 0,02` , `max_displaced_share 0,20`). z se lee sobre bruto (`judge_on: gross`); `e` , `ret` , `ret_build` sobre neto. Resultado de la batería sintética (300 madres sin ventaja): media z -0,09..+0,02, sd 0,93-1,04, rechazo al 5 % entre 3,0 y 5,7 % (`tests/test_crosstf_tf_null.py:30,98`).
- **Vecinas/lejana** (`config.yaml:185-186`): `neighbours` M15: M10, M30 · M30: M15, H1 · H1: M30, H4 · H4: H1, H12; `far` M15: H1 · M30: H4 · H4: M30 (H1 sin lejana). Las filas M15 y H4 fueron fijadas por Claude por analogía; **el dueño no las ha confirmado** (README crossTF). Cualquier otra hermana solo se anota (`role: note`).
- Cada ventana tiene sus operaciones: oos1 (paso 11) y `build` (11.7); las del build de la madre salen del `harvest` del gate (`Results`) por identidad.

T0 — Comparable

- **Pregunta:** ¿la hermana sigue siendo «la misma estrategia» tras la fabricación?
- **Definición:** `comparable` (`tf/rules.py:9`): razón = la primera de `unverified` , `clamped` , `rounding_shift > 0,25` . Además, `min_trades 30` en la ventana que cada test lee (build+oos1 juntos para T1). Una hermana no comparable da `limbo` con su razón en `*_why` .
- **Por qué:** una hermana con un periodo redondeado >25 % o con coeficientes no verificados es otra estrategia. **Punto débil:** ⚠ el dueño decidió el 2026-10-02 que redondeo y suelo **no** descartan (el estudio viejo `verdict.py`), pero T0 de la lectura nueva sí lee `clamped` y `rounding_shift > 0,25` : dos criterios de comparabilidad distintos conviven. Calibración vieja: hermanas redondeadas/recortadas no muestran inflación de FP (4,5-5,7 % vs 4,0-5,2 %; n pequeño, no se descarta un punto de diferencia).
- **Corta/Marca/Informa:** Marca (limbo). **Ejemplo:** USATEC H4: `rounding_shift 0,2 < 0,25` → comparable; M30: 0,0.

T1 — Frecuencia de operar (build + oos1)

- **Pregunta:** ¿la hermana opera con una frecuencia coherente con la de su madre y con la población de hermanas del mismo TF?
- **Definición:** `z_frec = (ln(n_hermana/n_madre) − m_pop) / sqrt(φ (1/n_hermana + 1/n_madre))` (`rules.z_frec` , l.51), `n` sobre build+oos1, `m_pop` = mediana (`rules.centre`) de ese log-ratio entre las hermanas comparables **de ese timeframe en la corrida** (mínimo `min_population 5` ; por debajo `population_below_min` → limbo), `φ` = sobredispersión de los conteos anuales de

la madre en el build acotada a [1,5; 3,5]. `t1`: `|z_frec| <= 1,5` pass; `> 3,5` fail; entre medias limbo (`rules.t1`). Una madre falla T1 si **alguna** vecina falla (`s1 = fail if any`), pasa solo con todas las vecinas esperadas medidas y todas pass. Fase 2 (necesita build): en fase 1 es «sin dato».

- **Por qué:** misma lógica que P2 del paso 10 (frecuencia entre mercados) aplicada a TF. **Puntos débiles:** el centro `m_pop` exige ≥5 madres comparables del mismo TF en la misma corrida; con la población real de hoy (2-3 madres) **T1 es siempre «sin dato»** (visto: `population_below_min` en las 4 hermanas de la corrida); un valor de φ casi siempre en el techo 3,5 (en la corrida: φ=3,5 las 4); frecuencia y rendimiento se mezclan para hermanas con dos TF distintos; sin multiplicidad.
- **Ejemplo:** USATEC: t1 = limbo (`population_below_min`) en ambas madres.

T2a — Retención en oos1

- **Pregunta:** ¿cuánto del exceso sobre el azar por operación de la madre conserva la hermana en oos1?
- **Definición:** `e = (T_real − media(T_azar))/n` en R neto por operación; `ret = e_hermana / e_madre` (vacío si `e_madre <= 0`, motivo `mother_e_not_positive`); `z_delta = (delta_real − media(delta_azar))/sd(delta_azar)`, con `delta = e_madre − e_hermana` por operación (judged: bruto), mismas corridas emparejadas (la correlación madre-hermana está dentro del sd; si la hermana ES la madre, z_delta=0). `t2a`: `ret >= 0,5` pass; `ret <= 0` y `z_delta >= 1,65` fail; lo demás limbo (`rules.t2a`). Para la madre: «pico aislado en oos1» (fail) = ambas vecinas fallan, o una falla y la otra tiene `ret < 0,25` (`retention.low`), con todas las vecinas esperadas medidas (`rules.mother`). Una vecina sola que falla (no hay dos) queda limbo con `t2a_lone_fail` (decisión K-2 abierta).
- **Por qué:** la pregunta ya no es «la hermana tiene z alto» (lectura vieja) sino «¿la ventaja de la madre es un pico aislado en su TF?». **Puntos débiles:** (a) `ret` se mide en el R de **cada timeframe** (el tamaño por doctrina usa el ATR de cada TF: una hermana que gana el mismo precio por operación que una madre M30 lee `ret ≈ 0,57` en H1 y ≈ 1,70 en M15, `knowhow/research/crosstf-mother-sibling-dependence.md`; K-1 pendiente del dueño): el umbral 0,5 y 0,25 no son invariantes a escala; (b) cociente de dos estimaciones ruidosas, con un denominador (ventaja de la madre en oos1) que ya está seleccionado al alza por el paso 8; (c) sin multiplicidad sobre 2 vecinas × 3 tests.
- **Ejemplo:** USATEC 1.11.277: e_madre 0,0730; M30 e 0,0541 → ret 0,740 (pass); H4 e 0,0434 → ret 0,594 (pass); z_delta 0,69 / 0,83. 6.22.453: e_madre 0,0829; M30 ret 1,216 (pass), H4 ret 0,134 (limbo), z_delta 2,24.

T2b — Hermana frente al azar en el build (solo fase 2)

- **Definición:** `z_build` = z del total de la hermana (en su ventana build) frente a sus propias corridas al azar; `t2b`: `z_build >= 1,65` pass; `<= 0` fail; si no limbo. Para la madre: pico aislado en el build = **ambas** vecinas `<= 0` (`rules._both`). Sin build: `no_build_data`.
- **Por qué:** comprobar que la hermana funciona sola donde la madre se construyó. **Puntos débiles:** ⚠ todas las vecinas deben estar medidas y coincidir; con una madre y dos vecinas, un fail+limbo da limbo; el build de la hermana no es «libre» (comparte régimen con el build de la madre, optimizado sobre él): un z_build alto es parcialmente heredado. Bug corregido 2026-10-09: `z_build` de ±10-19 era cada databank de fase 2 mezclado bajo un solo nombre (`knowhow/pipeline/step12-zbuild-not-centred-and-d1-sister-empty.md`); los hechos de corridas anteriores a la corrección (n_build, e_build, z_build, ret_build, phi, z_frec, t1, t2b) son erróneos; z_delta, ret, t2a y z_new nunca se tocaron.
- **Ejemplo:** USATEC: z_build 1,24 / 0,87 (M30), 0,96 / 0,98 (H4): todas limbo (entre 0 y 1,65).

T3 — Freno de la lejana

- **Definición:** si la hermana `far` es comparable y su `t2a` falla, un PASS de la madre baja a LIMBO (`rules.mother`: `brakes`); solo baja, nunca sube. **Marca**. USATEC (H1): sin hermana `far` (H1 no tiene) → `t3 = pass`.

T4 — Solape (solo anota)

- `overlap_*` = proporción de entradas de la hermana a menos de **una barra de la madre** de una entrada de la madre (`rules.overlap`), por ventana. K-3 abierto (el dossier §0.1 usa «menos de una hora» para M30). Informa; no entra en `state`. Ejemplo: M30 0,61-0,62 (oos1), H4 0,006-0,009.

Combinación → estado de la madre

`state = peor(t1, t2a, t2b)` con T3 bajando pass a limbo; `phase1 = t2a` capado por T3 (lo que el 11.5 vería sin build). Ejemplo: USATEC 1.11.277 `phase1 = pass`, `state = limbo` (T1 y T2b sin dato); 6.22.453 `phase1 = limbo` (H4: ret 0,134 → limbo) `state = limbo`. **Hoy no se corta**: la regla lista para pegar vive en `docs/encargos/51-...md` (§4, «Paquete del corte»).

Coste. 11.5: 39 s (USATEC, 2 madres, 4 hermanas, B=5000 con y sin build); resto sin dato; 0 de SQX.

Discrepancias. ⚠ La lectura nueva está implementada pero **NO es la regla viva**; su validación con datos reales de SQX está pendiente (OPEN.md #108; frente 4 del encargo 51). ⚠ K-1/K-2/K-3 pendientes del dueño. ⚠ El rango vecinas/lejana de M15 y H4 no está confirmado.

PASO 11.7 — Retest crossTF en el build en SQX (fase 2)

Propósito. Obtener las operaciones de cada hermana en la ventana `build` (T1 y T2b las necesitan); solo para las madres que siguen en `CrossTF_Input` (sin corte hoy: todas). Se salta con una línea en un proyecto sin tareas de fase 2 (`plan.py`).

Implementación. `crosstfload.phase2_tasks(cfx)` → tareas `CrossTF <TF> build` (`sqx/projects/workflow.py:28` : `segment: build` , databank `CrossTF_<TF>_build`), gemelas de cada tarea de TF copiadas sobre la ventana `build` y a costes del build; mismo input. Se ejecutan con `stage.just` . **Criterios:** ninguno. **Ejemplo:** `CrossTF M30 build: 0 → 6` y `CrossTF H4 build: 0 → 6` (cada databank trae todo `CrossTF_Input` : 2 madres + 4 hermanas, ⚠ de ahí el bug de pooling corregido). **Coste:** 42 s (SQX 1 s, export 36 s).

PASO 12 — Análisis crossTFs en Python (el que corta)

Propósito. Juzgar por madre, con sus hermanas, si el edge «sobrevive» a otras temporalidades más allá de lo que explica su madre; y decidir qué pasa al 13. Cortar `CrossTF` antes de que el 13 copie las madres a `CrossTF_Mothers` : solo sigue la madre, en su timeframe, sin escalar; las hermanas siempre se van (dueño, 2026-10-01).

Entradas. Export de `CrossTF` (+ bases de fase 2 si existen), `scaling.parquet` , `blocks.json` , barras de cada TF. **Salidas.** Las mismas del 11.5 (con build) + `verdict.csv` del juez + hechos `crossTF.combined.*` , `crossTF.cells.<role>.<tf>.*` , `crossTF.tf.*` .

Criterio vivo — `crossTF.combined.z_new` (la ÚNICA regla de corte)

- **Pregunta:** ¿las hermanas escaladas de esta madre aportan, en conjunto, evidencia nueva más allá de lo que su madre ya explica?
- **Definición exacta** (`cells.py:162 beyond` , criteria.yaml:195-200):
- por celda (`cells.numbers` , l.32-): estadística `sharpe` por operación (`config.yaml:172`), precio revalorado por `engines.nulls.simulate` desde barras (nunca el P/L de SQX), nulo `timing` (baraja solo *cuándo* entra cada operación, en las barras de ese timeframe; `rung: timing`), `engines/nulls/config.yaml` : `draws` 100000 por defecto (⚠ el estudio usa su propio `nulls.draws` ; no verificado el valor efectivo en esta corrida), `seed` fresco por corrida (`nulls_seed 257690917` en el manifiesto) salvo `--seed` ; `z = (seen − media_null)/sd_null` , `p` empírico unilateral; mínimo `usable.min_trades 30` (menos → sin `p` ni `z`); celda sin operaciones → «no operó».
- `corr_vs_baseline` ρ = correlación de Pearson del P/L **mensual** de la hermana frente a su madre (meses sin operar = 0), sobre el span de cierres de todo el export (`cells.correlations`); recortada a `[0; 0,95]` (`MOST_ALIKE`), NaN → 0.
- `z_new = (z_hermana − ρ·z_madre) / sqrt(1 − ρ²)` por hermana escalada (`cells.py:184`).
- `combined_z_new` de la madre = suma de `z_new` de sus hermanas **comparables** (`unverified == 0` ; el `fair` filter no mira `clamped` ni `rounding_shift`) dividida entre `sqrt(n)` (`combined_n_tested = n`); `combined_z_new_max` = máximo (solo anota).
- Regla: pass si `combined_z_new >= 2.33` ; `else: limbo` (incluye sin dato). **Nunca fail.** Marca; el corte solo quita hermanas.
- **Por qué (justificación declarada):** una hermana re-lee el mismo activo en los mismos años; bajo no-edge su z sigue la de su madre con correlación ρ (0,5-0,95 para el factor 2; 0,3-0,7 para H4); `z_new` es N(0,1) bajo no-edge sea cual sea la suerte de la madre; dos hermanas aportan cantidades casi no correlacionadas bajo el nulo (0,03 y 0,07), de modo que su suma/√n es una z. 2,33 = 1 % unilateral (Council); «Raw z never counts».
- **Medido (calibración 2026-10-02, 68 trabajos de nulo, semilla 20261002; `knowhow/research/crosstf-mother-sibling-dependence.md`):** z sin corregir da 17-31 % de FP donde se espera 5 % (dada una madre afortunada); `z_new` devuelve la tasa a 2,7-6,3 %; `combined.z_new > 1,65` en 3,6-5,7 %. Una banda ρ > 0,9: `z_new` 10 % FP con madre afortunada (n = 39). A 2,33: **sin FP medidos**. La calibración **ya no está en disco** (`AlgoData/scratch/calib_step12/` no existe): las cifras solo se pueden citar de la ficha.
- **Puntos débiles estadísticos visibles:** 1. ρ sale de ~48 meses de P/L mensual (error ≈ 0,15 cerca de 0; ≈ 0,03 con ρ = 0,9) y es la correlación del dinero, no la de las z; pareja a pareja es ruidoso (`z_new` explota si ρ sube a 0,95 y el denominador √(1−ρ²) = 0,31). 2. Con ρ bajo, `z_new ≈ z` y lo que se mide es otra lotería del 5 % de la hermana, no confirmación de la ventaja de la madre; con ρ alto no se puede aprobar (dossier §5.7, razonamiento no medido). 3. La combinada **premia tener menos pruebas**: una hermana muda/sin datos/no utilizable no entra; la suma/√n con n=1 (1,72) supera a n=2 (1,67 y 0,16 → 1,30); no mira `combined_n_tested` . 4. Con las dos hermanas habituales (M30 y H4 de una madre H1) la regla pide `(z1+z2)/√2 >= 2,33` = suma ≥ 3,30: casi inalcanzable (en 10 madres NIKKEI M30: máximo +1,72, mediana -0,05; 0 pass, 8 limbo con dato, 2 sin dato). 5. El nulo `timing` conserva el conjunto de oportunidades; el baraje depende de la semilla (con 2.500 monos el error de media/sd del nulo ≈ 2 %); sin `--seed` una madre pegada a la línea puede cruzarla de una corrida a otra. 6. El ρ y el z_madre vienen del mismo oos1 en el que la madre ya pasó el paso 8: el z_madre ya está seleccionado al alza (p=0,033 en las dos madres de la corrida; 1,85 y 1,86 de z), lo que baja `z_new` de las hermanas por el término `ρ·z_madre` : por ejemplo con ρ=0,90 y z_madre=1,85, una hermana con z=1,36 da

z_new=-0,72. 7. Comparabilidad: `combined` excluye `unverified` pero incluye hermanas `clamped` y con `rounding_shift` hasta lo que sea (`fair = scaled & unverified==0`); T0 (lectura nueva) sí las trata distinto. 8. Sin corrección por multiplicidad entre madres, ni entre este paso y el 10 («los pasos 10 y 12 son una sola prueba», dossier 12 §2.11: nada los cuenta juntos). 9. El P/L de SQX en el sentido del signo `fails` (lectura de hermanas) usa Sharpe revalorado sin spread ni slippage (dossier §5.1).

- **Ejemplo real (USATEC 2026-10-11, `paso-12/verdict.csv` , 6 filas):** madre 1.11.277 (H1 baseline: 258 op., sharpe/op. 0,137, p 0,0332, z 1,85): M30 escalada 272 op., z 1,36, p 0,089, ρ 0,90, `z_new -0,715` ; H4 escalada 168 op., z 1,33, p 0,094, ρ 0,57, `z_new +0,332` ; `combined.z_new = (−0,715+0,332)/√2 = −0,271` → limbo (MANTENER). Madre 6.22.453 (z 1,86): M30 z 1,68 p 0,048 `z_new −0,069` ; H4 z 0,40 `z_new −1,588` → `combined −1,172` → limbo. Las 4 hermanas: DESCARTAR «solo sigue la madre». Lectura por hermana: las 4 «inherited». **Cadena: 6 → 2** (2 madres en limbo, 4 hermanas fuera por regla de hermanas); 0 madres eliminadas. Sin hermana «survives» (p<0,05 **y** `z_new>1,645`).
- **Coste:** 41 s (python) + 3 s (juez) en esa corrida; XAUUSD H1 wf4s: 57 s; 4-54 s en otras. Cores/RAM: sin dato. Cada celda reprecia sus barras y ejecuta el nulo timing: coste crece con (nº de hermanas × nº de madres × `draws`); sin dato de escalado.

Lecturas por hermana (`verdict.py`) — informan, no cortan

Seis lecturas, en orden: `unusable` (longitud de bloque custom sin regla, `unverified`), `silent` (0 operaciones), `no_data` (<30 operaciones o `z_new` NaN), `survives` (`p < 0,05` **y** `z_new > 1,645` **y** `seen > 0`), `inherited` (`seen > 0` sin más), `fails` (`seen <= 0`). El primer trío significa «la prueba no ocurrió». `n_survives` no lo lee nadie (dossier §5.2). El **control** (parámetros de la hermana sobre las barras de la madre) se muestra y no se lee: falló 29 de 46 lecturas reales y se invierte con baseline negativo.

Discrepancias del paso 12 y de 10.5-11.7

- ⚠ **El dossier `criterios-paso-12-...` describe la lectura vieja (`z_new`)**, que sigue viva; **no cubre** la lectura nueva (T0-T4) ni 11.5/11.7. El dossier menciona D1 y la propiedad «el stop no se escala»; **ambos caducados** (D1 retirado 2026-10-09; escalado `bar/c` desde 2026-10-07).
- ⚠ **`verdict.py` comenta «5 %»** donde lo medido es 2,7-6,3 % (dossier §5.8); el README mezcla cifra teórica (14-40 %) con la de la primera medición (30 % y 3,7 %). Las del dossier 12 §2.6: 17-31 % y 2,7-6,3 %.
- ⚠ Las 4 hermanas de la corrida salen `rounding_shift 0,2` en H4 y T0 pasa (límite 0,25); el estudio viejo documentaba umbral 0,15 retirado.
- ⚠ **El criterio vivo (`z_new >= 2,33` , combinada) y la lectura nueva (T0-T4) miden preguntas distintas** y la nueva espera «corta»; mientras, el 11.5 no corta nada y el 12 nunca elimina madres.
- ⚠ Los encargos 47 y 46 (fuente de la lectura nueva) ya no están; el plan vivo es `docs/encargos/51-...md` .
- ⚠ `z_build` y los hechos derivados de corridas anteriores a 2026-10-09 son inválidos (bug de pooling, ya corregido).
- ⚠ Cifra de «draws» de los nulos del paso 12 (`engines/nulls/config.yaml` : 100000 por defecto; el dossier cita «2.500 monos») no verificada contra la corrida.

Cadena de ejemplo completa (Test_USATEC_H1_ruptura_leaseWeb, custodio, 2026-10-11)

paso	qué	n entran → n salen	segundos
8 (dev)	juez: sorteo de 3 (semilla 2281916898), 3 en limbo	10000 → 3	18 (juez)
9	SQX Retest Markets - Family	0 → 3 (11 mercados, 10 válidos)	34
10	crossmarket + juez	3 → 2 (1 fail P3)	21 + 0
10.5 + 11	fabricar + SQX CrossTF	0 → 6 (2 madres + 4 hermanas)	88
11.5	crossTF fase 1 (sin corte)	6 → 6	39
11.7	SQX build (M30, H4)	0 → 6 cada una	42
12	crossTF fase 2 + juez	6 → 2 (4 hermanas fuera, 2 madres limbo)	41 + 3

El run completo duró 167,2 min (SQX 38,0 min, 77 % el resto), de los cuales los pasos 9-12 son ≈ 4,5 min (270 s); la parte pesada de la corrida fue el paso 8 (falsePositives 4170 s). Heap: custodio 80 g, resto de RAM/cores: sin dato en los logs.

Notas de población (importante para cualquier calibración)

- Todos los runs reales de 9-12 tienen 1-25 estrategias (la mayor, 25 en DAX40 M30 cal5, 17 en GBPUSD H1 cal4); dev-sample 3 en el 8.
- La población de referencia (10, `Trade_NIKKEI225_M30_volatilidadL` , 2026-10-04): paso 10 → 0 pass, 5 limbo, 5 fail; paso 12 viejo → 0 pass, 10 limbo.
- Nada de `ledger/thresholds.yaml` toca estos pasos (no hay líneas de crossmarket/crossTF); todo vive en `criteria.yaml` y `config.yaml` de cada estudio (editables desde la ventana).

Pasos 13, 14, 15 y 16 — tal como están implementados y corridos (lectura 2026-10-11)

Fuentes: código (`studies/breakage/mcRetest/`, `studies/breakage/spp/`, `sqx/variants/neighbours*.py`, `pipeline/autopilot/{neighbours,judge,criteria}.py`), `pipeline/autopilot/criteria.yaml` (working tree = HEAD en estos bloques), `assets/_build.yaml`, `sqx/projects/stages.yaml`, y corridas en `AlgoData/autopilot/<P>/<stamp>/` y `AlgoData/raw/<P>/T1_Neighbours/`. Los dossiers `criterios-paso-14-mc-retest-2026-10-07.md` y `criterios-paso-16-spp-2026-10-07.md` se usan sólo para el porqué.

Nota de método sobre las corridas: de las 10 corridas recientes con paso 14, **solo 4 juzgaron de verdad** (`Test_NIKKEI225_H1_aroonL_mc1b` 2026-10-09, `Test_USA500_H1_tendencia_wf1`, `Test_DAX40_H1_reversion_wf2`, `Test_NIKKEI225_M30_volatilidad_wf3`, las tres de 2026-10-10). Las otras 6 (`wf4s`, `mc2`, `mc3`, `mc4`, `mc5`, `10k`) llevan carpeta `smoke/` (`--smoke`): `judge.settings()` filtra las reglas a `READABLE` (solo hechos de la puerta nivel 0, `judge.py:40-43`), y el resumen dice «ninguno: el paso no tiene criterio». En ellas yo he **reaplicado** las reglas a mano sobre los parquet (marcado «reaplicado»); no es el veredicto de la corrida.

PASO 13 — MC Retest en SQX (más el lote de vecinas, T1)

Propósito

Generar la evidencia de fragilidad que lee el 14: (a) T1, las «vecinas»: cada madre re-ejecutada con todos sus parámetros movidos a la vez, medida exactamente por Python; (b) T3 y T2, dos perturbaciones del propio SQX (`MCR 1 Bar`, `MCR 7 OHLC`). No corta nada.

Entradas

Madres que dejó el paso 12 en el databank CrossTF (sin las hermanas escaladas): `pipeline/autopilot/neighbours.py:entering`. Una sola población entra en todas las tareas MCR (no se encadenan).

Orden dentro del autopiloto

juez 12 → `13 neighbours` (Python + SQX, lote de vecinas) → `13 sqx` (MCR) → 14 python → 14 juez → 15. Una sola sesión en el custodio.

Sub-etapa 13a: lote de vecinas (T1) — `sqx/variants/neighbourbatch.py`, `neighbours.py`, `config.yaml:162-177`

- Definición: por madre, `n = 500` sorteos; **todos los parámetros declarados se mueven a la vez** (variable numérica con `<paramType>`; fuera: `MagicNumber`, los de tipo shift y los que casan `fixed`: `(Shift|Shf)\d*$` y selectores `(Method|Mth|Mode|Mde|Type|AppliedPrice|AppPrc|Fld|Str)\d*$`). Cada uno `x·(1+u)`, `u ~ U(−p, +p)` independiente, `p = 15 %` entrada/indicador (`entry_pct`), `30 %` salidas (`exit_pct`, tipo `ParamTypeExitUsed`). Enteros redondeados y con suelo 1 (`int_floor`); dobles a 6 cifras significativas.
- Duplicados (tupla igual a la madre o a una anterior tras redondear) se **descartan y no se re-sortean** (`n` es un tope). Semilla `[0, crc32(nombre madre)]`: cada madre su sorteo, reproducible.
- La madre viaja en el lote como vecina 0 (sus bytes intactos) para reprecificarse igual que las vecinas.
- Se re-testean en dos tramos: `WFC 1 IS` (build) y `WFC 2 OOS1`; jamás `WFC 3 OOS2` (`neighbours.legs`; el código aborta si el tramo está reservado, `neighbours.py:run`). Se usa el mecanismo de WFC legs del 16.5 sin los mercados adicionales.
- Por qué: rediseño del dueño 2026-10-07 («especificación» T1). Hipercubo uniforme de ±15 %/±30 % como noción de «mundo ligeramente distinto» de los parámetros. Punto débil: ver T1 en el paso 14.
- Corrida real (tiempos de `T1 make/execute/export/measure`):
- 3 madres, 1.503 ficheros: `wf4s` 4/42/37/4 s (total fase 94 s); `wf3` 4/40/35/3 s (90 s); `wf1` 4/38/36/3 s (88 s); `10k` 11/101/31/4 s (154 s).
- 2 madres/1.002 ficheros (`wf2`): 3/29/29/2 s (69 s).
- 9 madres/4.183 ficheros (`mc1b`): 7/92/66/6 s (179 s).
- 50 madres/20.866 ficheros (`mc2`): 33/447/290/34 s (844 s) = ~17 s por madre; escala casi lineal.
- Cores/RAM de la fase: sin dato (no hay `timing` por fase de cores/RAM; solo segundos de reloj en `tiempos.md` / `fases.csv`).
- Ficheros devueltos: la mediana por madre es 501 (500 + madre) pero hay madres con muchos menos: NIKKEI225 H1 `14.3.380` n=62, `1.8.268` n=299 (parámetros enteros pequeños: muchos redondeos colapsan en duplicados que se descartan; esa madre mide con una muestra de vecinas mucho menor).

Sub-etapa 13b: tareas MCR de SQX — `sqx/projects/mcretest.py`, `assets/_build.yaml:237-293`, `stages.yaml:21`

Etapa activa (dueño 2026-10-07): `[MCR 1 Bar, MCR 4 MinDist, MCR 7 OHLC]`. `MCR 2, 3, 5, 6, 8` siguen definidas en `_build.yaml` pero **no se corren** (un proyecto viejo las lleva apagadas).

tarea	perturbación (valores vigentes)	ventana	sims	precisión	para qué
MCR 1 Bar	`RandomizeStartingBar MaxChange 500`	build	200	1 (solo TF elegido)	T3: control de la barra inicial
MCR 4 MinDist	`RandomizeMinDistance` (rango del activo; en XAUUSD «SIN DECIDIR»)	build	100	1	solo si la población lleva órdenes stop/limit
MCR 7 OHLC	`RandomizeHistoryDataOHLC` Prob. O/H/L/C 30 %, `MaxChange 15`, ATR 14, `KeepConnected`, gap 30 % / 15 %	build	500	1	T2: histórico retocado

- Todas con aceptación silenciada (no filtran; son evidencia). `MCR 7 OHLC` va **la última** a propósito: el builder encadena `SPP IS` a la tarea anterior y el juez 14 corta ese databank.
- `MCR 4 MinDist`: en las 11 corridas vistas «NO configurada — la población no lleva órdenes stop ni limit» (órdenes a mercado, regla 15) → `0 → 0`, sin datos; el ingest sigue (solo `bar` es obligatoria).
- ⚠ Comentario inconsistente en `_build.yaml` (MCR 2 dice «un minuto — lento y fino» con `precision: 1`, que por su propia definición es «solo TF elegido»): sin efecto, esa tarea no corre.
- Punto débil: 200 y 500 simulaciones; el dossier avisa de que por debajo de ~200 el percentil 5 es ruido. T3 con 200 sims y MaxChange 500: ver abajo, no discrimina.
- Coste real (`tiempos.md`, fila «13 sqx»): `wf4s` 39 s totales (8 s de SQX, export 25 s) para 3 madres con 1.399-2.098 simulaciones; `USATEC wf` 1.399 simulaciones; `mc2` 125 s para 50 madres; `10k` 282 s. Ingest: «6 runs, 2.098 simulations» (3 madres) en `wf4s`. Cores/RAM: sin dato; `knowhow/perf/workflow-step-durations.md` (2026-09-29) dice que el MC Retest era el mayor coste SQX cuando corría con 8 tareas y 1.000 sims; con el rediseño ya no lo es.

Salidas

`raw/<P>/T1_Neighbours/<día>/neighbours.parquet` (una fila por fichero: `strategy, variant_id, neighbour, <seg>_excess/_profit/_drift/_trades`) y `raw/<P>/MCR_All/<día>/` (ingest con P/L por simulación, `original_pnl`, trades del build con fechas).

PASO 14 — Análisis MC Retest en Python (fragilidad = retención del EXCESO sobre la deriva)

Código: `studies/breakage/mcRetest/` (`neighbours/t1.py`, `neighbours/excess.py`, `model/retention.py`, `measure/excess.py`, `model/tails.py`, `contract/tabs.py`). Reglas: `pipeline/autopilot/criteria.yaml:201-224`. Juez: `pipeline/autopilot/judge.py`, `criteria.py`.

Propósito

Responder «¿es frágil?»: si la ventaja de la madre, medida como exceso sobre lo que el mercado paga por estar dentro, sobrevive cuando se perturban parámetros (T1), el histórico (T2) o la barra de arranque (T3). Pasar es «no es frágil», no «tiene ventaja».

Entradas

Las tablas de vecinas (13a), el ingest MCR (13b), el catálogo de barras (`barstore`) y los costes del activo (`reprice.costs`). Población juzgada: databank `MCR 7 OHLC` (`cut:`), `segment: oos1` (etiqueta del ledger; ver ⚠5).

Medida común: el «exceso»

- Por operación: `excess = profit_repreciado − drift`. El P/L se re-precia con el mismo repricer que el paso 8 (`engines.market.reprice.real`, costes de la ficha del activo por tramo); la deriva es `tradelevel.drift_term` con `drift.window = year`, `drift.variant = d` (`studies/screening/gate/config.yaml:57-64`: movimiento medio por año de calendario **y** operación ciega de la misma duración ponderada 1/ATR). T1 y el paso 8 usan literalmente la misma configuración (lee `GATE`), para restar lo mismo.
- Retención (`model/retention.py`): `ret_med = mediana(totales_re-ejecuciones) / total_original`, `ret_p5 = p5 / original`, `f_pos = share(re-ejecuciones con total > 0)`, `n`. Con original ≤ 0 o sin ejecuciones, las dos retenciones son NaN (sin dato, nunca fallo por sí mismo); `f_pos` solo es NaN si `n = 0`.
- Tratamiento de nulos en el juez (`criteria.py:judged`): un hecho NaN/ausente cae en la `missing:` de su regla (aquí `limbo`); un compuesto `all` toma el peor; el resultado de la estrategia es la peor de todas sus reglas (`outcomes`). Una regla con `pass` y `limbo` y sin `else`: lo que no llega a ninguno es **Falla**.

Criterio 14.1 — T1 build: «las vecinas conservan el exceso» (CORTA)

- Pregunta: ¿qué fracción del exceso del build de la madre conserva la vecina típica de sus parámetros?
- Definición: `all` de dos reglas, `mcRetest.fragilidad.t1.build.ret_med` y `.f_pos`, sobre las 500 vecinas re-ejecutadas en el tramo **build** (`criteria.yaml:213-216`):
- `ret_med`: Pasa ≥ 0,6; Limbo ≥ 0,3; Falla < 0,3; sin dato → Limbo.
- `f_pos`: Pasa ≥ 0,9; Limbo ≥ 0,7; Falla < 0,7; sin dato → Limbo.
- Combinación `all` = la peor. Resultado: Pasa solo con ret_med ≥ 0,6 **y** f_pos ≥ 0,9; Falla si cualquiera cae por debajo de su suelo de Falla (0,3 / 0,7).
- Facts calculados además y **no juzgados**: `t1.build.ret_p5`, `ret_benef` (retención de beneficio sin restar deriva), `beta_share` (deriva / beneficio de la madre; NaN si la madre no gana), `t1.rho_vec` (Spearman entre exceso build y oos1 de las vecinas), `n`.
- Por qué (declarado): umbrales de la propuesta del dueño, **PROVISIONALES, sin calibrar con población nula ni plantada** (cabecera del bloque). Restar la deriva evita que una meseta falsa «aguante» solo por el mercado (`ret_benef` alta con `ret_med` baja).
- Puntos débiles estadísticos: 1. Son umbrales sin calibrar; no hay tasa de falsos positivos ni potencia medida. 2. La retención es relativa a la madre, que está seleccionada como óptimo del build (sesgo de selección): una vecina promedio tiende a retener < 1 por regresión a la media aunque no haya fragilidad; y puede ser > 1 si la madre quedó lejos del óptimo (visto: `17.13.284` ret_med 1,28). 3. Sorteo uniforme independiente en un hipercubo de dimensión = nº de parámetros (5-10+): la vecina típica está lejos de la madre en norma conjunta (concentración de la medida); mezcla efecto de cada parámetro con interacciones, sin descomponer qué parámetro rompe. 4. Cociente de dos totales: inestable cuando el exceso de la madre es pequeño (hay valores de −44, −97 en `mc2`); el denominador puede ser mínimo y positivo. 5. Con la madre no positiva en exceso (`ret_med` NaN) manda `f_pos` (< 0,7 casi siempre) → **Falla**: mide que el exceso del build es ≤ 0, no fragilidad. En `mc1b` 5 de 9 y en `mc2` 29 de 50 madres tenían el exceso del build ≤ 0 (cola de índices alcistas largos: la deriva del build, 2012-19/2008-17 en largos, se come el beneficio; `beta_share` hasta 16). Esas estrategias pasaron el paso 8 (exceso medido en **oos1**) y mueren aquí por una medida **en build**. 6. `f_pos` con n = 500 tiene error estándar ≈ 0,013 en 0,9; con n pequeño (62, 299 por duplicados) más. Vecinas no independientes entre sí ni de la madre. 7. No hay corrección de multiplicidad entre estrategias (50 madres × 3-4 reglas).
- Ejemplo real (juzgado; `wf3`, NIKKEI225 M30 largas, 2026-10-10, 3 madres): `14.6.463` ret_med 0,3094 (Limbo) y f_pos 0,676 (<0,7 → Falla) → **Falla**; `17.13.284` 1,282/0,940 → Pasa; `20.11.454` 0,921/0,982 → Pasa. `wf2` DAX40 H1: `10.20.278` ret_med 0,7165 (Pasa) pero f_pos 0,646 → Falla; `14.16.487` 0,880/1,000 → Pasa. `wf1` USA500 H1: `18.5.471` 0,133/0,570 → Falla; `20.17.293` 0,599/0,964 → Limbo (a 0,001 del 0,6); `7.11.427` 0,786/0,992 → Pasa. `mc1b` NIKKEI225 H1 aroon: 9 de 9 Fallan (4 por ret_med < 0,3: −0,15, −0,09, −1,56, −1,02; 5 por f_pos con ret_med NaN).
- Reaplicado sobre tablas de otras corridas (smoke, no juzgadas por la corrida): `mc3` DAX40 M30 10 madres: 6 Pasan, 3 Limbo, 1 Falla; `mc4` 3 madres: 1/0/2; `mc2` 50 madres: 0 Pasan, 3 Limbo, 47 Fallan (30 por f_pos con ret_med no negativo o NaN; 17 por ret_med < 0,3); `wf4s` XAUUSD H1: 1 Pasa, 2 Limbo; `USATEC wf` 2 de 2 Pasan.
- Coste: ver 13a (`measure` 2-34 s). Es el criterio que más corta (en juzgadas, 3 de 4 corridas mataron al menos una madre por T1).

Criterio 14.2 — T1 oos1: «las vecinas siguen en positivo» (CORTA, solo descarta)

- Definición: `t1.oos1.ret_med > 0` → Pasa; si no, **Falla** (no hay línea de Limbo); sin dato → Limbo. Mismo cálculo que 14.1 pero con las vecinas re-ejecutadas en el tramo **oos1** y el exceso oos1 de la madre como denominador.
- Por qué: «la vecina típica no puede perder el exceso fuera de muestra».
- Débiles: (a) usa oos1, que ya ha servido para todos los pasos 8-12 (el dossier del paso 8 reserva oos2, no oos1), así que esto añade una mirada más sobre oos1 (multiplicidad no contada); (b) si el exceso oos1 de la madre es ≤ 0 → NaN → Limbo (en `mc3` 3 de 10, `wf4s` 3 de 3), con lo que el criterio queda desactivado justo donde sería más informativo; (c) con la madre apenas positiva el cociente es enorme y de signo arbitrario (−18,6, −10,4, +4,9 vistos): `> 0` solo mira el signo de la mediana, pero el signo de un cociente con denominador casi 0 depende del denominador no de la fragilidad; (d) mediana > 0 es un umbral laxísimo: en `wf2` `10.20.278` retiene 0,35 en oos1 y pasa.
- Ejemplo real: no fue decisivo en ninguna de las 4 corridas juzgadas (los 11 resultados `t1.oos1.ret_med` de `wf1/wf2/wf3` son todos > 0: 0,31-0,84); en las reaplicadas, `mc3` 4 madres con ret_med ≤ 0 (Fallarían por este criterio) y `mc4` 2.

Criterio 14.3 — T2 build: «aguanta otro histórico» (CORTA)

- Definición: `t2.build.ret_med_aprox`: Pasa ≥ 0,6; Limbo ≥ 0,25; Falla < 0,25; sin dato → Limbo (`criteria.yaml:218-219`). Fuente: `MCR 7 OHLC` (500 sims, build). Fórmula (`measure/excess.py:t2`): `retention(total_sim − deriva_original, P/L_original − deriva_original)`, es decir el exceso de cada re-ejecución es **su beneficio total menos la deriva que cobró el ORIGINAL** (una re-ejecución del MCR no trae fechas, así que no puede calcular la suya). «Aprox.» porque asume que la re-ejecución opera casi cuando el original. Soporte medido del «casi»: `count_dev_med` 0,2-0,8 % (mediana de la desviación del nº de operaciones respecto al original) y `same_count_frac` solo 5-29 % de las sims tienen exactamente el mismo nº de operaciones. Si las operaciones de la tabla fechada no coinciden una a una con el original SQX (tolerancia `recon.absolute_floor` 0,02 USD), no hay dato.
- Por qué: se pidió medir fragilidad ante histórico retocado con la misma métrica que T1.

- Débiles: aproximación declarada (la deriva no sigue a las operaciones que cambian de fecha); usa el P/L de SQX con sus costes, no el repricer común, mientras T1 sí; la perturbación (OHLC ±15 %, 30 % de barras) es una sola fuente; solo build; mediana ignora la cola (`ret_p5_aprox` se anota sin umbral); 500 sims.
- Ejemplo real (juzgado): `wf3` 0,848 / 1,050 / 1,034; `wf2` 0,894 / 0,989; `wf1` 0,788 / 0,875 / 0,955: todos Pasa. Distribución sobre las 10 corridas con hechos (52 estrategias con dato): mediana 0,954, p25 0,694, mín −14,5; 8 por debajo de 0,6 y 4 por debajo de 0,25 (las que fallarían). 36 de 88 sin dato (madre sin exceso positivo).
- Coste: cabe en «14 python» (≈ 31-38 s para 3-50 madres; `wf4s` 33 s, `mc2` 38 s, `10k` 30 s).

Criterio 14.4 — T3 build: «control de la barra inicial» (NUNCA corta)

- Definición: `t3.build.ret_med` : Pasa ≥ 0,8; todo lo demás Limbo (`else: limbo`); sin dato → Limbo. Fuente `MCR 1 Bar` (200 sims, `RandomizeStartingBar MaxChange 500` , build). Cálculo (`measure/excess.py:t3` , `model/tails.py`): como la re-ejecución carece de fechas, se **fecha por la cola que comparte con el original** (`common_suffix` , mínimo `fragility.min_tail = 20` operaciones; `window_excess` suma el exceso del original en la ventana común y el de la cabecera no compartida). La ventana común arranca `MaxChange` barras después de la primera barra del build. Sin dato si menos de `min_matched = 0,5` de las sims se pudieron fechar o si todas dan un único resultado.
- Por qué: control de que una perturbación irrelevante (donde mover la barra de arranque) no cambie el resultado; sirve de denominador de dispersión (`attribution.control_task: bar`) y de «prueba de que el método no se inventa fragilidad».
- Punto débil importante (medido): **no discrimina**. 58 valores en 10 corridas: mín 1,000, mediana 1,000, máx 1,108; `identical_frac` mediana 0,85 (las re-ejecuciones son idénticas al original dentro de la ventana común). Es estructural (`knowhow/sqx-format/mcr-start-bar-reruns-identical.md` , □ 2026-10-07: 24 de 24 con exceso positivo). Por eso la regla no puede fallar nunca y su única función real es dar Limbo por falta de datos. `ret_p5` varía (0,91-1,00) pero no se juzga.
- Ejemplo real: `wf3` 1,000/1,000/1,000 (`distinct` 53/41/38, `identical_frac` 1,0/1,0/0,88); `wf2` 1,002/1,022 (identical 0,355/0,18); `wf1` 1,000 en las tres.
- Coste: dentro del mismo «14 python».

Orden de aplicación en el juez

`judge.judge("14")` : reúne hechos (`facts.gather` , 298-1.043 hechos por corrida de 2-10 madres), parte los candidatos en madres y hermanas (hermanas escaladas ya salieron en el 12), evalúa las 4 reglas por madre y toma la peor; `Falla` ⇒ se **borra del databank** `MCR 7 OHLC` (`apply_verdict`); `Limbo` ⇒ sigue marcada (anotada en el resumen); fila en el ledger (paso 14, segmento `oos1` , `n_in` , `n_out` , sha256 de `criteria.yaml`). En modo desarrollo (`dev.on`), una muestra aleatoria recorta los supervivientes (no activo en las corridas reales vistas).

Lo que el estudio calcula y NO se juzga (histórico del veredicto propio)

`mcRetest.verdict.*` (`composite` , `score` , `produccion` , `ejecucion` , `especificacion` , `datos` , `pass`), 288 hechos por estrategia: cuatro preguntas (cola de pérdidas, modo de caída, atribución, evidencia) con vetos (`gates.py` : `min_pf 1,10` , `exec_keep_frac 0,70` , `collapse_frac 0,50` , `survival_dd_pct 0,25` , `psr_gate 0,90`) y nota compuesta (pesos 0,55/0,20/0,20/0,05; tiers 80/65/50). Sin la tarea `stress` (apagada el 2026-10-07) el veredicto del estudio es INCONCLUSO, nunca FALLA (`gates.production`): en `wf4s` `verdict.pass=0` , `produccion=0` , `score=44,5` en `1.14.418` . **No decide nada**; el paso lo juzgan las reglas de `criteria.yaml` .

Ejemplo de cadena (corridas juzgadas)

corrida	entran al 13	tras 14	detalle
`wf3` NIKKEI225 M30 (2026-10-10)	3	2	1 Falla (T1 f_pos)
`wf2` DAX40 H1 (2026-10-10)	2	1	1 Falla (T1 f_pos)
`wf1` USA500 H1 (2026-10-10)	3	2	1 Falla (T1), 1 Limbo (ret_med 0,5991) que sigue
`mc1b` NIKKEI225 H1 aroon (2026-10-09)	9	0	todas Fallan (ret_med < 0,3 o exceso build ≤ 0)

Tiempos y recursos (paso 13+14, 3 madres, `wf4s` 2026-10-11)

13 neighbours 94 s + 13 sqx 39 s (SQX 8 s) + 14 python 33 s + 14 juez 0 s ≈ 166 s. A 50 madres: 844 + 125 s + 38 s ≈ 17 min. Cores/RAM por fase: sin dato. El ingest usa `ingest.workers: 16` (medido 2026-09-25: 1,9 s y 1,9 GB con 40 runs).

Discrepancias (⚠)

1. ⚠ `docs/AgentPDFs/criterios-paso-14-mc-retest-2026-10-07.md` describe la regla antigua (`cut: MCR 8 Stress` , una sola regla `PF p5 del estrés ≥ 1,0` , 8 tareas, `MCR 8 Stress` como hecho). **El código y** `criteria.yaml` **ya no la tienen**: ahora T1/T2/T3 sobre `MCR 7 OHLC` , 3 tareas MCR. El dossier hay que leerlo solo para el porqué del estudio.

2. ⚠ `WORKFLOW.md` (fila 14) dice «T3 ... mediana 1,000 medida en 24 estrategias»; ahora hay 58 valores de 10 corridas, mínimo 1,000: confirma que T3 no discrimina.
3. ⚠ 6 de las 10 corridas recientes (todas las que llevan `smoke/`) **no juzgaron el 14** (ni el 12, ni el 10): «ninguno: el paso no tiene criterio». No es un fallo del código, es el modo `--smoke`; pero el único ejemplo de cadena con el filtro real es el de las 4 corridas sin `smoke`.
4. ⚠ Sin población nula ni plantada: los umbrales 0,6/0,3/0,9/0,7/0,25 y 0,8 no tienen tasa de falsos positivos medida (lo dice el propio YAML).
5. ⚠ `segment: oos1` en `criteria.yaml:204` es solo la etiqueta del ledger; el paso lee **build** (T1 build, T2, T3) y oos1 (T1 oos1). Ya señalado en el dossier §5.3.
6. ⚠ T1 build mide en build exceso sobre deriva, mientras el paso 8 mide R3 en oos1: una estrategia con buen exceso oos1 y exceso build ≤ 0 muere aquí (visto en NIKKEI225 H1 largas: 29/50 y 5/9) sin que sea fragilidad.

PASO 15 — SPPs en SQX

Propósito y definición

Dos tareas SPP (System Parameter Permutation) por proyecto, sin juicio: el perfil es un mapa, no un filtro (aceptación silenciada). `sqx/projects/spp.py`, `assets/_build.yaml:397-404`. - `SPP IS`: ventana build, lee el databank `MCR 7 OHLC` (los supervivientes del 14); `SPP OOS`: ventana oos1, encadenada a `SPP IS`. `precision: 1`. - Rejilla: cada parámetro se mueve ±35 % (`spread_pct 35`) en pasos de `step_pct 4` → 18 pasos (`Steps = 2·spread/step`); `max_tests: 15000` (el donante trae 1.000.000.001 = exhaustivo; hay que fijarlo). Con 5-9 parámetros el producto de niveles es astronómico, así que SQX **muestrea** 15.000 permutaciones; tras descartar duplicados (parámetros enteros redondean a la misma tupla) quedan `n_eff` ≈ 8.000-11.200 (ver 16). - Proyectos viejos tienen `SPP IS` apuntando a `MCR 8 Stress` (ya nada lo rellena): `python3 -m sqx.projects.spp <SYM> --cfx <cfx> --input "MCR 7 OHLC"` con el worker parado (WORKFLOW.md fila 15). - Una SPP a la vez por instalación (el perfil ocupa decenas de GB mientras se construye, medido 2026-09-22). Los tres ajustes `EvalProfitOptCheck`, `EvalAvgProfitCheck`, `EvalUniformDistrCheck`, `EvalTopProfitCheck` y las condiciones se apagan («0 condicion(es) y 2 chequeo(s) del SPP apagados»). - No corta; el databank de salida `SPP OOS` es el que cortará el paso 20 y leerá el 21. - Punto débil: la ventana IS repite el build en el que la madre se construyó y la OOS repite oos1 (no es fuera de muestra «virgen»: ya la vieron los pasos 8-12 y 14). El dossier lo llama «SPP OOS deja de ser OOS». - Costes reales (`tiempos.md` / `resumen.md`, fila «15 sqx»): `wf4s` 275 s (SQX 100 s, export 60 s) para 3 madres; `wf3` 295 s (2 madres); `wf2` 216 s (1); `wf1` 256 s (2); `10k` USDJPY M30 **2.606 s** (3 madres); sin dato de cores/RAM (el perfil es muy intensivo en RAM; en master medido «decenas de GB»).

PASO 16 — Análisis SPPs en Python

Código: `studies/breakage/spp/` (`run.py`, `one.py`, `model/{mapa,decay,profile,influence,combine}.py`, `verdict/noise.py`, `criteria.py`, `config.yaml`). Regla: `criteria.yaml:225-229` → `cut: SPP OOS`, `segment: oos1`, **`rules: []`** (comentario: «Council: FAIL-only on a peak solution / grid mean − 1 sd < 0; neither is exposed as a fact yet, so no rule»).

Propósito

(a) Reconocimiento de la rejilla SPP: qué parámetros mueven el resultado y cuáles son inertes (para el diseño de variantes del 16.5); (b) mapa de la ventana build y de oos1 con cuatro lecturas; (c) «métricas para criterios» (decaimiento IS→OOS1, original frente a su rejilla), calculadas **sin umbral y sin juzgar**.

Salida y decisión

El paso 16 no corta nada hoy: en las 10 corridas el juez dice «ninguno: el paso no tiene criterio» (con o sin `--smoke`). Produce hechos (707 por corrida de 3 madres), `design_brief_<estrategia>.json` (consumido por `sqx/variants/make.py`, 16.5) y `verdict.csv`. Ninguna regla del autopiloto lee `spp.*` (grep sobre `pipeline/autopilot`: sin coincidencias).

Criterio 16.1 — Veredicto de ruido (sólo INFORMA, y alimenta nada)

- Pregunta: ¿el mejor punto de la rejilla supera lo que daría buscar entre puro ruido? (`verdict/noise.py:noise_check`).
- Definición: sobre la métrica `read.metric = ReturnDDRatio` (Ret/DD, el Fitness de SQX), tuplas distintas (`dedupe.distinct`, por `NetProfit` + `NumberOfTrades`), `n_eff = nº de distintas`; `σ = std` (ddof 0) de los valores; `umbral = σ·√(2·ln n_eff)`; `exceso = max − media`; **PROCEED** (`pass=1`) si `exceso > umbral·margin` con `margin = 1,0`; si no **NOISE** (`pass=0`); `score = exceso/umbral`. Sin suelo de operaciones en este veredicto (dueño 2026-10-04, «sin suelo»; `knowhow/research/spp-noise-gate-centred-no-floor.md`). Se descartan las filas centinela (`RExpectancy` ±99999/-1; `SENTINEL_LIMIT 100`).
- Facts relacionados: `spp.verdict.{pass,score,area_de_meseta,max_mediana_iqr,tuplas_distintas_n_eff}`; `area_de_meseta` = share de tuplas con métrica > 0; `max_mediana_iqr` = (máx − mediana)/IQR («spike_ratio»).

- Por qué: cota de máximo de n normales independientes (`√(2 ln N)`): «si no hay parámetro que haga nada, el mejor de N sigue cayendo cerca de media + σ√(2 ln N)».
- Débiles: (1) los N puntos de una rejilla SPP son altamente dependientes (vecinas) y no normales: el máximo de un campo suave no se parece al de N independientes; la cota √(2 ln N) ≈ 4,2σ con N≈9.000 casi nunca se supera en una superficie suave, así que el veredicto tiende a NOISE (inferencia mía, no verificada con población nula); (2) usa `std` global incluyendo la dispersión estructural entre parámetros, no la de ruido; (3) los umbrales dependen de la métrica (Ret/DD está muy sesgada); (4) `margin 1,0` sin calibrar.
- Ejemplo real (`wf4s` , XAUUSD H1 ruptura, 2026-10-11, 3 madres, Ret/DD): `1.14.418` n_eff 8.693, score 0,842 → **NOISE** (`pass=0`); `5.2.399` n_eff 11.211, score 1,349 → PROCEED; `9.19.426` n_eff 8.001, score 1,160 → PROCEED. Misma comprobación aplicada a Sharpe sobre la rejilla recortada (`build.sobresale.grid.*` , y `oos1.sobresale.grid.*`): `pass=0` en las 6 (3 build + 3 oos1), score 0,54-0,87 build, 0,58-0,71 oos1: todas NOISE.
- Ningún corte depende de esto (no hay regla), así que el coste en estrategias es 0.

Criterio 16.2 — Mapa de la ventana (cuatro lecturas por ventana; sólo INFORMA)

`model/mapa.py` , `config.yaml: grid` , hechos `spp.build.*` y `spp.oos1.*` (los nombres de tabla son las claves). - **Rejilla recortada** (`floored`): sin la madre (permutación −1) ni centinelas; sin filas donde algún shift (`*Shift[0-9]` , `*Shf[0-9]` , `grid.fixed`) difiera del valor de la madre (SQX mueve los shifts 0-6 desde 1, un cambio mucho mayor que el ±35 %); sin duplicados exactos; sin permutaciones con menos de `grid.min_trades = 30` operaciones. Hechos asociados: `fijos_frac` , `excluidas_frac` , `n` . - **Lectura** (`reading`) por métrica ∈ { `benef_anual` (AvgProfitPerYear), `sharpe` (SharpeRatio), `profit_factor` , `drawdown` }: `real` (la madre), `median` , `iqr` , `d = (real − mediana)/IQR` (signo invertido en drawdown: alto = «sospechosamente bueno»; NaN si IQR ≤ `iqr_floor` `0,001` ·max(|mediana|,|real|)), `puesto` = share de la rejilla estrictamente peor que la madre. Extras: `benef_anual.s = mediana/real` (si real > 0), `p10` , `p5` de benef_anual (cola), `drawdown.m_dd = p95(DD)/DD_real` . - `frac_vecinos_rentables` = share con NetProfit > 0 (Kaufman; el dossier lo considera poco informativo). - Ejemplo real (`wf4s`): build Sharpe `1.14.418` real 1,010 vs mediana 0,640, d 0,755, puesto 0,899; `5.2.399` 0,670/0,325, d 0,802, puesto 0,872; `9.19.426` 1,080/0,720, d 1,091, puesto 0,933. benef_anual build `1.14.418` : real 6.431 vs mediana 3.515, puesto 0,966, d 1,445. En oos1 la mediana del Sharpe se hunde: 0,080 / −0,240 / 0,080 con reales 0,260/0,240/0,120 (puesto 0,77/0,885/0,55). - Débiles: `puesto` y `d` miden dónde está la madre en su rejilla, no si la rejilla tiene ventaja; la rejilla es un muestreo de 15.000 tuplas de un hipercubo ±35 % (no uniforme sobre «mundos plausibles»); permutaciones correlacionadas (misma serie de precios).

Criterio 16.3 — Decaimiento IS→OOS1 («ventanas»; sólo INFORMA)

`mapa.across` y `model/decay.py` ; hechos `spp.ventanas.decaimiento.sharpe.*` , `profit_factor_build_y_oos1.*` , `sharpe_build_y_oos1.*` (percentiles 1-99 de ambas). - `r_s = mediana(Sharpe oos1)/mediana(Sharpe build)` solo si la mediana build ≥ `criteria.is_floor.SharpeRatio = 0,1` ; `z_decay = (med_oos1 − med_build)/√(1/T_build + 1/T_oos1)` (T en años; error de ruido de un Sharpe anualizado de 1/√T); `a` = Vargha-Delaney A de oos1 sobre build (P(oos1 > build) + ½ empates, tamaño de efecto, no p-valor). - Además, en la pestaña «Métricas para criterios» (`criteria.py` , `verdict.csv`): Wasserstein W_1 entre IS y OOS1 en IQR de IS; razón de medianas por métrica con suelos (`is_floor` : Sharpe 0,1, Sortino 0,1, PF 1,0, R/Edge 1,0); `original` = (real − mediana)/|mediana| en % y en IQR con la banda 5-95 de la rejilla (`median_floor_iqr 0,25`); nulo de «familia sin ventaja» para Sharpe, Sortino y PF (barrido por 100 cuantiles de un normal: Sharpe 0 ± 1/√años; PF de una media por operación N(0, 1/√n)). Dueño 2026-10-04: «calculadas y mostradas, sin umbral, no juzgan nada». - Ejemplo real (`wf4s`): `z_decay` −1,022 / −1,032 / −1,168; `a` 0,090 / 0,137 / 0,023; `r_s` 0,125 / −0,738 / 0,111. Es decir, en las 3 madres la rejilla oos1 es sistemáticamente peor que la build (A muy por debajo de 0,5), el decaimiento es de la rejilla entera. - Débiles: IS y OOS1 **no comparten tuplas** (6 de ~11.600 en `17.9.39`), no hay emparejamiento por variante; un cociente de medianas explota cerca de 0; el nulo OOS1 de «sin ventaja» asume Sharpe 0 ± 1/√T con permutaciones correlacionadas (cota superior). Ninguno tiene umbral ni tasa de error.

Criterio 16.4 — Reconocimiento de parámetros para el 16.5 (diseño, no criterio)

`run.py:read` / `brief` , `model/{influence,profile}.py` . Para cada estrategia: η^2 (varianza explicada por cada parámetro) sobre Ret/DD y companions {NetProfit, PF, NumberOfTrades, Sharpe}; **test de duplicados** (un parámetro es inerte si cada uno de sus niveles repite exactamente el mismo backtest (NetProfit, NumberOfTrades) → «frozen»; no se decide por η^2); perfil marginal por parámetro sobre Sharpe (`profile.metric`), mediana por nivel, **meseta** = tramo contiguo de niveles a ≥ 50 % del rango mejor-peor (`plateau_share 0,5`), centro = punto medio de la meseta (no el argmax); niveles del diseño = 9 por parámetro vivo (`min_levels = max_levels = 9`), cobertura mínima ±30 % (`min_span`), ajustados a valores que el SPP exploró; shifts congelados en el valor de la madre; `spike` si la meseta tiene ancho 1. Salida: `design_brief_*.json` con `n_target 500` y estratos (vecindad 0,20 / factorial 0,60 / cobertura 0,20) que lee 16.5. Ejemplo `wf4s` : parámetros vivos 8 / 9 / 5, congelados 0 / 0 / 0. - Débiles: la meseta usa la mediana marginal (promediando sobre las demás dimensiones, rejilla desbalanceada); η^2 está sesgado al alza en parámetros inertes (por eso el congelado va por duplicados).

Orden de aplicación

Sin juez: `read` → `brief` → mapas build/oos1 → ventanas → criterios; `report` escribe `verdict.csv` (columna `verdict` = el veredicto de ruido), `distribuciones.parquet` , `design_brief_*.json` . Luego fila «16 judge» con 0 cortes.

Ejemplo de cadena

`wf4s` (3 madres): 14 (3 → 3, sin juzgar por smoke) → 15 (SPP IS 0 → 3, SPP OOS 0 → 3) → 16 (3 → 3) → 16.5 con las 3. `wf3`: 14 (3 → 2) → 15 (2) → 16 (2 → 2). `wf2`: 2 → 1 → 1 → 1. `wf1`: 3 → 2 → 2 → 2.

Tiempos y recursos (paso 16)

«16 python»: 9 s (3 madres, `wf4s`), 6 s (2, `wf3`), 4 s (1, `wf2`), 7 s (`wf1`), 8 s (`10k`, 3). En corridas viejas de más madres: 19-43 s para 1.500-3.000 hechos (NIKKEI225 M30 `simpleL`, 2026-10-04/05). Sin dato de RAM; la exportación del perfil SPP pesa «decenas de GB» en SQX pero la lectura Python es trivial.

Discrepancias (⚠)

1. ⚠ Dossier 16 y `README` del estudio llaman «veredicto» al `noise_check`, pero **no gobierna ninguna decisión**: ni el juez (`rules: []`) ni el 16.5 leen `spp.verdict.pass`; solo el `design_brief` (meseta, niveles) se consume. En `wf4s` `1.14.418` salió NOISE y siguió a 16.5 igual.
2. ⚠ El comentario del criterio 16 en `criteria.yaml` dice que los hechos «peak solution» y «grid mean − 1 sd < 0» no se exponen; hoy sí existen aproximaciones (`max_mediana_iqr`, `puesto`, `d`, `p5`), pero ninguna se cableó: la regla FAIL-only del Council sigue sin cargar.
3. ⚠ `WORKFLOW.md` fila 16 dice «□»: el análisis está construido y corre, pero el paso **no tiene criterio de corte**. El corte posterior está en el paso 20 (`SPP OOS`).
4. ⚠ El nombre de la regla (`segment: oos1`) vs lo que se lee (build y oos1 del SPP): solo etiqueta de ledger; ningún corte real hay que registrar.
5. ⚠ Los resultados del 16 sobre la población están medidos con la rejilla de 15.000 permutaciones de un único muestreo SQX (semilla de SQX no controlada: no verificado si es reproducible entre corridas).

Pasos 16.5 y 17 — tal como están implementados y corridos (2026-10-11)

Fuentes: código (rutas relativas a `/home/sergioguslw/Desktop/AlgoProject`), `ledger/thresholds.yaml`, `pipeline/autopilot/criteria.yaml`, `AlgoData/ledger/wfc_calibration.json`, y las corridas reales de `AlgoData/strategyPermutations/*/Strategy_*/` y `AlgoData/autopilot/*/`. Los dossiers `criterios-paso-16.5-nube-de-parametros-2026-10-07.md` y `criterios-paso-17-wfc-2026-10-07.md` describen el estado ANTERIOR al rediseño (estratos, A1/A3/B2/C1, intervalo de Fisher, veredicto Fiable/Indeciso/No fiable): **ya no existen en el código**. Lo vigente es lo que describen las `*-propuesta.md` más las decisiones del dueño del 2026-10-07/08 que están en el código. Cuando el código y la propuesta difieren, manda el código y va marcado con ⚠.

Población real usada para los ejemplos: 16 lotes con `variant_trades.parquet` (los más recientes: `Test_USATEC_H1_ruptura_leaseWeb` 2026-10-11, `Test_XAUUSD_H1_ruptura_wf4s` 2026-10-11, `Test_NIKKEI225_M30_volatilidad_wf3` 2026-10-11, `Test_USA500_H1_tendencia_wf1` y `Test_DAX40_H1_reversion_wf2` 2026-10-10, `Test_USDJPY_M30_doubleROCLong_10k` 2026-10-08). Los lotes DAX40 M30 `momentumL_mc3` solo tienen nube (sin `wfc.json`). Todos son proyectos `Test_*` de prueba; **no hay corrida de un `Trade_`** con la fábrica nueva (`Trade_NIKKEI225_M30_volatilidadL` es de 2026-10-04, de la fábrica antigua, sin `variant_trades.parquet`).

PASO 16.5 — fábrica de variantes y nube de parámetros

Propósito, entradas, salidas

- **Fábrica** (`sqx/variants/`, la lanza `pipeline/autopilot/variants.py:run`): de cada madre superviviente del paso 16 fabrica un lote de variantes (la misma estrategia con otros valores de parámetros), lo retestea en build, oos1 y oos2 en SQX y guarda todas sus operaciones. Es la materia prima de 17, 18, 18.5 y 19. **No corta nada, no juzga.**
- **Nube** (`studies/optimisation/cloud/`): un mapa diagnóstico, «¿qué parámetros mueven el exceso y se puede leer ese ranking?». `criteria.yaml` entrada `"16.5"`: `cut: null`, `rules: []` (`pipeline/autopilot/criteria.yaml:230-233`). Es la única pieza del paso que decide algo... nada: no hay veredicto (`one.py:69-70` pasa `None`).
- Entrada de la fábrica: el *design brief* del SPP del paso 16 (`studies/breakage/spp/run.py:100-130`): por parámetro `class` (live / shift), `plateau {from,to}`, `levels`, `original`; más la lista de congelados.

- Salida de la fábrica por madre (`AlgoData/strategyPermutations/<proyecto>/Strategy_<x>/`): `design.json` , `pilot.json` , `plan.csv` , `manifest.parquet` , `metrics.parquet` , `segments.parquet` , `equity.parquet` , `variant_trades.parquet` (230-350 MB), `retest_{build,oos1,oos2}.csv` .
- Salida de la nube: `estudios/cloud.{json,html,md}` + `distribuciones.parquet` ; hechos en `cloud.json:facts` (`vivos` , `coeficientes` , `variantes.build/oos1` , y por tramo `r2_ajustado` , `r2_barajado` , `legible` , `sobol_primer.<p>` , `sobol_total.<p>`).
- Orden: fábrica (por madre) → `collect` → export de operaciones → `varianttrades` → `mothertrades` → nube (`variants.py:104-106` , al final con SQX parado).

Criterios del paso: (A) tamaño del lote, (B) caja de parámetros, (C) diseño, (D) controles, (E) piloto, (F) recogida sin suelo, (G) filtros de la nube, (H) métrica de la nube (exceso por año), (I) superficie y R^2 ajustado, (J) referencia barajada y «legible», (K) índices de Sobol, (L) mapas de calor. Ninguno corta. Se numeran como criterios porque fijan lo que ven 17, 18, 18.5 y 19.

16.5-A. Tamaño del lote

- **Pregunta:** ¿cuántas variantes hacen falta para que la superficie cuadrática de la nube sea ajustable?
- **Definición:** `n = min(n_max, max(n_min, per_coefficient · p))` , con `p = 1 + 2k + k(k−1)/2` y `k` = nº de parámetros vivos (`sqx/variants/design/plan.py:13-26`). Valores (ledger, `set_by: dueño` , 2026-10-07): `variants.n_min = 1000` , `variants.n_max = 5000` , `variants.per_coefficient = 8` (`ledger/thresholds.yaml:1214-1233` ; cableados en `sqx/variants/config.yaml:18-23`). Si `8p > 5000` se avisa (`capped`). Solo es un **tope, no una cuota**: tras redondear, las tuplas duplicadas se descartan y se cuentan en `shortfall` (`plan.py:46-49, 83-87`).
- **Efecto práctico:** con k ≤ 11 (todas las madres reales hasta hoy: k = 4 a 11), `8p ≤ 624 < 1000` , así que **n = 1000 siempre**; el 8 por coeficiente nunca ata. Con k = 11, p = 78: 12,8 variantes/coeficiente; con k = 4, p = 15: 67. El lote tiene `n + controles` (1004 en los lotes reales: 1000 + origen + 3 controles).
- **Por qué (declarado):** convención de «≥ 8 observaciones por coeficiente» para una cuadrática; más variantes no reducen la incertidumbre de la ρ del WFC porque esa viene del único camino de precios (`σ² ≈ σ²_camino + (1−ρ²)²/n` ; propuesta 16.5 §2.4). Decisión abierta nº 2 de la propuesta: revisar el 8.
- **Punto débil:** (i) 1000 variantes de una rejilla discreta pequeña no son 1000 puntos distintos: en USATEC 1.11.277 el espacio vivo tiene `live_space = 3213` tuplas (`design.json`) y se muestrean 1000 (31 %); (ii) tras SQX, 273 de 1004 salen con la misma lista de operaciones (ver 17-filtros): el tamaño efectivo es menor; (iii) el 8/coef. es una regla de pulgar, no una potencia calculada para el ranking de Sobol.
- **Corrida real:** `design.json` USATEC 1.11.277: `live 6, asked 1000, n 1004, controls 4, shortfall 0, capped false` .
- **Coste:** ver al final (627 s para 2 madres).

16.5-B. La caja: meseta del SPP IS

- **Pregunta:** ¿dentro de qué rango se mueve cada parámetro vivo?
- **Definición:** `levels.live` (`sqx/variants/design/levels.py:8-30`): entero → **todos** los enteros de `[plateau.from, plateau.to]` ; decimal → los niveles del brief dentro de la meseta más ambos extremos. Un `shift` (`…Shift1` , `…Shf…`) y los **congelados** quedan **fijos en el valor de la madre en todas las variantes** (`plan.py:72` ; `config.yaml:6-9`). La meseta la calcula el SPP (`studies/breakage/spp/model/profile.py:29-59`): por parámetro, la mediana marginal de `SharpeRatio` (config `profile.metric`) por nivel; meseta = racha contigua de niveles con `mediana ≥ máx − 0,5·(máx − mín)` (`plateau_share: 0.5` , `studies/breakage/spp/config.yaml:33`), que contiene al argmax. «Vivo» o «congelado» lo decide el test de duplicados del SPP (mover el parámetro no cambia ninguna operación → inerte).
- **Solo build:** la caja sale del SPP IS (ventana build) para no contaminar la composición abierta del WFC (propuesta §2.1).
- **Por qué:** una muestra «representativa de lo que un optimizador podría elegir» dentro de la zona buena del IS.
- **Punto débil:** (i) la caja es el **producto cartesiano de mesetas marginales**: incluye combinaciones fuera de la meseta conjunta, y excluye todo lo que el SPP marginal dio por malo; (ii) la meseta se elige por el Sharpe IS → **el rango IS de las variantes está truncado por selección** (restricción de rango: baja la ρ IS↔OOS por construcción, el estadístico del paso 17 mide dentro de una zona ya filtrada por IS); (iii) umbral 0,5 no calibrado; (iv) un parámetro con meseta de ancho 1 (`spike`) no varía y queda fuera de `k` (la nube lo avisa: `contract.py:121-124`); (v) la métrica del SPP (Sharpe de SQX) no es la del mapa (exceso por año neto de deriva).
- **Corrida real:** USATEC 1.11.277: 6 vivos, `ATRRisingPeriod1` con niveles 128…(enteros consecutivos) (`pilot.json`).

16.5-C. El diseño: hipercubo latino

- **Definición:** `lhs.draw` (`sqx/variants/design/lhs.py:13-42`): `scipy.stats.qmc.LatinHypercube(d=k, seed=…)` , `seed 20260921` (`sqx/variants/config.yaml:10`). Entero: se ensancha la caja ±0,5 y se redondea al entero más cercano (los extremos pesan lo mismo que el centro); decimal: se redondea a 3 decimales; se recorta a [mín, máx]. Un nivel que el piloto descartó se mueve al nivel

conservado más cercano. Los duplicados tras redondear se eliminan sin reponer (`plan.py:73-81` , por `tuple_hash`).

- **Por qué:** cobertura uniforme; en el diseño antiguo (estratos vecindad 20 % / factorial 60 % / cobertura 20 %), la ρ cambiaba con el reparto de estratos (C: 0,83 factorial, 0,12 vecindad, 0,67 junto; propuesta §2.3). Decisión abierta 1: LHS frente a secuencia de Sobol (cualquiera cubre bien).
- **Punto débil:** (i) LHS es uniforme en el producto de mesetas, no en lo que un optimizador real elegiría (que se concentraría cerca del óptimo IS): el «top 20 %» del WFC es de una nube que **no pondera por plausibilidad**; (ii) con parámetros enteros de rango corto el LHS redondeado repite tuplas; (iii) no hay réplica ni semillas alternativas: una sola realización del diseño por madre.
- **Sustitución ⚠:** los dossiers antiguos describen el diseño por estratos y la «vecindad» a 2 pasos de la meseta; eso **ya no existe**. `ls` de `sqx/variants/design/` : `canaries, levels, lhs, pilot, plan` (no hay `strata.py`). El `README` de `studies/optimisation/cloud/` avisa: un lote del diseño antiguo no tiene `variant_trades.parquet` y no se puede leer.

16.5-D. Controles

- **Definición** (`design/canaries.py` , `plan.CONTROLS` , `filters.CONTROLS = ("canary","inert")`): la **madre** (`P00000` , estrato `origin` , entra en el análisis como una variante más); **3 canarios** (`canaries.n: 3` : tuplas del SPP con mayor NetProfit, menor NetProfit y menos operaciones; se esperan los valores guardados por SQX); **pares inertes** (la madre con un congelado llevado al extremo ±30 %, 5 niveles; debe salir idéntica a la madre). Los controles **no entran en ningún análisis** (`inputs/cloud.py:98` , `filters.py:71`).
- **Por qué:** comprobar que la cadena (rewrite → carga → retest → export) lee los parámetros; no tienen valor estadístico.
- **Corrida real:** `collected.json` USATEC: `canaries n 4, distinct_netprofit 4, all_identical false` . El `wfc.json` informa `controles: 3` en cada lote.

16.5-E. Piloto

- **Definición** (`design/pilot.py` , `sqx/variants/config.yaml:36-41`): 300 tuplas Sobol sobre los vivos (`pilot.sample 300` , `seed 20260929`), retestadas en build; por nivel de cada parámetro vivo se toma la mediana de operaciones; un nivel con `support ≥ 5` y mediana `< 30` operaciones (`min_trades: 30`) se **descarta del diseño** — salvo que el parámetro se quede con menos de 2 niveles, y nunca el valor de la madre.
- **Por qué:** evitar medio lote que opera demasiado poco para ser un punto de superficie (OPEN §41; 17.9.39: 1002 de 2000 con < 30 operaciones).
- **Punto débil:** la mediana marginal por nivel esconde interacciones (un nivel puede operar poco solo con ciertos valores de otro); umbral 30 y soporte 5 no calibrados.
- **Corrida real:** USATEC 1.11.277: `pilot_n 300, scored 300` , `pilot_dropped {}` (0 niveles descartados).

16.5-F. Recogida (`collect`) — sin suelo

- **Definición:** `sqx/variants/collect.py` (join en `:85-111`): `united.thin(rows, 0, segments)` → **suelo 0**: ninguna variante se descarta por operar poco; `usable` queda verdadero (lo lee `marketSurfaces`); `collected.json` da `floor: 0, thin: 0` . Cada estudio aplica su propio suelo.
- ⚠ **`assets/_build.yaml:369` sigue diciendo `wfc.min_trades_total: 100`** (el comentario `:354` y `engines/variants/config.yaml:8` dicen 50). **Ningún código lo aplica ya**: `grep min_trades_total` solo da el YAML, el visor de la ventana (`ui/daemon/sqxconfig/lists.py:56`) y el docstring de `united.thin` . Config muerta con tres valores distintos (50 / 100 / 0).

16.5-G. Filtros de entrada de la nube

- **Definición** (`studies/optimisation/cloud/inputs/cloud.py:82-113`): 1. **Controles fuera** (canary, inert). 2. **Suelo de dentro:** `build_trades ≥ 15·k` (`wfc.floor_is_per_param = 15` , dueño 2026-10-07, `ledger/thresholds.yaml:1235-1240`), con `k` = parámetros con `nunique > 1` en lo que queda tras controles. Si la **madre** no llega, se avisa y el suelo no se aplica. 3. **Duplicados exactos:** mismo `build_profit` **y** mismo `oos1_profit` redondeados a 2 decimales (`:110-112`); se conserva la madre. 4. Si `len(cloud) ≤ p` (variantes ≤ coeficientes) → «Sin dato», sin ajuste (`one.py:36-37`).
- ⚠ **Los duplicados de la nube y del WFC se definen distinto:** la nube compara beneficio a 2 decimales en build+oos1 (no ve oos2: no puede); el WFC compara huella de la lista de operaciones de los tres tramos (`engines/variants/fingerprint.py`). USATEC 1.11.277: nube −335 duplicados (quedan 666), WFC −273 (quedan 728).
- **Nunca lee oos2** (`LEGS = ("build","oos1")` , `inputs/cloud.py:39` ; filtro de parquet por tramo antes de cargar filas).
- **Corrida real:** USATEC 1.11.277: lote 1004 → −3 controles → −0 por suelo (90 operaciones) → −335 duplicados → 666 en el mapa. Otras madres: 746–1001 variantes.

16.5-H. La métrica: exceso por año, neto de deriva

- **Definición** (`engines/variants/excess.py:12-58` ; `inputs/cloud.py:44-79`): cada operación se reprecia (`engines.market.reprice.real` : costes de la ficha del activo y del tramo: spread, slippage, comisión, swap) y se le resta la deriva del mercado por estar dentro (`tradelevel.drift_term` con `drift.window: year` , `drift.variant: d` , **tomado de** `studies/screening/gate/config.yaml:57-64` , no copia). Exceso de una variante en un tramo = Σ exceso / años del tramo (365,25 d), en moneda de la cuenta por año. Variante sin operaciones en un tramo → 0.
- **Por qué:** quita lo que el mercado pagó solo por estar dentro (en periodos alcistas las variantes más expuestas ganan más dentro y fuera, lo que fabrica un orden que no viene de la lógica; propuesta 17 §3). Es la misma medida del paso 14.
- **Punto débil:** (i) el exceso se mide **neto de costes reconstruidos** y cada lote verifica si reproduce el NetProfit de SQX (`descuadre_sqx` , ver 17-J: llega a 0,87 en USDJPY); (ii) «moneda por año» no normaliza por riesgo ni por exposición: variantes que operan más tienen exceso de mayor varianza; (iii) la deriva `d` usa el año natural de la operación y es una convención.
- **Datos:** `variant_trades.parquet` (varias decenas de millones de filas por lote, 130–350 MB).

16.5-I. Superficie y R² ajustado

- **Definición** (`studies/optimisation/cloud/model/surrogate.py:6-57` , `model/space.py`): parámetros vivos (`nunique > 1`) reescalados a [0,1] por mínimo y máximo de **la propia nube**; superficie = **cuadrática completa** (intercepto, lineales, cuadrados, productos cruzados), mínimos cuadrados `lstsq` ; R² ajustado con los grados de libertad = **rango** de la matriz (`n − rank`), no su anchura. Se ajusta una vez por tramo (`build` y `oos1`) sobre el exceso por año de ese tramo (`one.py:39-45`).
- **Por qué:** una cuadrática no necesita hiperparámetros ajustados con los mismos datos que la juzgan (frente a un proceso gaussiano); el R² ajustado cuenta los coeficientes gastados.
- **Punto débil:** (i) R² dentro de muestra, sin validación cruzada: la cuadrática no recoge umbrales ni interacciones de orden > 2; (ii) el exceso por año tiene colas gruesas y entra directamente en `y.var()` ; (iii) el escalado min–max usa el rango observado, no la meseta; (iv) 28 a 78 coeficientes con n = 666–1001.
- **Corridas reales** (lote nube build / oos1, R² ajustado): USATEC 1.11.277: 0,768 / 0,660 (28 coef., 666 var.); USATEC 6.22.453: 0,754 / 0,658; DAX40 H1 14.16.487: 0,785 / 0,660 (55 coef.); XAUUSD 1.14.418: 0,779 / 0,720; XAUUSD 9.19.426: 0,699 / 0,352; NIKKEI wf3 17.13.284: 0,556 / 0,752; USDJPY 12.15.405: 0,261 / 0,244 (el más bajo).

16.5-J. Referencia barajada y «legible»

- **Definición** (`measure/reference.py:8-40`): se baraja el exceso entre las variantes `shuffles = 200` veces (`parameterCloud.shuffles` , dueño 2026-10-07, `thresholds.yaml:1333-1338`), se reajusta la misma cuadrática a cada baraje y se toma el **percentil 95** (`parameterCloud.shuffle_percentile` , `:1340-1345`) de los R² ajustados. `legible = R²_ajustado_real > ese percentil` . Si no: aviso «Sin estructura distinguible del ruido» (`contract.py:9, 114-116`) y el ranking de Sobol «no se lee». `seed: 0` (`cloud/config.yaml`).
- **Por qué:** los índices de Sobol son del modelo ajustado y una superficie que no explica nada reparte igualmente un ranking; una nula de barajado da el R² que dan los coeficientes por sí solos. Ruido puro sale «legible» ≈ 5 % (`tests/test_cloud_map.py`).
- **Punto débil (importante):** el R² ajustado barajado es, con n ≈ 1000, ≈ 0 (percentil 95 ≈ 0,01–0,02). **El umbral es trivialmente bajo:** cualquier estructura de R² > 0,02 es «legible», incluida una nube cuyo exceso es pura estructura de *dependencia común* (variantes casi idénticas que comparten el camino de precios). En las **16 nubes reales todas son legibles en build y oos1** (R² ≥ 0,21 vs. referencia 0,009–0,020): el criterio no ha discriminado nunca. Además es ajuste en muestra frente a barajado en muestra: no mide generalización (no hay CV).
- **Corrida real:** USATEC 1.11.277: build 0,768 vs ref 0,016 → legible; oos1 0,660 vs 0,0127 → legible. DAX40 14.16.487: 0,785 vs 0,010; USDJPY 12.15.405: 0,261 vs 0,012.

16.5-K. Índices de Sobol

- **Definición** (`model/sensitivity.py:9-39`): sobre la **cuadrática ajustada** (no sobre los datos), secuencia de Sobol con scrambling `scipy.stats.qmc.Sobol(d=2k, scramble=True, seed=0)` , `2^12 = 4096` puntos base (`sobol_power: 12`), cubo **uniforme** [0,1]^k; primer orden = estimador de Saltelli (`mean(yB·(yAB_i − yA))/Var`), total = Jansen (`mean((yA − yAB_i)²)/(2 Var)`). Se calculan por tramo. Es aritmética de la superficie: coste ≈ 1–2 s.
- **Por qué:** decir qué parámetros mueven el exceso, solos o combinados; si Σ primeros ≈ 1 cada parámetro actúa solo; si Σ totales ≫ 1, dependencia de combinaciones finas (señal de fragilidad a revisar a mano; propuesta §3.3).
- **Punto débil:** sin intervalos de confianza (ni bootstrap sobre variantes ni sobre semillas); depende de que la cuadrática represente bien la superficie; el cubo uniforme [0,1] asume que cada parámetro se mueve por todo su rango observado con la misma probabilidad (la caja es la meseta marginal, no la zona operada); la comparación build/oos1 no tiene regla (el ranking de oos1 «no decide»).

- **Corrida real** (USATEC 1.11.277, build): primer orden `ExitAfterBars1` 0,700; `BBBarOpensDeviation1` 0,116; `BBBarOpensPeriod2` 0,115; `BBBarOpensPeriod1` 0,057; `ATRRisingPeriod1` 0,002; `BBBarOpensDeviation2` 0,000; Σ primeros 0,990, Σ totales 1,01 (apenas interacción). **En oos1 el ranking se redistribuye**: `ExitAfterBars1` 0,368, `ATRRisingPeriod1` 0,253 (de 0,002 en build) — ningún criterio dice si eso importa.

16.5-L. Mapas de calor

- `heatmaps.py` (`config heatmap_pairs: 4` → 6 mapas): mediana del exceso por celda en 6 bins de igual anchura por eje para los pares de los 4 parámetros de mayor Sobol total; rango de color compartido; madre marcada. Solo informa. Sin dato propio de calidad estadística (medianas por celda, sin n por celda visible).

Orden de aplicación (16.5)

controles fuera → suelo 15·k en build (si la madre lo cumple) → duplicados por (build, oos1) → ¿n > p? → ajuste cuadrático por tramo → barajado ×200 → Sobol 4096 → mapas. Nada corta.

Ejemplo de cadena (USATEC 1.11.277, H1, 2026-10-11)

Lote fabricado 1004 (1000 LHS + origen + 3 canarios) → 1004 devueltas por SQX (3 tramos × 1004 en disco) → nube: −3 controles → −0 suelo → −335 duplicados → **666** en el mapa (6 vivos, 28 coeficientes, 23,8 variantes/coef.) → R^2 0,768 (build) → legible → `ExitAfterBars1` 0,70. 6.22.453: 1004 → 999 en el mapa (7 vivos).

Tiempos y recursos (16.5)

- `AlgoData/autopilot/Test_USATEC_H1_ruptura_leaseWeb/20261011-071421/tiempos.md`: «16.5 variants» **627 s** (fábrica + SQX + exportación) para 2 madres: 306 s y 299 s por madre; de ellos 278 s en export de operaciones. XAUUSD wf4s (3 madres): 963 s (export 409 s; `execute` 122 s, `export_retest` 120–153 s por madre). El 16.5 es el 6 % de los 10.034 s de la corrida; SQX (`custodio` , un solo trabajo a la vez, RAM del custodio 80 g por la regla 3) no se atribuye aparte («sqx 0» en la fila: está dentro de `execute`).
- La nube: **2,1–2,2 s por madre** (`wall_s` de `cloud.json`).
- Disco: 324 MB por madre (USATEC 1.11.277), de ellos 232 MB `variant_trades.parquet` .
- Núcleos usados por la fábrica en SQX / RAM de la nube: **sin dato**.
- ⚠ **Variante sin retest completo:** las 11 mercados adicionales se retestean en el mismo lote (`collected.json: markets`) pero la nube solo lee `Main` .

Discrepancias 16.5 (⚠)

1. `docs/AgentPDFs/WORKFLOW.md:45` dice «□ ningún lote real con `variant_trades.parquet` »: **hay 16** (`find AlgoData -name variant_trades.parquet`), fechados 2026-10-08 a 2026-10-11.
2. Dossier `criterios-paso-16.5-nube-de-parametros-2026-10-07` : describe A1/A3/B2/C1, vecindad, pestañas, estratos; todo eliminado. Dice «el autopiloto no lanza la nube»: **sí la lanza** (`pipeline/autopilot/variants.py:105-106`) y la nube sí escribe hechos (`one.py:75-76`), aunque el paso no tiene regla.
3. ⚠ `assets/_build.yaml:369` `min_trades_total: 100` (comentario: 50) no lo aplica ningún código (ver 16.5-F).
4. ⚠ Duplicados: definición distinta en nube y en WFC (16.5-G).
5. `studies/optimisation/cloud/README.md:48` y la nube leen `floor_is_per_param` de `engines/variants/config.yaml` , pero en la nube se compara **solo build** (`build_trades`), mientras en el WFC la ventana de dentro es build+oos1 de la composición decisoria: el mismo `15·k` se pide sobre ventanas distintas.
6. Umbral «legible» ≈ 0,01–0,02 (16.5-J): documentado como p95 de barajados; en la práctica nunca discrimina con n ≈ 1000.

PASO 17 — Walk Forward Correlation (WFC)

Propósito, entradas, salidas

- **Pregunta del paso** (propuesta §0): «si ordeno las variantes de una madre por lo que ganaron dentro de muestra, ¿ese orden se parece al de fuera? Y lo que elegiría un optimizador, ¿gana fuera?».
- **Entrada:** el lote del 16.5 (`metrics.parquet` , `variant_trades.parquet`). Es el **primer paso que lee oos2** (bloque ciego 17, 18, 18.5, 19): abre `oos2` solo si `ledger.gate.seal` lo permite (criterio escrito en `criteria.yaml` + calibración registrada con `criteria_sha256` igual al hash de la entrada `"17"` + `abre: true` ; `ledger/gate.py:86-123`). Hoy el hash coincide (`815e4960…`) y

`abre: true` (fecha 2026-10-07).

- **Salida:** `estudios/wfc.{json,html,md}`, `wfc.json` raíz (n, rho, call, p_mas, p_menos), `distribuciones.parquet` (los nulos), y **tres filas por composición en el ledger** (un «look» por tramo leído; `look.log_all`). Hechos `wfc.c1.*`, `wfc.c2.*`, `wfc.c3.*` (`report.py:43`), que el juez lee.
- **Veredicto:** `estado` ∈ {pasa, limbo, no_pasa} **de c1 solamente**; `criteria.yaml`: `"17": cut: null, segment: oos2, regla wfc.c1.estado, pass ">= 1", limbo ">= 0", missing: limbo` (`pipeline/autopilot/criteria.yaml:234-250`; pasa = 1, limbo = 0, no_pasa = −1). **No elimina ninguna estrategia** (`cut: null`): 18, 18.5 y 19 corren para toda superviviente y el paso 20 lee NO PASA como fallo.
- **Tiempo:** 6 s «17 blind» para 2 madres (`tiempos.md`); `wall_s` por madre 4,5–4,7 s.

Criterios del paso 17: (A) composiciones, (B) filtros de variantes y suelos, (C) la medida (exceso neto y bruto), (D) ρ de Spearman y nulo de volteo semanal de signos, (E) clasificación del orden, (F) nivel del top y su nulo, (G) tabla PASA/LIMBO/NO PASA, (H) etiqueta «fuerte», (I) anotaciones (bloque de 2 semanas, R_sel, ρ parcial, cuadrantes, ρ sin suelos, deriva previa), (J) comprobación contra SQX, (K) sello de oos2 y calibración.

17-A. Las tres composiciones

- **Definición** (`engines/variants/panel.py:19-21`): c1 = dentro **build+oos1**, fuera **oos2** (**decide**, `wfc.deciding = build+oos1__oos2`, `thresholds.yaml:1319-1324`); c2 = dentro build, fuera oos1+oos2; c3 = dentro build, fuera oos2 (se anotan). Cada una tiene su propio nulo y su p; el estado solo se dice de c1. `build → oos1` sola se quitó.
- **Por qué:** el WFC imita a un optimizador que ya ha visto build (donde optimizó) y oos1 (con el que el paso 8 la eligió); lo único no visto es oos2.
- **Punto débil:** las tres comparten oos2: que coincidan **no es confirmación** (propuesta §1); c2 y c3 comparten además el lado de dentro build; solo c1 cuenta. Una sola ventana de fuera (≈ 2,75 años) → un solo camino de precios para todas las variantes.
- **Corrida real** (USATEC 1.11.277): c1 ρ = −0,055 (neutro), c2 −0,163, c3 −0,310; c3 con p− = 0,105.

17-B. Filtros de variantes y suelos

- **Definición** (`engines/variants/filters.py:25-83`, orden): (1) controles fuera; (2) **suelo de dentro** = `15·k` operaciones en la ventana de dentro (`wfc.floor_is_per_param = 15`, columna `NumberOfTrades (build+oos1)`), con `k` = parámetros vivos del lote (`batch.live`: `nunique > 1` sin controles, `inputs/batch.py:105-117`); (3) **sello de oos2** (`gate.seal`; todo lo que sigue lee oos2); (4) **duplicados exactos** = misma huella de operaciones (`fingerprint.prints`) en los tramos leídos (la madre se conserva); (5) **suelo de fuera** = `10` operaciones **por año** de la ventana de fuera (`wfc.floor_oos_per_year = 10`; oos2 2,75 años → 28; `thresholds.yaml:1242-1247`); (6) variantes sin valor se descartan (`sin_valor`). Si la madre no llega a un suelo, se avisa y ese suelo **no se aplica**. Si quedan `< 30` variantes (`wfc.min_variants = 30`), `sin_dato` → LIMBO. Si la madre no tiene operaciones fuera → `sin_dato` (IDLE).
- **Por qué:** el suelo de dentro es de grados de libertad (con pocas operaciones por parámetro, una variante se ajusta al ruido); fuera no se ajusta nada, solo hace falta poder medir el exceso por año.
- **Punto débil:** `15·k` es una convención; el suelo de dentro cuenta operaciones **de la variante** pero no la dependencia **entre** variantes (mismo camino de precios): 700–1000 «puntos» no son 700–1000 observaciones. La nulo de volteo lo corrige parcialmente (17-D); el suelo no.
- **Corridas reales** (c1): USATEC 1.11.277: 1004 → −3 controles → −0 → −0 → **−273 duplicados** → 728; USATEC 6.22.453: −2 duplicados → 999; XAUUSD 1.14.418: −7 → 994; USDJPY 12.15.405: `suelo_dentro 67, suelo_fuera 145` → 789 (las únicas con suelo mordiendo); USDJPY 12.21.437: 19 y 189 → 792. DAX40 H1, NIKKEI wf3, USA500 wf1: 0 duplicados, 0 suelos (1001 quedan). El suelo de fuera de 28 operaciones elimina hasta 189 de ~1000 en USDJPY.

17-C. La medida: exceso por año, neto y bruto

- **Definición** (`one.py:24-49`, `inputs/batch.py:54-102`, `engines/weekly/panel.py:37-54`): exceso de cada operación (precio repreciado menos deriva, igual que 16.5-H) → **neto** (`net`, tras costes) y **bruto** (`gross = net + cost`: sin comisión, swap, spread ni slippage). Por variante y lado: Σ / años de la ventana. El **orden (ρ) se mide en bruto; el nivel (mediana del top, p_nivel) en neto**.
- **Por qué:** en bruto, porque las variantes que operan más pagan más costes en ambos lados, lo que por sí solo las ordena igual dentro y fuera, y un volteo de signos convierte ese coste en ganancia y lo pierde del nulo (`measure/reading.py:13-17`); el nivel es dinero, tras costes.
- **Punto débil:** dos medidas distintas para el orden y el nivel de una misma variante; el bruto elimina justo el efecto que más diferencia a las variantes (cuánto operan), pero el top del nivel se elige con el **neto** dentro (`inside = e_is net`, `reading.py:61-62`) mientras la ρ usa el **bruto**: el top y la ρ ordenan con rankings distintos.
- **Datos reales de la distinción:** en USATEC 1.11.277, c1: `nivel_top_dentro` 3660, `nivel_todas_dentro` 3240 (neto, dentro) frente a `nivel_top` 774 y `nivel_todas` 1051 fuera (el top gana menos que la mediana de todas).

17-D. ρ de Spearman y nulo de volteo semanal de signos

- **Definición:** `ρ = Spearman(e_IS_bruto, e_OOS_bruto)` sobre las variantes que pasan los filtros (`measure/reading.py:49` ; `engines/weekly/flip.py:61-75` , empates con rango medio). **Nulo** (`flip.py:9-27` , `reading.py:46-52`): el panel de fuera es *semanas* × *variantes* (cada operación **entera en la semana de cierre**, lunes como ancla, `engines/weekly/panel.py:57-87`); a cada semana se le sortea un signo ±1 **igual para todas las variantes**; el lado de dentro queda fijo; `ρ^(b) = Spearman(e_IS, signs @ semanas_bruto)` ; `B = 2000` (`wfc.null_draws` , `thresholds.yaml:1256-1261`), `block_weeks = 1` , semilla 0 (`SeedSequence(0).spawn(2)` ; `wfc.seed`). `p+ = (1 + #{null ≥ ρ})/(1 + B)` y `p− = (1 + #{null ≤ ρ})/(1 + B)` (`flip.py:43-58`); se guardan también p5 y p95 del nulo (`nulo_p5` , `nulo_p95`).
- **Por qué:** el intervalo de Fisher trataba 300 variantes como 300 observaciones independientes (σ_ρ real 0,09–0,41, mediana 0,26, frente a 0,05–0,07 de la fórmula; N efectivo 8–145, mediana 17; propuesta §4.2). El volteo conserva cómo se parecen entre sí las variantes (misma señal en toda la familia) y elimina la ventaja. (La propuesta describía el volteo **por día**; el código es **por semana**, para no romper las operaciones largas.)
- **Punto débil:** (i) el nulo supone **simetría** de las semanas bajo H0 (exceso de signo intercambiable) y **independencia entre semanas**: autocorrelación de volatilidad y rachas lo violan; (ii) el nulo volteado tiene una desviación de ρ de **0,09 a 0,45** en lotes reales: `nulo_p5/p95` van de ±0,20 (USDJPY) a **±0,74 (DAX40)**, es decir **la prueba casi nunca puede rechazar** (con ρ = +0,53 en DAX40, p+ = 0,24); (iii) mide el orden **en una sola ventana** de fuera (≈ 143 semanas): el N efectivo no lo da n sino el camino de precios; (iv) la ρ IS↔OOS en una nube cuya caja ya está truncada por Sharpe IS (16.5-B) parte con rango restringido; (v) el nulo supone que el lado de dentro no tiene relación con fuera salvo por la familia, pero la deriva restada se mide con barras de la propia ventana (`variant d` : año natural), que añade dependencia.
- **Corrida real** (c1): ρ de −0,22 a +0,53 en 12 lotes; **p+ ≤ 0,05 en ninguno** (mínimo 0,086, NIKKEI 20.11.454, ρ = 0,34 con nulo ±0,40); p− ≤ 0,05 en ninguno (mínimo 0,082, USDJPY 12.21.437, ρ = −0,219).
- **Coste:** 2000 × 1000 variantes × ~143 semanas, matriz de signos @ panel y rangos: segundos (4,5 s todo el paso).

17-E. Clasificación del orden

- **Definición** (`verdict/state.py:17-31`): `positivo` si `p+ ≤ 0,05` (`wfc.alpha` , `thresholds.yaml:1298`), `perverso` si `p− ≤ 0,05` , `neutro` en otro caso, `sin_dato` si ρ no existe (NaN). No se corrige por multiplicidad: dos colas por composición y tres composiciones.
- **Por qué:** «por construcción, una familia cuyo orden es ruido sale perversa como mucho el 5 % de las veces» (propuesta §4.4).
- **Punto débil:** α = 0,05 por cola sin corrección; tres composiciones × dos colas = 6 contrastes por madre (solo c1 decide); α no está calibrado contra la dependencia real (la calibración sintética con `own_noise 0.4` da sd de ρ ≈ 0,22; los lotes reales llegan a 0,45).
- **Corrida real: 12 de 12 lotes: c1 = «neutro»**; c2 positivo en 1 (NIKKEI 17.13.284, ρ = 0,50, p+ ≈ 0,02); nunca perverso en c1 (USDJPY 12.21.437 queda en p− = 0,082).

17-F. Nivel del top y su nulo (`p_nivel`)

- **Definición** (`measure/level.py:6-31` , `reading.py:61-78`): **top** = variantes con `e_IS_neto ≥ cuantil (1 − top_share)` , `top_share = 0,20` (`wfc.top_share` , `:1312` ; con empates es algo más del 20 %: 146 de 728 en USATEC 1.11.277, 201 de 1001 típico). **`nivel_top`** = mediana de `e_OOS_neto` del top (moneda/año). **`p_nivel`** = `(1 + #{draws con mediana del top ≥ observada}) / (1 + B)` , con los **mismos** signos semanales del nulo de la ρ aplicados al panel neto (`flat = signs @ semanas_neto / años`).
- **Por qué:** la ρ mide inclinación de la nube, no en qué cuadrante está (en la corrida de referencia, G tiene 324 de 332 variantes ganando dentro y fuera y ρ = 0,01; 13.3.390 pierde fuera con ρ = +0,35); el «nivel» responde si lo que un optimizador elegiría gana fuera.
- **Punto débil (el más importante del paso):** (i) este nulo de volteo mantiene el signo común a toda la familia, así que `p_nivel` es en la práctica **un contraste de que la media del exceso neto de la familia en oos2 es positiva** (un t-test de volteo con un solo camino), no un contraste de que «elegir por IS ayuda»: no compara el top contra el resto; (ii) la mediana del top dentro del nulo de volteo con n ≈ 1000 variantes casi idénticas es de varianza grande; (iii) top 20 % por neto IS no es el criterio que usa la ρ (bruto).
- **Corrida real** (c1): `p_nivel` ≤ 0,05 en **ninguno** de 12 lotes (mínimo 0,074, XAUUSD 5.2.399, top +2002/año; siguientes 0,144, 0,191…). Los casos con top ≤ 0: DAX40 −3203 (todas pierden fuera: `cuadrante_pp 0/1001`), USATEC 6.22.453 −91, USDJPY 12.21.437 −913.

17-G. La regla: PASA / LIMBO / NO PASA

- **Definición** (`verdict/state.py:34-56` , tabla `:5-13`), solo c1:
- `sin_dato` , o `n < 30` variantes → **LIMBO**;
- `nivel_top ≤ 0` → **NO PASA** (sin mirar la cola inferior);
- `nivel_top > 0` y orden **perverso** → **LIMBO**; `nivel_top > 0` y `p_nivel > 0,05` → **LIMBO**;

- `nivel_top > 0` y orden positivo/neutro y `p_nivel ≤ 0,05` (`wfc.level_alpha = 0,05`, `:1382-1387`) → **PASA**. «PASA es un veto levantado, no un voto a favor». Dueño 2026-10-07; la significancia del nivel, 2026-10-08.
- **Por qué (declarado):** E y G (todas ganan, el orden da igual) salían «No fiable» y se descartaban; con esta tabla, si su top gana en oos2 y supera el nulo, PASA (meseta plana). «Ningún orden salva a una familia cuyo top pierde fuera.» No hay «fabrica más puntos».
- **Punto débil:** (i) NO PASA es **irreversible** (el paso 20 lo lee como fallo); su tasa de falso NO PASA con ventaja Sharpe 0,5 es del 22 % a 2,75 años (`falso_no_pasa` del registro); (ii) PASA es muy raro con ventaja moderada (con Sharpe anual 0,5 y 2,75 años: **20–43 %** PASA, ~50–58 % LIMBO; ver 17-K); (iii) el LIMBO de `p_nivel > 0,05` agrupa casos de ventaja débil con casos sin ventaja; (iv) «perverso con top ganando = LIMBO» casi nunca se activa porque el orden perverso casi nunca sale significativo.
- ⚠ **`WORKFLOW.md:46` dice «el top gana fuera cuando su mediana neta es > 0, sin pedir significancia (`p_nivel` se anota y ya no decide)».** El **código** (`state.py:56`), `criteria.yaml:238-241` y `config.yaml:28-31` dicen lo contrario: PASA exige `p_nivel ≤ level_alpha`. El hash firmado en el registro de calibración corresponde a la entrada con significancia.
- **Corrida real** (c1, 12 lotes con `wfc.json`): **PASA 0, LIMBO 9, NO PASA 3**. NO PASA: DAX40 14.16.487, USATEC 6.22.453, USDJPY 12.21.437. LIMBO: los 9 restantes (todos con top > 0 y `p_nivel` 0,074–0,377, orden neutro).

17-H. Etiqueta «fuerte»

`fuerte = (orden == positivo) ∧ (ρ ≥ 0,30)` (`wfc.rho_strong = 0,3`, el 0,30 de Tinsley). Solo etiqueta, no mueve estado. En ninguno de los 12 lotes c1 salió fuerte (c1 siempre neutro). Punto débil: umbral heredado de la literatura, aplicado a ρ de una nube con N efectivo ≪ n.

17-I. Lo que se anota sin decidir

- **Bloque de 2 semanas** (`wfc.straddle_warn = 0,1`, `straddle_block_weeks = 2`): si más del 10 % de las operaciones de fuera cierran en lunes o martes tras pasar el fin de semana abiertas, se redibuja el nulo con bloques de 2 semanas y se anota qué clase tendría el orden (`reading.py:55-60`). En los lotes reales `fines_de_semana` ≤ 0,009: nunca se disparó (≤ 1 %).
- **R_sel** (`level.retention`): `(prima fuera del top)/(prima dentro del top)`, `prima = mediana(top) − mediana(todas)`; «sin dato» salvo que la prima de dentro supere `r_sel_min_gap = 0,05` del rango intercuartílico de dentro (provisional, el Council no dio número). Se compara con p5/p95 del mismo nulo. Reales: de −0,76 a **+1,35** (USATEC 6.22.453: R_sel = 1,35 con top ≤ 0, porque la mediana de todas es más negativa), con bandas de nulo de ±0,7 a ±1,5: **ninguno distinguible**.
- **ρ parcial** (`measure/order.py`): Spearman de las dos ordenaciones tras quitar los rangos de operaciones de cada variante (`rho_parcial`), sobre bruto. USATEC 1.11.277: ρ −0,055 → ρ parcial −0,102.
- **Cuadrantes** (`cuadrante_pp/pn/np/nn`): cuántas variantes ganan/pierden dentro y fuera (neto). DAX40: pp 0 / pn 1001; USATEC 6.22.453: pp 357 / pn 642; NIKKEI 17.13.284: pp 963 / pn 38.
- **ρ sin suelos** (`rho_sin_suelos`, `n_sin_suelos`) solo en la composición decisoria; igual a ρ cuando no hay suelo activo (mayoría de lotes); USDJPY 12.15.405: 0,08 → 0,05.
- **Deriva previa** (`nivel_top_deriva_previa`, solo cuando fuera es oos2: c1 y c3 (`comp[1] == OOS2`, `one.py:111`)): mediana del top con la deriva medida solo con barras previas (`PRIOR = {window: year, variant: c}`). USATEC 1.11.277 c1: 2229 frente a nivel_top 774.
- Todo se escribe en `wfc.json:facts`; el juez solo mira `wfc.c1.estado`.

17-J. Comprobación contra SQX

- **Definición** (`one.py:139-150`, `engines/weekly/panel.py:103-121`): por tramo, la fracción de variantes cuya suma de P/L repreciado difiere de `NetProfit (leg)` de SQX en más de `1,0 + 0,01·|NetProfit|` moneda (`wfc.sqx_gap_tolerance = 1.0`, `sqx_gap_relative = 0.01`). Se toma el **peor tramo** → `descuadre_sqx`; si > 0, se añade un aviso a la nota de c1 (`GAP`). **No cambia el estado.**
- **Por qué:** única comprobación de que los costes reconstruidos (`reprice`) reproducen los de SQX; sin esto el neto del nivel no se sostiene.
- **Corrida real:** USATEC 1.11.277 y 6.22.453: **0,000**; NIKKEI wf3: 0,013 y 0,061; DAX40 H1: 0,088; USA500 wf1: 0,142 y **0,244**; XAUUSD wf4s: 0,590, 0,276, 0,454; **USDJPY 10k: 0,853 y 0,868**. ⚠ En 8 de 12 lotes la reconstrucción de costes no reproduce SQX en > 5 % de las variantes (hasta el 87 %) y el estado se emite igual; `nivel_top` y `p_nivel` (neto) dependen de esa reconstrucción. Causa del descuadre: **no verificado**.

17-K. Sello de oos2 y calibración

- **Definición:** `gate.seal(17, "build+oos1" + "oos2", símbolo)` exige (a) paso 17 en la reserva de la política del activo (`assets/_policy.yaml`), (b) regla escrita en `criteria.yaml`, (c) registro `AlgoData/ledger/wfc_calibration.json` con `criteria_sha256 == hash(entrada "17")` y `abre: true` (`ledger/gate.py:86-123`). Se pregunta **antes** de abrir ninguna fila de oos2 (`one.py:77`, `report.py:59`). El registro lo produce `python3 -m studies.optimisation.wfc.calibrate`

(`calibrate/README.md`): familias sintéticas de 300 variantes, 15 años dentro, fuera 1/2,75/5 años, ventaja = Sharpe anualizado del exceso semanal {0,25; 0,5; 1,0}, costes {−0,1; −0,25; −0,5}, `own_noise 0,4` , `order_spread 0,5` , 300 familias por escenario. `abre` se calcula con dos celdas: falso PASA sin ventaja ≤ 0,40 (`wfc.calibration.max_false_pass`) y falso NO PASA con ventaja 0,5 y 2,75 años ≤ 0,25 (`max_false_fail`); registrado: **0,02 y 0,22** → **`abre: true`** .

- **Tasas medidas** (`wfc_calibration.json` , 2,75 años fuera, `own_noise 0,4` , 300 familias/celda, criterio completo / sin nulo en el nivel): | escenario | PASA | LIMBO | NO PASA | (sin nulo en nivel: PASA) | |---|---|---|---|---| | sin ventaja (Sharpe 0) | 0,02 | 0,50 | 0,48 | 0,49 | | coste −0,25 | 0,04 | 0,31 | 0,65 | 0,31 | | ventaja 0,25 sin orden | 0,13 | 0,60 | 0,27 | 0,70 | | ventaja 0,5 sin orden | 0,20 | 0,58 | 0,22 | 0,75 | | ventaja 0,5 con orden | 0,43 | 0,49 | 0,09 | 0,91 | | ventaja 1,0 sin orden | 0,51 | 0,42 | 0,07 | 0,89 | | ventaja 1,0 con orden | 0,92 | 0,08 | 0,00 | 1,00 | Con `own_noise` 1,0 (variantes menos parecidas) y ventaja 0,5 con orden: PASA 0,58; con 0,2 (más parecidas): 0,47. **Potencia: con una ventaja de Sharpe 0,5 el criterio da PASA solo en el 20–43 % de los casos y NO PASA en 9–22 %.** Un NO PASA es irreversible.
- ⚠ **El registro se midió con un criterio distinto del que corre:** `calibrate/README.md:38-43` dice que el registro en disco (2026-10-07: 0,02 y 0,22) se midió «con familias dibujadas y volteadas día a día», mientras el código vuela por **semana**; la re-medición con `--set seed=48` y el nulo semanal (falso PASA 0,03 / 0,44 sin nulo; falso NO PASA 0,13) **no está registrada**. El hash lo liga solo al texto de la entrada `"17"` de `criteria.yaml` , no a la versión del código ni de `config.yaml` . El `WORKFLOW.md` lo resume: «midió el nulo del nivel, no el criterio tal como se escribió».
- ⚠ **Ensayo con lotes reales vacío:** `tasas.ensayo_build_oos1 = {lotes: [], tasas: null}` ; la calibración nunca se ha contrastado con la distribución real de ρ de lotes con `variant_trades.parquet` (ahora existen 16).
- ⚠ La calibración sintética (`sd_rho` p50 ≈ 0,22, rango 0,05–0,42) subestima la dependencia de los lotes reales (`nulo_p5/p95` de ±0,20 a ±0,74 ⇒ sd de ρ hasta ≈ 0,45).

Orden de aplicación (17)

`gate.allow/seal` → leer `metrics.parquet` y las operaciones de los 3 tramos → exceso por operación (neto, bruto) → **por composición** (c1, c2, c3): filtros (controles, suelo dentro, sello, duplicados, suelo fuera, sin valor) → ¿≥ 30 y la madre opera fuera? → nulo semanal (2000) → ρ, p+, p−, clase del orden → top 20 % y `p_nivel` → anotaciones → solo c1: `estado` (tabla) y `rho_sin_suelos` → `descuadre_sqx` → escribir `wfc.json` , `estudios/wfc.*` y 3 filas del ledger por composición (`look.log_all` ; los tramos leídos).

Ejemplo de cadena (USATEC 1.11.277, H1, 2026-10-11)

16.5: 1004 variantes → 17: −3 controles → −0 suelo dentro → −0 suelo fuera → −273 duplicados → **728 variantes** (6 vivos; suelo `15·6 = 90` ; `n_top 146`). c1: ρ = −0,055, nulo [−0,394; +0,416], p+ = 0,578, p− = 0,422 → **neutro**; `nivel_top` +774/año (top dentro 3660; todas fuera 1051), `p_nivel` = 0,404, R_sel = −0,66 → top > 0 pero no supera el nulo → **LIMBO**. USATEC 6.22.453: 1004 → 999; ρ = +0,177 (p+ = 0,218, neutro); `nivel_top` = −91 → **NO PASA**. Resumen del autopiloto: «17 judge: 396 hechos; … 1 en limbo» (`resumen.md`).

Población, 12 lotes con `wfc.json` : **PASA 0 · LIMBO 9 · NO PASA 3**.

Tiempos y recursos (17)

`wfc.json` `wall_s` 4,55–4,69 s por madre (lectura de operaciones de 3 tramos, nulo 2000×, 3 composiciones); fila «17 blind» = 6 s para 2 madres. El paso no usa SQX. RAM/núcleos: **sin dato** (no hay medida en `perf/` ; el catálogo `profiling/budgets.csv` solo contiene la fila antigua `variants`).

Discrepancias 17 (⚠)

1. `WORKFLOW.md:46` (p_nivel «ya no decide») frente a código, `criteria.yaml` y registro (decide).
2. Dossier `criterios-paso-17-wfc-2026-10-07` : describe cuatro composiciones, Fisher, Fiable/Indeciso/No fiable, persistencia del top, ratio de Sharpe: **todo eliminado**; solo sirve para el porqué.
3. Propuesta §4.3: nulo **por día**; código: **por semana** (`flip.signs(block_weeks=1)` sobre `engines/weekly`).
4. Propuesta §6 tabla: neutro/positivo + top gana = PASA; código añade `p_nivel ≤ 0,05` (decisión 2026-10-08).
5. Registro de calibración medido con el volteo por día; ensayo con lotes reales vacío (17-K).
6. `descuadre_sqx` alto en 8 de 12 lotes (hasta 0,87) sin consecuencia en el estado (17-J).
7. Duplicados: 273 de 1004 en USATEC 1.11.277 (27 %) → n efectivo menor; el `k` de `15·k` sale de `nunique > 1` de la tabla de parámetros (no del brief del SPP).
8. Dossiers antiguos hablan de que la nube/ el 17 «no escriben hechos / el juez no los lee»: ya los escriben y el juez los lee (396 hechos en `resumen.md`).

Pasos 18 (CSCV), 18.5 (superficies por mercado) y 19 (Walk Forward Matrix): criterios tal como están implementados y corridos

Fecha de lectura: 2026-10-11. Autoridad: código y `ledger/thresholds.yaml` > corridas en `AlgoData` > dossiers. Rutas relativas a `/home/sergioguslw/Desktop/AlgoProject` salvo `AlgoData/` (= `~/Desktop/AlgoData`). «Hecho» = el nombre con que el piloto lee el número (`pipeline/autopilot/criteria.yaml`).

0. Lo que hay que saber antes de leer nada (resumen para el estadístico)

1. Los tres pasos leen un panel SEMANAL de exceso neto por operación (P/L reprecificado con costes del tramo, menos la deriva del mercado del término `d` de la puerta), cada operación entera en la semana en que cierra, semanas sin operación = 0. Sólo el 19 usa el P/L de SQX (con recargo de costes) en lugar de reprecificar.
2. Ningún paso corta: los tres tienen `cut: null` en `criteria.yaml` (anotan y juzgan, no borran). Los cuatro estados (PASA / LIMBO / NO PASA / sin dato=LIMBO) los consume el paso 20 con la lectura provisional `peor` (WORKFLOW línea 50; el 20 sí corta lo que sale DESCARTAR desde 2026-10-11).
3. Los tres leen `oos2` (reservado, bloque ciego 17/18/18.5/19), cada uno con su ventana (§1, §2, §3).
4. Corridas reales con el código actual (todas las del 2026-10-10 y 11, `Test_*`, 1 batch = 1 madre, 700-1.000 variantes): CSCV 10 madres, 10 PASA, 0 LIMBO, 0 NO PASA. 18.5 13 madres con `variant_trades.parquet` (más 3 USDJPY del 10-08 sin oos2), 0 NO PASA, 2 LIMBO. WFM 10 madres, 7 PASA («no descarta»; ninguna «reoptimizar gana»), 3 LIMBO, 0 NO PASA. Los criterios casi no discriminan hasta ahora: ver cada «punto débil».
5. Casi todos los umbrales son del dueño pero marcados «provisional hasta calibrar» (CSCV `p_perdida_fail`, 18.5 `min_weight` y `premium_percentile`, WFM `skew_warn`). La calibración con población sin ventaja y ventaja plantada que se prometió no consta como hecha salvo la tabla del README del CSCV (paneles sintéticos).

Orden de la cadena (WORKFLOW.md:47-50, 139, 166; `.../tiempos.md` de la corrida USATEC)

16.5 variantes (`variant_trades.parquet`) → **18.5 lectura que decide (build+oos1, sin oos2)** → bloque ciego: 17 (WFC), **18**, **18.5 oos2** → juicios (18.5, 17, 18) → **19** (reapertura de SQX, tarea WFM, export, estudio Python) → 20 (análisis conjunto). Ningún paso filtra a otro (WORKFLOW:168: «No se filtra con la WFM antes de correr las demás»).

PASO 18 — CSCV y PBO

Propósito. «¿Elegir la mejor variante del pasado sirve fuera?» Método de Bailey-Borwein-López de Prado-Zhu (PBO) sobre el lote de variantes de UNA madre (`studies/optimisation/cscv/README.md`). Puede descartar, nunca confirma: PASA = «no descarta».

Entradas. `<batch>/variant_trades.parquet` (leg × market `Main`; lo escribe `sqx.variants.varianttrades`), `metrics.parquet` (contrato C3: `variant_id`, `stratum`, `param_*`), el activo (`assetdata.load`), barras del TF del lote y `studies/screening/gate/config.yaml#drift` (`window: year`, `variant: d`). Código: `studies/optimisation/cscv/{report,many,contract}.py`, `inputs/batch.py`, `measure/{cscv,null,outputs,queue}.py`, `verdict/state.py`; config `studies/optimisation/cscv/config.yaml` (los cortes son filas `ledger:` de `ledger/thresholds.yaml:47-103`). **Salidas.** En `<batch>/estudios/`: `cscv.json` (contrato + `facts` + `rendimiento`), `.html`, `.md`, `distribuciones.parquet` (los nulos), `cscv_particiones.parquet`, `cscv_sorteos.parquet`; caché `cscv_weekly.parquet` (semanas × (net|gross, variante), clave = hash de {mtime/size de trades, drift, costes, ventana, TF}) y `cscv_variants.parquet`. Ficheros por paso del piloto: `autopilot/<P>/<run>/paso-18/{hechos.parquet,verdict.csv}`. **Ventana.** TODO el histórico, build+oos1+oos2: de `assetdata.segment_start(build)` al fin de oos2 (`inputs/batch.py:34`). Ej. USATEC H1: 768 semanas, 2012-01-16 a 2026-09-28. `criteria.yaml` `"18"`: `segment: build+oos1+oos2`, `cut: null`.

18.0 Construcción del panel (previo a los criterios; `inputs/batch.py`)

- Excess por operación = `engines.variants.excess.per_trade`: P/L reprecificado (`reprice.real`, costes del tramo de cada pierna, `reprice.costs(feed, leg)`) menos `tradelevel.drift_term` (deriva del mercado en la ventana natural del año, variante `d` = movimiento por ventana + operación ciega de misma duración; dueño 2026-10-07, «d deja 0,017 de t de sesgo por lado»). Neto = exceso; bruto = neto + coste (comisión+swap+spread+slippage) (`engines/weekly/panel.py:52`).
- Se agrega por semana de cierre (lunes de la semana natural; `panel.monday`), cada operación entera, 0 donde no cerró nada. No hay marcado a mercado de operaciones abiertas.

- Filtros (`batch.screen` , `inputs/batch.py:123`): controles fuera (canarios, pares inertes; `filters.CONTROLS`), variantes sin operaciones fuera, duplicados exactos (misma lista de operaciones, `engines.variants.fingerprint` , la madre primero) fuera, suelo `cscv.floor_per_param = 15` operaciones en el histórico completo por parámetro VIVO (7 vivos → 105), suelo `cscv.floor_per_year = 10` operaciones/año. Si la MADRE no llega a un suelo, avisa y ese suelo no se aplica. Menos de `cscv.min_variants = 30` tras filtros → «sin dato», estado LIMBO (`state.no_data`). La madre siempre entra. Los tres suelos son del dueño (2026-10-07).
- Ejemplo real `Test_USATEC_H1_ruptura_leaseWeb/Strategy_6.22.453` (2026-10-11 09:32): lote 1.004 (1.000 LHS + 3 canarios + 1 origen), 3 controles fuera, 0 sin operaciones, 2 duplicados (0,2 %), 0 bajo suelo → 999 entran. La otra madre del mismo lote (1.11.277): 728 entran, 27,3 % de duplicados.

18.1 Núcleo: CSCV (`measure/cscv.py`)

- Se corta el histórico en `cscv.blocks = 16` bloques contiguos de longitud casi igual (`np.array_split` , no se pierde ninguna fila: ≈48 semanas/bloque en USATEC). Particiones: C(16,8) = 12.870 (se calculan las C(15,7)=6.435 mitades que contienen el bloque 0, leídas dos veces, dentro/fuera intercambiados). Dueño sube 12 → 16 el 2026-09-30 (C(12,6)=924 antes).
- En cada partición: Sharpe SEMANAL por variante en la mitad «dentro» (media/desv. con ddof=1, desde sumas de bloque; 0 si la mitad no se movió); elige `argmax` (primer empate); se mide su Sharpe «fuera» y su rango relativo ω = (1 + nº de variantes con Sharpe fuera estrictamente menor) / (N+1).
- Semilla raíz `cscv.seed = 20260922` ; cada sorteo siembra su propia corriente desde (semilla, tipo, índice): resultado idéntico con 1, 8 o 96 procesos.

18.2 Criterios (numerados) y orden de aplicación

Orden de decisión en código (`verdict/state.py:9-27`): (a) `n < 30` → LIMBO; (b) `p_perdida ≥ 0,5` O `pbo_sobre_p95` → NO PASA; (c) `p_perdida_sobre_p95` → LIMBO; (d) si no, PASA. En `criteria.yaml` (`"18"` , líneas 252-267) es la misma gramática: la regla sin `else` falla, la que tiene `else: limbo` degrada, `missing: limbo` .

C18-1. Probabilidad de pérdida de la elegida (`cscv.p_perdida`) — CORTA NO PASA (lo marca; `cut: null`)

- **Pregunta:** ¿la variante elegida dentro pierde fuera (Sharpe ≤ 0) en la mitad de las particiones o más?
- **Definición:** `p_perdida = media(sr_out ≤ 0)` sobre las 12.870 particiones (`outputs.py:68`). **NO PASA si ≥ `cscv.p_perdida_fail = 0,5`** (dueño 2026-10-07; `criteria.yaml:262-263` `pass: "< 0.5"` , sin `else` = fallo). Sin dato → LIMBO. Población: los ~1.000 del lote (una madre). Escala: absoluta, NO se lee contra el nulo.
- **Por qué:** «si la elegida pierde fuera en la mitad, elegir es peor que cara o cruz» (propuesta §6.1). Ledger lo marca «provisional hasta calibrar con población sin ventaja y con ventaja plantada».
- **Punto débil:** con un panel sin ventaja y de Sharpe semanal bajo el corte 0,5 suspende por azar: README mide 14,7 % de NO PASA con 60 columnas independientes de Sharpe semanal 0,03, 29,3 % con columnas correladas 0,6 y Sharpe 0,03 (las variantes de una madre SON correladas); WORKFLOW:47 ⚠ lo cita (29 %). Un Sharpe medio positivo en todas las variantes hace p_perdida ≈ 0 aunque no haya nada que elegir (es el nivel, no el orden).
- **Ejemplo real:** 10 madres: p_perdida 0,002 a 0,318 (mediana ≈0,08); ninguna ≥ 0,5. Máxima: `Test_XAUUSD_H1_ruptura_wf4s/Strategy_5.2.399` 0,318 (n=1.001). Mínima: NIKKEI `Strategy_17.13.284` 0,002.
- **Coste:** ver «Tiempos».

C18-2. P de pérdida contra el p95 de su propio nulo (`cscv.p_perdida_sobre_p95`) — MARCA LIMBO

- **Pregunta:** ¿elegir pierde más a menudo que el 95 % de las elecciones entre variantes que valen TODAS el Sharpe mediano del panel?
- **Definición:** `int(p_perdida_real > quantile(nulo.p_perdida, 95/100))` (`state.py:58,63` ; `cscv.p_perdida_null_pct = 95` , dueño 2026-10-08; estricto `>`). Nulo = 200 sorteos (`cscv.null_draws` , dueño 2026-10-07). `criteria.yaml:264-265` `pass: "< 1"` , `else: limbo` . Sustituye al corte fijo 0,25 (`cscv.p_perdida_limbo` , 2026-10-07) que quedaba bajo la mediana del nulo sin ventaja (0,36-0,38).
- **Por qué:** distinguir pérdida «por el balancín» (con medias iguales en dos mitades, la elegida dentro tiende a quedar mala fuera) de pérdida de más.
- **Punto débil:** el propio README mide que NUNCA supera su 5 %: sobre su nulo la regla rechaza 5,2 % ± 1,1 (independientes, Sharpe 0,10), 0,5 % (correladas 0,2) y 0 % con correladas 0,6; «en lotes reales espera LIMBO casi nunca». El nulo se centra en la mediana MUESTRAL del panel, así que la pérdida real queda en el centro del nulo («lo que comparten las variantes mueve ambos juntos»). Con p95 del nulo ≥ 0,5 no puede dispararse (salta antes C18-1). 200 sorteos → el p95 es la estadística de orden 190, error típico ≈0,03 (config `null.draws`).
- **Ejemplo real:** 0 de 10. p95 del nulo 0,14 (USATEC 6.22.453) a 0,92 (XAUUSD 5.2.399, donde la regla es inalcanzable). Real/p95-nulo p.ej. 0,0098 / 0,1436.

C18-3. PBO contra el p95 de su propio nulo (`cscv.pbo_sobre_p95`) — CORTA NO PASA (marca)

- **Pregunta:** ¿la variante elegida dentro cae en la mitad mala fuera más a menudo de lo que lo hace con nada que elegir?
- **Definición:** `PBO = media(ω ≤ 0,5)` sobre las 12.870 particiones; `pbo_sobre_p95 = int(PBO_real ≥ percentil95(PBO_nulo))` con `np.percentile` de los 200 sorteos (`flip.tails`, `state.py:54,62`). `criteria.yaml:266-267` `pass: "< 1"`, sin `else` = NO PASA; `missing: limbo`. No hay umbral numérico fijo: el corte es el p95 de su nulo.
- **Nulo** (`measure/null.py`): por sorteo, cada columna se estandariza (media 0, desv. 1, ddof=1), todas reciben la MISMA media = mediana del Sharpe semanal del panel (`m`, «el Sharpe mediano del panel en semanas»; lectura elegida, pregunta abierta al dueño), se re-sortean semanas con bootstrap estacionario de Politis-Romano (bloques geométricos de media `cscv.null_block_weeks = 4`; las MISMAS filas para todas las columnas, conserva la dependencia entre variantes) y se pasa el CSCV entero. Se hace en neto y en bruto (5 × 200 CSCV por madre con 3 Sharpe plantados).
- **Por qué:** el PBO sin nada que elegir ronda 0,50 con desviación grande (dossier: media 0,50, sd 0,19, y >0,5 en el 51 % de sorteos): un corte fijo 0,5 suspende «a cara o cruz», por eso se lee contra su nulo. Banda de control: la mediana del PBO del nulo debe caer en [0,4; 0,6] (`null.median_low/high`) o avisa («el remuestreo no deja el panel sin nada que elegir»). En las 10 corridas estaba en banda (0,48-0,52).
- **Punto débil:** (i) una sola cola superior al 5 % por madre sin corregir multiplicidad entre madres ni entre los ~3 criterios; (ii) al ser sólo p95 de un nulo de 200, el 5 % de falsos NO PASA es nominal y el README lo mide por debajo (conservador); (iii) potencia baja: con un Sharpe anual plantado de 0,1 en el 10 % de las columnas la potencia (PBO real < p5 del nulo) sale 0,05-0,40 (10 corridas), 0,26-0,99 con 0,2 y 0,80-1,00 con 0,3 (`potencia.d01/d02/d03`); (iv) el PBO depende del orden, no del tamaño de las diferencias (un orden estable con diferencias minúsculas da PBO bajo; un panel plano da ≈0,5).
- **Ejemplo real:** PBO real 0,279 a 0,825 (mediana ≈0,56) contra p95 del nulo 0,809-0,871: 0 de 10 lo alcanza. Ej. USATEC 6.22.453: PBO 0,557, nulo p5 0,138 / mediana 0,480 / p95 0,824, `pbo_p`=0,657. Notar que PBO real > 0,5 ocurre en 5 de 10 (0,727; 0,825; 0,741; 0,557; 0,580) y es PASA.

Anotaciones (nunca deciden; `facts` en `cscv.json`)

- `pbo_p` (cola inferior: (1+#null ≤ real)/(1+B), el real contado) y `pbo_p_mas`; `bruto.pbo` y su nulo en bruto; `orden_estable = int(PBO ≤ p5 del nulo en NETO y en BRUTO)` (si 1, el PASA se escribe «orden estable»): en las 10 corridas = 0, nunca se alcanzó.
- Recta de degradación (Sharpe fuera sobre Sharpe dentro, un punto por partición): `beta`, `alfa`, `r2` ajustado. β es negativa por construcción (las dos mitades suman el mismo total): se lee contra su nulo (`beta_p`); real −0,68 a −1,01, nulo mediana ≈ −0,96.
- Recorte del Sharpe `hc = 1 − mediana(Sharpe fuera)/mediana(Sharpe dentro)` de la elegida (NaN si la mediana dentro ≤ 0): 0,34-0,81 (mediana ≈0,52).
- Dominancia estocástica de elegir sobre elegir al azar (CDF del Sharpe fuera de las elegidas vs todas las variantes, `dominance.tolerance = 0,01` para el primer orden, segundo orden estricto sobre integrales acumuladas, 512 bins): sale 1ª orden en 3 de 10; el nulo la da en ≈42 % de sorteos.
- Selección: `distintas` (166-297 variantes distintas elegidas al menos una vez) y `peso` de la más elegida (0,054-0,397; la máxima, DAX40 14.16.487, 39,7 %).
- La madre: `puesto_mediano`, `sobre_mediana`, `p_perdida` propia (0,004-0,132) y dominancia; ⚠ la madre se eligió mirando build, que está «dentro» en muchas particiones (lo avisa el código).
- Potencia: Sharpe anual δ ∈ {0,1; 0,2; 0,3} plantado en el 10 % de columnas igualadas (`power.share = 0,10`), δ/√52 semanal, redibujado por sorteo.

18.3 Ejemplo de cadena (corrida 2026-10-10/11, 10 madres de 5 lotes)

lote (madre)	n entran	estado	p_pérdida (p95 nulo)	PBO (p95 nulo)
USA500 H1 `20.17.293`	1.001	pasa	0,190 (0,650)	0,466 (0,809)
USA500 H1 `7.11.427`	1.001	pasa	0,062 (0,457)	0,726 (0,825)
DAX40 H1 `14.16.487`	1.001	pasa	0,154 (0,702)	0,327 (0,827)
NIKKEI M30 `17.13.284`	1.001	pasa	0,002 (0,191)	0,384 (0,818)
NIKKEI M30 `20.11.454`	1.001	pasa	0,005 (0,273)	0,279 (0,829)
XAUUSD H1 `1.14.418`	994	pasa	0,174 (0,397)	0,580 (0,843)
XAUUSD H1 `5.2.399`	1.001	pasa	0,318 (0,919)	0,292 (0,829)
XAUUSD H1 `9.19.426`	996	pasa	0,108 (0,283)	0,825 (0,871)
USATEC H1 `1.11.277`	728	pasa	0,024 (0,215)	0,741 (0,869)
USATEC H1 `6.22.453`	999	pasa	0,010 (0,144)	0,557 (0,824)

Entran 10 madres, salen 10 PASA (0 LIMBO, 0 NO PASA, 0 «orden estable»). Fuente: `AlgoData/strategyPermutations/Test_*/Strategy_*/estudios/cscv.json`. En el paso 20 del piloto el verdict.csv del 18 sale `pass` (USATEC) y `MANTENER`. Corridas anteriores (2026-10-04/06, otro código: lectura sobre curva diaria, argmax/plateau, PBO>0,5 = fail) dejaron 199 `cscv.json` de formato antiguo en `AlgoData/strategyPermutations`; no son comparables (p. ej. NIKKEI `Trade_…volatilidadL`: 8 pass / 2 fail con PBO 0,642 y 0,653 según el dossier).

18.4 Coste

- Medido (README, 2026-10-07, 48 núcleos físicos con carga 105): 300 variantes × 200 sorteos 3,5 s reloj / 72 s CPU; 1.000 × 200 = 8,9 s / 243 s; 1.000 × 1.000 = 40,5 s / 1.195 s. Un CSCV de 1.000 variantes = 0,18 s en un hilo; 107 MB máx. por proceso (README) — en corridas reales `mem_proceso_max_mb` 547-1.001 MB y padre ≈2 GB.
- Reales (`rendimiento` de los 10 `cscv.json`): `wall_s` 3,6-6,6 s (cuando hay que reprecificar semanas: `panel_s` ≈2,4 s; con caché 0), 48 procesos (`core.fanout` nunca usa más de los físicos aunque `perf.workers: 96`), 1.001 tareas, `cpu_s` : nulo ≈64 s, nulo bruto ≈55 s, cada δ ≈54 s. Presupuesto `perf.budget_s: 60` .
- Paso en el piloto (USATEC): «18 blind» 18 s; XAUUSD 32 s; DAX40 9 s; NIKKEI 21 s; USA500 21 s (incluye el reprecio de semanas la primera vez).
- RAM del orden de GB; sin dato de disco salvo `variant_trades.parquet` 232 MB por lote (USATEC) y `cscv_weekly.parquet` 9,4 MB.

18.5 Discrepancias del paso 18 (⚠)

- ⚠ El dossier `docs/AgentPDFs/criterios-paso-18-cscv-2026-10-07.md` describe el estudio ANTERIOR (curva diaria `equity.parquet` , reglas «Argmax»/«plateau», DSR, `pbo_argmax > 0,5 = fail` , «ninguna regla en criteria.yaml», 8 pass/2 fail en `Trade_NIKKEI225_M30_volatilidadL`). El código actual (commit `ca7dd56` , «studies: rework cloud, CSCV…») usa exceso semanal, 16 bloques y cortes contra el nulo propio, y `criteria.yaml` YA tiene la entrada `"18"` . Manda el código.
- ⚠ WORKFLOW:47 y los README de `cscv` / `marketSurfaces` dicen «sin lote real con `variant_trades.parquet` » / «ningún lote real lo tiene»: obsoleto, hay 10 lotes con él (2026-10-10/11).
- ⚠ Comentario de `criteria.yaml:255` «Los cortes 0,25 y 0,5 son provisionales»: el 0,25 ya no existe (sustituido por el p95 del nulo, 2026-10-08).
- `cscv.p_perdida_fail = 0,5` sigue «provisional» en el ledger; no consta la calibración que se prometió. La tabla del README (paneles sintéticos de 60 variantes × 400 semanas, 100 sorteos, 300-400 paneles por celda) es la única medida de falsos positivos; no hay medida con paneles reales de 1.000 variantes correladas como las del lote.

PASO 18.5 — Superficies por mercado

Propósito. «¿El orden de las variantes en el mercado principal se repite en los demás?» (README `studies/optimisation/marketSurfaces/README.md` ; reescrito 2026-10-07 desde la propuesta del dueño). No habla con SQX ni corre backtests; reprecifica las operaciones de cada mercado en sus propias barras con su propia ficha de costes. **Entradas.** `variant_trades.parquet` (`leg ∈ {build, oos1, oos2}` , `market ∈ {Main, <feeds>}`), `segments.parquet` (NetProfit SQX por variante/tramo/mercado, para la comprobación de costes), `metrics.parquet` , `assets/_markets.yaml` (mercados declarados por activo, categoría `family` / `structural` …, `data_from` y grupos `near_identical`), `studies/screening/gate/config.yaml#drift` , suelos de `engines/variants/config.yaml#panel` (filas `wfc.*` del ledger). **Salidas.** `estudios/marketSurfaces.{json,html,md}` , `marketSurfaces_detalle.json` , `mercados.csv` , `verdict.csv` , `distribuciones.parquet` . Hechos aplanados `marketSurfaces.*` (`orden.estado` como número: positivo=1, neutro=0, perverso=−1; `pipeline/autopilot/batchfacts.py:10`). **Partido en dos** (dueño; `criteria.yaml` `"18.5"` líneas 268-286): lectura que DECIDE (build+oos1 en el principal y en los demás), se corre y se juzga tras el 16.5 y ANTES del bloque ciego, sin tocar oos2 (la pierna oos2 ni siquiera se pide al parquet); lectura oos2 (principal en build+oos1, otros en oos2) se calcula dentro del bloque ciego con `--oos2` y sólo puede añadir LIMBO. El oos2 del principal no se lee nunca. `cut: null` ; `segment: build+oos1` .

18.5.0 Qué se mide (previo a los criterios)

- **Exceso** por mercado = P/L reprecificado a mercado (`engines/weekly/panel` , `reprice.costs(feed, leg)`) menos deriva `d` ; BRUTO (antes de costes) ordena las variantes; NETO se usa sólo en la prima.
- **Variantes utilizables en el principal:** no duplicado exacto (huella de las operaciones `Main` en build+oos1, la madre primero) Y al menos `floor_oos_per_year = 10` operaciones/año de ventana Y `floor_is_per_param = 15` × parámetros vivos (filas `wfc.floor_*` , dueño 2026-10-07). Conjunto común de un mercado m: además llegan al suelo por año en m (`one.py:62-67`). Un mercado se acorta por `data_from` : años = años de la pierna con datos.

- **Mercado entra en la media** si tiene ≥ `markets.min_common = 50` variantes comunes y como máximo `markets.max_ties = 0,5` de ellas empatadas (empate = otra variante con exactamente el mismo exceso bruto; `order.tie_share`) (`reading.py:79-80`).
- **Pesos** (`order.weights`): 1; los mercados de un grupo `near_identical` presentes en la media comparten un peso total 1 (p. ej. [USA500, USATEC, DJ30] pesan 1/3 cada uno, 1/2 si quedan dos de tres). Por eso «mercados (peso)» de las corridas es 9-16 con 11-16 mercados declarados.
- **rho_m** = Spearman (rango medio en empates) entre el exceso bruto en el principal y en m sobre las variantes comunes; **rho conjunta** = media ponderada de rho_m (los mercados no finitos salen con su peso).

18.5.1 Criterios

C185-1. Orden conjunto (`marketSurfaces.orden.estado`) — NO PASA / LIMBO / PASA, DECIDE

- **Pregunta:** ¿lo mejor en el principal tiende a ser lo mejor (positivo) o lo peor (perverso) en los demás, más allá del ruido?
- **Definición** (`verdict/call.py:10-26`): `sin_dato` si rho no finita o `peso < markets.min_weight = 5,0`; si no, `positivo` si `p_mas ≤ alpha`, `perverso` si `p_menos ≤ alpha`, `neutro` en otro caso, con `alpha = marketSurfaces.alpha = 0,05` (una cola cada lado). Estado del paso: perverso → **NO PASA**; sin dato → LIMBO; positivo o neutro → PASA («la zona viaja» / «no descarta»). `criteria.yaml:279-280`: `fact: marketSurfaces.orden.estado, pass: ">= 0", missing: limbo` (perverso = −1 falla, ausente = LIMBO).
- **Nulo** (`engines/weekly/flip.py`): `null.draws = 2.000` vectores de signos; un signo ±1 por SEMANA (bloques de `null.block_weeks = 1` semanas), el MISMO para todas las variantes y todos los demás mercados; el principal no se voltea; las semanas son las de la ventana de las piernas del principal en cada mercado (la semana que comparten dos piernas toma un signo). `p_mas = (1+#nulo ≥ real)/(1+B)`, `p_menos = (1+#nulo ≤ real)/(1+B)`. Semilla `marketSurfaces.seed = 0` (hijas por lectura).
- **Por qué:** una rho sobre variantes de una misma estrategia mide si un parámetro bueno lo es también en otro mercado; voltear las semanas de los demás mercados preserva la estructura entre variantes y rompe la relación con el principal.
- **Punto débil:** (i) potencia baja: las curvas plantadas del propio estudio (`potencia.r01/r02/r03`, familia con rho verdadera 0,1/0,2/0,3 mezclada por copula gaussiana, 200 familias) dan en las 13 corridas 0,135-0,845 (r=0,1), 0,200-1,0 (r=0,2) y 0,395-1,0 (r=0,3): con una transmisión de 0,1-0,2 el paso casi nunca la ve; (ii) la rho propia del principal consigo mismo (build contra oos1, `auto.rho`) ya es ≈0 en varias madres (−0,06, −0,004, 0,004, −0,02), así que «neutro» es lo esperado incluso en una estrategia con ventaja; (iii) 11-15 mercados casi independientes pero muchos de la misma familia (índices) → correlación entre rho_m no modelada salvo por el peso de los grupos `near_identical`; (iv) un lado «perverso» y uno «positivo» con 0,05 cada uno son dos pruebas; (v) el peso mínimo 5 es una lectura del agente que implementó el paso (la propuesta decía «menos de 5 mercados en la media (con los pesos)») y el ledger lo marca provisional, «pendiente de que el dueño lo confirme»; (vi) correlación entre semanas vecinas (cierre de operaciones a caballo de un fin de semana): si más de `straddle_max = 0,1` de las operaciones de los otros mercados cierran lunes/martes tras un fin de semana abierto se repite el nulo en bloques de 2 semanas (`orden.estado_2sem`), sólo avisa y no cambia el estado.
- **Ejemplo real (13 madres, 2026-10-10/11):** las 13 resultan `neutro`. rho conjunta −0,064 a +0,185; p_mas 0,085 (XAUUSD `1.14.418`, rho +0,185, la más cercana a «positivo») a 0,751; p_menos máximo 0,915. USATEC `6.22.453`: rho +0,018, p+ 0,396, p− 0,604, peso 10, 11 mercados, 999 variantes. 0 perversas, 0 positivas.

C185-2. Prima del 20 % mejor (`marketSurfaces.prima.bajo_p5`) — MARCA LIMBO

- **Pregunta:** ¿lo que elegiría un optimizador en el principal (su top 20 %) rinde en los otros mercados más que la mediana, en unidades de dispersión?
- **Definición** (`measure/premium.py`): top = variantes utilizables con exceso NETO del principal ≥ cuantil (1 − `premium.top_share = 0,2`) de las utilizables; prima por mercado = (mediana del exceso neto del top − mediana de todas) / rango intercuartílico, sobre el conjunto común; prima conjunta = media ponderada sobre los mismos mercados; nulo = los mismos 2.000 signos aplicados a los paneles netos. `bajo_p5 = (prima ≤ percentil` premium.percentile = 5`del nulo)`. `criteria.yaml:281-282`: `pass: "< 1", else: limbo, missing: limbo`. Sólo puede dar LIMBO.
- **Punto débil:** el top se elige UNA vez sobre el exceso neto del principal y se interseca con cada conjunto común (no se re-elige por mercado; lectura abierta del dueño); la prima mezcla rangos entre mercados muy distintos; el percentil 5 «sin calibrar» (propuesta §10.1; ledger lo dice); la prima neta incluye costes de cada mercado y las comprobaciones de coste fallan (ver ⚠).
- **Ejemplo real:** 12 de 13 por encima del corte; `Test_USDJPY_M30_doubleROCLong_10k/Strategy_12.21.437` (2026-10-08, 16 mercados, peso 16): prima −0,203 ≤ nulo p5 −0,124 → LIMBO. USATEC `6.22.453`: prima +0,043, corte −0,135 (pasa).

C185-3. Orden en oos2 no perverso (`marketSurfaces.oos2.perverso`) — MARCA LIMBO (bloque ciego)

- **Definición:** misma regla que C185-1 sobre oos2 de los otros mercados (principal en build+oos1), mismos `alpha`, `min_weight`; semilla aparte (hija 1), sin curva de potencia. `perverso = 1` → LIMBO (`criteria.yaml:283-284`: `pass "< 1", else limbo, missing pass`). En una lectura oos2 sin dato escribe `oos2.sin_dato = 1` y NO `oos2.perverso` (dueño 2026-10-08).
- **Ejemplo real:** 0 perversas en 10 lecturas con oos2.

C185-4. oos2 se pudo leer (`marketSurfaces.oos2.sin_dato`) — MARCA LIMBO

- **Definición:** `sin_dato = 1` si la suma de pesos de los mercados con ≥ 50 variantes comunes en oos2 (y empates ≤ 0,5) queda bajo 5 (`criteria.yaml:285-286` , `else: limbo, missing: pass` , dueño 2026-10-08: una lectura corrida y no legible es LIMBO, no PASA).
- **Ejemplo real:** USATEC `1.11.277` : sólo peso 1,0 en oos2 (rho +0,200, p− 0,759) → `sin_dato = 1` → LIMBO (la otra madre del mismo lote, `6.22.453` , lee oos2 con peso suficiente y pasa). Motivo del peso bajo: no verificado; presumible el suelo de 10 operaciones/año × años de oos2 (2024-2026-09 ≈2,75 años, 28 operaciones) con 666 variantes comunes en build+oos1.

Anotaciones (no deciden)

`auto.rho` (Spearman del principal build contra oos1) y `auto.cociente = rho conjunta / rho propia` sólo si `rho_auto > reference.min_rho = 0,1` ; `potencia.r01..r03` ; `variantes` utilizables; `duplicados` (cuota quitada); `costes.descuadre` (máximo del grupo de variantes cuyo P/L reprecificado no reproduce el NetProfit de SQX: tolerancia 1 unidad de cuenta + 1 % de |NetProfit|, aviso `cost_gap` si > `costs.max_share` = 1 % de las variantes de algún mercado); `orden.finde` (cuota «a caballo de fin de semana»); categorías por tipo de mercado (rho por categoría, sólo informativo; una «llamada por categoría» es una mejora argumentada y no hecha).

18.5.2 Ejemplo de cadena

lote (madre)	mercados (peso)	variantes	orden	prima bajo p5	oos2	estado
DAX40 H1 `14.16.487`	11 (9,0)	1.001	neutro, rho +0,081	0	rho −0,211	pasa
NIKKEI M30 `17.13.284`	11 (9,0)	989	neutro +0,143	0	rho +0,187	pasa
NIKKEI M30 `20.11.454`	11 (9,0)	1.001	neutro −0,049	0	−0,129	pasa
USA500 H1 `20.17.293`	11 (10)	1.001	neutro −0,001	0	−0,079	pasa
USA500 H1 `7.11.427`	11 (10)	1.000	neutro −0,055	0	−0,080	pasa
USATEC H1 `1.11.277`	11 (10)	666	neutro −0,064	0	`sin_dato`	**LIMBO**
USATEC H1 `6.22.453`	11 (10)	999	neutro +0,018	0	−0,030	pasa
XAUUSD H1 `1.14.418`	15 (13)	993	neutro +0,185	0	−0,032	pasa
XAUUSD H1 `5.2.399`	15 (13)	1.001	neutro −0,041	0	+0,133	pasa
XAUUSD H1 `9.19.426`	15 (13)	995	neutro +0,099	0	−0,057	pasa
USDJPY M30 (3 madres, 2026-10-08)	16 (16)	787-930	neutro (−0,035 a +0,055)	1 (`12.21.437`)	no corrido	pasa, pasa, **LIMBO** (`12.21.437`)

Entran 13 madres (las 3 USDJPY sin oos2), salen 11 PASA y 2 LIMBO, 0 NO PASA. Lote 1.004 → 666-1.001 variantes utilizables según suelos y duplicados (USATEC `1.11.277` : 666 por 27 % de duplicados + suelos por parámetro vivo). Corridas viejas (2026-10-05, `Test_NIKKEI225_M30_simpleL_*`) tienen `marketSurfaces.json` sin `facts` (formato antiguo, `state: fail`); el dossier decía que 10 de 10 madres fallaban en `Trade_NIKKEI225_M30_volatilidadL` : no es reproducible con el código actual.

18.5.3 Coste

- README: 500 variantes × 11 mercados × 620 semanas × 2.000 sorteos = 18 s (28 s con `--oos2`), lectura (reprecio de operaciones) aparte; `workers: 12` (un mercado cada uno, un hilo BLAS).
- Reales (`summary.segundos`): USATEC 40,6-41,9 s, DAX40 9,0 s, USA500 12,1-15,4 s, NIKKEI 31,8-34,2 s, XAUUSD 89,5-103,8 s (15 mercados), USDJPY 103-107 s (16 mercados). En el piloto: «18.5 blind» 82 s (USATEC), 209 s (XAUUSD), 89 s (DAX40), 106 s (NIKKEI), 17-32 s (USA500 con caché). Memoria: sin dato.
- El juicio en sí: <1 s (columnas «judge» 0 s).

18.5.4 Discrepancias del paso 18.5 (⚠)

- ⚠ El dossier `criterios-paso-18.5-superficies-por-mercado-2026-10-07.md` (1.381 líneas) describe el estudio ANTERIOR (pares de mercados, Spearman bruto y «sin exposición», decil superior + Jaccard, `core/surface/` , «falla en las diez»). El código actual es rho conjunta ponderada + prima del top 20 % + oos2; las 13 corridas actuales no dan ningún NO PASA. Manda el código.
- ⚠ `costes.descuadre` es 0,925-1,0 en las 13 corridas (máximo sobre mercados) y todas llevan el aviso `cost_gap` («El P/L reconstruido no reproduce el NetProfit de SQX en más del 1 % de las variantes de: USA500_M1, DJ30_M1, DAX40_M1, … El exceso bruto de esos mercados no es de fiar», estado `watch`). El aviso es sólo texto: no cambia el estado. Es decir, el ordenamiento

bruto de los otros mercados descansa en una reprecificación que casi nunca reproduce el NetProfit de SQX. Causa: no verificada (candidatas: moneda de cuenta, ficha de costes del tramo, tolerancia). Es el punto estadístico más serio de este paso.
- ⚠ README del 18.5 dice «Not run on a real batch» (2026-10-07): obsoleto, hay 13 lotes reales.
- `markets.min_weight = 5,0` (suma de pesos, no nº de mercados) y `premium_percentile = 5` quedan como lecturas provisionales; `power`: familia plantada por copula gaussiana, alternativa «sembrar la transmisión en las semanas» sin hacer (`POSSIBLE_IMPROVEMENTS.md`).
- Duplicados se deciden sólo sobre las operaciones del principal en build+oos1 (para que la lectura que decide no abra oos2).

PASO 19 — Walk Forward Matrix (WFM) en SQX

Propósito. «Si en lugar de fijar los parámetros se hubieran vuelto a optimizar periódicamente y se hubiese operado con lo recién optimizado, ¿habría ganado la estrategia en `oos2`?» Dos piezas: la TAREA «WFM» de SQX (`sqx/projects/wfm.py`, `/wfm`) y su LECTURA en Python (`studies/optimisation/wfm/`). **Entradas.** Export de la tarea (`AlgoData/raw/<P>/WFM/<día>/wfm/`: `cells.parquet`, `steps.parquet`, `trades.parquet` de las 30 celdas, `main.parquet` con las operaciones de la madre con parámetros fijos, `conditions.parquet`, reglas); `assets` (costes por tramo, `sqx.export.export_wfm`); barras del TF; config `studies/optimisation/wfm/config.yaml` (cortes `ledger:wfm.*`, líneas 1389-1430 de `thresholds.yaml`) y `assets/_build.yaml:448-` (tarea). **Salidas.** `AlgoData/reports/<P>/WFM/<día>/wfm/{verdict.csv,cell_excess.csv,distribuciones.parquet,wfm.json,.html,.md,estrategias/}`; hechos `wfm.*` para el piloto y paso 20 (`autopilot/<P>/<run>/paso-19/{hechos.parquet,verdict.csv}`). **Ventana.** SQX corre sobre `build..oos2` entera (`_build.yaml:450`); el estudio sólo cuenta las operaciones que CIERRAN en `oos2` (2024-01 → 2026-09-30 ≈ 2,75 años en índices; 2023 → 2026-09-30 ≈ 3,75 en divisas; `assets/_policy.yaml`). Entre 67 y 79 % del «fuera de muestra» de SQX cae en build y oos1 (datos ya vistos por la construcción y el paso 8): por eso se juzga sólo oos2. `criteria.yaml` `"19"`: `segment: oos2`, `cut: null`.

19.0 La tarea SQX (qué se fija; `sqx/projects/wfm.py`, `assets/_build.yaml:448-`)

- Rejilla 30 celdas: `runs` 6-16 paso 2 (6 valores: 6, 8, 10, 12, 14, 16 pasadas) × `oos_pct` 20-40 paso 5 (5 valores) = 30; «% OOS» es porcentaje de la ventana de optimización de cada pasada, `period_type: percent`, `optimization_type: floating` (ventana deslizante).
- Fidelidad `wf_type: exact_is_exact_oos` (decisión del dueño 2026-09-24: cada tramo OOS se vuelve a backtestear de verdad y cada paso hace su propia optimización), `max_tests: 5000` por paso y celda (el donante traía 500; sólo se guarda el ganador de cada paso), `distribution_pct: 35`, `step_pct: 4` → `maxSteps = round(2·35/4) = 18`; `precision: 2`.
- Costes: la tarea cobra la ventana entera al spread de **oos1** (`costs_segment: oos1`), no al del último tramo.
- Condiciones por celda (10, `conditions` de `_build.yaml`, umbrales calibrados en 150 celdas de 5 estrategias del ORO del maestro, 2026-09-24; el comentario avisa «en otro activo esos % no valen y los umbrales no se han recalibrado»): oos NetProfit > 0, oos ProfitFactor > 1,05, stability PF > 60, stability NetProfit > 20, stability DrawdownPct < 150, score PF > 85, special WFPctOfProfitableRuns > 50, WFMaxProfitByRunInPct < 50, WFMinTradesInRun > 20, WFMaxPctDDbyRun ≤ 25. Una celda «pasa» con `threshold_pct = 80` % de ellas cumplidas. La estrategia «pasa» el área de SQX si en algún rectángulo `grid_passing_size = 4` × 4 hay `min_squares = 12` o más celdas que pasan (6 posiciones posibles en 6 × 5). SQX MARCA, no borra (`DeleteFailedStrategies=false`) y NO juzga: el estudio decide.
- Todas las demás condiciones de aceptación del donante se apagan (`silence`) antes de escribir éstas (si no, la matriz pasaría todo al 100 %).

19.1 Criterios del estudio (qué decide, qué anota)

C19-1. L1: mediana de las 30 celdas del exceso neto por año en oos2 contra un nulo de volteo semanal — DECIDE

- **Pregunta:** ¿la estrategia reoptimizada gana en el tramo que nadie vio, más allá del ruido?
- **Definición exacta** (`measure/levels.py:10-26`, `run.py:36-78`, `verdict/call.py:9-38`): 1. Por operación de celda (`sample == OOS`, todas las celdas de la madre): `net = Profit/Loss(SQX) − recargo − deriva`, donde `recargo = lotes × valor del punto × (Δspread + 2·Δslippage) + Δcomisión` de oos1 (con que SQX cobró) a oos2 (`excess.recharge`; el swap no cambia; primer orden: stops y objetivos no se mueven, sin redondeo de tick); la deriva es `tradelevel.drift_term` (puerta `d`, medida con las barras de la ventana natural de la operación). El recargo sólo se aplica a lo que cierra en oos2 (los tramos build y oos1 de la tabla por segmento NO se recargan). Segmento = día de CIERRE. 2. Panel semanal oos2: semanas × 30 celdas, cada operación entera en la semana en que cierra (`panel.weekly`). 3. `L1_obs = mediana_sobre_celdas( suma_semanas(celda) ) / años_oos2` (USD de cuenta por año, no normalizado: depende del tamaño/lotaje de la madre). 4. Nulo: `wfm.null_draws = 2.000` vectores de signos semanales (un signo por semana, el mismo para las 30 celdas; `engines.weekly.flip.signs`), `L1_nulo = mediana(signos @ panel)/años`. Semilla `wfm.seed = 0`, una hija por madre (posición en el export, orden alfabético). 5. `p_menos = (1 + #{nulo ≤`

`obs})/(1 + 2.000)` y `p_mas` análogo. 6. Estado: **NO PASA si `p_menos ≤ wfm.alpha = 0,05`**; si no, **LIMBO si `mediana_exceso ≤ 0`** (no gana pero no se puede afirmar que pierda); si no **PASA**, con etiqueta «reoptimizar gana» si `p_mas ≤ 0,05`, «no descarta» si no. `criteria.yaml:297-300`: `wfm.l1.p_menos pass "> 0.05" missing limbo` y `wfm.l1.mediana_exceso pass "> 0" else limbo missing limbo`.

- **Por qué esa estadística:** las celdas son casi copias unas de otras (§19.2), así que se resume con una MEDIANA y no con un recuento (README `wfm`); NO PASA exige significancia porque «una pérdida sin significancia es LIMBO: el WFC ya mata sobre los mismos 2,75 años» (docstring de `call`). El nulo de volteo semanal de signo común conserva la dependencia entre celdas y asume simetría del exceso semanal.
- **Punto débil:** (i) con sólo 2,75-3,75 años de oos2 (≈143-195 semanas) la potencia es baja: NO PASA exige p− ≤ 0,05; en las 10 corridas p− mínimo = 0,066 (DAX40 `14.16.487`, mediana −5.730 USD/año, que SI el alpha fuese 0,07 sería NO PASA) y ninguna pierde con p− ≤ 0,05 aunque 3 de 10 tienen mediana negativa; (ii) se promedia 30 celdas con ≈1,4-4,8 celdas efectivas (§C19-4), la mediana no añade información que no estuviera en 2-3 curvas; (iii) la asimetría semanal rompe el nulo simétrico (el README del motor semanal cita rechazo del 13 % al 5 % nominal en la cola baja con exceso de asimetría positiva): `skew_warn = 1,0` sólo avisa; (iv) multiplicidad: tres criterios por madre más 17, 18, 18.5 sin corrección conjunta (se leen todos por «peor de los cuatro» en el 20); (v) el recargo primer orden y la deriva que se resta del P/L de SQX (que no es el P/L reprecificado) mezcla dos fuentes de precio (SQX para el P/L, Python para recargo y deriva).
- **Ejemplo real:** `Test_XAUUSD_H1_ruptura_wf4s` (2026-10-11): `1.14.418` L1 +1.304 USD/año, p+ 0,363, p− 0,637 → PASA «no descarta»; `5.2.399` +143, p− 0,554 → PASA; `9.19.426` +1.563, p− 0,710 → PASA. `Test_DAX40_H1_reversion_wf2/14.16.487` −5.730, p+ 0,934, p− 0,066 → LIMBO (`l1_no_gana`). `Test_USA500_H1_tendencia_wf1/20.17.293` −4.153, p− 0,146 → LIMBO; `7.11.427` +1.796, p− 0,718 → PASA. `Test_USATEC_H1_ruptura_leaseWeb/1.11.277` −31, p± = 0,500 → LIMBO; `6.22.453` +1.478, p+ 0,326 → PASA. NIKKEI M30 `17.13.284` +255 y `20.11.454` +1.024 → PASA.

C19-2. Reglas del modo `reoptimizado` — sólo pueden dar LIMBO; INEXISTENTES en `fijo` (por defecto)

- **Definición:** sólo con `modo_real: reoptimizado` (config.yaml; hoy `fijo`; dueño 2026-10-07) el estudio emite `wfm.reopt.*`: (a) `centro_exceso` = exceso por año en oos2 de la celda central (`wfm.centre_runs = 10`, `wfm.centre_oos_pct = 30`, fijada antes de correr; nunca la mejor celda en oos2): ≤ 0 → LIMBO; (b) `l2_p_menos` = p− de L2 (suma semanal de (referencia − madre fija)/años contra los mismos signos): ≤ 0,05 → LIMBO («mejor operar la madre fija»). `criteria.yaml:301-304`, `missing: pass` (en `fijo` no existen y no cuentan). En `fijo` la referencia de L2 es la serie semanal MEDIANA entre celdas (no la mediana de sumas de L1, de modo que L1 − madre ≠ delta de L2; pregunta abierta nº1 en `POSSIBLE_IMPROVEMENTS.md`).
- **Por qué:** la WFM sólo es prueba de lo que se opera si se opera reoptimizando; si se opera la madre fija es evidencia indirecta.
- **Estado real:** `modo_real = fijo` en las 10 corridas; los hechos `centro` y `l2` se anotan pero no deciden. Ej. L2: USATEC `6.22.453` delta +2.429, p− 0,849; NIKKEI `17.13.284` delta −1.560, p− 0,247 (reoptimizar peor que fija pero no significativo).

Anotaciones (nunca deciden)

- **C19-3. L2** (reoptimizar contra congelar): ver C19-2; en todas las corridas p− ∈ [0,247; 0,849], ninguna ≤ 0,05.
- **C19-4. Celdas efectivas** (`levels.effective`): $(\Sigma\lambda)^2/\Sigma\lambda^2$ de los autovalores de la matriz de correlación semanal de las 30 celdas (las que no se movieron se excluyen); 1 = idénticas, 30 = independientes; sobre ≈143 semanas incluso celdas independientes leerían ≈25. Reales: 1,44-4,80 (mediana ≈1,7). «Fracción de celdas positivas» 0,00-0,93.
- **C19-5. Área corregida de SQX** (`measure/gaterule.py`): recalcula las condiciones de la familia `score` sobre el OOS cosido solo (reescala por oos/all de la misma métrica porque el numerador de SQX incluye la primera ventana optimizada): □ 2026-10-07, 300 celdas: la cumple el 71 % tal como la calcula SQX y el 34 % corregida. Se reaplica `score_pct = floor(met/active·100+0,5)`, umbral `threshold_pct` y el rectángulo 4×4 (primera mejor en orden de filas gana el empate; si no cabe, SQX toma la mejor celda suelta y cuenta 1). Reales: `area_celdas` 2-28, `area_pasa` False en 3 de 10 (DAX40 `14.16.487` 2, USA500 `20.17.293` 4, XAUUSD `5.2.399` 11); coinciden con `sqx_failed = True`.
- **Deriva** de parámetros (cuota mediana de parámetros que cambian por reoptimización, `measure/drift.py`): 0,80-1,00 en las 10 (es decir, casi todo cambia en cada paso) — el propio README advierte que cuenta cambios, no distancia a la meseta.
- **Asimetría** semanal de la mediana de celdas (−1,87 a +1,33) y `straddle` (cuota de operaciones a caballo de fin de semana; si > `wfm.straddle_max = 0,1` se repite L1 con bloques de 2 semanas y avisa sin cambiar el estado).
- **Recobro/recargo** (`recobro_oos2` en el CSV): dinero recargado en total sobre todas las operaciones de las 30 celdas que cierran en oos2: 17.832 USD (USATEC `1.11.277`) a 85.663 USD (XAUUSD `1.14.418`). Es una magnitud de 1-3 órdenes mayor que L1 (−31 a +1.796 USD/año): el resultado depende mucho de que el recargo de spread/slippage de oos1 a oos2 sea correcto (aproximación de primer orden, sin mover stops).
- Tabla por segmento (exceso/año de cada celda en build/oos1/oos2 sobre los años que sus pasadas cubren; build y oos1 NO recargados), pestaña «Los objetivos» (cada condición por celda), equity acumulada de celda vs madre fija.

19.2 Orden de aplicación

(1) L1 p− ≤ 0,05 → NO PASA (`l1_pierde`); (2) si no, `mediana_exceso ≤ 0` → LIMBO (`l1_no_gana`); (3) en `reoptimizado` , `centro_exceso ≤ 0` o `L2 p− ≤ 0,05` → LIMBO (`centro_no_gana` , `reoptimizar_empeora`); (4) si no, PASA («reoptimizar gana» si p+ ≤ 0,05, si no «no descarta», que «no cuenta como evidencia»). El peor estado activo gana. Datos ausentes (sin trades de la madre) no disparan (`missing: pass` en las reglas de reopt, `limbo` en las L1). El área SQX se anota, no juzga.

19.3 Ejemplo de cadena (lotes 2026-10-10/11; el WFM sólo lo corren las madres que llegan vivas del 16.5)

lote	madres	PASA	LIMBO	NO PASA	«reoptimizar gana»
DAX40 H1 `reversion_wf2`	1	0	1	0	0
NIKKEI M30 `volatilidad_wf3`	2	2	0	0	0
USA500 H1 `tendencia_wf1`	2	1	1	0	0
USATEC H1 `ruptura_leaseWeb`	2	1	1	0	0
XAUUSD H1 `ruptura_wf4s`	3	3	0	0	0
Total	**10**	**7**	**3**	**0**	**0**

Referencia anterior (README, 2026-10-07, `Trade_NIKKEI225_M30_volatilidadL` , 10 madres, `fijo`): 4 PASA (todas «no descarta»), 5 LIMBO, 1 NO PASA (`18.5.355` , L1 −6.505 USD/año, p− 0,004); celdas efectivas 1,4-4,1; área corregida: 89 de 300 celdas pasan, 2 madres hallan el 4×4; 3 s por madre. El WORKFLOW (línea 49) cita esta única corrida real; hay más desde entonces. La del 2026-10-04 (dossier) no es comparable (otro criterio).

19.4 Coste

- Tarea SQX en el custodio (80 GB de heap, todos los núcleos, regla 3): `sqx` de la fase «19 sqx» = 196 s (DAX40, 1 madre), 268 s (NIKKEI, 2), 343 s (USA500, 2), 346 s (USATEC, 2), 648 s (XAUUSD, 3) de «Project finished»; a eso se suma ≈575-627 s de «reapertura» del proyecto por el bloque ciego y 78-233 s de exportes (`export.trades` , `export.extra` del WFM 14-18 s). Paso 19 entero (reloj): 983 s (DAX40) a 1.511 s (XAUUSD). `AlgoData/autopilot/task-durations.json` guarda un promedio «WFM» de 3.623 s (una corrida anterior con más madres; sin desglose).
- Lectura Python: 3-8 s por lote (`wfm.json` `wall_s` 6,57 s con 3 madres), un proceso; ≈3 s por madre. «19 kept/judge/ledger» ≈0 s.
- Memoria/disco del estudio: sin dato (sólo `distribuciones.parquet` ≈51 kB, `cell_excess.csv` ≈9 kB).

19.5 Discrepancias del paso 19 (⚠)

- ⚠ El dossier `criterios-paso-19-wfm-2026-10-07.md` (1.071 líneas) describe el estudio ANTERIOR: veredicto por ρ de Spearman IS→OOS («predice / ciega / perversa»), «ninguna regla en criteria.yaml» y «no deja `hechos.parquet` ». El código actual: L1 por volteo semanal de signos, `criteria.yaml` `"19"` con 4 reglas, hechos `wfm.*` y `hechos.parquet` (autopilot `paso-19/`). Manda el código.
- ⚠ `docs/AgentPDFs/curvas-wfm-2026-10-01.md` (§4) dice «Construido: pestaña “Contra el activo y el azar”» (Sharpe del original contra el activo comprado y contra 10.000 traders al azar). Esa pestaña NO existe ya en `studies/optimisation/wfm/` (tabs: exceso por celda, exceso acumulado vs madre fija, L1, L2, deriva, por segmento); `core/wfmatrix.py` no tiene importadores (sin verificar si es residuo). Las 5 respuestas del dueño (referencia = estrategia original, sólo oos2, contra el oro y el azar) ya no están implementadas en este paso. Además esas cinco madres (XAUUSD, Donchian, una plantilla y un activo) no son una población.
- ⚠ Umbrales de las 10 condiciones por celda calibrados sólo con ORO (5 estrategias, 150 celdas) y usados para todos los activos (aviso en `_build.yaml`); además el área SQX incluye un 67-79 % de datos ya vistos y el recorte `oos_only` sólo corrige la familia `score` , no las otras (`stability` , `special` , `oos` siguen leyendo el OOS cosido completo, build y oos1 incluidos). Por eso el área sólo anota.
- `wfm.skew_warn = 1,0` está «provisional, propuesto por el agente que implementó el paso» (no fijado por el dueño); `modo_real` por defecto `fijo` : el criterio de «lo que se opera» (centro, L2) no se aplica hoy.
- `Fitness` es 0 en todos los pasos exportados y los pasos que corren más allá del final de los datos se descartan (`read.drop_future: true` , «nunca false»).
- WORKFLOW:49 («una corrida real: 10 madres del Nikkei») está desfasado frente a las 10 madres actuales de 5 lotes.

Resumen de umbrales vigentes (todos en `ledger/thresholds.yaml` salvo indicación)

clave	valor	quién/cuándo	estado
`cscv.blocks`	16	dueño 2026-09-30	firme
`cscv.p_perdida_fail`	0,5	dueño 2026-10-07	provisional
`cscv.p_perdida_null_pct`	95	dueño 2026-10-08	
`cscv.null_draws` / `null_block_weeks`	200 / 4	dueño 2026-10-07	
`cscv.floor_per_param` / `floor_per_year` / `min_variants`	15 / 10 / 30	dueño 2026-10-07	
(config) `cscv.seed`, `null.median_low/high`, `power.deltas/share`, `dominance.tolerance/bins/reach`	20260922, 0,4/0,6, [0,1;0,2;0,3]/0,10, 0,01/512/6,0	`studies/optimisation/cscv/config.yaml`	no están en el ledger
`marketSurfaces.null_draws` / `null_block_weeks` / `alpha` / `seed`	2.000 / 1 / 0,05 / 0	dueño 2026-10-07	
`marketSurfaces.min_common` / `max_ties` / `min_weight`	50 / 0,5 / 5,0	dueño 2026-10-07; `min_weight` lectura del agente	`min_weight` provisional
`marketSurfaces.top_share` / `premium_percentile`	0,2 / 5	dueño 2026-10-07	percentil sin calibrar
`marketSurfaces.auto_min_rho` / `power_draws` / `cost_max_share` / `straddle_max`	0,1 / 200 / 0,01 / 0,1	dueño 2026-10-07	
`wfm.alpha` / `null_draws` / `seed` / `straddle_max`	0,05 / 2.000 / 0 / 0,1	dueño 2026-10-07	
`wfm.skew_warn`	1,0	agente 2026-10-07	provisional
`wfm.centre_runs` / `centre_oos_pct`	10 / 30	dueño 2026-10-07	
`_build.yaml` wfm: rejilla 6×5, `threshold_pct` 80, 4×4, `min_squares` 12, `max_tests` 5.000, `wf_type` exact_is_exact_oos		dueño 2026-09-24	umbrales de condiciones calibrados con oro
# Pasos 20-23: como estan implementados y se han corrido (estado del arbol al 2026-10-11)			

Aviso de estado. El arbol de trabajo tiene cambios SIN commit del 2026-10-11 que rediseñan 20 y 21 y BORRAN 22 y 23 (`git status`: `studies/readings/conditionalMap/` y `studies/readings/structure/` marcados `D`, `studies/closing/exposure/verdict.py` borrado, `atrCalculator` borrado). Lo que se describe de 20 y 21 es el codigo actual del arbol; lo de 22 y 23 es el ultimo codigo (git HEAD) mas el dossier del 2026-10-08. Todas las corridas reales guardadas en AlgoData son ANTERIORES al rediseño (2026-09-26 a 2026-10-06): ninguna usa el benchmark nuevo del paso 20 ni la version sin veredicto del 21.

Ficheros de referencia: `studies/closing/blindJoint/`, `studies/closing/exposure/`, `engines/inference/snooping/superior.py`, `pipeline/autopilot/{tail,blind,closing}.py`, `pipeline/autopilot/criteria.yaml`, `ledger/thresholds.yaml`, `assets/_policy.yaml`, `docs/AgentPDFs/criterios-paso-{21,22,23}-*.md`, `docs/AgentPDFs/WORKFLOW_TIMING_BASELINE.md`.

PASO 20 — Analisis conjunto de 17, 18, 18.5 y 19, y SPA/StepM contra un trader al azar con la misma huella

Proposito

Dos cosas en un solo comando (`python3 -m studies.closing.blindJoint.report`, lanzado por el autopiloto en `pipeline/autopilot/blind.py:134-140` y planificado en `tail.py:146`): 1. Leer a la vez los cuatro estados que 17 (WFC), 18 (CSCV), 18.5 (superficies de mercado) y 19 (WFM) dejaron sin leer, y combinarlos en una llamada por madre. 2. Contestar la pregunta que ninguno hace: ¿bate la madre, sobre `oos2`, a un trader que toma SUS MISMAS operaciones (numero, lado, lotes, duracion, costes) con entradas al azar? Con SPA de Hansen para la familia y StepM de Romano-Wolf para nombrar madres. Esta segunda parte es hoy informativa: no decide (ver orden de aplicacion).

Entradas

- Puerta ciega: `inputs.blind_door` (`inputs.py:37-49`) -> `ledger.gate.allow_read`: exige filas de 17, 18 y 19 en el ledger del estudio (activo+TF+familia). "Ejecutado" = presente en el ledger, no en disco. 17-19 no escriben fila solas: `pipeline/autopilot/blind.py:register` (o `ledger.backfill --blind`) las reconstruye.

- Piezas por madre (`pieces.py`): `estudios/{wfc,cscv,marketSurfaces}.json` en el lote de variantes `AlgoData/strategyPermutations/<proyecto>/<madre>/` y `estrategias/<madre>.json` del `wfm/` mas reciente. Solo se abre el JSON pequeño y se lee su `verdict.state` (pass/watch/fail = PASA/LIMBO/NO PASA); el paso 20 NO recalcula umbrales. Una madre solo se lee si tiene las cuatro (`pieces.py:93-99`); si falta alguna se lista como `incompleta` y no se lee.
- Excepcion WFC: `pieces.said` (`:64-66`) toma el estado del `estado` de su composicion decisoria y lo traduce con `WFC = {pasa: pass, limbo: watch, no_pasa: fail}` (`pieces.py:16` , dueño 2026-10-07).
- Para el SPA/StepM: `trades.parquet` de cada lote (solo la madre; lo escribe el 16.5 desde 2026-10-04), barras M1 del feed, ficha de costes del activo (`reprice.costs(feed,"oos2")`), ventana oos2 de `assets/_policy.yaml` .
- Puerta oos2: `gate.allow(20,"oos2",symbol)` (`inputs.py:52-68`). `assets/_policy.yaml:48` lista hoy `reserved_for: [WFC, CSCV, MarketSurfaces, WFM, BlindJoint, ATRStop]` ; el dueño añadio BlindJoint el 2026-10-04. Solo ata a un agente autonomo (`ALGO_AUTONOMOUS=1`); un humano nunca es rechazado, pero el ledger cuenta cada mirada.

Salidas

`reports/<P>/<databank WFM>/<dia>/blindJoint/` : `blindJoint.{json,html,md}` , `estrategias/<madre>.{json,html}` , `readings.csv` (llamada de cada madre bajo cada lectura), `verdict.csv` (columnas `strategy,identity,verdict,reason,wfc,cscv,marketSurfaces,wfm`), `manifest.json` , los remuestreos del SPA (`core.study.draws`). Una fila de ledger paso 20 (`report.py:ledger_row`) SOLO si se abrio oos2. Lo que consume el autopiloto: `blindJoint.verdict_csv.drop` (=1 si el verdict.csv dice DESCARTAR).

Criterios numerados

C20.1 — Llamada conjunta de los cuatro estados, lectura `peor` (la que decide) - Pregunta: ¿hay algun estudio ciego que suspenda o dude de la madre? - Definicion: `readings.worst` (`readings.py:22-24`): DESCARTAR (`fail`) si alguno de [WFC, CSCV, superficies, WFM] es `fail` ; si no, `pass` solo si los cuatro son `pass` ; si no `watch` (REVISAR). `report.VERDICT = {fail: DESCARTAR, watch: REVISAR}` (`report.py:22`), todo lo demas MANTENER. Config: `joint.pieces: peor` (`config.yaml:40`), PROVISIONAL del dueño 2026-10-07 «manda el peor de los cuatro» «hasta que el dueño rediseñe el paso 20». - Corta o marca: CORTA solo DESCARTAR. En el autopiloto, `criteria.yaml:300-314` : paso 20, `cut: SPP OOS` (el mismo databank que el 16; el 21 sigue leyendo de ahi), regla `blindJoint.verdict_csv.drop < 1` , `missing: limbo` . REVISAR y MANTENER no cortan (REVISAR "marca"). Una madre incompleta sale MANTENER con motivo «incompleta: no leida» (`report.py:67`). - Poblacion/ventana: las madres que llegaron al 20 con las cuatro piezas; cada pieza con su propia ventana (el WFC/CSCV/superficies/WFM leen oos2 o build+oos1+oos2 segun su paso: `criteria.yaml:287` ya marca 18.5 con oos2). - Nulos: un estado ausente en una pieza hace la madre incompleta, no la cuenta como fallo. - Justificacion declarada: dueño 2026-10-07, provisional. Es la lectura mas dura de la rejilla: un solo NO PASA de cualquiera de cuatro estudios, cada uno ya con su propio umbral, descarta. Puntos debiles: (a) los cuatro estudios no son independientes (todos leen las mismas variantes y la misma madre), asi que "cuatro opiniones" cuenta menos de cuatro; (b) la probabilidad de descartar a una madre buena crece con el numero de piezas (union de cuatro tests, cada uno con su error tipo I, sin correccion conjunta); (c) la asimetria PASA/LIMBO/NO PASA se hereda de cada paso: el WFC convierte "limbo" en `watch` , pero pieza sin dato = madre no leida, no LIMBO; (d) no distingue pesos: un `fail` de superficies 18.5 (diagnostico) pesa igual que un `fail` del WFM. - Alternativas implementadas y no activas, visibles en `readings.csv` : `unanimidad` (los cuatro `pass`), `sin_fallo` (ningun `fail` ; fue la lectura vigente 2026-10-04..07), `ninguna` (piezas como contexto, decide solo el StepM). `readings.names()` (`:46-54`) produce 6 columnas: unanimidad/sin_fallo x {supervivientes, entrantes}, `ninguna+entrantes` , `peor+entrantes` . - Ejemplo real (lectura vigente entonces, `sin_fallo+entrantes` ; MISMOS estados que daria `peor`): `Test_USA500_H1_indicesIBS` (USA500 H1, 2026-10-06 19:31): 6 madres, 3 completas (a las otras 3 les falta la WFM). Estados (WFC, CSCV, superficies, WFM): 1.13.354 = fail,fail,fail,watch; 18.15.392 = fail,fail,fail,watch; 5.13.334 = watch,fail,fail,watch. Bajo `peor` serian 3 de 3 DESCARTAR; bajo el `sin_fallo` de entonces tambien (`verdict.csv` : DESCARTAR x3, MANTENER x3 incompletas). `Test_NIKKEI225_M30_simpleL_fin` (2026-10-05): 10 de 10 completas, 0 pasan; solo 18.16.329 tiene WFC=pass y CSCV=pass, pero superficies=fail. En las tres corridas leidas (NIKKEI, USA500, USDJPY doubleROC) ninguna madre pasa las cuatro; no se han leido las otras 7. Bajo `peor` las 10 NIKKEI y las 3 USA500 serian DESCARTAR (cada una tiene al menos un `fail`). - ⚠ No se han corrido nunca los resultados bajo `peor` : las corridas guardadas son de `sin_fallo` o `sin_regla` (ver discrepancias).

C20.2 — Benchmark: el trader al azar con la huella de la madre (nuevo, 2026-10-11) - Pregunta: ¿gana la madre en oos2 mas que sus propias operaciones entradas al azar? - Definicion (`benchmark.py:10-62` , `inputs.random_traders`): por madre, `benchmark.draws = 2000` traders toman sus operaciones de oos2 — mismo numero, lado, lotes, tiempo de tenencia (en barras M1) de cada una, mismos costes — con barras de entrada uniformes sobre TODO oos2 (no de su primera a su ultima operacion). Generador: `engines.nulls.model.RUNGS["timing"]` (`config.yaml:18` ; la misma escalera del estudio `monkey`), semilla fija `benchmark.seed=0` . Precio: `engines.market.reprice.pnl` con `reprice.costs(feed,"oos2")` (spread+slippage+comision+swap); no el `calibrate.charged` de la escalera, que no lleva spread. El benchmark es la MEDIA de los 2000 traders del P&L por dia, marcado al ultimo cierre M1 de cada dia, como el panel de la madre. Como la media es lineal, no se precia cada trader por dia sino su posicion media por barra (`_per_bar`); la serie suma exactamente el neto medio. - Nulo que implica (README `blindJoint:75-81`): `E[madre_k − azar_k] <= 0` para todo k, condicional a su huella (cuantas, cuanto duran, que tamaño): solo se contrasta CUANDO entra. - Error de Monte Carlo: sd(neto de un trader)/sqrt(draws)/dias = `mc_day` ; ~1/sqrt(2000) = 2,2 % del error estandar de la madre (□ medido en 300 madres sinteticas de 150 trades:

mediana 1,2 %, peor 1,5 %), 0,05 % mas de varianza; no se remuestrea (offset fijo). - Coste en CPU: `# ponytail` en `benchmark.py:42`: todos los draws se precian a la vez, ~100 bytes por draw x operacion (200 MB con 2000 x 1000 operaciones). - Corta o marca: no corta nada hoy (ver C20.4/C20.5). Informa en `table` (`random_day`, `excess_day`, `mc_day`). - Justificacion: sustituye al buy and hold a igual riesgo (dueño 2026-10-11): una estrategia 7 % del tiempo en el mercado se medía contra una posicion dimensionada para ser tan volatil como ella (`equal_risk`), lo que le exigia batir a un largo apalancado. El nuevo descuenta deriva, lado, tiempo en mercado y costes. - Puntos debiles visibles: (a) las entradas son uniformes sobre cada barra M1 del feed, no solo las horas en que ella opera (limite declarado `engines/nulls/README.md`); (b) las operaciones al azar pueden solaparse donde las de ella no; (c) los lotes de ella se conservan tal cual (llevan la volatilidad de sus entradas reales); (d) usa los costes de la ficha del activo para oos2: una ficha recalibrada despues del retest mueve el benchmark; (e) 2000 draws y semilla 0 fijos: el benchmark es reproducible pero el offset de MC no esta en el bootstrap; (f) la madre se mide con su P&L cerrado reconstruido (ver C20.3) y el benchmark con el mismo marcado, asi la comparacion es coherente, pero un aviso: el nulo condiciona a que la madre YA fue elegida por 17-19 sobre oos2 en parte (ver multiplicidad). - Ejemplo numerico: SIN CORRIDA REAL. Ninguna corrida guardada usa este benchmark (las 10 `blindJoint` de AlgoData son del 2026-09-29 al 2026-10-06 y dicen `"benchmark": "equal_risk"` en el ledger). Hay prueba de respuesta conocida `tests/test_blindjoint.py` (2026-10-11), no verificada su ejecucion aqui.

C20.3 — Panel de la madre en oos2 (cierre a cierre) y su licencia - Definicion (`inputs.panel`, `:134-158`): P&L diario marcado a mercado reconstruido de las operaciones de la madre en oos2 y las barras M1 (`portfolio.common.construct.equity.closecurve.panel`). Licencia: se rechaza (`ValueError`) salvo que los minimos diarios reconstruidos coincidan con la curva de SQX (`TOLERANCE = 1.0` en moneda de cuenta) en `RECONCILED = 0.99` de los dias (mediana sobre madres). La curva de SQX (`mother_low`) es el MINIMO diario de equity, no marca a mercado, por eso solo licencia y nunca alimenta. Dias de solape de patas (calentamiento de 2 meses) quedan fuera con la ventana oos2 de la politica; si un recorte captura fechas duplicadas se rechaza. - Estatus: control de calidad (si falla, no se lee oos2: el veredicto sigue con los cuatro estados, sin fila de ledger, aviso «Falta el StepM»). - Debil: operaciones solapadas o madre con un lote antiguo sin `trades.parquet` -> rechazo (`trades_door`), no se estima.

C20.4 — SPA de Hansen (familia) - Pregunta: ¿alguna de las K madres bate a su trader al azar, pagando la busqueda sobre las K? - Definicion (`superior.spa`, `:25-52`): exceso diario `excess = madre − benchmark` (K columnas); `arch.bootstrap.SPA(benchmark=0, models=-excess, bootstrap="stationary", reps=1000, seed=0, studentize=True, nested=False)`; las columnas con desviacion 0 (no operaron) se excluyen (`measure.py:78`). Longitud de bloque: `block_length` = mediana por columna del `optimal_block_length` estacionario de Politis-White, redondeada, minimo 1, un unico valor para todo el panel (el remuestreo toma dias enteros de todas las columnas a la vez, conservando la correlacion). Salida: tres p, `lower`, `consistent` (la de Hansen), `upper` (White). Umbral de lectura: ninguno; no corta. Se escribe en el ledger (`note`) y se imprime. - Corta o marca: INFORMA. Con `peor` no decide nada (`peor` "does not wait for the StepM"). Solo las lecturas elegidas distintas de `peor` lo necesitarian (`report.need_stepm`). - Justificacion: un test de familia sobre varias candidatas evita el sesgo de coger la mejor; Hansen estudentiza y poda las claramente peores (consistent). Puntos debiles: (a) con K = 2 a 15 mothers el SPA tiene muy poca potencia y las madres de un mismo proyecto suelen ser casi la misma estrategia (□ 1.28.59 y 1.29.55 USDJPY: 98,8 % de los dias oos2 con P&L identico, rho 0,978: K = 2 en el papel, 1 en realidad); (b) el nulo de "una de K" se evalua con una K que ya es una seleccion (K = supervivientes de pasos 7-19, elegidos en parte con oos1/oos2) y la poblacion `entrantes` (dueño 2026-10-04, P2) paga la busqueda de las que entraron, no de la que habria entrado sin pasar los filtros; (c) series diarias de ~700-1000 dias con 1-3 bloques de media: p muy ruidosa; (d) estacionariedad/dependencia debil sobre oos2, que puede incluir un solo regimen (NIKKEI 2024-26); (e) reps = 1000 limita la resolucion de p a ~0,001 y la comparacion de dos p cercanas; (f) la semilla fija hace la p reproducible pero no mide su variabilidad de remuestreo. - Ejemplo real (BENCHMARK ANTIGUO, buy and hold a igual riesgo, 10 corridas guardadas; ninguna con el nuevo): USA500 H1, `Test_USA500_H1_indicesIBS`, 2026-10-06, K = 3, 709 dias oos2 (2024-01-01..2026-10-01), bloque medio 1: p lower 0,547, consistent 0,815, upper 0,815; ningun StepM. NIKKEI225 M30 `..._simpleL_fin`, 2026-10-05, K = 10, 709 dias, bloque 3: 0,499 / 0,731 / 0,731. USDJPY M30 `Test_USDJPY_M30_doubleROCLong`, 2026-10-06, K = 3, 974 dias: 0,403 / 0,579 / 0,579. Otras (USDJPY M30/H1 donchian, 2026-09-29/30, `sin_regla`): consistent 0,864 (K = 5) y 0,749 (K = 15, upper 0,772). Ninguna p de ninguna corrida baja de 0,40: en ninguna ocasion el SPA rechazo la nula; el exceso medio diario contra el buy and hold a igual riesgo fue negativo en 10 de 10 madres NIKKEI, 3 de 3 USA500 y 2 de 3 USDJPY (la unica positiva: 3.13.366, +9,60 USD/dia).

C20.5 — StepM de Romano-Wolf (nombrar madres) - Definicion (`superior.screen` / `stepm`, `:55-103`; envoltorio `measure.stepm`): paso descendente escrito sobre `SPA.better_models(fwer, pvalue_type="consistent")` (por defecto de arch) con `fwer = 0,05` (`ledger/thresholds.yaml:30-35`, clave `snoopingScreen.stepm.fwer`, dueño 2026-09-25: «probabilidad de nombrar al menos una que no bate a su operador al azar de igual huella»). Se repite: nombradas se quitan, se vuelve a correr hasta que no nombre ninguna. Implementacion propia porque `arch.StepM` 7.2.0 falla cuando las rondas nombran todas las columnas (con K = 2..5 ocurre, a K = 200 nunca; en 80 paneles donde StepM no falla nombra exactamente lo mismo). `population: entrantes` (`config.yaml:47`): K = todas las madres que llegaron al 20 (menos las planas); `supervivientes` = solo las que las piezas dejan. - Corta o marca: INFORMA hoy (`peor`). Bajo otra lectura, una madre pasa si las piezas la dejan Y el StepM la nombra. - Debil: potencia ridicula con K de un digito y series de dos-tres años; controla FWER sobre el exceso medio, no sobre la significancia economica; `reps` = 1000 compartido con el SPA; el FWER de 0,05 no esta calibrado con ninguna poblacion de control. - Ejemplo real: en las 10 corridas, StepM nombra a 0 madres de K (todas «no»). Estado del propio ledger: `n_out = 0` en NIKKEI (n_in 10), USA500 (3), USDJPY M30 (3). - Coste: 1000 remuestreos por ronda; 5.130 estrategias = 65 s por bootstrap (□ nota de `superior.screen`, 2026-10-01); con 3-15 madres, ~segundos. Tiempo total del paso medido en

la referencia `Test_NIKKEI225_M30_simpleL_ref` (10 madres, 2026-10-04/05): **15 s**, sin SQX, 0 cores dedicados (`WORKFLOW_TIMING_BASELINE.md:34`), pero con el benchmark ANTIGUO. Con 2000 draws por madre (benchmark nuevo) el tiempo y la RAM (~200 MB por madre en el pico) no estan medidos.

Orden de aplicacion

1. `blind_door` (ledger de 17-19). 2. `pieces.population` (completas/incompletas). 3. `segment_door` (politica oos2), luego "alguna madre completa", luego `trades_door`. Cualquier rechazo del 3 se convierte en el aviso «Falta el StepM» y NO aborta (salvo lecturas distintas de `peor`: `need_stepm` levanta `PermissionError`). 4. Panel + benchmark + SPA/StepM (solo si ninguna puerta rechazo). 5. Llamada por madre bajo cada lectura, `verdict.csv` con la lectura `peor`. 6. Fila de ledger (solo si se leyo oos2): `n_in = K completas`, `n_out = madres != DESCARTAR`, `criterion = blindJoint/peor+entrantes`, `thresholds` con fwer, rung, draws, seed, `seeds [0,0]`. 7. El autopiloto aplica `criteria.yaml` paso 20 y corta DESCARTAR de `SPP OOS`.

Cadena de ejemplo (n entran -> n salen)

Corrida guardada mas reciente de cada tipo, con las lecturas antiguas: NIKKEI225 M30, 10 completas -> 0 pasan `sin_fallo+entrantes` (todas con superficies = fail). USA500 H1: 6 madres, 3 completas (3 sin WFM) -> 0 pasan. USDJPY M30 doubleROC: 6 madres, 3 completas -> 0 pasan. Bajo `peor` lo esperable es el mismo corte: todas las madres con algun `fail` caen (las 16 completas de esas tres corridas tienen al menos un `fail`); las incompletas siguen MANTENER sin leer (⚠). Con la regla actual del autopiloto «missing: limbo» una madre incompleta sin `verdict.csv` queda en LIMBO, no cortada: no verificado el comportamiento exacto.

Tiempos y recursos

15 s para 10 madres (referencia 2026-10-04/05, benchmark antiguo, sin SQX). Python puro, un nucleo (arch + numpy); con el benchmark nuevo, 2000 trades x madres operaciones en un array de enteros por rung: sin dato. Ledger/informes: ~KB.

Discrepancias ⚠

- ⚠ README de `blindJoint` (`:48-52`) dice que el paso 20 «no esta listado» en `_policy.yaml` y que el SPA «no se lee»; el YAML ya lo lista (`reserved_for ... BlindJoint`). Y `WORKFLOW.md:50` dice «`BlindJoint` fuera de `reserved_for`». Codigo manda: dentro.
- ⚠ Ninguna corrida real usa la lectura `peor` ni el benchmark `random_trader`; todas las `blindJoint` guardadas son `sin_fallo+entrantes` (≥ 2026-10-05) o `sin_regla` (09-29/30) con `equal_risk`. La lectura vigente nunca se ha ejecutado de extremo a extremo con datos reales.
- ⚠ `report.py` ledger: `n_out` cuenta `!= fail` para `peor`: pasan REVISAR, que se quedan (coherente con criteria, pero `n_out` ≠ numero de MANTENER).
- ⚠ `config.yaml` `pieces: peor` es la unica lectura que no usa StepM, de modo que el estudio gasta CPU en SPA+StepM+benchmark (2000 draws) y la llamada no depende de ellos; el SPA solo aparece en informe y nota de ledger.
- ⚠ La descripcion del WORKFLOW (`:50`) «el SPA sobre oos2 se lee para un humano; solo un agente autonomo lo encuentra cerrado» ya no es cierta con BlindJoint en `reserved_for`.
- ⚠ `bin/workflow.sh:88` pasa `--panel 21=CrossTF_Mothers --panel 22=Results`, pero `tail.PANEL` solo tiene el 21 (`tail.py:15`): el `--panel 22` ya no tiene paso; no verificado si `run.py` lo rechaza o lo ignora.

PASO 21 — Exposicion contra el buy and hold

Proposito

Describir (no juzgar) la FORMA del retorno de cada superviviente: cuanto tiempo de mercado costo, cuanto rinde por hora expuesta, que pasaria con un buy and hold a distintos tamaños, y que parte del movimiento del mercado ocurrio con posicion. Encargo del dueño 2026-09-24 («5 % con una hora a la semana…»). Desde 2026-10-11: «solo describe», sin veredicto ni umbral de eficiencia (`exposure/verdict.py` borrado; `config.yaml` ya no tiene `min_efficiency`).

Entradas

- Export de trades del databank `--panel 21=<databank>` (en `bin/workflow.sh:88`: `CrossTF_Mothers`) — `raw/<P>/<DB>/<fecha>/trades.parquet`, la mas reciente (`inputs.newest`), muestra `Sample type = OOS1` (`config.yaml: study.sample: OOS1`, `segment: oos1`). Una fila por operacion con `Open time, Close time, Size, Type, Profit/Loss, Balance`.
- Barras del feed re-muestreadas a `study.timeframe` (config `M30`; el lanzador de la ventana lo sustituye por el TF del export, `ui/daemon/runner/readings.py:79-83` segun el dossier) y cortadas a la ventana del segmento segun `assets/_policy.yaml` (nunca de la primera/ultima operacion). `point_value` de la ficha del activo.

- No lee oos2. Autopilot: accion `reading` tras el 20 (`tail.py:147-150`); `closing.py:13` corre un estudio por madre.

Salidas

`reports/<P>/<DB>/<dia>/exposure/estrategias/<madre>.{json,html}` por madre (el JSON lleva `summary` con ~44 cifras planas); al correr la poblacion: pagina `exposure.*` , `verdict.csv` (sin columna `verdict` , solo `many.CSV` : strategy, identity, n, exp_share, exp_hours_per_week, strat_return_pct, return_per_exposure_pct, efficiency, return_ratio, dd_ratio, captured, aligned, notes) y manifest. ⚠ En las corridas guardadas del autopiloto solo estan los JSON por madre, no `verdict.csv` ni pagina de poblacion. No hay regla en `criteria.yaml` para el 21 (no hay clave "21"), ni umbral en `ledger/thresholds.yaml` , ni fila de ledger.

Criterios (todos INFORMAN; ninguno corta ni marca estado salvo `notes` , que son avisos `watch`)

C21.1 — Ocupacion / exposicion (`occupancy.measure` , `occupancy.py:52-78`) - Pregunta: ¿que parte del tiempo de mercado tuvo posicion y cuanto capital comprometio? - Definicion: `held` (`:10-36`) suma lotes sobre las barras (cumsum de eventos de entrada/salida; una operacion que cierra en la barra donde abre ocupa esa barra: la salida se empuja una barra, `leave = max(leave, entry+1)` ; operaciones solapadas se suman). `share` = media de `gross > 1e-9` (umbral del residuo de coma flotante: ya corregido); `hours_per_week` = barras con posicion x minutos de barra /60 / (ventana en semanas) con minutos = mediana de los saltos entre barras; `avg_notional_pct` = media de lotes x close x point_value / equity inicial x 100; `notional_when_in_pct` lo mismo solo dentro; `tilt` = lotes con signo / lotes brutos (+1 siempre largo). Equity inicial = `Balance − Profit/Loss` de la primera operacion de la muestra (la oos continua la cuenta IS: no es el deposito). - Denominador: la ventana de la politica, no las operaciones (una estrategia que no operó en 2018 no acorta su ventana). - Debil: «horas fuera» = 168 − horas con barra con posicion; incluye horas con posicion abierta sin barras (85 % de las operaciones de USA500 cruzan al menos un cierre diario: el hueco de reapertura no se ve). El timeframe de las barras fija la resolucion: `config.yaml` dice M30 pero el lanzador de la ventana usa el del export (dossier 5.2; no verificado en el codigo actual del lanzador). En rejillas gruesas con operaciones solapadas el residuo flotante inflaba la ocupacion (dossier 5.1); el codigo actual ya usa `> 1e-9` (`occupancy.py:70`) pero `presence` usa `lots != 0` (`occupancy.py:95`): sin parchear en esa linea. - Ejemplo (oos1, USA500 H1, `Test_USA500_H1_indicesIBS` , 3 madres, `CrossTF_Mothers` 2026-10-06, 1456 dias): ocupacion 22,6 % / 20,9 % / 16,4 %, 25,6 / 23,7 / 18,6 horas por semana, 129 / 123 / 95 operaciones. NIKKEI225 M30 `..._simpleL_fin` (2026-10-05, 10 madres, 1456 dias): ocupacion 5,6-7,2 %, 6,2-7,9 h/semana, 102-106 operaciones.

C21.2 — Serie diaria de la estrategia y estadisticos (`benchmark.daily` , `compare.stats`) - Definicion: P&L por dia natural en que cerro cada operacion (el dia de cierre, el que reporta SQX) sobre los dias de la serie de movimientos del activo; el P&L de un dia sin barra cae al siguiente dia con barra. `stats` : `net` , `return_pct` = net/equity inicial, `cagr_pct` = ((equity final/inicial)^(1/años) − 1), `maxdd_pct` sobre el pico corriente de ese saldo, `vol` = sd diaria (ddof 1), `sharpe` = mean/sd x sqrt(252) (convencion de SQX), todo contra benchmark cero; denominador = dias de calendario de la ventana, dias planos incluidos. - Debil: P&L al dia de cierre, no marcado a mercado: la serie es mas «a saltos». La nota del codigo (`benchmark.py:98-102`) y del README dicen que eso infla la sd y vuelve la comparacion conservadora para la estrategia; el dossier del 2026-10-08 midio lo contrario (sd marcada a mercado MAYOR en las seis madres probadas, +1,0 a +2,4 %): el efecto real es que la eficiencia sale ~2 % a favor de la estrategia (dossier 5.5). Correlacion con el activo: 0,07 con la serie del estudio y 0,49 marcada a mercado (1.13.354): el estudio no calcula correlacion, pero la «antesala de alfa/beta» daria beta casi nula con esta serie (dossier 5.6). - Ejemplo: ver C21.3 y C21.4.

C21.3 — «Buy and hold» en tres convenciones (`benchmark.py:19-70`) - `one_lot` = 1 lote; `avg_size` = media de `Size` de la estrategia; `equal_risk` (cabecera, `config.yaml: headline`) = `std(P&L diario estrategia, ddof1) / (std(movimiento diario del cierre, ddof1) x point_value)` : los lotes de un largo siempre >0 cuya sd diaria iguala la de la estrategia (incluyendo dias planos). Rentabilidad del benchmark: `moves x lotes x point_value` , sin spread ni swap (decision declarada: «una entrada es redondeo»; el swap es una decision del benchmark). - Corta o marca: INFORMA. `equal_risk` es siempre un LARGO: una estrategia corta ganadora en un activo que cae se compara con un largo y da razon de rendimientos negativa (dossier 5.3; no hay corrida real corta). - Justificacion: solo la convencion a igual riesgo vuelve «rinde menos que el buy and hold» una afirmacion sobre la ventaja y no sobre el tamaño. Debil: (a) el sd diario de la estrategia incluye lumpiness del dia de cierre (C21.2); (b) el benchmark no paga swap: en indices con 8-9 lotes el swap de un largo (5,72 % anual en USA500, peor caso FTMO/Hantec, `assets/symbols/USA500.yaml`) cuesta ~7.000-8.300 USD por madre frente a 11.400-13.600 USD de neto del benchmark (estimacion del dossier 5.7, cota, no medida por el estudio): sesga CONTRA la estrategia; (c) con 3-5 años y Sharpe del activo de ±0,45-0,49 de error tipico (dossier 5.13) toda comparacion es ruidosa. - Ejemplo: USA500 H1, madre 1.13.354: lotes `equal_risk` = 8,91, `avg_size` = 20,47; benchmark a igual riesgo: +13,45 % de retorno; estrategia +16,66 % (max DD 5,45 %). NIKKEI M30: lotes `equal_risk` 12,7-14,3 (vs `avg_size` ~60).

C21.4 — Dicotomia retorno/exposicion (`compare.dichotomy` , `compare.py:51-76`) - Definicion: `return_per_exposure_pct = return_pct / share` (extrapolacion «si estuviera dentro todo el tiempo al ritmo al que rinde cuando esta»); `efficiency = return_per_exposure_pct / return_pct del benchmark equal_risk` (1,0 = «tan productivo por hora como tener el activo»); `return_ratio = strat return_pct / bench return_pct` ; `dd_ratio = maxdd estrategia / maxdd benchmark` ; `hours_off = 168 − hours_per_week` . Sin veredicto ni umbral desde 2026-10-11. - ⚠ Es un cociente de RETORNOS dividido por la ocupacion, no de Sharpes como dice el dossier del 2026-10-08 (§2.5, que describe el codigo anterior); el codigo actual usa `return_pct` . - Debil: (a) la eficiencia sale negativa o se dispara si el benchmark rinde ~0 o pierde (aviso `bench_loses`); (b) es lineal en 1/ocupacion y no tiene suelo: con ocupacion

5 % el valor «de azar» ya es mas alto (el nulo de colocacion al azar con misma exposicion da eficiencia ≈ 1/sqrt(p); con p = 22,6 % → 2,1; p = 5 % → 4,5; dossier 5.12, calculo del revisor): la cifra no es comparable entre estrategias de ocupaciones distintas; (c) no hay intervalo ni nulo: dossier 5.13 midio un IC95 % de la eficiencia de [−32, +45] para 1.13.354; (d) se lee sobre OOS1, el tramo que ya selecciono a las madres en 7-16. - Ejemplo real (`Test_USA500_H1_indicesIBS` , `CrossTF_Mothers` , 2026-10-06; oos1 = 2020-2023): 1.13.354: ocupacion 22,6 %, retorno 16,66 %, rend. por hora expuesta 73,7 %, eficiencia 5,49, return_ratio 1,24, dd_ratio 0,54; 18.15.392: 20,9 %, 9,53 %, 45,6 %, 3,36, 0,70, 0,92; 5.13.334: 16,4 %, 1,92 %, 11,7 %, 1,03, 0,17, 1,18. NIKKEI225 M30 `..._simpleL_fin` (2026-10-05): 9 de 10 madres con retorno negativo en oos1 (−1,5 a −5,3 %) y eficiencia de −2,8 a −9,4; una sola con +1,0 % (19.16.284): eficiencia 1,99, return_ratio 0,12.

C21.5 — Presencia en el movimiento del mercado (`occupancy.presence` , `occupancy.py:81-97`) - Definicion: `captured` = suma \| Δprecio por barra\| donde hay posicion / suma total de \|Δprecio\|; `aligned` = (signo de lotes · Δprecio donde hay posicion) / suma total de \| Δprecio\|. `captured` alto con ocupacion baja y `aligned ≤ 0` es «estar presente en tramos buenos» y no ventaja propia (aviso `timing` si `captured > 2 x share` y `aligned <= 0` , `one.py:38`). - Ejemplo: USA500: captured 27,9 / 25,9 / 22,0 %, aligned 0,9 / 0,4 / 0,4 %. NIKKEI: captured ≈ ocupacion (5,4-7,2 %), aligned ≈ 0 (−0,0 a +0,1 %): estar dentro no capta mas movimiento que la fraccion de tiempo. - Debil: sin nulo ni intervalo, sin separacion de deriva; `lots != 0` con residuos flotantes (ver C21.1).

C21.6 — Avisos de legibilidad (`one.notes` , `config.yaml: notes`) Avisos `watch` , sin efecto: pocas operaciones (`min_trades: 30` , en toda la ventana, no 35/año del dueño), ventana corta (`min_days: 250`), la estrategia pierde, el benchmark pierde, `timing` (C21.5). Dossier 5.9: 21 de las 29 fichas del dossier tienen menos de 35 operaciones/año.

Orden de aplicacion

Carga de export mas reciente de la muestra OOS1 -> ventana de politica -> barras -> por madre: equity inicial, serie diaria, ocupacion, presencia, estadisticos, tres convenciones, dicotomia, avisos -> JSON por madre. Nada corta. No hay llamada de ledger ni juez.

Cadena de ejemplo

USA500 H1: 3 madres entran, 3 salen (sin corte); NIKKEI225 M30: 10 entran, 10 salen. Lo que 21 recibe lo decide el 20: el 20 corta `SPP OOS` , y el 21 sigue leyendo del databank que el dueño nombra con `--panel 21=` (`CrossTF_Mothers` en `bin/workflow.sh`). ⚠ No verificado si `CrossTF_Mothers` contiene solo las que sobrevivieron al 20 o las de 12.

Tiempos y recursos

6 s para 10 madres (`WORKFLOW_TIMING_BASELINE.md:35` , 2026-10-04/05, sin SQX); un nucleo; RAM despreciable (series diarias y barras de un TF). Lectura de un `trades.parquet` de ~1 M filas una vez por poblacion (`inputs.sample` lo lee entero). Tiempo de la corrida individual `reports/.../exposure/estrategias` : sin dato por madre.

Discrepancias ⚠

- ⚠ Dossier `criterios-paso-21-…2026-10-08.md` (588 lineas) describe un veredicto `worth_it` con `min_efficiency: 2.0` , `exposure.verdict.pass` , «cociente de Sharpes»: el codigo actual no tiene nada de eso (borrado 2026-10-11). Sus §4.3, 5.3, 5.13, 5.16 y 6 hablan de un veredicto que ya no existe; sirven para el porque, no para el estado.
- ⚠ `README.md` dice «Owner, 2026-10-11… no worth_it», y `studies/closing/README.md` ya lo describe como solo-describe: coherente con el codigo. `WORKFLOW.md:51` tambien.
- ⚠ Dossier 5.1 (residuo flotante en la ocupacion): corregido para `share` (`occupancy.py:70` usa `> 1e-9`), no para `presence` (`:95` , `lots != 0`).
- ⚠ Comentario de `benchmark.py:98-102` y README dicen que el dia de cierre es «conservador hacia la estrategia»; el dossier midio el efecto contrario (pequeño, ~2 % a favor).
- ⚠ Las corridas reales guardadas son de la version CON veredicto (2026-10-05/06); no hay ninguna corrida de la version actual.
- ⚠ `config.yaml: timeframe: M30` frente al TF del export que usa el lanzador (dossier 5.2; no verificado en el codigo actual).

PASO 22 — Mapa condicional (ELIMINADO el 2026-10-11)

Estado

Sin codigo vigente. `WORKFLOW.md:56` : «eliminado (2026-10-11): el estudio y su paso se borraron del codigo; no se renumera ningun paso». `studies/readings/conditionalMap/` (README, cells.py, config.yaml, contract.py, inputs.py, one.py, regime.py, report.py, sessions.py, tooltips.py) y `tests/test_conditionalmap.py` marcados `D` en el arbol sin commit. `studies/readings/README.md:11` : « `structure/` (step 23) and `conditionalMap/` (step 22) were deleted on 2026-10-11». `tail.PANEL` solo conserva el 21. Se documenta el ultimo estado (git HEAD, `studies/readings/conditionalMap/…` , y el dossier `criterios-paso-22-…2026-10-08.md`) por si el agente estadistico necesita saber que se quito.

Que era

Estudio descriptivo, nunca un filtro ni un veredicto (`verdict: null` , todos los bloques `state: info`). Pregunta: ¿de que estado del mercado en el momento de entrar depende que la estrategia gane? Por operacion: etiqueta de volatilidad (ATR de 20 dias diario, `volatility.atr_period: 20` , via `engines.regimes.regime.atr`), de tendencia (ratio de eficiencia de Kaufman a 20 dias, `trend.window: 20` , sin signo), sesion (Asia / solape Asia-Londres / Londres / solape Londres-NY / NY / fuera; horas locales Tokio 9-18, Londres 8-17, NY 8-17) y dia de la semana.

Criterios que tenia (ninguno cortaba)

- **C22.1 — Sin look-ahead**: ambas series retrasadas un dia (`.shift(1)`): una operacion del dia D lee el valor al cierre de D−1; hallado en revision 2026-09-26 (79 % de las barras de un dia real eran posteriores a la entrada). Cortes por tercil CONGELADOS sobre el segmento `build` de la politica (cuantiles 1/3 y 2/3 de las series entre las dos fechas de build) y aplicados a cualquier muestra. Debil: en unidades de precio, no se mueven con la volatilidad relativa: en USA500 el 99,7 % de los dias de oos1 cae en volatilidad «alta»; NIKKEI225 95,9 %; USDJPY 76 % en «baja» (dossier 5.1, calculo del autor). En los cuatro activos medidos la fila «media» tiene menos del 9 % de los dias fuera de build.
- **C22.2 — Celdas**: 4 vistas por muestra = 9 (vol x tendencia) + 30 (sesion x dia) + 6 (sesion) + 5 (dia) = 50 celdas por muestra y 100 por estrategia (dos muestras: OOS1 y «Completa» = build + oos1). Por celda con ≥ 30 operaciones: n, P&L medio en USD sin normalizar por tamaño, intervalo bootstrap percentil 95 % (`bootstrap.resamples: 9999, confidence: 0.95, seed: 0` , `core.surface.dedupe.bootstrap_ci`) y proporcion de ganadoras (sin intervalo). Debil: el intervalo es contra CERO, no contra el resto de la estrategia (en «Completa» la estrategia gana de media porque se construyo ahi, asi que muchas celdas «excluyen el cero» solo por eso); operaciones de una celda no independientes (la celda mas llamativa de 3.13.366 OOS1: 32 operaciones, 9 episodios de mercado, 94 % en 2022, 3 operaciones = 68 % de la ganancia).
- **C22.3 — Suelo de 30 operaciones**: prestado de `engines/nulls/config.yaml:42` (`verdict.min_trades`), no derivado de una cuenta sobre una celda; con 30 operaciones el IC mide ±348 a ±491 USD frente a un P&L medio de 46-120 USD (dossier 5.4).
- **C22.4 — Aviso de comparaciones multiples**: texto fijo, sin correccion. Con una estrategia sin ventaja la probabilidad de ver ≥ 1 celda «significativa» es 41-56 % en OOS1, 69-78 % en la Completa (calculo del autor del dossier, no en el codigo).
- **Regla dura 15**: 41 de 50 celdas son de reloj/calendario (sesion, dia), que la regla 15 prohibe al constructor.

Corrida real

`Test_USDJPY_M30_doubleROCLong` (3 madres, USDJPY M30 doble ROC largo, `reports/.../conditionalMap/` , autopiloto `22 | reading | 8 s | conditionalMap sobre Results: 3 madre(s)` ; tiempo por madre 1,89-2,50 s). En total 137 celdas legibles de 300 posibles y 12 con IC95 % fuera de cero (todas por arriba; 8 de reloj/calendario). Esperable por azar con la media real de cada estrategia conservada: 12,6 (6,3 con media cero); en OOS1: 1 vs 2,7 esperadas; en Completa: 11 vs 9,8. Por madre (OOS1/Completa): 10.23.380: 402 ops, −61,20 USD / 1138, +45,81, 19 celdas legibles de 50, 0 fuera de cero / 33, 3; 3.13.366: 331, +0,89 / 933, +80,22, 16, 1 / 29, 3; 4.25.395: 240, +67,75 / 660, +119,76, 13, 0 / 27, 5. Referencia de tiempos (NIKKEI225 M30 `..._simpleL_ref`): 14 s para 10 madres, sin SQX (`WORKFLOW_TIMING_BASELINE.md:36`).

Que lo reemplazo

Nada. `WORKFLOW.md:135` («build — sólo el paso 6… El mapa condicional (22, eliminado)…») conserva solo la mención.

Discrepancias ⚠

- ⚠ `bin/workflow.sh:88` aun pasa `--panel 22=Results` (ver paso 20).
- ⚠ `docs/AgentPDFs/README.md:61-62` aun lista los dossiers de 22 y 23 como «provisional»; `criterios-paso-22/23` se conservan en disco, y los manuales del paso 22 y 23 (`06-lecturas.pdf` , cap. 53-mapa-condicional y estructura) no verificados si estan retirados.
- ⚠ `docs/SKILLS.md` , `OPEN.md:955` («Steps 22, 23…») mencionan estos pasos; no verificado el estado exacto.

PASO 23 — Estructura: ablacion por condicion de entrada e inversion (ELIMINADO el 2026-10-11)

Estado

Sin codigo vigente. `WORKFLOW.md:57` : «eliminado (2026-10-11): el estudio y las variantes estructurales (`make` , `keep` , `plan`) se borraron; solo queda `sqx/structural/logic.py` , que lee la logica de una estrategia». `studies/readings/structure/` completo marcado `D` ; `sqx/structural/` conserva `logic.py` y README. En el arbol sin commit tambien salen modificados `pipeline/autopilot/closing.py` y `tests/test_autopilot_closing.py` . Descrito por git HEAD (`studies/readings/structure/{config.yaml,reading.py,measure.py,one.py,report.py}`) y el dossier `criterios-paso-23-…2026-10-08.md` (692 lineas).

Que era

Diagnostico por madre (`verdict: info`, «Diagnostico, nunca seleccion», sin estados ni corte): ¿cual de sus condiciones de entrada elige buenas operaciones y gana por la direccion en que entra? Cadena de 5 comandos (`closing.py`): `sqx.structural.make` (fabrica ficheros S00O00 madre reconstruida, S00A01.. una ablacion por condicion de entrada borrada, S00I00 inversion), `sqx.variants.execute` (3 patas WFC en SQX, custodio), `keep` (verifica que SQX conservo la edicion), `export_retest --batch structure`, `studies.readings.structure.report`. Lectura SOLO de `build` y `oos1` (`STRUCTURE = ("build","oos1")`), nunca oos2 en el estudio (aunque el lote se reprueba tambien en oos2: dossier punto 7, 105 versiones sin fila de ledger).

Criterios que tenia (ninguno cortaba; palabras, no supervivientes)

- **C23.1 — Ablacion y recorte al azar** (`measure.ablation`, `subset_null`, `config.yaml: subsets.draws: 10000, seed: 0; verdict.alpha: 0.05`): por condicion y tramo, `delta_E = expectativa/operacion de la madre − la de la ablacion` (por operacion, nunca beneficio total, porque quitar un filtro cambia el numero de operaciones; tambien Sharpe por operacion mean/sd). Nulo: 10.000 subconjuntos al azar de `len(madre)` operaciones sacados de la lista de la ablacion; p = (1 + #(azar ≥ observado)) / (draws+1) sobre la expectativa. Palabras (`reading.condition`): `aporta` si ΔE ≥ 0 y p < 0,05; `no se distingue` si p ≥ 0,05; `resta` si ΔE < 0 (sin prueba, cualquier negativo); `redundante` si la ablacion tiene las mismas operaciones; `no es un filtro` si la ablacion no opera mas que la madre; `no aplicada` si `keep` detecta que SQX perdio la edicion. Debil: (a) p NO es exacta cuando la madre no opera un subconjunto de la ablacion (una estrategia con una posicion abierta a la vez, sin el filtro, entra antes y ya esta dentro cuando llegaria la entrada de la madre: «gemelas» del 20 al 77 %); con 5 % nominal el falso «aporta» sube a 15-20 % en el peor caso (dossier 2.5, calculado por el autor, no es cota demostrada); (b) «aporta» en `build` no es evidencia: el generador eligio la condicion y la estrategia por mejorar build; (c) 30 condiciones con una prueba cada una, α = 0,05 sin correccion (esperable 1,5 «aporta» por azar a 5 %).
- **C23.2 — Inversion** (`measure.inversion`, `reading.direction`, `controls.min_paired: 0.99`): invierte `#Direction#` y las palabras Long/Short sin tocar las señales (mismas barras de entrada y salida, lado contrario; lanza `ValueError` y tumba el lote si la madre lleva stop, target, trailing o break-even). Empareja operaciones en orden y exige mismas instantes, mismo tamaño, lado contrario (≥ 99 % emparejadas, si no `no invertida`). Descompone: `mid` (media de Δmid de la madre; el de la invertida es su negativo), `spread` = −(gross_m + gross_i)/2, `carry` swap+comision. Palabra: `en la direccion` si `mid > 0` (madre gana a precio medio), `sin filo a precio medio` si no (corte en cero, sin prueba). Debil: la inversion sin stops es el espejo exacto de la madre a precio medio (aritmetica: «gana la madre = pierde la invertida»); solo informa el desglose; la deriva del activo no se separa (en oos1 USDJPY ≈ 8.000 USD de los 19.700-26.600 que las madres ganan a precio medio); «en la direccion» salio en 65 madres de 65 en build.
- **C23.3 — Controles de identidad** (`controls.identity_tolerance: 0.005`): la madre reconstruida debe reproducir el resultado guardado en el `.sqx` (n y neto con ±0,5 %). Se compara con la muestra «IS» del fichero; con el fichero que copia el autopiloto (databank `SPP OOS`, tarea sobre oos1) esa muestra es oos1, no build: 59 de 62 madres dan «igual: False» por esa razon (dossier punto 2); por tanto el control del tramo build sale siempre `watch` en el autopiloto.

Corrida real (dossier, no verificada en disco por mi)

3 madres USDJPY (plantilla doble ROC + hueco aleatorio BBWRPercent o WassersteinShiftHigh; build 2008-2017, oos1 2018-2022), 2 condiciones cada una: `build` «aporta» 5 de 6, `oos1` 0 de 6 (5 «no se distingue», 1 «resta»); inversion «en la direccion» 3/3 en build, 2/3 en oos1. En los cinco lotes con ablaciones en disco (15 madres, 30 condiciones): `build` 27 de 30 «aporta», `oos1` 1 de 30 (azar esperado ≥ 1,5; hasta 3,6 con nulo estrecho). Sin plantilla (Simple Strategy, p.ej. NIKKEI225) la fabrica no ablaciona nada (18 de 50 madres de NIKKEI con 2 condiciones y 0 ablaciones; el AND `generated="random"` se toma por una sola condicion, `sqx/structural/logic.py:10-11`) y el informe no avisa.

Coste

Referencia `Test_NIKKEI225_M30_simpleL_ref` (10 madres, 2026-10-04/05): **260 s**, de ellos SQX 76 s (calculando), resto 29 s Python + 34 s de carga + ~121 s de otras fases (columnas del cuadro: total 260, sqx 76, otros 13/44/64/0): ver `WORKFLOW_TIMING_BASELINE.md:37` y su cabecera de columnas; no verificado el significado exacto de cada columna intermedia. Corrida de 3 madres USDJPY: 143 s (12 ficheros x 3 patas x 17 mercados porque las patas WFC llevan los mercados adicionales; el estudio solo lee el principal). Python: unos segundos por madre.

Discrepancias ⚠

- ⚠ `docs/AgentPDFs/criterios-paso-23-…` (dossier del 2026-10-08) describe las 12 cosas que no cuadran sobre un estudio ya borrado; su valor actual es historico.
- ⚠ `OPEN.md:587` («Step 23 control “madre reconstruida” printed an empty table») y el control de identidad (C23.3): nunca se corrigio antes del borrado.
- ⚠ `closing.py` modificado sin commit: el comentario `closing.py:16` y `STRUCTURE` pueden haber cambiado; no verificado.

- ⚠ Los PDFs del manual `06-lecturas.pdf` y `10-cierre.pdf` aparecen como modificados en `git status`; no verificado si ya no describen 22 y 23.

Pasos 24, 25 y 26 — tal como están implementados y corridos (lectura del 2026-10-11)

Fuentes verificadas: código de `studies/readings/edgeCost/`, `engines/market/reprice.py`, `mt5/verify/`, `mt5/compare.py`, `pipeline/autopilot/{tail,finalists,survivors}.py`, `ledger/thresholds.yaml`, `pipeline/autopilot/criteria.yaml`; datos de `AlgoData/{reports,pool,mt5/verify,harvest}`. Los dossiers `criterios-paso-24/25/26-*-2026-10-08.md` describen un estado anterior (con stop del paso 24); donde discrepan, manda el código y va marcado con ⚠.

PASO 24 — Stop ATR (ELIMINADO, 2026-10-11)

Estado: no existe. El paso, su estudio (`atrCalculator`) y su injerto en SQX se borraron (WORKFLOW.md:58 y :105-114). No hay código ni criterio vigente. Restos: `AlgoData/atrCalculator/` y `AlgoData/reports/atrCalculator/` (datos de corridas antiguas), cartas de `knowhow/` que lo mencionan, y las columnas heredadas `stop_percentile` y `stop_x` (siempre vacías) en `pool/*.csv` (`mt5/verify/pool.py:19-20, 87`).

Qué fue y por qué se quitó (para el agente estadístico)

- Diseño: un stop lejano `SL = k·X·ATR(20)` con X = percentil 90 del MAE/ATR de las ganadoras de `build` de cada madre, injertado antes del 16.5, porque MT5/la cuenta de fondeo exigía stop. La cadena construye sin stop a propósito (cada parámetro es sitio para sobreajuste).
- Criterio preregistrado (`portfolio/DECISIONS.md`, 2026-10-11; dossier `medicion-stop-lejano-2026-10-11.md`): con k=2 fijado antes de mirar, si el stop 2·X toca < 1 % de las operaciones (cota superior del intervalo de Wilson 95 %, todas las madres, `build` + `oos1` juntos, `oos2` no se lee) se injerta con X fijo; si no, ningún stop por operación.
- Resultado de la medición: 7 proyectos, 56 madres, 17.718 operaciones (build+oos1). Toques a k=1: 32,96 % (Wilson 32,27–33,66); k=1,5: 18,75 %; **k=2: 11,55 % (11,09–12,03) → NO PASA** (límite 1 %). Por tramo a k=2: build 9,27 % (1.026/11.066), oos1 15,33 % (1.020/6.652). Por madre 6,2–16,7 %; ninguna baja de 6,2 %. Derivación calibrada a k=1: 33,0 % derivado vs 32,7 % real de SQX (válida para tasas de toque; no para dinero: neto derivado 2.183.070 vs 2.058.370 real, suelo del daño).
- Coste histórico del stop injertado (dossier 2026-10-08): p90 activo en 23–41 % de operaciones y ~32 % del neto de oos1; su retirada devuelve ese ~+32 %.
- Puntos débiles declarados de la medición (medicion-stop-lejano §6): las madres hermanas comparten operaciones, así que el Wilson (independencia) es optimista; sólo largos; día cortado en la medianoche del feed; comisión sin flotar; el MAE se deriva, no se retestea.
- Cuenta (informa, no decide): al tamaño exportado entre el 49,8 % y el 76,5 % de las cuentas simuladas (100.000 arranques) rompen un límite de prueba **sin stop**; es problema de tamaño de posición, no de stop (pendiente en `portfolio/`, OPEN.md #113).

Consecuencias vivas en 25 y 26 (⚠)

- ⚠ `pipeline/autopilot/finalists.py:19,27` aún dice «mothers after step 25 … the mother that reached step 26 with no stop»: coherente con la ausencia de stop, pero «after step 25» ya es falso (el 25 no corre; ver abajo).
- ⚠ `judge.py` fila 2 usa R = pérdida media de SQX «porque no llevan stop» (`mt5/verify/judge.py:106-107`); el dossier 26 §5.7 decía que ese supuesto era falso por el stop del 24. Con el 24 eliminado, el supuesto vuelve a ser cierto (pero R sigue sin confirmar como unidad; ver 26-C2).
- Sin stop por operación, el 26 recibe el `.sqx` de la madre tal cual (`finalists.py:27-31`).

PASO 25 — Edge por coste, por estrategia

Propósito y estado real

Pregunta: ¿cuántas veces cabe el coste pagado en la ventaja bruta de cada operación, en `oos1`? Es el mismo estudio que corre el paso 8 en modo población.

⚠ **Estado: el paso 25 está FUERA de la cadena desde 2026-10-11** (WORKFLOW.md:59). Ninguna etapa lo corre: `pipeline/autopilot/tail.py` tras el 21 salta a `finalists` (26) y `PANEL = {"21": "exposure"}` (`tail.py:15`); no hay regla del 25 en `criteria.yaml` (de hecho allí no hay ni paso 25 ni `edgeCost.*`: la línea 68 anota «Fuera desde la conmutación: … spread. *y edgeCost.*»).

Sólo queda el estudio, usable (a) en modo población sobre la cosecha del 8 (se corrió el 2026-10-11 en dos proyectos Test_), (b) con `--strategy` como orden suelta. Lo que sigue documenta ese estudio tal cual, y su gemelo con efecto: R5c del paso 8.

Entradas / salidas

- Entrada: cosecha del gate `AlgoData/harvest/<proyecto>/<databank>/<fecha>/` (`trades.parquet` con `identity`, `sample` IS/OOS, `Type`, `Open/Close time`, `Size`, `Close type`, `Close price`; `metrics.parquet` con `strategy` y `TimeFrame [IS]`) + barras del feed (`core.barstore.read(feed, tf)`) + ficha del activo (`assets/symbols/*.yaml`, costes del tramo oos1). `inputs.newest` toma la última carpeta fechada (`inputs.py`).
- Salida: `AlgoData/reports/<proyecto>/<databank>/<fecha>/edgeCost/{edgeCost.json,.html,.md,manifest.json,verdict.csv}`; `verdict.csv` = `strategy, identity, edge_mean, verdict` (en el manifiesto: columnas `identity, edge_mean, verdict`; el índice es el nombre).
- Comando: `python3 -m studies.readings.edgeCost.report --project P --databank D --feed XAUUSD_M1 [--strategy NOMBRE] [--set verdict.min_edge_spreads=…]` (`report.py:20-27`).

Criterios

C1. Edge medio sobre el coste pagado (el único criterio del estudio) - Pregunta: ¿el movimiento de precio bruto por operación supera varias veces lo que cuesta operar? - Definición exacta: por estrategia, sólo operaciones `sample == "OOS"` (oos1; `costs.per_trade`, `costs.py`), cada una **repreciada** por `engines.market.reprice.real` con los costes declarados del segmento oos1 de la ficha del feed (`reprice.costs(feed, "oos1")`): entrada = apertura de barra + side·(spread + slippage), salida = apertura de barra − side·slippage (o, si `Close type` ∈ AT_PRICE —salida a precio, p. ej. stop/TP— el precio de cierre de SQX corregido por slippage), redondeo al `tick_step`; `cost = point_value·size·(spread + 2·slippage) + swap por noche (sobre el nocional, % o puntos; rollover y triple) + comisión (PercentageBased sobre nocional de entrada, o SizeBased por lote)` (`reprice.pnl`, `reprice.py:75-118`); `pnl` = neto redondeado al céntimo. Entonces `edge_mean = mean(pnl + cost) / mean(cost)` (media sobre todas sus operaciones de oos1; cociente de medias, no media de cocientes) (`one.measure`, `many.run`; idéntico a `onepass.edge_mean`, `studies/screening/analysis/onepass.py:11-21`, el hecho `gate.scorecard.edge_mean_oos`). 1 = punto muerto; es también el múltiplo del coste al que el neto se anula. - Umbral vigente: `verdict.min_edge_spreads = 2.0` (`ledger/thresholds.yaml:198-206`; fijado por el agente 2026-09-26 «antes de mirar», confirmado definitivo por el dueño 2026-09-29, OPEN.md §48); `verdict.action = mark` (`:208-216`). Se lee en `studies/readings/edgeCost/config.yaml` vía `ledger:` y `thresholds.fill`. - CORTA/MARCA: **MARCA o INFORMA.** `many.run`: `verdict = DESCARTAR` sólo si `edge_mean < 2.0` **y** `action == "drop"`; con `mark` todos salen `MANTENER` (aunque el panel pinta en rojo los < 2). `one.run` emite un veredicto «por encima/por debajo del umbral» con estado pass/fail, no elimina. Nulos: si una estrategia no paga coste, `mean(cost)` = 0 → inf/NaN (sin tratamiento propio; `onepass.edge_mean` lo documenta como «inf or NaN»). Una `identity` sin nombre en `metrics.parquet` queda con `strategy` NaN (no verificado qué ocurre). - Por qué: es el orden de magnitud de partida del encargo 11 («2-3 spreads por operación»); hacer la ventaja robusta a un coste 2× mayor. Punto débil visible: (i) el nombre del umbral/clave dice «spreads» pero el denominador es el coste total pagado (spread + 2 slippage + swap + comisión) ⚠ (dossier 25 §5.2); (ii) cociente de medias dominado por colas (pocas operaciones grandes) y sin intervalo ni error de muestreo: el dossier calculó que con un `oos1` igual de válido «5 de 10 por encima de 2» sería entre 2 y 8 (IC 95 %, remuestreo por semanas); (iii) sólo `oos1` (un tramo, un año malo, incluye la subida del índice); (iv) multiplicidad: se mira por estrategia sin corrección sobre poblaciones de miles; (v) los edges de la lista de supervivientes están sesgados al alza, porque pasar neto > 0 en oos1 es equivalente a edge > 1 por construcción; (vi) no mide la ventaja neta ni su significancia (para eso está `t_exceso_k` en el gate). - Ejemplo real (2026-10-11, población del gate sobre `Results`, oos1, `code_version` abca93c-dirty / 78f2bfe-dirty): - `Test_USATEC_H1_ruptura_leaseWeb`: 10.000 estrategias; `edge_mean` media 2,415, mediana 2,398, rango −5,55…10,34; 5.866 ≥ 2, 4.134 < 2, 2.556 < 1,25, 2.113 < 1, 970 < 0. Veredicto del paso 25: 10.000 MANTENER (acción `mark`). - `Test_XAUUSD_H1_ruptura_wf4`: 10.000; media 0,746, mediana 0,786, rango −2,89…3,61; sólo 232 ≥ 2, 9.768 < 2, 7.786 < 1,25, 6.475 < 1, 1.221 < 0. 10.000 MANTENER. - Lectura: un mismo umbral 2,0 deja pasar el 59 % en USATEC (un índice, coste relativo pequeño) y el 2 % en XAUUSD: el umbral es de nivel absoluto, no relativo al coste/volatilidad del activo. - Sin corrida real sobre una versión con stop (la medición vieja de 10 versiones NIKKEI M30 del 2026-10-04 dio 5 de 10 ≥ 2 con stop, 9 de 10 sin él; está en el dossier, la cosecha `Stop_p90` ya no existe como paso). - Coste: el cálculo en sí es ligero (`edgeCost.json` registra `wall_s` = 0,58 s y 0,39 s, pero es sólo el tramo posterior al repreciado; el repreciado de 10.000 estrategias no está cronometrado ⚠ sin dato de extremo a extremo; sin dato en `perf/`). Sin SQX, un solo proceso, RAM sin dato. Fuera del cálculo, `edgeCost.html` pesa ~2 MB y el JSON ~1,8 MB para 10.000 filas. - ⚠ La carpeta de cosecha actual `harvest/Test_XAUUSD_H1_ruptura_wf4/Results/2026-10-11/` contiene sólo 572 operaciones y 1 identidad, no las 10.000 estrategias que reflejó el informe: la cosecha se sobrescribió después (no verificado cuándo). El informe ya no es reproducible desde esa cosecha.

C1'. Gemelo con efecto: R5c del paso 8 (para contexto, sin documentar el paso 8 entero) - `pipeline/autopilot/criteria.yaml:91-94`: Pasa con `edge_mean_oos >= 2` **y** `t_exceso_k >= 1.65`; LIMBO si `edge_mean_oos >= 1.25` (o la t baja); Falla con `edge_mean_oos < 1.25`. Es decir, aquí el mismo número sí corta, con umbral de fallo 1,25 y de pase 2,0, que no coinciden con el 2,0/mark del estudio ⚠ (los dos «2» viven en ficheros distintos: `thresholds.yaml` y `criteria.yaml`). Con las poblaciones de arriba, sólo por R5c fallarían 2.556 (USATEC) y 7.786 (XAUUSD) de 10.000.

Orden de aplicación y cadena

Un solo criterio. Cadena de ejemplo (informativa, no corta): 10.000 entran → 10.000 salen (`mark`); marcadas en rojo 4.134 (USATEC) / 9.768 (XAUUSD).

Tiempos / recursos

Sin dato fiable (ver C1). Sin SQX, sin CPU pesada.

Discrepancias del 25 (⚠ resumen)

1. El paso está «fuera de la cadena» pero `finalists.py` y la documentación del estudio (`README.md` : «step 25 (the version that will be traded)») lo siguen nombrando; el README del estudio habla de una versión con stop que ya no existe.
2. `criteria.yaml` no tiene regla del 25; el 25 nunca cortó nada en la práctica (dossier 25 §5.7: `action: drop` no habría hecho nada).
3. Umbral «spreads» mide coste total; dos «2» en dos ficheros; `edgeCost` aparece en `catalogue.py:37` como paso «8».
4. Cosecha de referencia sobrescrita (arriba).

PASO 26 — Validación en MT5 con la feed de cada empresa → pool validado

Propósito

Comprobar que SQX, con los costes y el reloj de cada empresa de fondeo, reproduce el backtest del mismo EA en MT5 en la cuenta de esa empresa. No elige por rendimiento (dueño, 2026-09-29): sólo valida que la historia de SQX vale para esa cuenta. Si valida, la estrategia entra en el **pool** de esa empresa, lo único que lee la cartera (paso 27).

Entradas

- Lista de finalistas `finalistas.json` (`pipeline/autopilot/finalists.py`): **todas las madres supervivientes que llegan al 26** (`survivors.mothers` , «never filtered by 17-19», dueño 2026-09-29), cada una con `strategy, identity, mother, sqx (ruta absoluta), symbol, timeframe` . `unfit()` (`mt5/verify/autopilot.py:23-45`) exige que activo, TF e identidad coincidan con lo que dice el propio `.sqx` , y que el activo tenga ficha.
- Parámetros que el dueño da cada vez, **sin defecto** (`autopilot.py:15-18`): `start` (AAAA-MM-DD o `MT5` = historia común más profunda de todos los servidores), `end` , `model` (`every_tick` , `ohlc_m1` , `open_prices` , `real_ticks` ; `tester.MODELS`), `firms` ; opcional `--also-real-ticks` (segunda pasada en Real ticks). El raíl se detiene si falta alguno (`tail.py:158-163`).
- Por empresa: cuenta MT5 guardada (`core.paths.MT5_ACCOUNTS` : hoy `ftmo` y `hantec`), símbolo de la empresa en `assets/symbols/<activo>.yaml#mt5.<firma>` , comisión confirmada en `costs.commission.brokers.<firma>` ; sin cualquiera de las tres, esa empresa se rechaza («nunca se adivina») y no da veredicto (`firms.usable` , `conditions.for_sqx` , `autopilot.uncommissioned`).

Mecánica (`mt5/verify/run.py`)

1. Regla 5 del activo; por empresa se lee **en vivo** `symbol_info` del servidor: spread actual, point, swap (modo, largo, corto, día de triple swap), apalancamiento, primera barra mensual (`conditions.read`).
2. Se traducen costes a SQX (`conditions.for_sqx`): spread = `info.spread·point/tick_size` (el spread del momento de la lectura, «spread actual del símbolo», dueño 2026-09-29, sin factor); comisión de la ficha; swap en modo points/money/percent (modos MT5 0,1,4,5,6; otros → Refused); **slippage de SQX = 0** (`config.yaml: sqx.slippage: 0` , porque el tester de MT5 no desliza).
3. Proyecto `Test_MT5Verify_<activo>_<TF>_<hhmmss>` clonado del donante en el **conductor** (regla 3/10), una tarea retest por empresa con sus costes y exportación del `.mq5` (`sqxside.build/run` , `sqx.projects.mt5verify.configure`); la ventana es la pedida, recortada a un día antes del último dato de SQX (`run.latest`); tamaño = el FixedSize propio del `.sqx` (otro método → SystemExit; `mt5verify.own_sizing`); depósito = `InitialCapital` del retest (por defecto 100.000). El proyecto se retira al acabar (`sqxside.retire`).
4. El EA exportado se compila con lote fijo = el de la estrategia (`mt5side.compile_ea` , reescribe `mmLots` o `UseMoneyManagement=false` + `mmLotsIfNoMM`), y se backtestea en MT5 por empresa en su cuenta, apalancamiento y modelo elegido (`mt5side.backtest` ; límite 60 min por pasada, 240 en Real ticks).
5. Ambas listas se recortan a operaciones abiertas en [Desde, Hasta] (`compare.window` , por hora de apertura, antes de convertir reloj en SQX), SQX se convierte trade a trade del huso de su feed (`feeds.timezone`) al del servidor (`EETUS` = Nueva York + 7 h; horas inexistentes/repetidas → NaT, no se emparejan; `compare.to_zone`), y ambas se expresan **en puntos** (`compare.in_points` : P&L = movimiento de precio × point_value × tamaño, un único valor de punto para los dos lados; comisión y swap fuera del P&L comparado).
6. `judge.firm_result` busca el desplazamiento horario entero (±3 h, `clock.max_offset_h`) que empareja más (primero «pares dentro de media tolerancia», luego pares, luego menor distancia, luego menor |h|; `judge.clock`), empareja y aplica las filas.

7. Una fila de ledger por comparación (`record.log` , `step 26` , criterio «SQX ↔ MT5 en ()», `n_in = 1` , `n_out = 1` si pasa); y el pool (`pool.record`).

Salidas

- `AlgoData/mt5/verify/<fecha-hora>_<estrategia>/{run.json,result.json,mq5/,sqx/,in/}` (28 carpetas hoy); informes de MT5 en `AlgoData/mt5/tests/` (161).
- Pool: `AlgoData/pool/<firma>.csv` , **append-only**; `pool.read(firma)` devuelve, por (identidad, desde, hasta, modelo), la fila cuyo **primer veredicto** (por `verified_on`) sea `pass` : una repetición queda en el histórico pero no cambia nada (dueño 2026-10-11, `pool.py:read`). Una verificación sin veredicto (`none` , pocas operaciones) no se anota (`pool.rows_of`). Columnas: identidad, ventana, modelo, veredicto, hash de umbrales (leído de la fila del ledger de la corrida, no del YAML de hoy), `sqx_trades` , `mt5_trades` , `clock_h` , `row_1, row_2, row_3a, row_3b, row_4, row_5` , y las dos columnas heredadas de stop. ⚠ **No guarda `row_1b`** (ver C1b).
- Estados del veredicto: `pass` («Validada»), `fail` («No validada»), `none` («Sin veredicto»: SQX < `trades_min` operaciones en la ventana). Una empresa rechazada («sin verificar») no es veredicto.

Criterios (todos MARCAN el veredicto de esa empresa; el conjunto decide; ninguno elimina estrategias de SQX)

Regla de síntesis (`judge.firm_result`): `Validada` ⟺ **todas** las filas `pass` y SQX tiene ≥ `trades_min` operaciones; una fila que no se puede calcular **falla** (`judge.one` : «un chequeo que no se pudo hacer no se ha pasado»); umbrales leídos de `ledger/thresholds.yaml` vía `mt5/verify/config.yaml:thresholds` , valores dueño 2026-09-29 salvo los dos provisionales.

C0. Reloj (no es fila, pero decide el resto) - Qué: desplazamiento entero de horas tras la conversión de huso. Definición: `judge.clock` (arriba). Salida registrada: `clock_h` (en los datos: 0 en NIKKEI; −1, 0 o +1 en USDJPY, p. ej. Hantec +1 en dos corridas). - Débil: el desplazamiento se elige **mirando** los emparejamientos que luego se juzgan (selección sobre el mismo dato que se evalúa; hasta 7 candidatos); y es el mismo para toda la ventana, no por tramo.

C1. Fila 1 — operaciones emparejadas, en el peor de los dos sentidos - Pregunta: ¿el EA de MT5 hace las mismas operaciones que SQX? - Definición: `compare.pair` empareja **voraz en el orden de SQX**: para cada operación de SQX toma la operación de MT5 libre del mismo `Type` con menor |diferencia de hora de entrada| ≤ `entry_tolerance_bars · minutos(TF)` ; cada de MT5 se usa una vez. `matched = parejas / nº SQX` , `matched_mt5 = parejas / nº MT5` ; la fila es `min(matched, matched_mt5)` (`judge.rows`). **No compara precio ni salida** en esta fila. - Umbral: `matched_min = 0.95` (dueño 2026-09-29, `thresholds.yaml:1041`); `entry_tolerance_bars = 1` (`:1048`). Pasa si `>= 0.95` . - Por qué: valor de partida aceptado por el dueño. Débil: (i) potencia binomial muy dependiente de n (dossier 26 §4.4, supone independencia): con 105 operaciones y fidelidad real del 97 % falla 1 de cada 10, un 93 % pasa 1 de cada 4; hacen falta ~300 operaciones para que un 93 % real pase < 10 %; las huérfanas llegan en rachas (más optimista aún); (ii) la tolerancia decide: `16.9.76` /FTMO valida sólo por cinco parejas a una barra (con tolerancia 0 quedaría en 366/387 y fallaría); S00V014 pasa de 77 % (0 barras) a 93 % (8); (iii) el emparejado voraz depende del orden y no es óptimo; (iv) con tolerancia 1 barra una entrada desplazada una barra cuenta igual que una exacta. - Ejemplos reales: NIKKEI225 M30, 2024-01-01→2026-09-30, `every_tick` , 2026-10-04, S00V014: FTMO 0,867 (105 SQX / 101 MT5), Hantec 0,876; S01V014: 0,836 / 0,738; S02V014: 0,68 / 0,672; S03V014: 0,650 / 0,600; S04V014: 0,546 / 0,462. USDJPY H1 2022-06→2026-09 (`modelo ohlc_m1` , 2026-09-30, `16.9.76`): FTMO 0,9587 (387 SQX / 399 MT5, pasa), Hantec 0,9638 (393). EURUSD M30 2024-01→2025-06 ohlc_m1 (2026-10-09, Strategy 3.24.322): FTMO 0,8235 (51/52). - Coste: ver «Tiempos».

C1b. Fila 1b — parejas que cierran a la misma hora - Definición: de las parejas, fracción con `Close time_mt5 == Close time` (igualdad exacta de hora de cierre tras el desplazamiento de reloj) (`judge.rows`). Pasa si `>= exit_same_min = 0.95` (`thresholds.yaml:1083` , **provisional**, propuesto por el agente del paso 26 el 2026-10-11, cambio aprobado por el dueño; «igual que matched_min; pendiente de la corrida de referencia por activo × empresa»). - Débil: igualdad exacta de timestamp (sin tolerancia de salida), cuando la entrada sí la tiene; en salidas de barra a barra es razonable, pero una salida un minuto distinta por reloj la falla; umbral sin calibrar. - Ejemplo real: **sin corrida real**: las 28 corridas y el pool son anteriores a 2026-10-11 (la fila no existe en `result.json` ni en `pool/*.csv`). ⚠ `pool.FIGURES` no incluye `row_1b` , de modo que el pool no registrará la fila aunque se corra (`pool.py:17-18`).

C2. Fila 2 — diferencia media de P&L por operación emparejada, en R - Definición: `mean|PL_MT5 − PL_SQX| de las parejas / R` , con P&L en puntos × valor de punto fijo × tamaño (sin comisión ni swap); `R = media de las pérdidas de SQX` (`|P&L| de las operaciones perdedoras` , todas, sin stop). Si no hay pérdidas o parejas, la fila falla. Umbral `trade_gap_max_r = 0.05` (`:1055`); pasa si `<= 0.05` . - Por qué: que el EA gane/pierda lo mismo por operación, medido contra la unidad de riesgo típica. Débil: «R» no es un stop real (la pérdida media de SQX, no el riesgo planeado); sensible a la media de pérdidas (colas); una media sobre las parejas ignora las huérfanas (que la fila 1 ya castiga); el sesgo firmado se muestra pero no se juzga. - Ejemplos: NIKKEI `every_tick` : 0,23–1,03 (todas fallan, p. ej. S02V014 FTMO 1,028; S00V014 0,266 / 0,230). USDJPY (`ohlc_m1`): 0,035 (FTMO `16.9.76`), 0,0206 (Hantec), 0,015 y 0,009 (`21.9.59`): pasan. EURUSD: 0,058 (falla por poco).

C3a. Fila 3a — correlación del P&L diario - Definición: P&L por día del servidor (día de **cierre**, medianoche del servidor), en % de la cuenta (`Profit/Loss / deposit·100`), cada lado reindexado a la unión de días con relleno 0; correlación de Pearson (`judge.daily` , `rows`). Se necesitan > 2 días y desviación > 0 en ambas series, si no, falla. Umbral `daily_corr_min = 0.95` (`:1062`). - Débil: Pearson

sobre días mayoritariamente 0 está dominado por pocos días grandes; no es robusta a un día desplazado; con 105 operaciones en 2,75 años hay pocos días con valor; dossier §4.5 calculó su precisión. Ejemplos: NIKKEI 0,50–0,96 (sólo S01V014 pasa: 0,965 FTMO / 0,954 Hantec); USDJPY 0,957 / 0,973 / 0,971; EURUSD 0,944 (falla por 0,006).

C3b. Fila 3b — diferencia media absoluta del P&L diario, % de la cuenta - Definición: `mean|daily_SQX − daily_MT5|` en puntos de % de la cuenta, sobre los mismos días. Umbral `daily_mad_max_pct = 0.1` (`:1069`). - Débil/⚠: con depósito de 100.000 USD y operaciones de 0,1 lote la diferencia diaria es del orden de 0,001–0,005 % (NIKKEI 0,0005–0,0017; USDJPY 0,0019–0,0042; EURUSD 0,005): **no puede fallar a ese tamaño** (dossier 26 §5.1/5.16); es un umbral sin potencia, no un control.

C4. Fila 4 — peor día: diferencia en puntos de % - Definición: `|min(daily_SQX) − min(daily_MT5)|` en puntos de % de la cuenta. Umbral `worst_day_gap_max_pp = 0.5` (`:1076`). - Débil/⚠: igual que 3b, con estos tamaños el peor día mide ~0,001 %; valores observados 0,0001–0,0020 (todas pasan). Sin potencia.

C5. Fila 5 — drawdown máximo: diferencia relativa - Definición: `max_dd = max((pico − equity)/pico)·100` sobre el **balance al cerrar cada operación** (suma acumulada del P&L en puntos, pico recortado a ≥ depósito); fila = `|dd_SQX − dd_MT5| / max(dd_SQX, dd_MT5)` ; 0 si ambos son 0. Umbral `dd_rel_gap_max = 0.10` (`:1097`). - ⚠ **No es el drawdown intradía sobre flotante** que dicen WORKFLOW.md:52 («drawdown intradía») y el encargo 34: es el del balance al cierre; la nota de la propia fila lo dice («el flotante intradía con M1 está pendiente», `judge.py` fila 5; dossier 26 §5.8). Para una cuenta de fondeo manda el flotante (memoria del dueño). - Débil: cociente relativo de dos números pequeños, inestable cuando el DD es chico. Ejemplos: NIKKEI FTMO 0,002–0,24 (S00V014 0,002 pasa; el resto falla), Hantec 0,004–0,38; USDJPY 0,073 (FTMO `16.9.76` pasa), 0,157 (Hantec falla), 0,024 y 0,013 (pasan); EURUSD 0,188 (falla).

C6. Mínimo de operaciones de SQX para dar veredicto (`trades_min`) - Definición: si `len(sqx) < trades_min = 30` (`:1090` , **provisional**, propuesto por el agente 2026-10-11, aprobado por el dueño; pendiente de corrida de referencia) → estado `none` «Sin veredicto»: ni pasa ni falla, no se anota en el pool. Se evalúa sobre las operaciones de SQX **ya recortadas a la ventana**. - Débil: 30 operaciones dan una fila 1 sin potencia (con 27 y 1 huérfana permitida, un EA con fidelidad real del 97 % pasa sólo el 81 %). Ejemplo real: 2026-09-29 `Strategy 3.48.75` XAUUSD H1 2025-01→2026-09: 28 operaciones SQX / 23 MT5 → habría sido «sin veredicto» (entonces falló: fila 1 0,643, fila 2 0,138, fila 4 0,593, fila 5 0,336).

Orden de aplicación

1) Preflight (parámetros dados, cuenta + símbolo + comisión por empresa, conductor libre, terminal de MT5 cerrado); 2) coste y retest SQX; 3) backtest MT5; 4) recorte de ventana → huso → puntos → desplazamiento de reloj → emparejado; 5) filas 1, 1b, 2, 3a, 3b, 4, 5 → todas deben pasar; 6) `trades_min` ; 7) ledger + pool. Una estrategia que no valida no detiene el lote (`autopilot.run` sigue con la siguiente); una verificación que se cae sí detiene el autopiloto.

Ejemplo de cadena real (pool a 2026-10-11)

- Pool FTMO: 5 estrategias verificadas (`S00V014…S04V014` , NIKKEI225 M30 `every_tick` , 2024-01-01→2026-09-30, corrida 2026-10-04), **0 validadas**; Hantec: 5 verificadas, **0 validadas**. Filas que fallan: fila 1 en 5/5 en ambas empresas, fila 2 en 5/5, fila 3a falla 4/5 en FTMO y 4/5 en Hantec (pasa S01V014), fila 5 falla 4/5 en FTMO y 3/5 en Hantec; filas 3b y 4 pasan en todas. (Calculado aplicando los umbrales vigentes a los valores guardados; el pool guarda los valores, el veredicto se tomó con los umbrales del momento.) Una sexta corrida (`S05V014` , 2026-10-04) cayó (`failed`).
- Otras corridas con `result.json` (no en el pool): USDJPY H1 2022-06-01→2026-09-24, `ohlc_m1` , 2026-09-30, 3 estrategias × 2 empresas por vez, repetidas hasta 3 veces cada una: `16.9.76` y `10.9.72` validan en FTMO (`pass`) y fallan en Hantec; `21.9.59` falla en ambas. Valores `16.9.76` /FTMO: 387/399 operaciones, filas 0,9587 / 0,035 / 0,957 / 0,003 / 0,0004 / 0,073. En esa fecha aún no existía la fila 1b ni `trades_min` ni «primer veredicto». Los veredictos de repetición no cambiaron (tester determinista sobre el mismo histórico).
- Resumen de las 28 carpetas de `mt5/verify` : 9 `failed` (caídas de infraestructura, sin veredicto), 19 `done` : 11 USDJPY H1 (2 empresas; FTMO `pass` en 5 de ellas, Hantec 0), 6 NIKKEI M30 `every_tick` (5 en el pool, ninguna validada), 1 EURUSD, 2 XAUUSD (ninguna validada). Sólo USDJPY H1 produce `pass` , y sólo en FTMO. No hay ninguna corrida real con el flujo completo del autopiloto (ningún `finalistas.json` real consumido; «falta el pool, y una superviviente real», WORKFLOW.md:52).

Tiempos / recursos

- Duración de una verificación (de `run.json` `started→ended`): USDJPY H1 `ohlc_m1` , 2022→2026: ~2 min (3 min la primera); NIKKEI M30 `every_tick` , 2,75 años, 2 empresas: 3–5 min; XAUUSD H1 `ohlc_m1` 2025→2026: 85 min la corrida 2026-09-29 20:15 (con reintentos de infraestructura, no representativa), 1 min otra; EURUSD M30 `ohlc_m1` 1,5 años, 1 empresa: ~1 min. Límite configurado: 60 min por pasada, 240 min Real ticks (`config.yaml: mt5.*`).
- Cores/RAM: SQX en el conductor (16 g de heap), un `Test_` proyecto por estrategia; MT5 bajo Wine, una pasada por empresa en serie, un terminal (no se solapan). Sin medida de RAM/cores propios del paso en `perf/` (sin dato).

- Escala: 1 estrategia × E empresas por proceso; el autopiloto verifica los finalistas **en serie**; con 1.000+ supervivientes el coste crece lineal (≥ 2–5 min por estrategia → 30–80 h para 1.000; no verificado como estimación del dueño, derivada de los tiempos medidos).

Discrepancias del 26 (⚠ resumen)

1. **Fila 1b y `trades_min` están en código y umbrales pero sin corrida real** y el pool no guarda `row_1b` (`pool.FIGURES`).
2. **Fila 5 no es intradía/flotante** (WORKFLOW.md:52 y encargo 34 §3 lo prometen).
3. **Pool**: WORKFLOW y el dossier 26 §5.17/5.10 hablaban del último/«cualquier» pass; el código vigente toma el **primer** veredicto por (identidad, ventana, modelo) (`pool.read`). Los 10 registros existentes se tomaron con umbrales anteriores a 1b/`trades_min`.
4. Filas **3b y 4 sin potencia** a tamaño 0,1 lote y cuenta de 100.000 USD (no pueden fallar); sólo la 1, 2, 3a y 5 discriminan.
5. Umbrales de 1b y `trades_min` provisionales («pendiente de la corrida de referencia por activo × empresa»); el resto son «valor de partida» del dueño, nunca calibrados con una corrida con pares verdaderos conocidos; potencia de la fila 1 sin planear (con ≤ 150 operaciones separa mal 93 % de 95 %).
6. **Spread «actual» leído del servidor el día de la verificación**, no el histórico de la ventana (dossier §5.3): una ventana de años se valida con el spread de hoy; el swap del servidor también es de hoy.
7. Slippage = 0 en SQX y el tester de MT5 no desliza: la comparación no valida el slippage que luego se paga en real; comisión y swap «se traducen y no se juzgan» (el P&L comparado va en puntos, sin ellos).
8. Selección del desplazamiento de reloj sobre el mismo dato que se juzga (C0).
9. `finalists.py` sigue diciendo «after step 25»; el paso 25 ya no está en la cadena; el paso 24 no existe, así que «el .sqx de la madre tal cual» es la entrada.
10. Un único primer veredicto cuenta: no hay límite documentado de intentos con otros umbrales o ventanas (el ledger anota cada comparación, nada los cuenta como ensayos; dossier §5.9, no verificado en código).